



2ND EDITION

Cloud Security Handbook

Effectively secure cloud environments
using AWS, Azure, and GCP

EYAL ESTRIN

Cloud Security Handbook

Second Edition

Effectively secure cloud environments using
AWS, Azure, and GCP

Eyal Estrin



Cloud Security Handbook Second Edition

Copyright © 2025 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Portfolio Director: Vijin Boricha

Relationship Lead: Niranjana Naikwadi

Project Manager: Gandhali Raut

Content Engineer: Tazeen Shaikh

Technical Editor: Nithik Cheruvakodan

Copy Editor: Safis Editing

Indexer: Pratik Shirodkar

Proofreader: Tazeen Shaikh

Production Designer: Shankar Kalbhor

Growth Lead: Ankita Thakur

First published: April 2022

Second edition: April 2025

Production reference: 1250425

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83620-001-7

www.packtpub.com

To my beloved wife, Diana, whose unwavering support and encouragement have been the bedrock of my journey. You have always pushed me to chase my dreams, believing in me even when I doubted myself.

To my parents, Avi and Yael, thank you for instilling in me the values of hard work and perseverance. Your love and guidance have shaped me into the man I am today.

This book is dedicated to all of you, as a testament to your love and the profound impact you have had on my life.

Eyal Estrin

Contributors

About the author

Eyal Estrin is a seasoned cloud security architect and an influential voice in the field of cloud computing and information security. With over 20 years of experience in the IT industry, he has been working with public clouds since 2015, gaining hands-on experience with AWS, Azure, and GCP. Eyal is also recognized as an AWS Community Builder, sharing his insights through public columns and engaging with the tech community about cloud security and cloud adoption. His books provide comprehensive comparisons of major cloud providers and best practices for securing cloud environments.

To connect with Eyal and explore more of his work and online presence, visit:

<https://linktr.ee/eyalestrin>

About the reviewers

Chandrapal Badshah is a cloud security consultant specializing in defending cloud environments for start-ups. With a background in product security, he has worked extensively on securing cloud-native infrastructures, improving security postures, setting up guardrails, and operationalizing cloud security. He has delivered talks and trainings on both offensive and defensive cloud security at international security conferences, sharing his expertise in securing and attacking cloud environments.

He writes blogs at <https://badshah.io>.

Vasileios Giannakopoulos is an experienced information security and privacy professional with over 15 years of experience in IT risk management, cybersecurity, IT infrastructures, and data protection. He specializes in integrating security and privacy, implementing ISO 27001, and ensuring compliance with GDPR and global standards. Vasileios has held senior roles including information security and privacy officer and senior security architect, demonstrating expertise in infrastructure security, risk assessment, and privacy governance across diverse industries.

Table of Contents

Part 1: Securing Infrastructure Cloud Services

1

Introduction to Cloud Security		3
What is a cloud service?	5	AWS and the shared responsibility model 10
What are the cloud deployment models?	5	Azure and the shared responsibility model 11
		GCP and the shared responsibility model 11
What are the most common cloud service models?	6	Command-line tools 13
Why we need security	7	AWS CLI 13
What is the shared responsibility model?	8	Azure CLI 13
		Google Cloud SDK 14
		Summary 14

2

Securing Compute Services – Virtual Machines		15
Securing VMs	15	Azure VMs 19
Fundamental best practices for securing VMs	16	GCE 19
Amazon EC2	17	Best practices for securing network access to an instance 20
Azure VMs	17	Amazon EC2 21
GCE	18	Azure VMs 23
Best practices for authentication	18	GCE 24
Amazon EC2	18	

Best practices for conducting patch management	25	Best practices for securing backups	26
Amazon EC2	25	AWS Backup	27
Azure VMs	25	Azure Backup	28
GCE	26	Google Cloud Backup and DR	29
		Summary	29

3

Securing Compute Services – Containers and Kubernetes 31

Securing containers	31	Implementing secure configuration in Kubernetes	49
Securing Kubernetes	33	Securing non-Kubernetes workloads	50
Kubernetes terminology	33	Securing Amazon Elastic Container Service (ECS)	51
General best practices for Kubernetes	35	Securing Azure Container Apps	54
Best practices for identity and access management in Kubernetes	37	Securing Google Cloud Run	57
Best practices for data protection in Kubernetes	41	Summary	59
Best practices for auditing and security monitoring in Kubernetes	46		

4

Securing Compute Services – Serverless and FaaS 61

Securing serverless/FaaS	61	AWS Lambda	69
High-level recommendations for securing FaaS	63	Azure Functions	70
AWS Lambda	64	Google Cloud Run functions	72
Azure Functions	64	Best practices for auditing and security monitoring in FaaS	72
Google Cloud Run functions	64	AWS Lambda	73
Best practices for IAM in FaaS	64	Azure Functions	74
AWS Lambda	65	Google Cloud Run functions	75
Azure Functions	67	Best practices for compliance in FaaS	75
Google Cloud Run functions	68	AWS Lambda	76
Best practices for data protection in FaaS	68	Summary	76

5

Securing Storage Services 77

Securing object storage	78	Best practices for securing Google's Persistent Disk	96
General best practices for securing object storage	78	Securing file storage	97
Best practices for IAM in object storage	80	General best practices for securing file storage	98
Best practices for data protection in object storage	83	Best practices for IAM in file storage	99
Best practices for auditing and security monitoring in object storage	88	Best practices for data protection in file storage	103
Securing block storage	92	Best practices for auditing and security monitoring in file storage	107
General best practices for securing block storage	93	Securing the CSI	110
Best practices for securing Amazon Elastic Block Store	93	General best practices for securing a CSI driver	110
Best practices for securing Azure-managed disks	95	Securing Amazon EKS and the CSI driver	111
		Securing AKS and the CSI driver	112
		Securing GKE and the CSI driver	113
		Summary	114

6

Securing Networking Services – Part 1 115

Securing virtual networking	115	Securing AWS Site-to-Site VPN	136
Amazon Virtual Private Cloud	117	Securing AWS Client VPN	138
Azure VNet	118	Securing Azure VPN Gateway (Site-to-Site)	140
Google Cloud VPC	119	Securing Azure VPN Point-to-Site	141
General best practices for securing virtual networks	120	Securing Google Cloud VPN	142
Best practices for monitoring virtual networks	126	Securing Zero Trust services	143
Securing DNS services	130	Securing AWS Verified Access	144
Securing Amazon Route 53	131	Securing Global Secure Access	146
Securing Azure DNS	133	Securing BeyondCorp	148
Securing Google Cloud DNS	134	Summary	150
Securing VPN services	135		

7

Securing Networking Services – Part 2 151

Using DDoS protection services	151	Using AWS WAF	159
Using AWS Shield	153	Using Azure Application Gateway WAF v2	162
Using Azure DDoS Protection	155	Using Azure Front Door	163
Using Google Cloud Armor (for DDoS protection)	157	Using Google Cloud Armor (as a managed WAF)	165
Using WAF services	158	Summary	166

8

Securing Generative AI Services 167

Introduction to GenAI	167	Amazon Bedrock	174
Foundation models	167	Azure OpenAI	174
Large language models	168	Google Vertex AI	175
Prompt	168	Best practices for data protection in GenAI services	176
Response	168	Amazon Bedrock	177
Example of a GenAI workflow	169	Azure OpenAI	177
Introduction to managed GenAI services	170	Google Vertex AI	178
Considerations for using data to train managed GenAI services	170	Best practices for audit and security monitoring in GenAI services	179
Risks related to the use of GenAI	171	Amazon Bedrock	180
High-level best practices for securing GenAI applications	171	Azure OpenAI	180
Amazon Bedrock	172	Google Vertex AI	181
Azure OpenAI	173	Summary	182
Google Vertex AI	173		
Best practices for IAM in GenAI services	173		

Part 2: Deep Dive into IAM, Auditing, and Encryption

9

Effective Strategies for Implementing IAM Solutions 185

Introduction to IAM	185	Securing IAM in GCP	199
Failing to manage identities	186	Securing directory services	203
Identities in cloud environments	187	Securing AWS Directory Service	205
Authentication process	187	Securing Microsoft Entra Domain Services	207
Authorization process	187	Securing Google Managed Service for Microsoft Active Directory	208
Securing cloud-based IAM services	188	Summary	209
Securing AWS IAM	189		
Securing Microsoft Entra ID	194		

10

Auditing and Threat Management in Cloud Environments 211

Maintaining audit trails	211	Conducting threat detection and response	224
General best practices for audit trails in cloud environments	212	Amazon GuardDuty	224
AWS CloudTrail	213	Microsoft Defender for Cloud	226
Azure Monitor	215	Google Security Command Center	229
Google Cloud Audit Logs	217	Managing cloud-native SIEM	232
Controlling access to CSP support engineers	219	Microsoft Sentinel	233
AWS Support	219	Google Security Operations	237
Customer Lockbox (in Azure)	221	Summary	239
Google Access Transparency and Access Approval	223		

11

Applying Encryption in Cloud Services		241
Introduction to encryption	242	Google Cloud KMS 257
Symmetric encryption	243	
Asymmetric encryption	245	
Best practices for using encryption in transit	247	Best practices for deploying secrets management services 259
Internet Protocol security (IPSec)	247	AWS Secrets Manager 260
Transport Layer Security (TLS)	247	Azure Key Vault (as a secrets management service) 261
Encryption at rest	249	Google Secret Manager 262
Best practices for deploying KMSs	250	Encryption in use 264
General best practices for securing KMSs	250	AWS Nitro Enclaves 264
AWS KMS	251	Azure confidential computing 266
Azure Key Vault (as a key management service)	254	Google Confidential Computing 268
		Summary 270

Part 3: Threat and Vendor Management

12

Understanding Common Security Threats to Cloud Services		275
Data breaches in cloud services	276	Example of a real event – Football Australia 284
Common consequences of data breaches	277	Best practices for detecting and mitigating insufficient IAM, secrets, and key management in cloud environments 285
Example of a real event – Drizly	277	
Best practices for detecting and mitigating data breaches in cloud environments	278	
Misconfigurations in cloud services	280	Account hijacking in cloud services 287
Common consequences of misconfiguration	281	Common consequences of account hijacking 287
Example of a real event – CommuteAir	281	Example of a real event – Retool MFA 288
Best practices for detecting and mitigating misconfigurations in cloud environments	282	Best practices for detecting and mitigating account hijacking in cloud environments 288
Insufficient IAM, secrets, and key management	284	Insider threats in cloud services 290
Common consequences of insufficient IAM and key management	284	Common consequences of insider threats 291
		Example of a real event – First Republic Bank 291
		Best practices for detecting and mitigating insider attacks in cloud environments 291

Insecure APIs in cloud services	294	Common consequences of abuse of cloud services	298
Common consequences of insecure APIs	294	Example of a real event – UNC2903	298
Example of a real event – Trello	294	Best practices for detecting and mitigating abuse of cloud services	299
Best practices for detecting and mitigating insecure APIs in cloud environments	295	The MITRE ATT&CK framework	301
The abuse of cloud services	297	Summary	302

13

Engaging with Cloud Providers 303

Choosing a cloud provider	304	What is a cloud provider questionnaire?	309
What is the most suitable cloud service model for our needs?	304	What are SOC reports?	316
Data privacy and data sovereignty	306	Tips for contracts with cloud providers	317
Auditing and monitoring	307	Conducting penetration testing in cloud environments	319
Migration capabilities	308	Summary	322
Authentication	309		

Part 4: Advanced Use of Cloud Services

14

Managing Hybrid Clouds 325

Hybrid cloud strategy	327	Managing identity over hybrid Azure environments	333
Cloud bursting	327	Managing identity over GCP hybrid environments	335
Backup and disaster recovery	327	Network architecture for hybrid cloud environments	337
Archive and data retention	328	Connecting the on-premises environment to AWS	338
Distributed data processing	328	Connecting the on-premises environment to Azure	340
Application modernization	328	Connecting the on-premises environment to GCP	342
Identity management over hybrid cloud environments	329		
Best practices for identity in a hybrid environment	330		
Managing identity over hybrid AWS environments	330		

Storage services for hybrid cloud environments	344	Using computing services over AWS hybrid environments	349
Connecting to storage services over AWS hybrid environments	344	Using computing services over Azure hybrid environments	351
Connecting to storage services over Azure hybrid environments	346	Using computing services over GCP hybrid environments	352
Connecting to storage services over GCP hybrid environments	348	Securing hybrid cloud environments	354
Computing services for hybrid cloud environments	349	Securing AWS hybrid environments	354
		Securing Azure hybrid environments	355
		Securing GCP hybrid environments	356
		Summary	357

15

Managing Multi-Cloud Environments **359**

Multi-cloud strategy	360	Creating network connectivity between AWS and Azure	375
Freedom to select a cloud provider	361	How to create network connectivity between Azure and GCP	377
Freedom to select your services	361	Data security in multi-cloud environments	378
Data sovereignty	361	Data classification and government	378
BC and DR	362	Encryption in transit	379
Improving reliability	362	Encryption at rest	379
Identity management	362	Encryption in use	380
Data security	363	Cost management in multi-cloud environments	380
Asset management	363	CSPM	382
Skills gap	364	CIEM	383
IAM over multi-cloud environments	364	Patch and configuration management in multi-cloud environments	384
Managing identity in AWS over multi-cloud environments	366	The monitoring and auditing of multi-cloud environments	387
Managing identity in Azure over multi-cloud environments	368	Summary	388
Managing identity in GCP over multi-cloud environments	370		
Network architecture for multi-cloud environments	372		
Creating network connectivity between AWS and GCP	373		

16

Implementing DevSecOps 389

Introduction to DevSecOps	390	Implementing security as part of the test phase	398
DevOps and CI/CD	390	Implementing security as part of the release phase	399
The goals of DevSecOps	392	Implementing security as part of the deployment phase	400
DevSecOps best practices – people	393	Implementing security as part of the maintenance phase	400
Adopting a security-by-design approach	393	DevSecOps best practices – technology	401
Shared responsibility	393	Amazon Q Developer	401
Cultural shift	394	GitHub Copilot	401
Empowering development and operations teams	394	Amazon Inspector	402
DevSecOps best practices – processes	395	Microsoft Defender for Cloud	402
Implementing security as part of the planning phase	396	Google Artifact Analysis	402
Implementing security as part of the coding phase	396	Open source tools	403
Implementing security as part of the build phase	397	Summary	405

17

Security in Large-Scale Environments 407

Managing governance and policies at a large scale	408	Azure Resource Manager (ARM) templates	426
Governance in AWS	410	Google Cloud Deployment Manager	428
Governance in Azure	415	HashiCorp Terraform	429
Governance in Google Cloud	419	HashiCorp Sentinel policies	430
Automation using IaC and PaC	422	Summary	431
IaC	422	What is next?	431
PaC	423	Plan ahead	431
Common best practices for IaC and PaC in AWS, Google, and Azure	423	Automate	431
AWS CloudFormation	424	Think big	432
		Continue learning	432

Index	433
Other Books You May Enjoy	456

Preface

Cloud Security Handbook provides complete coverage of security aspects when designing, building, and maintaining environments in the cloud. This book is filled with best practices to help you smoothly transition to the public cloud while keeping your environments secure. You do not have to read everything – simply find out which cloud provider is most widely used at your workplace, or which cloud provider you wish to focus on, and feel free to skip the rest.

Who this book is for

This book is designed for IT and information security professionals who are beginning their journey into the public cloud or transitioning existing environments to the cloud. Additionally, DevOps practitioners, cloud engineers, and cloud architects responsible for managing production environments in the cloud will find this book valuable.

What this book covers

Chapter 1, Introduction to Cloud Security, provides a comprehensive overview of cloud security concepts, including **Infrastructure as a Service (IaaS)**, **Platform as a Service (PaaS)**, **Software as a Service (SaaS)**, private cloud, public cloud, hybrid cloud, multi-cloud, and the Shared Responsibility Model. This chapter, along with the others in the book, will equip you with the knowledge needed to implement security measures across different cloud environments.

Chapter 2, Securing Compute Services – Virtual Machines, explains how **Amazon Web Services (AWS)**, **Microsoft Azure**, and **Google Cloud Platform (GCP)** set up virtual machines. It also discusses the best ways to keep these virtual machines secure.

Chapter 3, Securing Compute Services – Containers and Kubernetes, explains how AWS, Azure, and GCP use containers and Kubernetes. It also shares the best practices for keeping these technologies secure.

Chapter 4, Securing Compute Services – Serverless and FaaS, explains how AWS, Azure, and GCP use **Function as a Service (FaaS)**. It also outlines the best practices for keeping these serverless services secure.

Chapter 5, Securing Storage Services, explains how AWS, Azure, and GCP handle different types of storage, including object storage, block storage, and file storage, as well as Container Storage Interface (CSI). It also discusses the best practices for keeping these storage services secure.

Chapter 6, Securing Networking Services – Part 1, explains how AWS, Azure, and GCP set up virtual networking, DNS services, and VPN services. It also discusses the best practices for keeping these services secure.

Chapter 7, Securing Networking Services – Part 2, explains how AWS, Azure, and GCP provide protection against DDoS attacks and use web application firewalls. It also covers the best practices for keeping these services secure.

Chapter 8, Securing Generative AI Services, explains how AWS, Azure, and GCP use Generative AI services. It also discusses the best practices for keeping these services secure.

Chapter 9, Effective Strategies for Implementing IAM Solutions, explains how AWS, Azure, and GCP use **Identity and Access Management (IAM)** systems, cloud-based IAM services, and directory services. It also discusses how to secure these services effectively.

Chapter 10, Auditing and Threat Management in Cloud Environments, explains how AWS, Azure, and GCP set up audit tools. It covers how to control access to cloud service provider support engineers, detect and respond to threats, manage cloud-native **Security Information and Event Management (SIEM)** systems, and ensure the security of these monitoring and auditing services.

Chapter 11, Applying Encryption in Cloud Services, explains when to use two types of encryption in cloud environments: symmetric (using one key) and asymmetric (using two keys). It also describes the different options for managing encryption keys and secrets in AWS, Azure, and GCP. The chapter covers best practices for keeping data secure when it is moving between places (encryption in transit) and when it is stored (encryption at rest), as well as how to secure data in use.

Chapter 12, Understanding Common Security Threats to Cloud Services, explains the typical security risks in public cloud environments. It describes how to identify these threats and discusses ways to protect against them using built-in security services from AWS, Azure, and GCP.

Chapter 13, Engaging with Cloud Providers, guides you through the process of selecting the right cloud service provider by exploring essential aspects such as understanding cloud provider questionnaires, deciphering SOC reports, identifying critical contractual agreement elements, and learning how to perform effective penetration testing in cloud environments. The chapter provides comprehensive insights into making informed decisions when partnering with cloud service providers, ensuring that organizations can evaluate, negotiate, and secure their cloud infrastructure strategically.

Chapter 14, Managing Hybrid Clouds, explains how to set up key features such as IAM, network architecture, storage services, and compute services in hybrid cloud environments. It also discusses the best practices for securing these hybrid setups, helping organizations effectively manage and protect their resources across both on-premises and cloud platforms.

Chapter 15, Managing Multi-Cloud Environments, explains how to handle important topics such as IAM, network architecture, data security, cost management, and various management practices such as **Cloud Security Posture Management (CSPM)**, **Cloud Infrastructure Entitlement Management (CIEM)**, patch and configuration management, and monitoring and auditing in multi-cloud settings. This chapter provides guidance on effectively managing and securing resources spread across multiple cloud platforms.

Chapter 16, Implementing DevSecOps, offers a comprehensive overview of DevSecOps, explaining it as a collaborative approach that integrates security practices throughout the software development life cycle. The chapter explores DevSecOps as a cultural transformation, demonstrating how to incorporate security measures into **Continuous Integration** and **Continuous Deployment (CI/CD)** pipelines. It also highlights specific DevSecOps services from major cloud providers such as AWS, Azure, and GCP, and showcases open source tools that can be integrated into development workflows to enhance security.

Chapter 17, Security in Large-Scale Environments, explores how to manage security in complex cloud setups across AWS, Microsoft Azure, and GCP. The chapter explains key concepts such as Landing Zones, **Infrastructure as Code (IaC)**, and policy as code, and provides practical guidance on implementing security best practices for managing large-scale cloud environments.

To get the most out of this book

Software/hardware covered in the book	Operating system requirements
An up-to-date web browser	Windows, macOS, or Linux
Credentials to access the AWS, Azure, and GCP web consoles	

Download the example code files

Check out this book's GitHub repository for updates and errata related to the book: <https://github.com/PacktPublishing/Cloud-Security-Handbook>.

We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out.

Conventions used

There are a number of text conventions used throughout this book.

CodeInText: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and X handles. For example: “Execute the `terraform graph` command.”

Any command-line input or output is written as follows:

```
aws ec2 enable-ebs-encryption-by-default --region <Region_Code>
```

Bold: Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: “Azure CLI can be installed on Windows, Linux (**Ubuntu**, **Debian**, **RHEL**, **CentOS**, **OpenSUSE**), and macOS.”

Tips or important notes

Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: Email feedback@packtpub.com and mention the book's title in the subject of your message. If you have questions about any aspect of this book, please email us at questions@packtpub.com.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit <http://www.packtpub.com/submit-errata>, click **Submit Errata**, and fill in the form. We ensure that all valid errata are promptly updated in the GitHub repository: <https://github.com/PacktPublishing/Cloud-Security-Handbook>.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packtpub.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packtpub.com/>.

Share your thoughts

Once you've read *Cloud Security Handbook Second Edition*, we'd love to hear your thoughts! Please [click here](#) to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/9781836200017>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

Part 1:

Securing Infrastructure Cloud Services

Cloud security is essential as organizations rely on virtual machines, containers, serverless computing, storage, networking, and generative AI services. Each of these technologies brings unique security challenges that require strong protection. This part covers key strategies and best practices to help secure cloud infrastructure and stay ahead of evolving threats.

This part comprises the following chapters:

- *Chapter 1, Introduction to Cloud Security*
- *Chapter 2, Securing Compute Services – Virtual Machines*
- *Chapter 3, Securing Compute Services – Containers and Kubernetes*
- *Chapter 4, Securing Compute Services – Serverless and FaaS*
- *Chapter 5, Securing Storage Services*
- *Chapter 6, Securing Networking Services – Part 1*
- *Chapter 7, Securing Networking Services – Part 2*
- *Chapter 8, Securing Generative AI Services*

Introduction to Cloud Security

This book, *Cloud Security Handbook*, is meant for various audiences. You could be taking your first steps working with cloud services, or you could be coming from an IT perspective and want to know about various computing and storage services and how to configure them securely. You might be working in information security and want to know the various authentication, encryption, and audit services and how to configure them securely, or you might be working with architecture and want to know how to design large-scale environments in the cloud in a secure way.

Reading this book will allow you to make the most of cloud services while focusing on security aspects. Before discussing cloud services in more detail, let me share my opinion regarding cloud services.

The world of IT is evolving. For decades, organizations used to purchase physical hardware, install operating systems, and deploy software. This routine required a lot of ongoing maintenance (such as patch deployment, backup, hardening, monitoring, and so on).

The traditional data center had many constraints, including the following, among others:

- **Physical hardware:** Organizations were bound to the hardware they purchased. In case hardware capacity was not enough, organizations had to purchase additional hardware, up to the physical space limit in their data centers.
- **Utilization:** In most cases, hardware was not utilized 100% (i.e., not running 24x7, using 100% of hardware resources) or overprovisioned (i.e., provision more hardware resources than actually needed by the applications).
- **Cost:** The purchase of new hardware was done several years in advance, regardless of the actual use.
- **Maintainability:** Organizations had to maintain everything from the physical data centers, hardware, software, and their configurations.

The cloud introduced a new paradigm – that is, the ability to build modern applications or consume managed services to achieve the same goal of running software (from file servers to **enterprise resource planning (ERP)** or **customer relationship management (CRM)** products) while using the expertise of the hyper-scale cloud providers.

Some well-known customer stories of cloud computing are as follows:

- **Netflix:** One of the largest video streaming services worldwide. It uses **Amazon Web Services (AWS)** to run its media streaming services: <https://aws.amazon.com/solutions/case-studies/innovators/netflix/>.
- **Mercedes-Benz:** One of the most famous automotive brands. It uses Azure OpenAI service to transform the in-car experience for drivers: <https://azure.microsoft.com/en-us/blog/mercedes-benz-enhances-drivers-experience-with-azure-openai-service/>.
- **Home Depot:** The largest home improvement retailer in the United States. It uses Google Cloud to run its online stores: <https://www.googlecloudpresscorner.com/2024-04-11-The-Home-Depot-Extends-Relationship-with-Google-Cloud-to-Drive-Innovation-in-Interconnected-Retail>.

Securing cloud environments can be challenging, depending on your previous knowledge in IT, information security, or cloud services in general. Reading this book will assist you in gaining the necessary knowledge of how to secure cloud environments, regardless of your role in the organization or your previous experience.

In this book, we will compare various aspects of cloud computing (from fundamental services, such as computing, storage, to networking, and best practices for building and securely maintaining large-scale environments), while reviewing the different alternatives offered by **AWS**, **Microsoft Azure**, and **Google Cloud Platform (GCP)**.

It does not matter which organization you are coming from – this book will allow you to have a better understanding of how to embed security in any of the large hyper-scale cloud providers.

You do not have to read everything – simply find out which cloud provider is common at your workplace or which cloud provider you wish to focus on, and feel free to skip the rest.

In this chapter, we will cover the following topics:

- What is a cloud service?
- What are the cloud deployment models?
- What are the cloud service models?
- Cloud deployment models
- The shared responsibility model

What is a cloud service?

As part of this introduction, let us set the terminology to make sure we are all on the same page.

The **National Institute of Standards and Technology (NIST)** defines **cloud** as a technology that has the following five characteristics:

- **On-demand self-service:** Imagine you wish to start a blog, and you need computing resources. Instead of purchasing hardware and waiting for the vendor to ship it to your office and having to deploy software, the easier alternative can be a self-service portal, where you can select a pre-installed operating system and content management system that you can deploy within a few minutes by yourself.
- **Broad network access:** Consider having enough network access (the type that large **internet service providers (ISPs)** have) to serve millions of end users with your application.
- **Resource pooling:** Consider having thousands of computers, running in a large data center, and being able to maximize their use (from CPU, memory, and storage capacity), instead of having a single server running 10% of its CPU utilization.
- **Rapid elasticity:** Consider having the ability to increase and decrease the amount of computing resources (from a single server to thousands of servers, and then back to a single server), all according to your application or service needs.
- **Measured service:** Consider having the ability to pay for only the resources you consumed and being able to generate a billing report that shows which resources have been used and how much you need to pay for the resources.

Further details relating to the NIST definition can be found at the following link: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>.

Now that we understand what the cloud characteristics are, let's talk about cloud deployment models.

What are the cloud deployment models?

To have a better understanding of the cloud, let us review the different cloud deployment models:

- **Private cloud:** An infrastructure deployed and maintained by a single organization. A private cloud, as opposed to the traditional data center, offers scalability (i.e., add/remove resources as needed), cost savings (i.e., no need for upfront investment), and better manageability (i.e., the ability to manage all resources from a single portal, including deployment and monitoring). Let's say we are a large financial organization (such as a bank or insurance organization); we would like to serve various departments in our organization (from HR, IT, sales, and so on) and we might have regulatory requirements to keep customers' data on-premises – a private cloud can be a suitable solution.

- **Public cloud:** An infrastructure deployed and maintained by a service provider for serving single or multiple customers and/or organizations, mostly accessible over the internet. Naturally, this book will focus on the public cloud model, regarding various services offered by AWS, Azure, and GCP.
- **Hybrid cloud:** A combination of a private cloud (or on-premises cloud) and at least one public cloud infrastructure. I like to consider the hybrid cloud as an extension of the local data center. We should not consider this extension as something separate, and we should protect it the same way we protect our local data center.
- **Multi-cloud:** This is preferred in a scenario where our organization is either using multiple managed services (see the definition of **software as a service (SaaS)** in the next section) or using multiple public cloud infrastructures (see the definition of **infrastructure as a service (IaaS)** and **platform as a service (PaaS)** in the next section) to share information between the various cloud providers (such as a database from one cloud provider, and an analytic service in another cloud provider).

Next, let us look at some cloud service models, including some examples of common cloud service models.

What are the most common cloud service models?

An essential part of understanding clouds is understanding the three most common cloud service models:

- **IaaS:** This is the most fundamental service model, where a customer can select the virtual machine size (in terms of the amount of CPU and memory), select a pre-configured operating system, and deploy software inside the VM instance according to business needs (services such as **Amazon Elastic Compute Cloud (EC2)**, **Azure Virtual Machines**, and **Google Compute Engine**).
- **PaaS:** This type of service model varies from managed database services to managed application services (where a customer can import code and run it inside a managed environment), and more (services such as **AWS App Runner**, **Azure App Service**, and **Google App Engine**).
- **SaaS:** This is the most widely used service model – a fully managed software environment where, as a customer, you usually open a web browser or a mobile app, log in to an application, and consume services. These could be messaging services, ERP, CRM, business analytics, and more (services such as **Microsoft 365**, **Google Workspace**, **Salesforce CRM**, **SAP SuccessFactors**, and **Oracle Fusion Cloud HCM (Human Capital Management)**).

Understanding the cloud service models will allow you to understand your role as a customer, as explained later in the *What is the shared responsibility model?* section. For now, let us try to understand why we need cloud security.

Why we need security

As mentioned previously, we can see clear benefits of using cloud services that enable our business to focus on what brings us value (from conducting research in a pharmaceutical lab to selling products on a retail site, and so on).

But what about security? Specifically, cloud security? Why should our organization focus on the overhead called *information security* (and, in the context of this book, *cloud security*)?

The cloud has changed the paradigm of organizations controlling their data on-premises (from HR data to customers' data) and investing money in maintaining data centers, servers, storage, network equipment, and the application tier. Using public clouds has changed the way organizations look at information security (in the context of this book, cloud security).

The following are a few common examples of the difference between on-premises data solutions and the cloud:

	Traditional data center	Cloud environment
<i>Who is responsible for physical security?</i>	The organization	The cloud service provider (CSP)
<i>Where is the data located?</i>	On-premises data center	Cloud provider's data center
<i>Who controls access to the data?</i>	The organization	Both the organization and the CSP
<i>Who is responsible for vulnerability management and patch management?</i>	The organization	IaaS – the organization PaaS – the cloud provider SaaS – the cloud provider
<i>Who is in charge of incident response?</i>	The organization	Both the organization and the CSP
<i>Who is responsible for law, privacy, and regulation?</i>	The organization	Both the organization and the CSP

Table 1.1 – Differences between on-premises data solutions and the cloud

In the past, organizations were unwilling to migrate to a public cloud for security reasons because the physical servers were located outside of the organization's direct control, and sometimes even outside their physical geography.

Here are a few questions often asked by organizations' management:

- Are my servers going to behave the same as if they were on-premises?
- How do I protect my servers outside my data center from a data breach?
- How do I know the cloud provider will not have access to my data?
- How can I ensure the cloud provider does not copy data between different locations (with different data protection laws)?
- Do my employees have enough knowledge to work in new environments such as the public cloud?

Perhaps the most obvious question asked is this: *Is the public cloud secure enough to store my data and how much more (or less) secure than on-premises is the cloud?*

By design, the hyper-scale cloud providers invest billions of dollars to protect their data centers and build secure services, investing in employee training, locating security incidents, and remediating them fast. This is all with much higher investment, attention, and expertise than most organizations can dedicate to protecting their local data centers.

The reason for this is simple – if a security breach happens to one of the hyper-scale cloud providers, their customers' trust will be breached, and the cloud providers will run out of business.

At the end of the day, cloud security enables our organization to achieve (among other things) the following:

- **Decreased attack surface:** Using central authentication, data encryption, **Distributed Denial of Service (DDoS)** protection services, and more
- **Compliance with regulation:** Deploying environments according to industry best practices
- **Standardization and best practices:** Enforcing security using automated tools and services

Reading this book will allow you to have a better understanding of various methods to secure your cloud environments – most of them using the cloud vendor's built-in services and capabilities.

Next, let us understand what the shared responsibility model is, and how it relates to the cloud service models.

What is the shared responsibility model?

When speaking about cloud security and cloud service models (IaaS/PaaS/SaaS), one thing that we all hear about is the **shared responsibility model**, which tries to draw a line between the cloud provider and the customer's responsibilities regarding security (but also resiliency, sustainability, and more).

As you can see in the following diagram, the cloud provider is always responsible for the lower layers – from the physical security of their data centers, through networking, storage, host servers, and the virtualization layers:

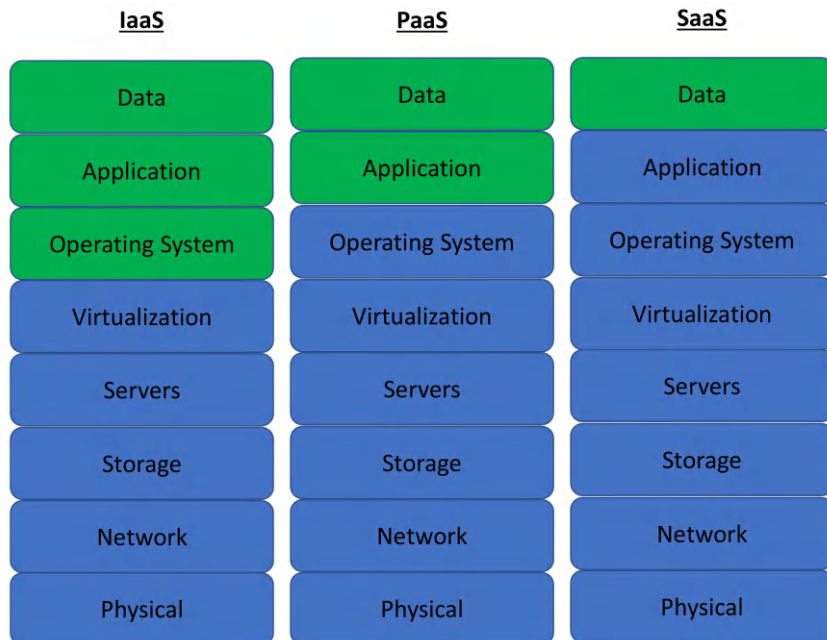


Figure 1.1 – The shared responsibility model

Above the virtualization layer is where the responsibility begins to change (and even shared between the cloud provider and its customers).

The line between the CSP's responsibilities and the customer's responsibilities may not be clear enough (from security, availability, sustainability, etc.). Organizations need to take a risk-based approach to mitigate the risks of the shared responsibility model.

When working with IaaS, we, as the customers, can select a pre-installed image of an operating system (with or without additional software installed inside the image), deploy our applications, and manage permissions to access our data.

When working with PaaS, we, as the customers, may have the ability to control code in a managed environment (services such as AWS App Runner, Azure App Service, or Google App Engine) and manage permissions to access our data.

When working with SaaS, we, as the customers, receive a fully managed service, and all we can do is manage permissions to access our data and decide which data to store in the cloud.

As a customer of cloud services, reading this book will allow you to gain the essential knowledge and best practices for using common cloud services (including computing, storage, networking, authentication, and so on) in a secure way.

Next, we will look at how the various cloud providers (AWS, Azure, and GCP) look at the shared responsibility model.

For more information on the shared responsibility model, you can check out the following link: <https://cloudsecurityalliance.org/blog/2021/04/06/what-an-auditor-should-know-about-cloud-computing-part-1>.

AWS and the shared responsibility model

Looking at the shared responsibility model from AWS's point of view, we can see the clear distinction between AWS's responsibility for the security *of* the cloud (physical hardware and the lower layers such as host servers, storage, database, and network) and the customer's responsibility for security *in* the cloud (everything the customer controls – operating system, data encryption, network firewall rules, and customer data). The following diagram depicts AWS and the shared responsibility model:

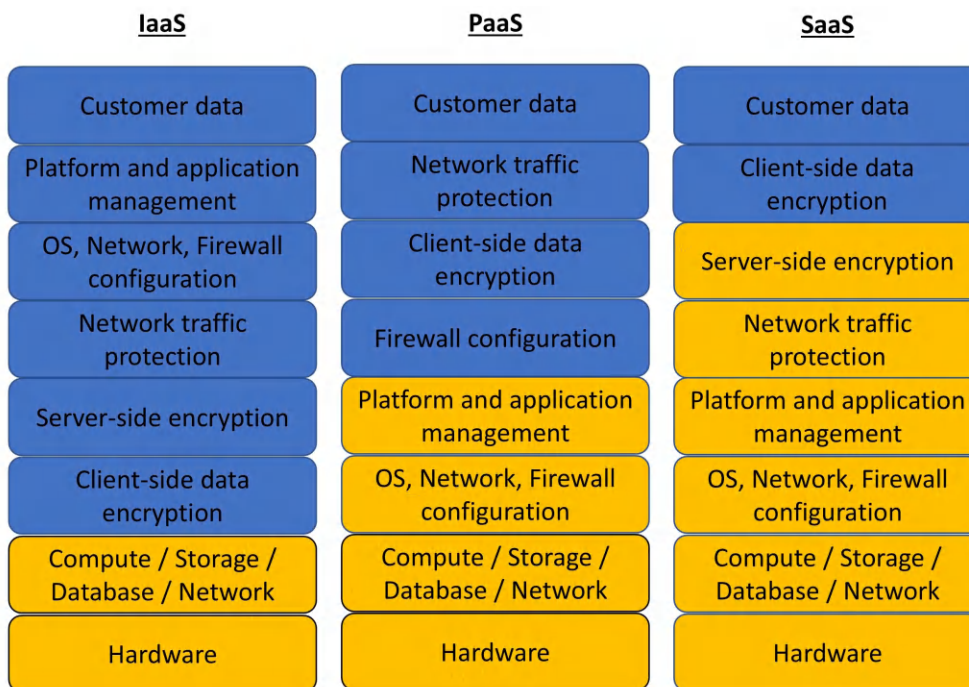


Figure 1.2 – AWS and the shared responsibility model

More information on the AWS shared responsibility model can be found at the following link: <https://aws.amazon.com/blogs/industries/applying-the-aws-shared-responsibility-model-to-your-gxp-solution/>.

Azure and the shared responsibility model

Looking at the shared responsibility model from Azure's point of view, we can see the distinction between Azure's responsibility for its data centers (physical layers) and the customer's responsibility at the top layers (identities, devices, and customers' data). In the middle layers (operating system, network controls, and applications), the responsibility changes between Azure and the customers, according to various service types. The following diagram depicts Azure and the shared responsibility model:

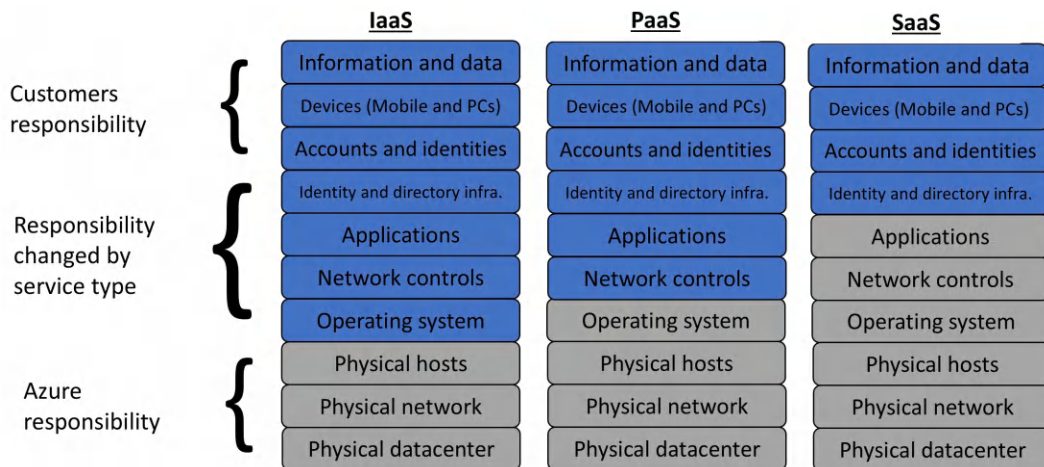


Figure 1.3 – Azure and the shared responsibility model

More information on the Azure shared responsibility model can be found at the following link: <https://docs.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility>.

GCP and the shared responsibility model

Looking at the shared responsibility model from GCP's point of view, we can see that Google would like to emphasize that it builds its hardware, which means it controls the hardware, boot, and kernel of its platform, including the storage layer encryption, network equipment, and logging of everything that Google is responsible for.

When looking at things that the customer is responsible for, we can see a lot more layers, including everything from the guest operating system, network security rules, authentication, identity, and web application security, to things such as deployment, usage, access policies, and content (customers data). The following diagram depicts GCP and the shared responsibility model:

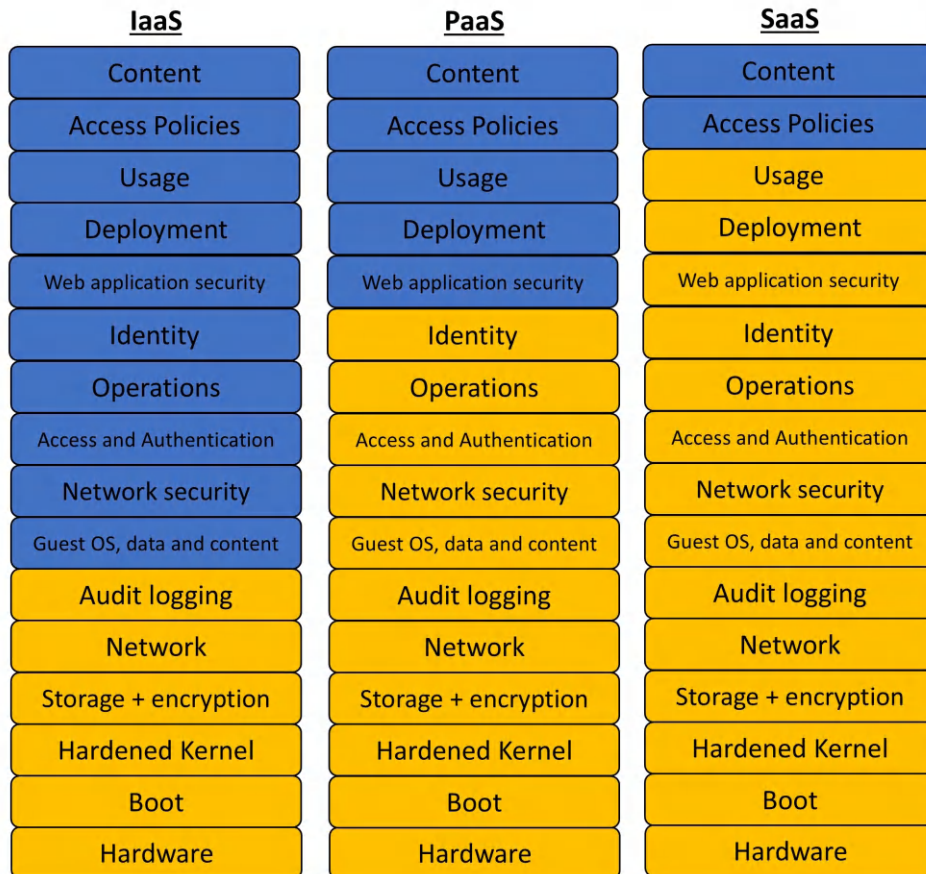


Figure 1.4 – GCP and the shared responsibility model

Google takes the shared responsibility model one step further by defining a new term called *shared fate*, where both Google and its customers are partnered in improving the security of GCP. Google does so by offering its customers a set of blueprints with recommendations for the secure deployment of various services, according to Google's best practices.

More information about the GCP shared responsibility model can be found at the following link: <https://cloud.google.com/architecture/framework/security/shared-responsibility-shared-fate>.

As a customer, understanding the shared responsibility model allows you, at any given time, to understand which layers are under the cloud vendor's responsibility and which layers are under the customer's responsibility. Next, we will introduce command-line tools to automate repetitive tasks while managing cloud environments.

Command-line tools

One of the things that make cloud environments so robust is the ability to control almost anything using the **application programming interface (API)** or the command line.

Most mature cloud providers have already published and maintained their **command-line interface (CLI)** to allow customers to perform actions in an easy and standard way.

An alternative to using the command line to interact with the cloud provider's API is using a **software development kit (SDK)** – a method to control actions (from deploying a virtual machine to encrypting storage), query information from a service (checking if auditing is enabled for my customers' logging into my web application), and more.

Since this book doesn't require previous development experience, I will provide examples for performing actions using the command-line tools.

During various chapters of this book, I will provide you with examples of commands that will allow you to easily implement the various security controls over AWS, Azure, and GCP. I highly recommend that you become familiar with those tools since they allow you to achieve common tasks faster and more efficiently than using the web console.

AWS CLI

AWS CLI can be installed on Windows (64-bit), Linux (both x86 and ARM processors), macOS, and even inside a Docker container.

The AWS CLI documentation explains how to install the tool and provides a detailed explanation of how to use it. The documentation can be found at <https://aws.amazon.com/cli>.

Azure CLI

Azure CLI can be installed on Windows, Linux (**Ubuntu, Debian, RHEL, CentOS, OpenSUSE**), and macOS.

The Azure CLI documentation explains how to install the tool and provides a detailed explanation of how to use it. The documentation can be found at <https://docs.microsoft.com/en-us/cli/azure>.

Google Cloud SDK

The Google Cloud command-line tool (**gcloud CLI**) can be installed on Windows, Linux (Ubuntu, Debian, RHEL, CentOS, Fedora), and macOS.

The Google Cloud SDK documentation explains how to install the tool and provides a detailed explanation of how to use it. The documentation can be found at <https://cloud.google.com/sdk>.

Summary

In this chapter, we learned the definition of a *cloud*, the different cloud deployment models, and the different cloud service models. We also learned what the shared cloud responsibility model is, and how AWS, Azure, and GCP look at this concept from their points of view.

Lastly, we had a short introduction to the AWS, Azure, and GCP built-in command-line tools, and, during the next chapters, I will provide you with examples of how to implement various tasks using the command-line tools.

The foundation set in this chapter will be referred to in the following chapters, where we will dive deeper into the best practices for securing cloud services using (in most cases) the cloud providers' built-in capabilities.

In the next chapter, we will review computing services in the cloud with a focus on Virtual Machines.

Securing Compute Services – Virtual Machines

Speaking about cloud services, specifically **Infrastructure as a Service (IaaS)**, perhaps the most used IaaS service is compute – from the traditional **virtual machines (VMs)** through modern compute services such as containers and, eventually, serverless.

Compute services allow us to deploy and run our applications from a single VM used for development purposes to a large production-scale environment.

This chapter will cover VM services and provide you with the best practices on how to securely deploy and manage each of them.

In this chapter, we will cover the following topics:

- Securing VMs
- Fundamental best practices for securing VMs
- Best practices for authentication
- Best practices for securing network access to an instance
- Best practices for conducting patch management
- Best practices for securing backups

Securing VMs

VMs have the same capabilities as physical machines on-premises, allowing us to deploy our applications while having full control over all aspects (from security to maintainability) inside the VM operating system.

VMs are commonly used for legacy applications or migrating on-premises applications to the cloud. Each cloud provider has its implementation of VMs (or virtual servers), but at the end of the day, the basic idea is the same:

1. Select a machine type (or size) – a ratio between the amount of **virtual CPU (vCPU)** and memory, according to their requirements (general-purpose, compute-optimized, memory-optimized, and so on).
2. Select a preinstalled image of an operating system (from Windows to Linux flavors).
3. Configure storage (adding additional volumes, connecting to file-sharing services, and so on).
4. Configure network settings (from network access controls to micro-segmentation, and others).
5. Configure permissions to access cloud resources.
6. Deploy an application.
7. Begin using the service.
8. Carry out ongoing maintenance of the operating system.

According to the **shared responsibility model**, when using IaaS, we (as the customers) are responsible for the deployment and maintenance of VMs, as explained in the coming section.

Next, we are going to see what the best practices are for securing common VM services in **Amazon Web Services (AWS)**, **Azure**, and **Google Cloud Platform (GCP)**.

Fundamental best practices for securing VMs

In the next couple of sections, we will review the best practices for securing VMs from different angles (authentication, network access, patch management, and backup).

Regardless of the cloud provider you are using for VMs, you should follow the fundamental best practices, which are relevant to any VM that you will be deploying based on any of the hyperscale cloud providers:

- Use the latest hardware generation of VMs to gain the most performance, cost, and security capabilities.
- For highly sensitive environments, use a VM family that supports confidential computing capabilities, to protect data in use (as explained in *Chapter 11, Applying Encryption in Cloud Services*).
- Use a VM image from a trusted source. Most **cloud service providers (CSPs)** maintain up-to-date images of both Windows and Linux flavors for use by their customers.
- When creating a custom VM image, use a minimal number of packages inside the VM image to lower the attack surface.

- Enforce encryption at rest for all disk volumes of VMs to protect data confidentiality (as explained in *Chapter 5, Securing Storage Services*).
- Enforce the use of SELinux on Linux VMs to protect the VMs from unauthorized activities at the kernel level. For more information, see <https://www.redhat.com/en/topics/linux/what-is-selinux>.
- Join Windows VMs to an **Active Directory (AD)** domain, and centrally enforce group policies to restrict OS capabilities from unauthorized access. For more information, see <https://www.microsoft.com/en-us/download/details.aspx?id=104005>.
- Use the CSP's built-in agents (backup, patch management, hardening, monitoring, and so on).
- Disable all unneeded services to decrease the attack surface on VMs.
- For highly sensitive environments, select a **Center for Internet Security (CIS)**-hardened image from the CSP's marketplace. For more information, see <https://www.cisecurity.org/cis-benchmarks>.

Now, let us look at the best practices for securing Amazon **Elastic Compute Cloud (EC2)**, **Azure Virtual Machines (Azure VMs)**, and **Google Compute Engine (GCE)** instances.

Amazon EC2

Amazon EC2 is the AWS implementation of VMs in the cloud.

Before we deep dive into security recommendations for Amazon EC2, it is recommended to consult the following reference to learn about the best practices for securing Amazon EC2 instances:

- *Security in Amazon EC2*: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security.html>

Azure VMs

Azure VMs is the Azure implementation of VMs in the cloud.

Before we dive deep into security recommendations for Azure VMs, it is recommended to consult the following references to learn about the best practices for securing Azure VMs:

- *Azure security baseline for Virtual Machines: Windows Virtual Machines*: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/virtual-machines-windows-virtual-machines-security-baseline>
- *Azure security baseline for Virtual Machines: Linux Virtual Machines*: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/virtual-machines-linux-virtual-machines-security-baseline>

GCE

GCE is the GCP implementation of VMs in the cloud.

Before we dive deep into security recommendations for GCE, it is recommended to consult the following reference to learn about the best practices for securing GCE:

- *Image management best practices*: <https://cloud.google.com/compute/docs/images/image-management-best-practices>

Best practices for authentication

Authentication refers to the action of performing login to a VM, using either local authentication (using **Secure Shell (SSH)** key authentication to Linux machines) or domain-joined authentication (that is, authentication against a central directory service).

CSPs do not have access to customers' VMs. When deploying a new VM instance, you need to configure credentials to access the VM for remote control, login (as a local or domain identity), and maintenance of the VM.

The most common way to locally authenticate to a Windows VM is using a username and password, while in a Linux VM, the most common way is to use SSH key authentication.

Amazon, Azure, and Google support the following methods to authenticate to EC2, Azure VMs, and GCE instances, respectively:

- **Local authentication**: This method is used for unmanaged EC2 instances
- **Central authentication**: This method is used for EC2 instances joined to a domain (as explained in *Chapter 9, Effective Strategies for Implementing IAM Solutions*)

Since local authentication does not scale (that is, the ability to manage local accounts for hundreds or thousands of VMs), due to the need to manage separate credentials for any VM in the organization, it is best practice to join the VM to a central **identity provider (IdP)** and avoid using local authentication.

Next, we look at the best practices for authentication in Amazon EC2, Azure VMs, and GCE.

Amazon EC2

Use the following best practices for authentication to Amazon EC2 instances:

- Local authenticating to Linux VMs is done using SSH key authentication. To avoid SSH key pair reuse by unauthorized parties, use AWS Secrets Manager to securely store and rotate SSH key pairs, as explained here: <https://aws.amazon.com/blogs/security/how-to-use-aws-secrets-manager-securely-store-rotate-ssh-key-pairs/>.

- Most organizations use a central directory service (such as Microsoft AD) to manage users, groups, and computer accounts, making it easy to log in with unique credentials for each user to all machines in the organization. For central authentication to VMs, join Windows or Linux instances to an AD domain and use your AD credentials to log in to EC2 instances (and avoid using local credentials or SSH keys completely). For more information, see the following links:
 - https://docs.aws.amazon.com/directoryservice/latest/admin-guide/launching_instance.html
 - https://docs.aws.amazon.com/directoryservice/latest/admin-guide/seamlessly_join_linux_instance.html

Azure VMs

Use the following best practices for authentication to Azure VMs:

- Authenticating Linux VMs is done using SSH key authentication. To avoid SSH key pair reuse by unauthorized parties, generate and store SSH keys in the Azure portal, as explained here: <https://docs.microsoft.com/en-us/azure/virtual-machines/ssh-keys-portal>.
- Most organizations use a central directory service (such as Microsoft AD) to manage users, groups, and computer accounts, making it easy to log in with unique credentials for each user to all machines in the organization. Follow these instructions to understand how to join Windows and Linux VMs to Microsoft Entra ID:
 - Join Windows VMs to Microsoft Entra ID and take the benefits of passwordless authentication: <https://learn.microsoft.com/en-us/entra/identity/devices/howto-vm-sign-in-azure-ad-windows>
 - Join Linux VMs to Microsoft Entra ID: <https://learn.microsoft.com/en-us/entra/identity/devices/howto-vm-sign-in-azure-ad-linux>

GCE

Use the following best practices for authentication to GCE:

- To perform login to a Linux VM (except Fedora CoreOS), you need to set up OS Login and OS Login with **two-factor authentication (2FA)**, as explained here: <https://cloud.google.com/compute/docs/oslogin/set-up-oslogin>.

- Most organizations use a central directory service (such as Microsoft AD) to manage users, groups, and computer accounts, making it easy to log in with unique credentials for each user to all machines in the organization. Follow these instructions to understand how to join Windows and Linux VMs to Google Managed Microsoft AD:
 - Join a Windows VM to a domain: <https://cloud.google.com/managed-microsoft-ad/docs/seamless-domain-join-gce>
 - Join a Linux VM to a domain: <https://cloud.google.com/managed-microsoft-ad/docs/quickstart-domain-join-linux>

In this section, we explained how to authenticate EC2, Azure VMs, and GCE. Next, let us see the best practices for securing network access to an instance.

Best practices for securing network access to an instance

Access to VMs is protected by *network-based access control mechanisms*. Network-based access controls are the equivalent of the on-premises layer 4 network firewall or access control mechanism.

Network access refers to the action of connecting to a VM over the network and protecting it from unauthorized access using **network access control lists (NACLs)** (such as the on-prem firewall).

Network access controls work in one of the following ways:

- **Stateless:** The receiver (for example, a firewall) does not maintain the previous connection's state (of the client). Suitable for scenarios that require high network speed. It requires the configuration of both inbound and outbound network rules.
- **Stateful:** The receiver (for example, a firewall) maintains the entire connection's state (of the client). Suitable for scenarios where we need to maintain information about connections between a client and a server (for example, to be able to block suspicious traffic). It requires the configuration of only inbound network rules.

As you will learn in *Chapter 6, Securing Networking Services – Part 1*, some CSPs implement a stateless NACL, while others implement a stateful NACL to allow network connectivity to resources such as VMs.

As a customer, you configure parameters such as source IP (or **Classless Inter-Domain Routing (CIDR)**), destination IP (or CIDR), destination port (or predefined protocol), and whether the port is **Transmission Control Protocol (TCP)** or **User Datagram Protocol (UDP)**.

The most basic thing about managing VMs is allowing remote access and management for the purpose of VM maintenance (such as software installation, patch management, and so on).

For remote access and management, the following applies:

- Windows VMs are using the **Remote Desktop Protocol (RDP)**. Limit inbound network access to TCP port 3389.

Note: This is the default port for Windows machines, and it can be changed to make it difficult for unauthorized parties to gain access.

- Linux VMs are using the SSH protocol. Limit inbound network access to TCP port 22.

Note: This is the default port for Linux machines, and it can be changed to make it difficult for unauthorized parties to gain access.

- For remote access protocols (SSH/RDP), limit the source IP (or CIDR) to well-known addresses.

Today, each of the hyperscale cloud providers offers secure methods to remotely connect and manage VMs. For file-sharing protocols (**Common Internet File System (CIFS)/Server Message Block (SMB)/File Transfer Protocol (FTP)**), limit the source IP (or CIDR) to well-known addresses. Limit the number of ports allowed in an inbound connection to the minimum required ports for allowing your service or application to function.

Let us look at how we can secure network access to an instance in the case of Amazon EC2, Azure VM, and GCE.

Amazon EC2

Use the following best practices for securing network access to Amazon EC2 instances:

- Use a NACL to restrict inbound and outbound traffic at the subnet level, as explained here: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html>
- Use security groups to restrict inbound traffic at the EC2 instance level, as explained here: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html>
- For granular access, use another security group as either the source or destination in a security group
- Set names and descriptions for security groups to allow a better understanding of the security group's purpose
- Use tagging (that is, *labeling*) for security groups to allow a better understanding of which security group belongs to which AWS resources

- Use **EC2 Instance Connect Endpoint** to remotely connect to an EC2 instance, as explained here: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connect-with-ec2-instance-connect-endpoint.html>

The following diagram explains how a user remotely connects to a Linux-based EC2 instance using EC2 Instance Connect Endpoint:

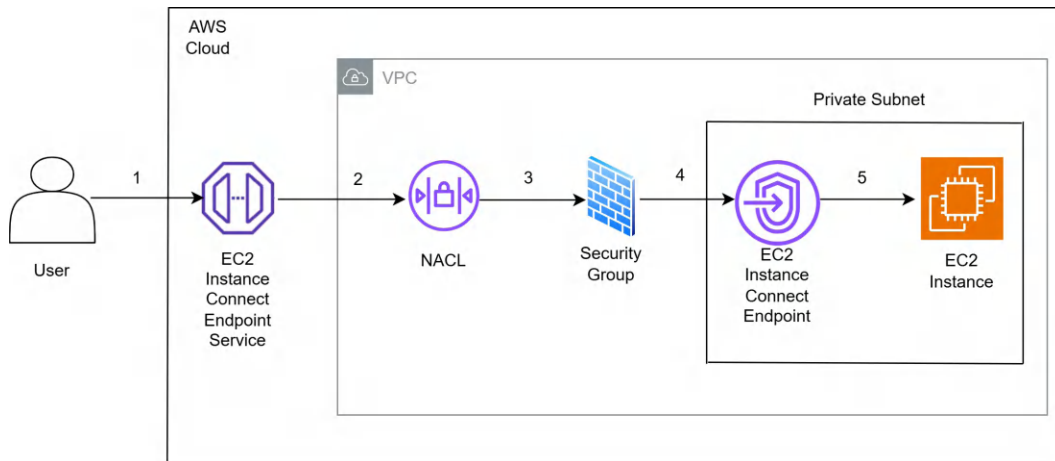


Figure 2.1 – Connection through EC2 Instance Connect Endpoint

In the preceding diagram, we have the following:

- The user connects to the EC2 Instance Connect Endpoint service from the public internet and authenticates using SSH key authentication.
- A secure connection passes from the EC2 Instance Connect Endpoint service to an NACL assigned at the **Virtual Private Cloud (VPC)** level and from the NACL to a security group assigned at the private subnet level to the EC2 Instance Connect Endpoint located in a private subnet.
- A secure connection passes from the EC2 Instance Connect Endpoint to the target EC2 instance, inside the private subnet.
- To keep the traffic between the Amazon VPC and the EC2 instance private (that is, within the AWS backbone), create an interface VPC endpoint, as explained here:

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/interface-vpc-endpoints.html>

Azure VMs

Use the following best practices for securing network access to Azure VMs:

- Use Azure **network security groups (NSGs)** to restrict both inbound and outbound traffic at either the subnet level or at the **network interface controller (NIC)** (VM) level, as explained here: <https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview>
- Set names for NSGs to allow a better understanding of the security group's purpose
- Use tagging (that is, *labeling*) for NSGs to allow a better understanding of which NSG belongs to which Azure resources
- Use **Microsoft Entra Privileged Identity Management (PIM)** to allow **just-in-time (JIT)** access to a remote VM, as explained here: <https://learn.microsoft.com/en-us/entra/id-governance/privileged-identity-management/pim-configure>
- Use **Azure Bastion** to remotely connect to an Azure VM, as explained here: <https://learn.microsoft.com/en-us/azure/bastion/bastion-overview>

The following diagram explains how a user remotely connects to an Azure VM using Azure Bastion:

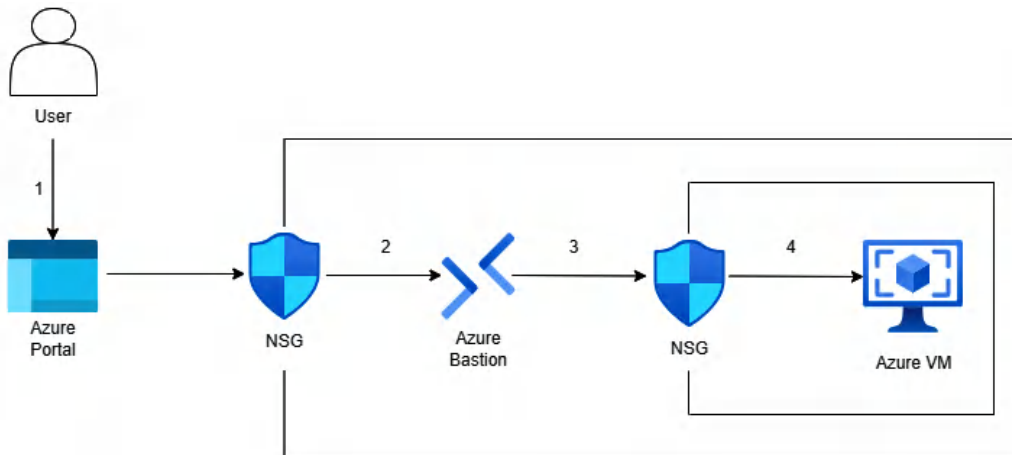


Figure 2.2 – Connection to Azure VM through Azure Bastion

In the preceding diagram, we have the following:

- The user logs in to the Azure portal and initiates a session to Azure Bastion
- A secure connection is initiated through an NSG to Azure Bastion
- Azure Bastion sends a secure RDP/SSH connection through an NSG to a private subnet
- The connection through the NSG connects to an Azure VM

GCE

Use the following best practices for securing network access to GCE:

- Use VPC firewall rules to restrict inbound traffic at the GCE level, as explained here: <https://cloud.google.com/firewall/docs/firewalls>
- Set names and descriptions for firewall rules to allow a better understanding of the security group's purpose
- Use tagging (that is, *labeling*) for firewall rules to allow a better understanding of which firewall rule belongs to which GCP resources
- Use **Identity-Aware Proxy (IAP)** to remotely connect to a Linux-based GCE, as explained here: <https://cloud.google.com/compute/docs/connect/ssh-using-iap>

The following diagram explains how a user remotely connects to a Google VM through an IAP:

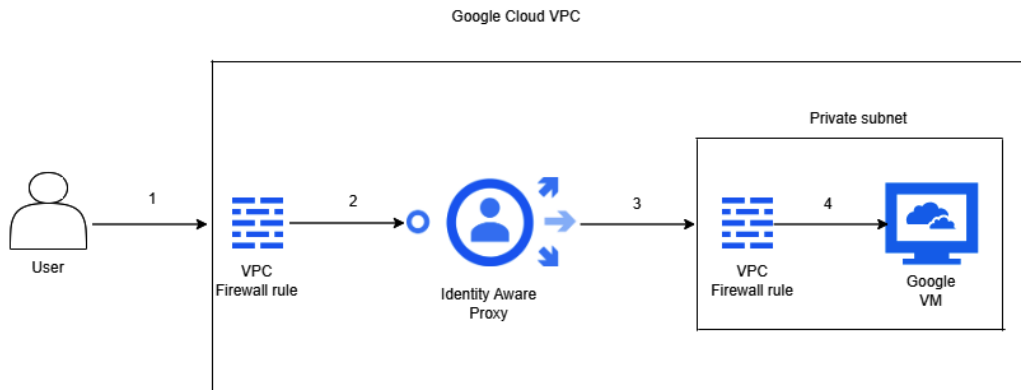


Figure 2.3 – Connection to a Google VM through Google IAP

In the preceding diagram, we have the following:

- The user initiates a secure connection to a Google IAP (located inside a Cloud VPC, behind a VPC firewall rule), and authenticates against the Google IdP
- The Google IAP passes the connection over a secure channel to a VPC firewall rule (located in a private subnet) and into a Google VM located in a private subnet
- To restrict access to GCE from trusted sources and reduce the risk of data exfiltration, use **VPC Service Controls**, as explained here: <https://cloud.google.com/compute/docs/instances/protecting-resources-vpc-service-controls>

In this section, we learned how to protect access to VMs at the network layer. Now, let's dive into how to conduct patch management.

Best practices for conducting patch management

Patch management is a crucial part of every instance of ongoing maintenance. To deploy security patches for either Windows- or Linux-based instances in a standard manner, it is recommended to use the following method:

1. Configure the patch baseline.
2. Use tagging (that is, *labeling*) for your VMs to allow patch deployment groups per tag (for example, prod versus dev environments).
3. Regularly scan your VMs for deviation from the patch baseline..
4. Install missing security patches on your VMs.
5. For stateless VMs (where no user session data is stored inside the VM), replace an existing VM with a new instance, created from an up-to-date operating system image (*note*: the process is known as immutable infrastructure).
6. Review patch deployment reports.

Now, let us see how each cloud platform implements patch management.

Amazon EC2

Use the following best practices for patch management of Amazon EC2 instances:

- Use AWS Systems Manager Patch Manager to deploy security patches for EC2 instances, as explained here: <https://docs.aws.amazon.com/systems-manager/latest/userguide/patch-manager.html>
- Create a group with minimal **Identity and Access Management (IAM)** privileges to allow only relevant team members to conduct patch deployment
- Use Amazon Inspector to search for missing security OS patches on EC2 instances and additional information such as **Common Vulnerabilities and Exposures (CVE)** and risk score, as explained here: <https://docs.aws.amazon.com/inspector/latest/user/scanning-resources.html>

Azure VMs

Use the following best practices for patch management of Azure VMs:

- Use Azure Update Manager to deploy security patches for Azure VMs, as explained here: <https://learn.microsoft.com/en-us/azure/update-manager/overview>
- Use minimal privileges for the account using Azure Update Manager to deploy security patches

- Configure automatic VM guest patching, as explained here: <https://learn.microsoft.com/en-us/azure/virtual-machines/automatic-vm-guest-patching>
- Use **Microsoft Defender Vulnerability Management** to search for missing security patches on Azure VMs, as explained here: <https://learn.microsoft.com/en-us/defender-vulnerability-management/defender-vulnerability-management>

GCE

Use the following best practices for patch management of GCE:

- Use VM Manager (Patch) to deploy security patches for GCE, as explained here: <https://cloud.google.com/compute/docs/os-patch-management>
- Use minimal privileges for the accounts using operating system patch management to deploy security patches, according to Google Cloud IAM roles
- Gradually deploy security patches zone by zone and region by region
- Integrate VM Manager with Google Security Command Center to generate vulnerability reports, as explained here: https://cloud.google.com/security-command-center/docs/concepts-security-sources#vm_manager
- Create patch jobs to deploy security patches on GCE: <https://cloud.google.com/compute/docs/os-patch-management/create-patch-job>
- Use the following best practices for OS updates at scale: <https://cloud.google.com/blog/products/management-tools/best-practices-for-os-patch-management-on-compute-engine>

In this section, we learned how to centrally deploy security hotfixes for EC2 instances, Azure VMs, and GCE and use a vulnerability management service to scan for missing security patches.

Best practices for securing backups

Backing up is crucial for VM recovery. Each of the hyperscale cloud providers offers a managed service for scheduling backups to VMs. Access to backups is governed using authorization rules, and backups are encrypted at rest by default.

Backups are part of **disaster recovery (DR)** processes, ensuring the organization will be able to survive data loss (from accidental deletion of a VM up to a hacker deliberately deleting data) or data inaccessibility (such as ransomware encrypting an entire VM).

As a best practice, regularly conduct restore VMs from backup to validate that the backups are functioning and that data can be restored (especially in production environments).

Let us see how this figures for each of the platforms.

AWS Backup

This is a fully managed backup service supporting many AWS services, from EC2 instances, storage services (Amazon **Simple Storage Service (S3)**, Amazon **Elastic Block Store (EBS)**, Amazon **Elastic File System (EFS)**, and so on), and database services (Amazon **Relational Database Service (RDS)**, Amazon Aurora, and so on).

Use the following best practices for securing Amazon EC2 instances while using AWS Backup:

- Refer to AWS' security considerations for using AWS Backup: <https://docs.aws.amazon.com/aws-backup/latest/devguide/security-considerations.html>.
- Configure the AWS Backup service with an IAM role to allow access to the encryption keys stored inside AWS **Key Management Service (KMS)** and to the backup vault, following the **principle of least privilege (PoLP)**.
- The AWS Backup service encrypts customers' backups at rest using AWS encryption keys, stored in AWS KMS (as explained in *Chapter 11, Applying Encryption in Cloud Services*). As an extra layer of protection, configure AWS Backup to perform backups of EBS volume snapshots.

Note: for sensitive environments, it is recommended to encrypt EBS volume snapshots using a customer-managed encryption key.

- Use tagging (that is, *labeling*) for backups to allow a better understanding of which backup belongs to which EC2 instance: <https://docs.aws.amazon.com/aws-backup/latest/devguide/creating-a-backup-plan.html>.
- To keep traffic between Amazon VPC and AWS Backup endpoints private (that is, within the AWS backbone), create an AWS Backup VPC endpoint, as explained here: <https://docs.aws.amazon.com/aws-backup/latest/devguide/backup-privatelink.html>.
- For a regulated environment, configure AWS Backup Vault Lock in compliance mode to keep backups immutable from changes or deletion: <https://docs.aws.amazon.com/aws-backup/latest/devguide/vault-lock.html>.
- For critical production environments, configure AWS Backup to create copies of backups to another AWS region (also known as cross-region replication): <https://docs.aws.amazon.com/aws-backup/latest/devguide/cross-region-backup.html>.
- Use AWS **Backup Audit Manager** to audit backup activity (for example, which resources have been backed up, which backups are encrypted, and so on): <https://docs.aws.amazon.com/aws-backup/latest/devguide/aws-backup-audit-manager.html>.

Azure Backup

This is a fully managed backup service, supporting many Azure services, from Azure VMs, storage services (Azure Blob Storage, Azure Files, and so on), database services (such as Azure Database for PostgreSQL), and **Azure Kubernetes Service (AKS)**.

Use the following best practices for securing Azure VMs while using Azure Backup:

- Follow the Azure security baseline for Backup, as explained here: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/backup-security-baseline>.
- Use Azure **role-based access control (RBAC)** to configure Azure Backup to have minimal access to your backups: <https://learn.microsoft.com/en-us/azure/backup/backup-rbac-rs-vault>.
- The Azure Backup service encrypts your backups in transit and at rest using Azure Key Vault (as explained in *Chapter 11, Applying Encryption in Cloud Services*). Azure Backup automatically encrypts backups using platform-managed keys (that is, encryption keys controlled by Azure). For sensitive environments, encrypt backups of VMs using customer-managed keys: <https://learn.microsoft.com/en-us/azure/backup/encryption-at-rest-with-cmk>.
- To keep traffic between the Azure VNet and Azure Backup private (that is, within the Azure backbone), create private endpoints for Azure Backup and deny public network access to the backup vault: <https://learn.microsoft.com/en-us/azure/backup/backup-azure-private-endpoints-concept>.
- Enable the *soft delete* feature to protect your backups from accidental deletion, as explained here: <https://learn.microsoft.com/en-us/azure/backup/soft-delete-virtual-machines>.
- For regulated environments, enable an immutable vault to keep backups immutable from changes or deletion, as explained here: <https://learn.microsoft.com/en-us/azure/backup/backup-azure-immutable-vault-how-to-manage>.
- For regulated environments, use Azure Policy to configure compliance controls for Azure Backup, as explained here: <https://learn.microsoft.com/en-us/azure/backup/security-controls-policy>.
- For critical production environments, configure Azure Backup to create copies of backups to a paired Azure region (also known as cross-region replication): <https://azure.microsoft.com/en-us/blog/cross-region-restore-crr-for-azure-virtual-machines-using-azure-backup/>.

Google Cloud Backup and DR

This is a fully managed backup service, supporting many GCP services, from GCE, storage services (such as Google Filestore), and database engines (such as Microsoft SQL, Oracle, MySQL, PostgreSQL, SAP HANA, and so on).

Use the following best practices for securing GCE while using the Google Cloud Backup and DR service:

- Follow the documentation for using Cloud Backup and DR to back up GCE: <https://cloud.google.com/backup-disaster-recovery/docs/concepts/backupdr-for-compute-engine>
- Use IAM permissions to configure Cloud Backup and DR to have minimal access to your backups, as explained here: <https://cloud.google.com/backup-disaster-recovery/docs/access-control>
- To restrict access to Cloud Backup and DR service from trusted sources and reduce the risk of data exfiltration, use VPC Service Controls: <https://cloud.google.com/backup-disaster-recovery/docs/configuration/vpc-sc>
- Use Security Command Center Premium to detect anomalous backup activity: <https://cloud.google.com/backup-disaster-recovery/docs/monitor-reports/alerts-scc>

In this section, we reviewed backup services in the context of configuring scheduled backups for EC2, Azure VMs, and GCE.

Summary

In this chapter, we learned how to securely maintain a VM, based on AWS, Azure, and GCP infrastructure.

In each section, we reviewed how to manage **identity management (IdM)** (for authentication and authorization), how to control network access (from access controls to network encryption), how to conduct patch management, and finally, how to configure backup for DR.

In the next chapter, we will review compute services in the cloud, with a focus on containers and Kubernetes.

3

Securing Compute Services – Containers and Kubernetes

In the previous chapter, we learned about security in VMs.

This chapter will cover containers and Kubernetes-managed services and provide you with best practices on how to securely deploy and manage each of them.

In this chapter, we will cover the following topics:

- Best practices for securing Kubernetes workloads (identity and access management, network access, auditing and security monitoring, and compliance)
- Best practices for securing non-Kubernetes workloads (identity and access management, network access, and auditing and monitoring)

Securing containers

Containers share some of the same characteristics as VMs but with a much smaller footprint. Instead of having to deploy an application above an entire operating system, you could use containers to deploy your required application, with only the minimum required operating system libraries and binaries.

Containers have the following benefits over VMs:

- **Small footprint:** Only required libraries and binaries are stored inside a container
- **Portability:** You can develop an application inside a container on your laptop and run it on a large scale in a production environment with hundreds or thousands of container instances
- **Deploy time:** Fast deployment and updates compared to VMs

The following diagram presents the architectural differences between VMs and containers:

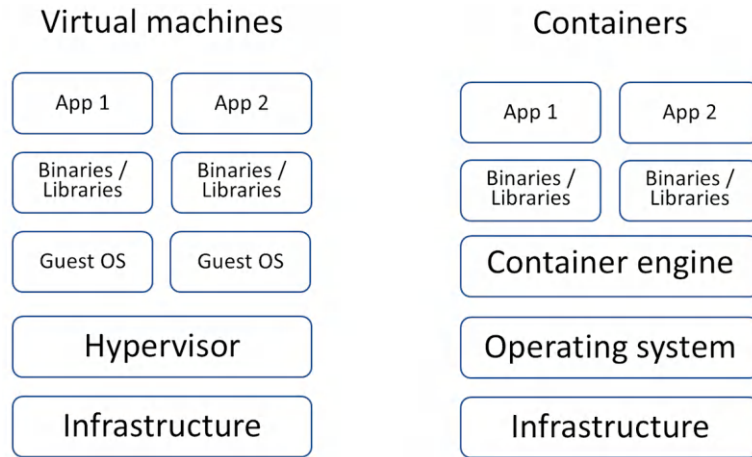


Figure 3.1 – VMs versus containers

If you are still in the development phase, you can install a container engine on your laptop and create a new container (or download an existing container image) locally, until you complete the development phase.

When you move to production and have a requirement to run hundreds of container instances, you need an **orchestrator** – a mechanism (or a managed service) for managing container deployment, health check monitoring, container recycling, and more.

Docker was adopted by the industry as a de facto standard for wrapping containers, and in the past couple of years, more and more cloud vendors have begun to support a new initiative for wrapping containers called the **Open Container Initiative (OCI)**.

Kubernetes is a containers orchestrator, based on an open source project (developed initially by Google) and is now considered the industry's de facto standard for orchestrating, deploying, scaling, and managing containers.

In this section, I will present the most common container orchestrators available as managed services – from Kubernetes to managed container orchestrator services on top of Amazon Web Services (AWS), Azure, and Google Cloud Platform (GCP) infrastructure.

You can use the following additional reading materials about containers, OCI images, and Kubernetes:

- *What is a container?*: <https://www.docker.com/resources/what-container>
- *Open Container Initiative*: <https://opencontainers.org/>
- *OCI artifact support in Amazon ECR*: <https://aws.amazon.com/blogs/containers/oci-artifact-support-in-amazon-ecr/>

- *Azure and OCI images:* <https://docs.microsoft.com/en-us/azure/container-registry/container-registry-image-formats#oci-images>
- *GCP and OCI image format:* <https://cloud.google.com/artifact-registry/docs/supported-formats#oci>
- *Kubernetes project:* <https://kubernetes.io/>

Next, we are going to see what the best practices are for securing common container and Kubernetes services from AWS, Azure, and GCP.

Securing Kubernetes

As mentioned earlier in this chapter, Kubernetes is the most widely used container orchestrator. In this section, we will learn how to secure managed Kubernetes services.

Kubernetes terminology

Before we begin the conversation about Kubernetes, let us define the various components of Kubernetes.

For **high availability**, Kubernetes is deployed in a **cluster** mode (hence, Kubernetes cluster). At a high level, a Kubernetes cluster has the following components:

- **Control plane components:**
 - **Kubernetes API server:** This main component of the Kubernetes cluster is used to expose Kubernetes APIs to incoming traffic for cluster maintenance and for in-cluster traffic between the various cluster components.
 - **etcd:** A key-value database for storing cluster configuration (such as the number of running worker nodes or deployed pods).
 - **kube-scheduler:** A component responsible for assigning Pods to nodes based on different scheduling factors such as resource requirements, affinity, and so on.
 - **kube-controller-manager:** A component in charge of running other controllers (such as node controller, replication controller, etc.).
- **Worker node components:**
 - **kubelet:** A component that controls the communication between the API server and the pods running on top of worker nodes.
 - **Kubernetes runtime:** Software for running containers, such as Docker or CRI-O.
 - **kube-proxy:** A component that manages network connectivity to Kubernetes services using filtering rules.
 - **Pod:** A unit that contains one or more containers. Applications are deployed into containers.

- **Ingress controller:** A component that controls inbound traffic to a Kubernetes service, including load balancing, network access rules (controlled using network policies), traffic encryption, and so on.
- **Kubernetes service:** A logical set of pods and policies for allowing access to deployed applications. Policies can define the number of required running pods, load-balance IP and port for inbound traffic, traffic load balancing, DNS name, and so on.
- **Namespace:** A logical way to create isolation between different applications deployed on top of the same Kubernetes cluster.

Here is a sample Kubernetes architecture diagram showing how each of the components integrates as part of the larger architecture:

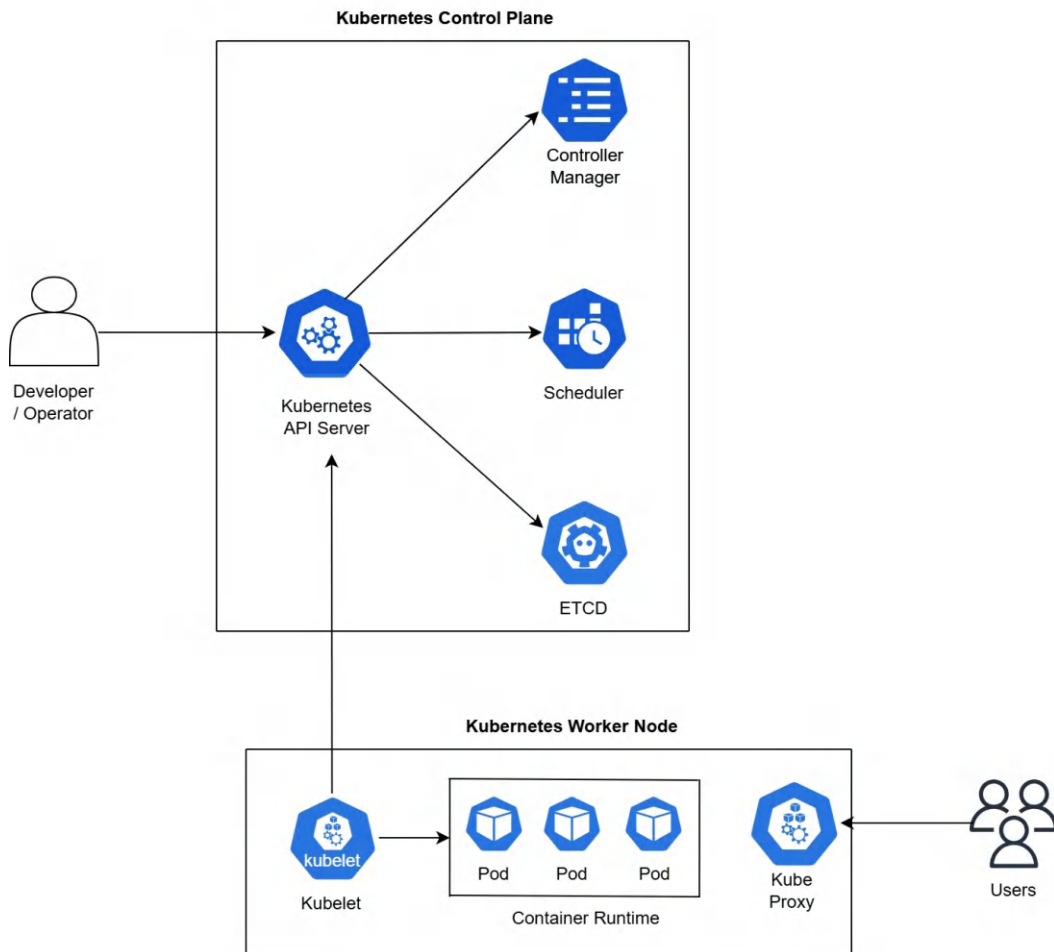


Figure 3.2 – Kubernetes architecture diagram

In the next couple of sections, we will review best practices for securing Kubernetes from different angles (from identity and access management, network access, auditing and monitoring, and finally, compliance).

General best practices for Kubernetes

Regardless of the cloud provider you are using when using Kubernetes, you should follow these best practices:

- For highly sensitive environments, use Kubernetes worker nodes from a VM family that supports confidential computing capabilities, to protect data in use (as explained in *Chapter 11, Applying Encryption in Cloud Services*)
- Create custom container images and use a minimal number of packages inside the container image to lower the attack surface
- Sign your container images using your private key, and check the signature before using a container image, as part of the deployment process, to verify its source
- Use container images from a trusted source and store them in a private container image registry (*Note:* most CSPs offer a managed container registry service)
- Scan your container images for vulnerabilities in libraries and binaries and update your images regularly
- Always run applications inside Kubernetes pods using a non-root user to avoid potential breaches to your Kubernetes environment, such as taking full control over a specific pod, other running pods on the same cluster, or even interfering with the worker node itself

Now, let us see the best practices to use Kubernetes in Amazon, Azure, and Google in particular.

Amazon Elastic Kubernetes Service (Amazon EKS)

EKS is the Amazon-managed Kubernetes orchestration service.

Read thorough documentation on security in Amazon EKS, which can be found here: <https://docs.aws.amazon.com/eks/latest/userguide/security.html>.

Use the following best practices for security in Amazon EKS:

- To create an isolated execution environment, protecting the Kubernetes environment, use AWS Nitro Enclaves with Amazon EKS, as documented here: <https://docs.aws.amazon.com/enclaves/latest/user/kubernetes.html>.
- Amazon EKS integrates with Amazon **Elastic Container Registry (ECR)** for retrieving container images. To protect the use of Amazon ECR, follow these best practices: <https://docs.aws.amazon.com/AmazonECR/latest/userguide/security.html>.

Azure Kubernetes Service (AKS)

AKS is the Azure-managed Kubernetes orchestration service.

Read thorough documentation on security in Azure AKS, which can be found here: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/azure-kubernetes-service-aks-security-baseline>.

Use the following best practices for security in Azure AKS:

- To create an isolated execution environment, protecting the Kubernetes environment, use Confidential Containers with AKS, as documented here: <https://learn.microsoft.com/en-us/azure/aks/confidential-containers-overview>.
- Azure AKS integrates with Azure Container Registry for retrieving container images. To protect the use of Azure Container Registry, follow these best practices: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/container-registry-security-baseline>.

Google Kubernetes Engine (GKE)

GKE is the Google-managed Kubernetes orchestration service.

Read thorough documentation on security in GKE, which can be found here: <https://cloud.google.com/kubernetes-engine/docs/concepts/security-overview>.

Use the following best practices for security in GKE:

- To create an isolated execution environment, protecting the Kubernetes environment, encrypt workload data in use with **Confidential Google Kubernetes Engine Nodes**, as documented here: <https://cloud.google.com/kubernetes-engine/docs/how-to/confidential-gke-nodes>.
- GKE integrates with **Google Artifact Registry** for retrieving container images. To protect the use of Google Artifact Registry, follow these best practices: <https://cloud.google.com/artifact-registry/docs/protect-artifacts>.
- **Binary Authorization** is a GCP capability that allows customers to make sure that container images are retrieved from trusted sources. GKE has built-in integration with Binary Authorization. Configure a Binary Authorization policy with GKE to govern the deployment of container images, as explained here: <https://cloud.google.com/binary-authorization/docs/configure-policy-gke>.

In this section, we reviewed high-level references to learn about the security of Amazon EKS, AKS, and GKE.

Best practices for identity and access management in Kubernetes

There are many aspects to review when talking about identity and access management in Kubernetes, including the following, among others:

- **Authentication:** The act of identifying against the Kubernetes API server, to take actions (such as read cluster configuration, add a worker node, deploy a new application, and more)
- **Authorization:** The act of validating an identity's permissions against the Kubernetes built-in role-based access control (the permissions define what type of actions an identity can take, such as deploying a new application, exposing an application to external customers, and more)
- **Permission of Pods:** Pods can have several types of access permissions: access to the Kubernetes API server (for example, for reading the amount of required CPU/memory for each pod), permissions to access other pods on the same cluster (such as different microservices part of the same application), and access to other resources on the CSPs ecosystem (for example, a pod consuming information from storage services), and more

Kubernetes supports the following types of identities:

- **User identity:** Although Kubernetes does not manage *user objects*, it allows you to connect the Kubernetes cluster to an external **OpenID Connect (OIDC)** provider (such as **Amazon Cognito**, **Microsoft Entra ID**, **Google Identity Platform**, etc.) and allows configuring permissions (i.e., role-based access control) to the Kubernetes cluster
- **Service account:** An identity created and managed locally by the Kubernetes cluster, to allow pods access to the Kubernetes API server (for actions such as querying for resources, updating configuration, etc.), and for enforcing which pod can take which actions (i.e., implementing the principle of least privilege)
- **Pod Identity:** A concept extending a *service account*, which allows you to assign a pod identity for each pod, to be able to gain access to the API server (through a mapping to a service account), and to external resources in the CSP ecosystem (such as object storage, databases, message queuing, etc.)

General best practices for securing IAM in Kubernetes (regardless of the CSP) are as follows:

- Connect the Kubernetes cluster to an external identity provider supporting the OIDC protocol, for central identity and access management. Once you complete connecting to the external OIDC provider, you will be able to grant permissions for identities (from human identities to service accounts/application accounts) from the central identity provider.
- Kubernetes has a built-in RBAC mechanism for controlling permissions to the cluster resources. When deploying a Kubernetes cluster as a managed service (in AWS, Azure, or GCP), each of the CSPs allow you to connect their identity and access management mechanisms to the Kubernetes RBAC, which allows you to set strict permissions to identities from your central identity provider.

- Create a unique service account for each namespace (instead of the default service account), and limit the privileges assigned to the newly created service account according to the application requirements (such as getting access to required resources, list services, etc.).
- Create a unique identity for every newly created Kubernetes cluster and application (also known as pod identity). Follow the principle of least privilege when granting permissions for pods to access resources in the CSP's ecosystem.
- **Secrets** (such as API keys or database credentials) are a common way to authenticate pods against other resources or applications. In case secrets leak, they will grant the person who holds them permission to access resources (such as other pods, other resources in the cloud environment, or on-premises). Avoid storing secrets as part of code or configuration files to decrease the chance of unauthorized access. Instead, use a managed vault service for the storage, retrieval, and rotation of secrets. Kubernetes supports a provider for the Secrets Store CSI Driver for connecting to managed vault services (as explained in *Chapter 5, Securing Storage Services*).

Let us see how IAM security best practices work in EKS, AKS, and GKE.

Amazon EKS

AWS IAM is the supported service for managing permissions to access and run containers through Amazon EKS (as explained in *Chapter 9, Effective Strategies for Implementing IAM Solutions*).

Read thorough documentation of Amazon EKS and identity management, which can be found at the following links:

- <https://docs.aws.amazon.com/eks/latest/userguide/security-iam.html>
- <https://aws.github.io/aws-eks-best-practices/security/docs/iam/>

Use the following best practices for authentication and authorization in Amazon EKS:

- Grant minimal IAM permissions for accessing and managing Amazon EKS, to avoid unauthorized access to the Kubernetes cluster, as explained here: https://docs.aws.amazon.com/eks/latest/userguide/security_iam_id-based-policy-examples.html.
- Use **EKS Pod Identities** to allow pods with minimal permissions to access resources in the AWS ecosystem, to avoid unauthorized access to AWS resources, as explained here: <https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html>.
- Store your container images inside **Amazon ECR** and grant minimal IAM permissions for accessing and managing Amazon ECR, to enforce the use of organization-trusted container image source: https://docs.aws.amazon.com/AmazonECR/latest/userguide/ECR_on_EKS.html.

- The content of a container image (such as application code) might be considered sensitive information (intellectual property). For sensitive environments, encrypt your container images inside **Amazon ECR** using CMK management (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as mentioned in the following documentation: <https://docs.aws.amazon.com/AmazonECR/latest/userguide/encryption-at-rest.html>.
- Use **AWS Secrets Manager** or **AWS Systems Manager Parameter Store** for storing, retrieving, and rotating secrets. Amazon allows you to integrate EKS with services such as AWS Secrets Manager using a Kubernetes Secrets Store CSI Driver, to avoid unauthorized access to static credentials, as explained here: https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating_csi_driver.html.

Azure AKS

Microsoft Entra ID is the supported service for managing permissions to access and run containers through AKS (as explained in *Chapter 9, Effective Strategies for Implementing IAM Solutions*).

Read thorough documentation of AKS and identity management, which can be found at the following links:

- <https://learn.microsoft.com/en-us/azure/aks/concepts-identity>
- <https://learn.microsoft.com/en-us/azure/aks/operator-best-practices-identity>

Use the following best practices for authentication and authorization in AKS:

- Integrate the AKS cluster with **Microsoft Entra ID** for central identity management (both for managing the AKS cluster and for pod identities to access resources in Azure), as explained here: <https://learn.microsoft.com/en-us/azure/aks/enable-authentication-microsoft-entra-id>
- Use **Microsoft Entra role-based access control (RBAC)** for Kubernetes authorization, to avoid unauthorized access to the Kubernetes cluster, and grant minimal permissions for accessing and managing AKS, as explained here: <https://learn.microsoft.com/en-us/azure/aks/manage-azure-rbac>
- Use **Microsoft Entra Workload ID** with AKS, and grant minimal permissions to allow AKS pods access to Azure resources, to avoid unauthorized access to Azure resources: <https://learn.microsoft.com/en-us/azure/aks/workload-identity-overview>
- Use **Azure Key Vault** for the storage, retrieval, and rotation of secrets. Azure allows you to integrate AKS with Azure Key Vault using a Kubernetes Secrets Store CSI Driver, to avoid unauthorized access to static credentials, as explained here: <https://learn.microsoft.com/en-us/azure/aks/csi-secrets-store-driver>

- Store your container images inside **Azure Container Registry (ACR)** and grant minimal permissions for accessing and managing Azure ACR, explained at the following link: <https://learn.microsoft.com/en-us/azure/container-registry/container-registry-roles>
- Use **Microsoft Entra ID managed identity** to authenticate from AKS to ACR to pull new container images, as explained here: <https://learn.microsoft.com/en-us/azure/aks/cluster-container-registry-integration>
- For sensitive environments, encrypt your container images inside ACR using customer-managed keys stored inside **Azure Key Vault** (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as mentioned here: <https://learn.microsoft.com/en-us/azure/container-registry/tutorial-enable-customer-managed-keys>

GKE

Google Cloud IAM is the supported service for managing permissions to access and run containers through GKE (as explained in *Chapter 9, Effective Strategies for Implementing IAM Solutions*).

Read thorough documentation of GKE and identity management, which can be found at the following links:

- <https://cloud.google.com/kubernetes-engine/docs/how-to/iam>
- <https://cloud.google.com/kubernetes-engine/docs/how-to/hardening-your-cluster>

Use the following best practices for authentication and authorization in GKE:

- Use GKE RBAC and grant minimal permissions for accessing and managing GKE, to avoid unauthorized access to the Kubernetes cluster, as explained here: <https://cloud.google.com/kubernetes-engine/docs/best-practices/rbac>.
- Use **Workload Identity Federation** for GKE, and grant minimal permissions to allow GKE pods access to GCP resources, to avoid unauthorized access to GCP resources, as explained here: <https://cloud.google.com/kubernetes-engine/docs/how-to/workload-identity>.
- Use the **Google IAM recommender** to set the minimal permissions for your GKE cluster, as explained here: <https://cloud.google.com/policy-intelligence/docs/role-recommendations-overview>.
- Use **Google Secret Manager** for the storage, retrieval, and rotation of secrets. GCP allows you to integrate GKE with Google Secret Manager using client libraries to avoid unauthorized access to static credentials, as explained here: <https://cloud.google.com/kubernetes-engine/docs/tutorials/workload-identity-secrets>.

- Store your container images inside **Google Artifact Registry** and grant minimal permissions for accessing and managing Artifact Registry, as explained here: <https://cloud.google.com/artifact-registry/docs/access-control>.
- For sensitive environments, encrypt your container images inside **Google Artifact Registry** using customer-managed encryption keys stored inside **Google Cloud KMS** (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as mentioned here: <https://cloud.google.com/artifact-registry/docs/cmek>.

In this section, we reviewed security best practices for IAM in Kubernetes. Now, let's see how we can secure network access to it.

Best practices for data protection in Kubernetes

There are many aspects to review when talking about data protection in Kubernetes. Here are some of them:

- External access to the Kubernetes API server, which may allow taking full control of a Kubernetes cluster
- Communication between the API server and the pods, which may allow unauthorized network access to the Kubernetes cluster from a compromised pod
- Pod-to-pod communication, which may allow unauthorized traffic between different pods within the same Kubernetes cluster from a compromised pod
- Communication between pods and other resources on the CSP ecosystem, which may allow unauthorized traffic from compromised pods to other resources in the cloud ecosystem.

Best practices for network access in Kubernetes are as follows:

- When deploying a Kubernetes cluster, create a **private mode** to decrease the attack surface on the Kubernetes cluster. Private mode means that all traffic to the Kubernetes API server must be done from authorized and private IP addresses.
- When deploying a Kubernetes cluster, deploy the worker nodes in a private subnet and the pods into another private subnet, segregated via a network access control list, to restrict access to the control plane of the Kubernetes, and avoid unauthorized access from a pod to the Kubernetes cluster itself.
- Kubernetes allows you to enforce restrictions on pods, such as access from pod to storage service, restrict the ability to use a privileged account (i.e., Root) inside a running pod, and more. The restrictions are called **Pod Security Admission**, and they are been enforced using a component of Kubernetes called **Pod Security Admission Controller**. Pod security supports *Privileged* profiles (completely unrestricted), *Baseline* profiles (minimal restriction, suitable for development environments), and *Restricted* profiles (fully restricted, recommended for sensitive or critical production applications).

As best practice, configure the *Restricted* profile for production applications. For more information, visit the following links:

- <https://kubernetes.io/docs/concepts/security/pod-security-admission/>
- <https://kubernetes.io/docs/concepts/security/pod-security-standards/>
- etcd contains the Kubernetes cluster configuration (current state, how many running pods, the desired number of running pods, pod locations, and more), and as such, it must be protected to avoid unauthorized access. Encrypt etcd at rest, as explained here: <https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>.
- Encrypt all traffic to the Kubernetes cluster and within the Kubernetes cluster using the TLS protocol and using your own TLS certificates, to keep data confidentiality in transit within the Kubernetes cluster.
- Use an ingress controller to restrict inbound access to the Kubernetes cluster, to restrict access to specific applications deployed as part of Kubernetes services from known IP addresses.
- To restrict access to the Kubernetes cluster and within the Kubernetes cluster (at layer 3 and layer 4 of the OSI model), configure network policies, which are network filtering rules allowing restricted access to resources from known IP addresses (or from known pods) at the network layer (i.e., source, destination, and port).
- To restrict access to the Kubernetes cluster and within the Kubernetes cluster (at layer 7 of the OSI model), use a service mesh solution, which allows you to restrict access to resources (such as pods), at the application layer (i.e., source, destination, and URL to a specific version of the application).
- To protect the Kubernetes cluster and the worker nodes from vulnerabilities, regularly update all components to the latest stable version (*note*: conduct tests to make sure new versions do not break the cluster or the applications).

Let us see how network-related security best practices work in EKS, AKS, and GKE.

Amazon EKS

In this section, we will review how to implement data protection for the EKS cluster and the pods deployed on top of the worker nodes (from network aspects, and data encryption).

Read thorough documentation of Amazon EKS and network access, which can be found at the following links:

- <https://docs.aws.amazon.com/eks/latest/userguide/eks-networking.html>

- <https://aws.github.io/aws-eks-best-practices/security/docs/network/>

To implement data protection in Amazon EKS, use the following best practices:

- Deploy EKS as a private cluster to restrict unauthorized network access to your EKS cluster, as explained here: <https://docs.aws.amazon.com/eks/latest/userguide/private-clusters.html>.
- Use VPC security groups to restrict network access to the Kubernetes API server, as mentioned here: <https://docs.aws.amazon.com/eks/latest/userguide/sec-group-reqs.html>.
- Use the **Amazon VPC CNI plugin** to configure security groups for restricting access between pods by setting network policies (at layers 3 and 4 of the OSI model) and avoid unauthorized network access to your pods: <https://docs.aws.amazon.com/eks/latest/userguide/managing-vpc-cni.html>.
- To restrict access between pods (at layer 7 of the OSI model) and avoid unauthorized network access to your pods, use **Istio**, as follows: <https://aws.amazon.com/blogs/opensource/using-istio-traffic-management-to-enhance-user-experience/>.
- Kubernetes ingress controller supports multiple ways to restrict inbound access to the Kubernetes cluster. One alternative is to use the **AWS Load Balancer Controller**, as explained here: <https://docs.aws.amazon.com/eks/latest/userguide/aws-load-balancer-controller.html>.
- To keep the traffic between the **Amazon Virtual Private Cloud (Amazon VPC)** and the Amazon EKS private (i.e., within AWS backbone), create an interface VPC endpoint, and avoid unauthorized network access to your EKS cluster, as explained here: <https://docs.aws.amazon.com/eks/latest/userguide/vpc-interface-endpoints.html>.
- Encrypt all EKS worker nodes volumes at rest (including etcd) to protect the configuration of the Kubernetes cluster, as explained here: <https://aws.github.io/aws-eks-best-practices/security/docs/data/>.
- Encrypt all inbound traffic to the Kubernetes cluster using the TLS protocol to protect data confidentiality. For highly sensitive environments, encrypt traffic between pods using **mutual TLS (mTLS)**, as explained here: <https://aws.github.io/aws-eks-best-practices/security/docs/network/>.

AKS

In this section, we will review how to implement data protection for the AKS cluster and the pods deployed on top of the worker nodes (from network aspects and data encryption).

Read thorough documentation of AKS and network access, which can be found at the following links:

- <https://learn.microsoft.com/en-us/azure/aks/concepts-network>
- <https://learn.microsoft.com/en-us/azure/aks/operator-best-practices-network>
- <https://learn.microsoft.com/en-us/azure/aks/operator-best-practices-cluster-isolation>

To implement data protection of AKS, use the following best practices:

- Deploy AKS as a private cluster to restrict unauthorized network access to your AKS cluster, as explained here: <https://learn.microsoft.com/en-us/azure/aks/private-clusters>.
- Use the **Azure CNI plugin** for restricting access between pods by setting network policies (at layers 3 and 4 of the OSI model) and avoid unauthorized network access to your pods: <https://learn.microsoft.com/en-us/azure/aks/azure-cni-overview>.
- To restrict access between pods (at layer 7 of the OSI model), use the **Open Service Mesh (OSM)** add-on and avoid unauthorized network access to your pods, as explained here: <https://learn.microsoft.com/en-us/azure/aks/open-service-mesh-about>.
- Kubernetes ingress controller supports multiple ways to restrict inbound access to the Kubernetes cluster. One alternative is to use the **Application Gateway Ingress Controller (AGIC)** add-on to restrict inbound access to the AKS cluster, as explained here: <https://learn.microsoft.com/en-us/azure/application-gateway/tutorial-ingress-controller-add-on-existing>.
- To keep the traffic between the Azure VNet and the Azure AKS private (i.e., within the Azure backbone), create a private endpoint and avoid unauthorized network access to your AKS cluster, as explained here: <https://learn.microsoft.com/en-us/azure/aks/private-clusters?tabs=azure-portal#use-a-private-endpoint-connection>.
- Encrypt all AKS worker nodes volumes at rest (including etcd) to protect the configuration of the Kubernetes cluster, as explained here: <https://learn.microsoft.com/en-us/azure/aks/use-kms-etcd-encryption>.

- Encrypt all inbound traffic to the Kubernetes cluster using the TLS protocol to protect data confidentiality. For highly sensitive environments, encrypt traffic between pods using mTLS, as explained here:
 - <https://learn.microsoft.com/en-us/azure/aks/ingress-tls>
 - <https://learn.microsoft.com/en-us/azure/application-gateway/for-containers/how-to-backend-mtls-gateway-api>

GKE

In this section, we will review how to implement data protection for the GKE cluster and the pods deployed on top of the worker nodes (from network aspects and data encryption).

To implement data protection of GKE, use the following best practices:

- Deploy GKE as a private cluster to restrict unauthorized network access to your GKE cluster, as explained here: <https://cloud.google.com/kubernetes-engine/docs/how-to/private-clusters>.
- Use **GKE Dataplane V2** to restrict access between pods, and avoid unauthorized network access to your pods: <https://cloud.google.com/kubernetes-engine/docs/how-to/dataplane-v2>.
- To restrict access between pods (at layer 7 of the OSI model), use **Cloud Service Mesh** and avoid unauthorized network access to your pods: <https://cloud.google.com/service-mesh/docs/onboarding/provision-control-plane>.
- Kubernetes ingress controller supports multiple ways to restrict inbound access to the Kubernetes cluster. One alternative is to use the **GKE Gateway controller** to restrict inbound access to the GKE cluster: <https://cloud.google.com/kubernetes-engine/docs/concepts/gateway-api>.
- Encrypt all GKE worker nodes volumes at rest (including etcd) to protect the configuration of the Kubernetes cluster, as explained here: <https://cloud.google.com/kubernetes-engine/docs/how-to/encrypting-secrets>.
- Encrypt all inbound traffic to the Kubernetes cluster using the TLS protocol to protect data confidentiality. For highly sensitive environments, encrypt traffic between pods using mTLS, as explained here:
 - <https://cloud.google.com/kubernetes-engine/docs/how-to/enable-inter-node-transparent-encryption>
 - <https://cloud.google.com/service-mesh/docs/security/configuring-mtls>

Now that we have reviewed security best practices for data protection, let us learn about auditing and monitoring in Kubernetes.

Best practices for auditing and security monitoring in Kubernetes

Auditing is a crucial part of data protection. There are many aspects to review when talking about auditing and security monitoring in Kubernetes, including the following:

- **Kubernetes control plane logs:** These contain information about Kubernetes' internal activity (such as API server, controller manager, scheduler, etc.)
- **Kubernetes cluster admin activity:** This contains information about cluster maintenance activity (such as cluster health, resource allocation, workload management, etc.)
- **Abnormal Pod activity:** This contains information such as pod eviction due to worker node low disk space, unauthorized admin activity attempt, violation of Kubernetes security policy, and so on

Follow these best practices for auditing and monitoring in Kubernetes:

- Although admin activity is automatically audited by the CSPs, its log history is limited. As best practice, send all audit and monitoring logs to a logging service for further analysis, incident response, and to comply with various regulations.
- Enable Kubernetes audit logs to capture all activity conducted on Kubernetes (who did what, and when) and integrate Kubernetes audit logs with the CSP built-in auditing services to be able to centrally collect logs, configure alerts on abnormal behavior (such as cluster changes in the middle of the night), and for incident response purpose. For more information, visit the following link: <https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/>.
- Add a threat detection solution to review Kubernetes logs, detect malicious activity (such as an attempt to run a pod with the context of a Root account or an attempt to run crypto mining within the context of a pod), and generate prioritized follow-up actions.

Now, let us check the specific best practices for auditing and monitoring in managed Kubernetes based on EKS, AKS, and GKE.

Amazon EKS

AWS allows you to enable logging and auditing using two built-in services:

- **Amazon CloudWatch:** A service that allows you to log EKS cluster activities and raise an alarm according to predefined thresholds (for example, low memory resources or high CPU, which requires up-scaling your EKS cluster)
- **AWS CloudTrail:** A service that allows you to monitor API activities (basically, any action performed on the EKS cluster)

In this section, we will review the capabilities offered by AWS to audit and monitor EKS activity.

Documentation of EKS and auditing/monitoring can be found at the following links:

- <https://docs.aws.amazon.com/eks/latest/userguide/eks-observe.html>
- <https://aws.github.io/aws-eks-best-practices/security/docs/detective/>

For auditing and monitoring of Amazon EKS, use the following best practices:

- Enable the Amazon EKS control plane when logging in to Amazon CloudWatch. It allows you to log API calls, audit, and authenticate information from your EKS cluster for further investigation and analysis of Kubernetes cluster activity, as explained here: <https://docs.aws.amazon.com/eks/latest/userguide/control-plane-logs.html>.
- **AWS CloudTrail** is enabled by default for Amazon EKS. To keep EKS API activity for longer than 90 days for further investigation and analysis, send CloudTrail logs to an Amazon S3 bucket, as explained here: <https://docs.aws.amazon.com/eks/latest/userguide/service-name-info-in-cloudtrail.html>.
- Enable **Amazon CloudWatch Container Insights** to collect information about the EKS activity for further investigation and troubleshooting, as explained here: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/deploy-container-insights-EKS.html>.
- Use **Amazon Managed Service for Prometheus** for collecting EKS metric data for further EKS cluster behavior monitoring, as explained here: <https://docs.aws.amazon.com/prescriptive-guidance/latest/implementing-logging-monitoring-cloudwatch/prometheus-monitoring-eks.html>.
- Use **GuardDuty Runtime Monitoring** to detect threats in workloads running on top of Amazon EKS: <https://docs.aws.amazon.com/guardduty/latest/ug/eks-runtime-monitoring-configuration.html>.

AKS

Azure allows you to enable logging and auditing using **Azure Monitor** for containers – a service that allows you to log container-related activities and raise an alarm according to predefined thresholds (for example, low memory resources or high CPU, which requires up-scaling your container environment).

In this section, we will review the capabilities offered by Azure to audit and monitor AKS activity.

For auditing and monitoring of AKS, use the following best practices:

- Thorough documentation of AKS and auditing/monitoring can be found at the following links:
 - <https://learn.microsoft.com/en-us/azure/azure-monitor/containers/kubernetes-monitoring-enable>

- <https://learn.microsoft.com/en-us/azure/azure-monitor/best-practices-containers>
- Enable diagnostic settings for AKS and send all logs to a Log Analytics workspace for further investigation and analysis of Kubernetes cluster activity, as explained here: <https://learn.microsoft.com/en-us/azure/aks/monitor-aks>
- Use **Azure Monitor Container Insights** to collect and analyze AKS logs: <https://learn.microsoft.com/en-us/azure/azure-monitor/containers/container-insights-overview>
- Use **Azure Monitor managed service for Prometheus** for collecting AKS metric data for further AKS cluster behavior monitoring, as explained here: <https://learn.microsoft.com/en-us/azure/azure-monitor/essentials/prometheus-metrics-overview>
- Use **Microsoft Defender for Containers** for vulnerability assessment and real-time threat protection on AKS workloads, as explained here: <https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-containers-introduction>

GKE

As with any other managed service, Google allows you to enable logging and auditing using the Google Cloud Logging service – a service that allows you to audit container-related activities.

In this section, we will review the capabilities offered by GCP to audit and monitor GKE activity.

For auditing and monitoring of GKE, use the following best practices:

- Thorough documentation of GKE and auditing/monitoring can be found at the following links:
 - <https://cloud.google.com/kubernetes-engine/docs/how-to/audit-logging>
 - <https://cloud.google.com/kubernetes-engine/docs/concepts/observability>
- Enable logging for GKE for further investigation and analysis of Kubernetes cluster activity, as explained here: <https://cloud.google.com/kubernetes-engine/docs/how-to/config-logging-monitoring>
- For sensitive/critical environments, enable data access audit logs (actions such as read configuration, create/modify/read user-provided resource data) for further investigation and analysis, as explained here: <https://cloud.google.com/logging/docs/audit/configure-data-access>

Note

Data access audit logs increase storage costs. Review the business requirements to enable it.

- Use **Google Cloud Managed Service for Prometheus** for collecting GKE metric data for further GKE cluster behavior monitoring: <https://cloud.google.com/stackdriver/docs/managed-prometheus>
- Use the **Google Security Command Center** to detect threats related to containers: <https://cloud.google.com/security-command-center/docs/concepts-container-threat-detection-overview>

In this section, we reviewed security best practices for auditing and monitoring in Amazon EKS, AKS, and GKE. Next, we will learn about how to ensure compliance with Kubernetes as per standard guidelines.

Implementing secure configuration in Kubernetes

Secure configuration and compliance with organization and industry best practices are important factors from a security point of view.

When talking about compliance with industry best practices, the most common standard for Kubernetes is the CIS Benchmark configuration guidelines published here: <https://www.cisecurity.org/benchmark/kubernetes>.

Run the **kube-bench** tool regularly to check for compliance with CIS Benchmarks for Kubernetes, as follows: <https://github.com/aquasecurity/kube-bench>.

Amazon EKS

AWS offers various ways to check for compliance with the EKS environment against industry best practices and standards. For compliance with EKS, use the following best practices:

- Thorough documentation of EKS and compliance can be found at the following links:
 - <https://docs.aws.amazon.com/eks/latest/userguide/compliance.html>
 - <https://aws.github.io/aws-eks-best-practices/security/docs/compliance/>
- Use **AWS Config** rules to check EKS configuration (such as EKS cluster version, whether the EKS cluster is configured as private, whether secrets are stored encrypted, etc.), as explained here: <https://docs.aws.amazon.com/config/latest/developerguide/security-best-practices-for-EKS.html>

- Use **Amazon Inspector** to scan for vulnerabilities in code and packages inside Amazon Container Registry, before being deployed to EKS: <https://docs.aws.amazon.com/inspector/latest/user/scanning-ecr.html>
- Use **AWS Security Hub** to check for compliance of the EKS cluster against industry standards (such as the NIST 800-53 standard), as explained here: <https://docs.aws.amazon.com/securityhub/latest/userguide/eks-controls.html>

AKS

Azure offers various ways to check for compliance with the AKS environment against industry best practices and standards. For compliance with AKS, use the following best practices:

- Use **Microsoft Defender for Cloud** to check for compliance against standards and regulations (such as the CIS Benchmarks), as explained here: <https://learn.microsoft.com/en-us/azure/defender-for-cloud/update-regulatory-compliance-packages>
- Information about the **Center for Internet Security (CIS)** Kubernetes benchmark and AKS can be found here: <https://learn.microsoft.com/en-us/azure/aks/cis-kubernetes>

GKE

Google offers various ways to check for compliance with the GKE environment against industry best practices and standards. For compliance with GKE, use the following best practices:

- Use the **GKE Compliance dashboard** to check for compliance against the CIS Benchmarks, as explained here: <https://cloud.google.com/kubernetes-engine/fleet-management/docs/about-compliance-dashboard>
- Information about the CIS Kubernetes benchmark and GKE can be found here: <https://cloud.google.com/kubernetes-engine/docs/concepts/cis-benchmarks>

In this section, we have learned how to securely maintain Kubernetes, based on AWS, Azure, and GCP infrastructure – from IAM to data protection, auditing, and security compliance against industry best practices. Next, let's see how we can secure non-Kubernetes workloads.

Securing non-Kubernetes workloads

In the previous section, we talked about Kubernetes workloads and how to secure managed Kubernetes services on top of AWS, Azure, and GCP. Kubernetes is not the only container orchestrator, and each of the hyperscale cloud providers offers other container orchestrators.

In the following section, we will discuss managed container orchestrators from AWS, Azure, and GCP.

Securing Amazon Elastic Container Service (ECS)

ECS is the Amazon-managed container orchestration service.

The main components of ECS are as follows:

- **ECS cluster (also known as ECS capacity):** The main component for running and managing containers. It can be deployed on top of the following:
 - **Amazon EC2 instances:** Customer needs to choose hardware resources (instance type) and take care of cluster size (number of running EC2 instances)
 - **Fargate:** A fully serverless solution, where AWS takes care of the entire control plane maintenance (from provisioning new cluster nodes to securing the control plane)
 - **Amazon ECS Anywhere:** Ability to run ECS on-premises
- **ECS task:** A logical grouping of containers. You configure task settings using task definitions (blueprints for running tasks).

The following is a sample Amazon ECS architecture diagram:

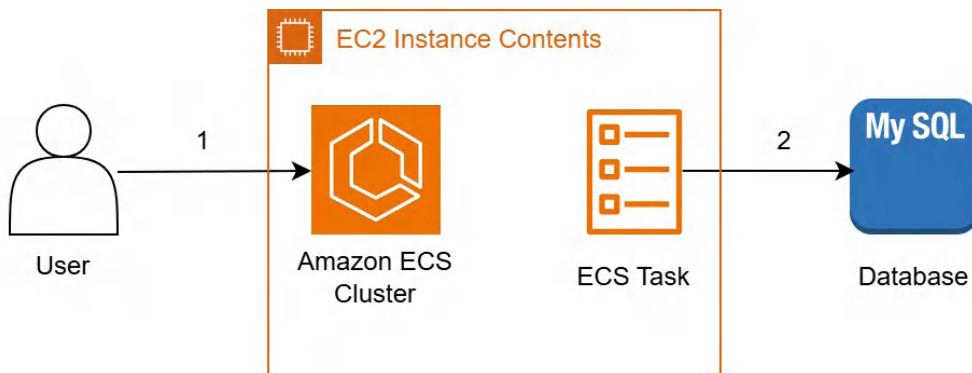


Figure 3.3 – Amazon ECS architecture diagram

As the preceding diagram shows, a *user* connects to an *ECS Cluster* (deployed on top of Amazon EC2), and within the cluster, there is an application deployed as an *ECS Task*, connected to a backend *Database*.

General best practices for securing Amazon ECS

In the next couple of sections, we will review best practices for securing Amazon ECS from different angles (from identity and access management, network access, and finally, auditing and monitoring).

The following resources offer comprehensive information on securing Amazon:

- Thorough documentation about the security of Amazon ECS can be found at the following link:
<https://docs.aws.amazon.com/AmazonECS/latest/bestpracticesguide/security.html>
- Information about how to secure ECS tasks and container images (for running inside an ECS task), can be found here:
<https://docs.aws.amazon.com/AmazonECS/latest/bestpracticesguide/security-tasks-containers.html>

Best practices for identity and access management

AWS IAM is the supported service for managing permissions to access and run containers through Amazon ECS (as explained in *Chapter 9, Effective Strategies for Implementing IAM Solutions*).

Use the following best practices for authentication and authorization in Amazon ECS:

- Thorough documentation of Amazon ECS and identity management can be found at the following link:
<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/security-iam.html>
- Grant minimal IAM permissions for managing Amazon ECS to avoid unauthorized access to your cloud environment, as explained here: <https://docs.aws.amazon.com/AmazonECS/latest/bestpracticesguide/security-iam.html>.
- Grant minimal IAM permissions for the Amazon ECS service (for running tasks, accessing S3 buckets, monitoring using CloudWatch Events, and so on) to avoid unauthorized access to AWS resources, as follows: <https://docs.aws.amazon.com/AmazonECS/latest/bestpracticesguide/security-iam-roles.html>.
- Store your container images inside Amazon ECR and grant minimal IAM permissions for accessing and managing Amazon ECR to avoid unauthorized access to your private container repository and use of container images from untrusted sources, as explained here: https://docs.aws.amazon.com/AmazonECR/latest/userguide/ECR_on_ECS.html.
- For sensitive environments, encrypt your container images inside Amazon ECR using CMK management (as explained in *Chapter 11, Applying Encryption in Cloud Services*), keeping container image content confidential, as explained here: <https://docs.aws.amazon.com/AmazonECR/latest/userguide/encryption-at-rest.html>.

- Secrets (such as API keys or database credentials) are a common way to authenticate ECS tasks against other resources or applications. Avoid storing secrets as part of code or configuration files; instead, use services such as **AWS Secrets Manager** or **AWS Systems Manager Parameter Store** for storing, retrieving, and rotating secrets to avoid unauthorized access to static credentials, as explained here: <https://docs.aws.amazon.com/AmazonECS/latest/bestpracticesguide/security-secrets-management.html>.
- Enforce the use of encryption in transit (i.e., the use of the TLS protocol) by integrating Amazon ECS with **Amazon ECS Service Connect** to keep traffic confidentiality, as explained here: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/service-connect-tls.html>.
- Use VPC security groups to allow access from your VPC to the **Amazon ECS VPC endpoint** and avoid unauthorized network access: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/get-set-up-for-amazon-ecs.html#create-a-base-security-group>.
- To keep the traffic between Amazon VPC and Amazon ECS private (i.e., within the AWS backbone), create an interface VPC endpoint: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/vpc-endpoints.html>.
- Use **AWS Systems Manager** to remotely connect and maintain ECS container instances for secure maintenance access, as explained here: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ec2-run-command.html>.

Best practices for conducting auditing and security monitoring in Amazon ECS

As with any other managed service, AWS allows you to enable logging and auditing using two built-in services:

- **Amazon CloudWatch:** A service that allows you to log containers' activities and raise an alarm according to predefined thresholds (for example, low memory resources or high CPU, which requires up-scaling your ECS cluster)
- **AWS CloudTrail:** A service that allows you to monitor API activities (basically, any action performed on the ECS cluster)

Use the following best practices for auditing, monitoring, and threat management of Amazon ECS activity:

- Enable Amazon CloudWatch alarms for high-performance usage (which may indicate an anomaly in the ECS cluster behavior), as explained here: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Best_Practice_Recommended_Alarms_AWS_Services.html#ECS.

- Send ECS container logs to **Amazon CloudWatch logs** for further investigation and troubleshooting: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/using_awslogs.html.
- Enable **Amazon CloudWatch Container Insights** to collect information about the ECS activity for further investigation and troubleshooting, as follows: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/deploy-container-insights-ECS-cluster.html>.
- **AWS CloudTrail** is enabled by default for Amazon ECS. To keep ECS API activity for longer than 90 days for further investigation and analysis, send CloudTrail logs to an Amazon S3 bucket. For more information about ECS and CloudTrail logs, see the following: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/logging-using-cloudtrail.html>.
- Use **GuardDuty Runtime Monitoring** to detect threats in workloads running on top of Amazon ECS, as explained here: <https://docs.aws.amazon.com/guardduty/latest/ug/runtime-monitoring.html>.

In this section, we have learned how to securely maintain Amazon ECS, based on AWS infrastructure – from identity and access management, securing network and remote access, to logging and auditing. Next, let us see how we can secure Azure Container Apps.

Securing Azure Container Apps

Container Apps is an Azure-managed serverless container orchestration service, based on Kubernetes technology. It can integrate with other Azure services, such as ACR for storing containers, Microsoft Entra ID for managing permissions, Azure Files for persistent storage, and Azure Monitor.

General best practices for securing Azure Container Apps

Since Azure Container Apps is a new service (announced in September 2023), its documentation by Microsoft is still a work in progress.

Follow the Azure security best practices for Container Apps, as explained here:

<https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/azure-container-apps-security-baseline>.

Best practices for identity and access management in Azure Container Apps

Microsoft Entra ID is the supported service for managing permissions to access and run containers through Azure Container Apps (as explained in *Chapter 9, Effective Strategies for Implementing IAM Solutions*).

Use the following best practices for authentication and authorization in Azure Container Apps:

- Thorough documentation of Azure Container Apps and identity management can be found at the following link: <https://learn.microsoft.com/en-us/azure/container-apps/authentication>
- For internal applications, enable **Microsoft Entra ID** for all container apps to allow internal user authentication using Microsoft Entra ID, as explained here: <https://learn.microsoft.com/en-us/azure/container-apps/authentication-azure-active-directory#enable-microsoft-entra-id-in-your-container-app>
- To avoid unauthorized access to Container Apps for internal applications, require authentication for any request to access an application based on container apps, as follows: <https://learn.microsoft.com/en-us/azure/container-apps/authentication#authorization>
- For internal applications, restrict access to end users based on their business needs to avoid unauthorized access: <https://learn.microsoft.com/en-us/entra/identity-platform/howto-restrict-your-app-to-a-set-of-users>
- For internal applications, enforce authorization using RBAC to avoid unauthorized access to your cloud environment: <https://learn.microsoft.com/en-us/entra/identity-platform/howto-implement-rbac-for-apps>
- For an external application that contains non-public information, enforce authentication and authorization using OIDC to avoid unauthorized access: <https://learn.microsoft.com/en-us/azure/container-apps/authentication-openid>
- Use managed identities to allow access from Azure Container Apps to resources in Azure and follow the principle of least privilege when granting access to resources to avoid unauthorized access: <https://learn.microsoft.com/en-us/azure/container-apps/managed-identity>
- Use managed identities to allow Azure Container Apps permissions to pull images from ACR to avoid unauthorized use of container images, as explained here: <https://learn.microsoft.com/en-us/azure/container-apps/managed-identity-image-pull>
- For sensitive environments, encrypt your container images inside ACR using customer-managed keys stored inside Azure Key Vault (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as mentioned here: <https://learn.microsoft.com/en-us/azure/container-registry/tutorial-enable-customer-managed-keys>
- Use Azure Key Vault for the storage, retrieval, and rotation of secrets. Azure allows you to integrate Azure Container Apps with Azure Key Vault using Dapr to avoid unauthorized use of static credentials, as explained here: <https://learn.microsoft.com/en-us/azure/container-apps/manage-secrets>

Best practices for data protection in Azure Container Apps

To implement data protection in Azure Container Apps, use the following best practices:

- Thorough documentation of Azure Container Apps and networking can be found at the following link:

<https://learn.microsoft.com/en-us/azure/container-apps/networking>

- For production applications, select the **Workload profile** environment type to secure outbound traffic (using the firewall capability) and enforce environment isolation (i.e., dedicated hardware and single tenant), as follows: <https://learn.microsoft.com/en-us/azure/container-apps/environment#types>
- Configure **user-defined routes (UDRs)** to restrict outbound traffic from Azure Container Apps using Azure Firewall, as explained here: <https://learn.microsoft.com/en-us/azure/container-apps/user-defined-routes>
- Use **Network Security Groups (NSGs)** to restrict access to Azure Container Apps and avoid unauthorized network access: <https://learn.microsoft.com/en-us/azure/container-apps/firewall-integration?tabs=workload-profiles>
- Enforce traffic encryption using the TLS protocol to protect traffic confidentiality: <https://learn.microsoft.com/en-us/azure/container-apps/ingress-overview>
- For internal applications, configure ingress to accept traffic from within your container app's environment to avoid unauthorized network access: <https://learn.microsoft.com/en-us/azure/container-apps/ingress-how-to>
- To limit access to internal applications from specific IPs, use IP ingress restrictions to avoid unauthorized network access: <https://learn.microsoft.com/en-us/azure/container-apps/ip-restrictions>
- To protect inbound traffic to Azure Container Apps from the public internet, use the **Web Application Firewall on Application Gateway** to protect against application layer attacks: <https://learn.microsoft.com/en-us/azure/container-apps/waf-app-gateway>

Best practices for conducting auditing and monitoring in Azure Container Apps

Azure allows you to enable logging and auditing using Azure Monitor. For the auditing and monitoring of Azure Container Apps, enable diagnostic settings for Azure Container Apps for further investigation and analysis and send all logs (specifically system logs, which contain authentication activities) to a Log Analytics workspace, as explained here: <https://learn.microsoft.com/en-us/azure/container-apps/log-options>.

In this section, we have learned how to securely maintain Azure Container Apps, based on Azure infrastructure – from identity and access management, securing the network, to logging and auditing. Finally, let us learn how to secure Google Cloud Run.

Securing Google Cloud Run

Cloud Run is a Google-managed serverless container orchestration service. It can integrate with other GCP services, such as **Artifact Registry** for retrieving containers, **Google Cloud IAM** for managing permissions, **Google Firestore** for persistent database storage, and more.

General best practices for securing Google Cloud Run

Follow GCP security best practices for Cloud Run:

- Thorough documentation of security in Google Cloud Run can be found at the following links:
 - <https://cloud.google.com/run/docs/securing/security>
 - <https://cloud.google.com/run/docs/tutorials/secure-services>
- Use **Google Binary Authorization** to deploy container images from a signed and trusted source, as explained here: <https://cloud.google.com/binary-authorization/docs/run/overview>
- For sensitive environments, encrypt your container images using customer-managed encryption keys (as explained in *Chapter 11, Applying Encryption in Cloud Services*), keeping container image content confidential as follows: <https://cloud.google.com/run/docs/securing/using-cmek>

Best practices for identity and access management in Google Cloud Run

Google Cloud IAM is the supported service for managing permissions to access and run containers using Google Cloud Run (as explained in *Chapter 9, Effective Strategies for Implementing IAM Solutions*).

Use the following best practices for authentication in Google Cloud Run:

- Thorough documentation of Google Cloud Run and identity management can be found at the following links:
 - <https://cloud.google.com/run/docs/authenticating/overview>
 - <https://cloud.google.com/run/docs/securing/managing-access>

- To allow access to Google Cloud Run, configure an IAM role to enforce permissions to avoid unauthorized access to your cloud environment, following the principle of least privilege, as explained here:
 - <https://cloud.google.com/run/docs/reference/iam/roles>
 - <https://cloud.google.com/run/docs/reference/iam/permissions>
- For internal users who need access to Google Cloud Run, use the **Identity-Aware proxy** to authenticate the end users to avoid unauthorized access, as explained here: <https://cloud.google.com/iap/docs/enabling-cloud-run>
- For an external application that contains non-public information, enforce authentication using **Identity Platform** to avoid unauthorized access: <https://cloud.google.com/run/docs/tutorials/identity-platform>
- For each application, create a unique Cloud Run service identity, following the principle of least privilege, to avoid unauthorized access, as explained at the following links:
 - <https://cloud.google.com/run/docs/securing/service-identity#per-service-identity>
 - <https://cloud.google.com/run/docs/configuring/jobs/service-identity>
- Use **Google Secret Manager** for the storage, retrieval, and rotation of secrets. GCP allows you to integrate Google Cloud Run with Google Secret Manager using client libraries and avoid unauthorized access to static credentials: <https://cloud.google.com/run/docs/configuring/jobs/secrets>

Best practices for securing network access to Google Cloud Run

Google Cloud Run exposes services (among others, to the internet) – for that reason, it is important to plan before deploying each Google Cloud Run connected to Google Cloud VPC (as explained in *Chapter 6, Securing Networking Services – Part 1*).

Follow these best practices to secure network access to Google Cloud Run:

- To protect Google Cloud Run inbound access and avoid unauthorized network access, follow these best practices:
 - <https://cloud.google.com/run/docs/securing/ingress>
 - <https://cloud.google.com/run/docs/securing/private-networking>
 - <https://cloud.google.com/run/docs/configuring/networking-best-practices>

- To restrict access to Google Cloud Run from trusted sources and reduce the risk of data exfiltration, use **VPC Service Controls**, as explained here: <https://cloud.google.com/run/docs/securing/using-vpc-service-controls>
- To protect inbound traffic to Google Cloud Run from the public internet, use **Google Cloud Armor** to protect against application layer attacks: <https://cloud.google.com/armor/docs/integrating-cloud-armor#serverless>

Best practices for conducting auditing and security monitoring in Google Cloud Run

Google allows you to enable logging and auditing using **Google Cloud audit logs**.

The following resources offer comprehensive information on the best practices for auditing and monitoring Google Cloud Run:

- Thorough documentation of Google Cloud Run and identity management can be found at the following link: <https://cloud.google.com/run/docs/audit-logging>
- To forward container logs to **Google Cloud Logging** for further investigation and analysis, follow this documentation: <https://cloud.google.com/run/docs/logging#container-logs>

In this section, we have learned how to securely maintain Google Cloud Run, based on GCP infrastructure – from identity and access management, securing the network, to logging and auditing.

Summary

In this chapter, we have focused on container orchestrator services in AWS, Azure, and GCP.

In each section, we have reviewed how to manage identity management (for authentication and authorization), how to implement data protection (from access controls to networks and encryption), how to configure auditing and logging, and finally, how to implement industry best practices.

Next, we are going to see what the best practices are for securing common serverless/functions as a service from AWS, Azure, and GCP.

4

Securing Compute Services – Serverless and FaaS

In the previous chapter, we learned about security in containers and Kubernetes.

This chapter will cover serverless computing or **function as a service** (FaaS) and provide you with best practices on how to securely deploy and manage each of them.

Although there are many cloud-managed services (such as object storage, API gateways, load balancers, etc.) that are considered serverless, in this section, I will present the most common serverless computing services/FaaS. Following that, we are going to see what the best practices are for securing common FaaS deployments from AWS, Azure, and GCP. For each of these platforms, we will review best practices for securing FaaS from different angles (from IAM to network access, auditing, monitoring, and finally compliance).

In this chapter, we will cover the following topics:

- High-level recommendations for securing FaaS
- Best practices for identity and access management in FaaS
- Best practices for data protection in FaaS
- Best practices for auditing and security monitoring in FaaS
- Best practices for compliance in FaaS

Securing serverless/FaaS

Although the name implies that there are no servers, the term **serverless** means that you, as a customer of the service, are not responsible for the underlying compute infrastructure (operating system maintenance, scale, runtime management, and so on) – you simply import your code (according to the language supported by FaaS in each cloud provider), select your preferred runtime, select the amount of required memory per function (which affects the amount of CPU), and set the trigger to invoke the function.

In virtual machines (VMs), a customer deploys a guest OS on top of a managed hypervisor, and inside the OS, the customer deploys software (containing code, binaries, and libraries), and on top of it, different applications.

In containers, a customer deploys a container image on top of a container engine. The container image is a package of software (containing code, binaries, and libraries), and usually, each container is dedicated to a specific application.

In serverless/FaaS, a customer is using a managed FaaS, and inside it, the customer selects a runtime (such as Java, Python, Go, etc.) and deploys the function's code (and its dependencies). Each function serves a specific goal (such as querying a database, reading a file's metadata, etc.).

The following diagram presents the architectural differences between VMs, containers, and serverless:

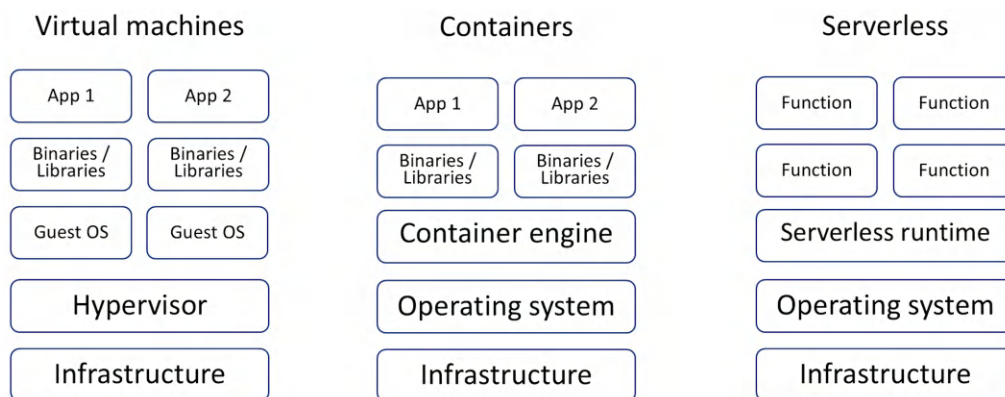


Figure 4.1 – VMs versus containers versus serverless

Most FaaS platforms support the following invocation modes:

- **Synchronous invocation**, such as an HTTP request, triggers the function to execute. In such a scenario, functions might be exposed to attacks from the public internet, and we need to protect the function using API gateways, WAF, and more.

In the following diagram, we can see that a **User** sends an HTTP request (1) through an **API Gateway** (2), which executes a **Function** to read information from a **Database** (3):

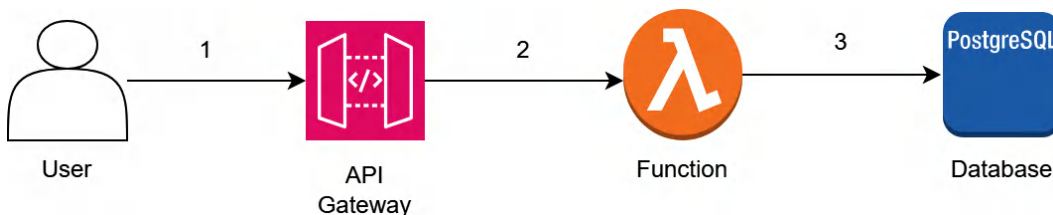


Figure 4.2 – Function invoked in synchronous mode

- **Asynchronous invocation**, such as an event in one service, triggers the function to execute and take some actions. Such scenarios usually happen inside private subnets.

In the following diagram, we can see that a **User** uploads an image to a **File storage** service (1), and the action of the file upload triggers a **Function** (2), which sends the file to a **Vision API** service for further analysis (3) and displays to the user information about the image (such as objects in the image – cars, buildings, people, etc.):

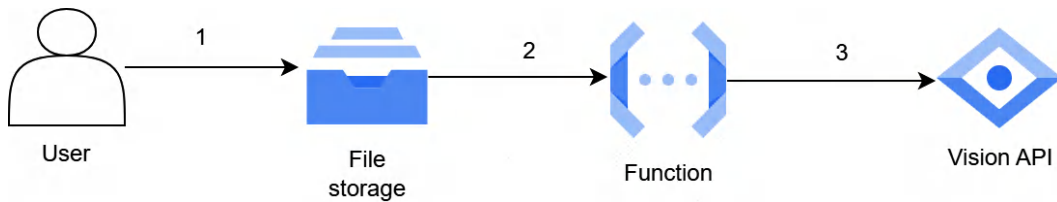


Figure 4.3 – Function invoked in asynchronous mode

Now that we are familiar with the basics of serverless/FaaS, let's see the best practices for securing them.

High-level recommendations for securing FaaS

Regardless of the cloud provider you are using when using FaaS, you should follow the following best practices:

- When running containers inside FaaS to execute functions, use container images from a trusted source and store them in a private container image registry to avoid the use of container images with malicious or vulnerable code.

Note that most CSPs offer a managed container registry service.

- When creating custom container images, use a minimal number of packages inside the container image to lower the attack surface.
- Scan your function's code for vulnerabilities in code, libraries, and binaries regularly.
- Keep your runtime environment and dependencies up-to-date with the latest security patches and updates.
- Conduct regular security audits and vulnerability assessments to identify and mitigate potential network security issues.
- Validate all inputs to your functions to prevent injection attacks and other common vulnerabilities.
- Use an API Gateway to protect inbound access to functions from the public internet by using configuration authorization access rules.

Let us check out the best practices that are particular to each platform.

AWS Lambda

AWS Lambda is Amazon's serverless service. It can integrate with other AWS services, such as **AWS IAM** for managing permissions to AWS Lambda, **Amazon CloudWatch** for monitoring AWS Lambda, and **Amazon S3** and **Amazon EFS** for persistent storage.

Thorough documentation on security in AWS Lambda can be found at the following links:

- <https://docs.aws.amazon.com/lambda/latest/dg/lambda-security.html>
- <https://docs.aws.amazon.com/whitepapers/latest/security-overview-aws-lambda/security-overview-aws-lambda.html>

Azure Functions

Azure Functions is Azure's function as a service. It can integrate with other Azure services, such as **Microsoft Entra ID** for managing permissions to Azure Functions, **Azure Monitor Application Insights** for monitoring Azure Functions, and **Azure Blob Storage** for persistent storage.

Thorough documentation on security in Azure Functions can be found here:

<https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/functions-security-baseline++>

Google Cloud Run functions

Google Cloud Run functions is GCP's function as a service. It can integrate with other GCP services, such as **Google Cloud IAM** for managing permissions to Google Cloud Run functions, **Google Cloud Audit Logs** for monitoring Google Cloud Run functions, and **Google Cloud Storage** for persistent storage.

Thorough documentation on security in Google Cloud Run functions can be found here:

<https://cloud.google.com/functions/docs/securing>

In this section, we were introduced to FaaS based on AWS, Azure, and GCP infrastructure. Next, we will have a deep-dive session about IAM, network access, auditing, and compliance.

Best practices for IAM in FaaS

There are many aspects to review when talking about identity and access management in FaaS, including, among others, the following:

- Authentication to a FaaS platform, which is usually done by other cloud services (such as API Gateways, message queuing services, message streaming services, and more)

- Authorization to maintain a FaaS, to upload an updated version of code for a deployed FaaS
- Permission to deploy and update a function (human for console/CLI/SDK, and service account for deployment as part of CI/CD pipeline)
- Permission of functions to access other resources on the CSP ecosystem

High-level best practices for IAM in FaaS are as follows:

- For interactive login (through the console/CLI/SDK) to deploy and maintain the FaaS, enforce the use of **multi-factor authentication (MFA)** to protect the FaaS from unauthorized access.
- Use identity federation to manage user identities across multiple services and platforms, ensuring consistent and secure access management.
- Use the CSP's built-in RBAC mechanism to restrict which identity has permissions to deploy or maintain functions, following the principle of least privilege.
- Follow the principle of least privilege when granting permissions to manage FaaS and for allowing functions access to resources in the CSP's ecosystem to avoid unauthorized access.
- Regularly review and update IAM policies and permissions to ensure they are still appropriate and secure. Remove any unnecessary permissions.
- Ensure that functions have the minimum permissions necessary to perform their tasks. Avoid granting broad permissions that could be exploited.
- Use temporary credentials for functions to access other resources. This reduces the risk associated with long-term credentials.
- Segment environments (development, testing, and production) to ensure that access controls are appropriately applied, and that sensitive data is protected.
- Secrets (such as API keys and database credentials) are a common way to authenticate functions against other resources and applications. Avoid storing secrets as part of code or configuration files; instead, use a managed vault service for storing, retrieval, and rotation of secrets to avoid unauthorized access to static credentials.

Now let us see the best practices in AWS Lambda, Azure Functions, and Google Cloud Run functions.

AWS Lambda

AWS IAM is the supported service for managing permissions to AWS Lambda (as explained in *Chapter 9, Effective Strategies for Implementing IAM Solutions*).

Read the documentation on IAM in AWS Lambda, which can be found at the following link: <https://docs.aws.amazon.com/lambda/latest/dg/security-iam.html>

Use the following best practices for authentication and authorization in AWS Lambda:

- Use identity-based IAM policies to grant permissions to access Lambda functions and maintain them, following the principle of least privilege, to avoid unauthorized access to Lambda functions. Visit the following link for more information:

`https://docs.aws.amazon.com/lambda/latest/dg/access-control-identity-based.html`

- Assign each Lambda function a unique IAM role (Lambda execution role) with permissions to access and use AWS resources according to business needs, to avoid unauthorized access of Lambda functions to AWS resources, as explained here:

`https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html`

- Use resource-based policies with AWS services to grant them permissions to invoke Lambda functions (for running tasks, accessing S3 buckets, monitoring using **CloudWatch Events**, and so on). See this link for more information:

`https://docs.aws.amazon.com/lambda/latest/dg/access-control-resource-based.html`

- Use **IAM Access Analyzer** to review which identities have access to Lambda functions and limit the permissions according to business needs, to avoid unauthorized access of Lambda functions to AWS resources, as explained here:

`https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-resources.html#access-analyzer-lambda`

- Use **AWS Secrets Manager** for the storage, retrieval, and rotation of secrets. Amazon allows you to integrate AWS Lambda with AWS Secrets Manager using the **AWS Parameters and Secrets Lambda Extension** to avoid unauthorized access to static credentials. For more information, read the following:

`https://docs.aws.amazon.com/secretsmanager/latest/userguide/retrieving-secrets\_lambda.html`

- For sensitive environments, encrypt Lambda environment variables using customer-managed keys (as explained in *Chapter 11, Applying Encryption in Cloud Services*) to keep environment variables (such as connection strings) confidential, as explained here:

`https://docs.aws.amazon.com/lambda/latest/dg/configuration-envvars-encryption.html`

Azure Functions

Microsoft Entra ID is the supported service for managing permissions to your Azure Functions (as explained in *Chapter 9, Effective Strategies for Implementing IAM Solutions*).

Use the following best practices for authentication and authorization in Azure Functions:

- For internal applications, enable **Microsoft Entra ID** for all Azure Functions to allow internal user authentication using Microsoft Entra ID, as explained here:

<https://learn.microsoft.com/en-us/azure/app-service/configure-authentication-provider-aad>

- For external applications (based on Azure Functions) that contain non-public information, enforce authentication and authorization using **OpenID Connect (OIDC)** to avoid unauthorized access. For more information, read the following:

<https://learn.microsoft.com/en-us/azure/app-service/configure-authentication-provider-openid-connect>

- Disable basic authentication to Azure Functions to avoid unauthorized access, as explained here:

<https://learn.microsoft.com/en-us/azure/app-service/configure-basic-auth-disable>

- Use managed identity to grant Azure Functions permissions to access Azure resources to avoid unauthorized access by Azure Functions to Azure resources:

<https://docs.microsoft.com/en-us/azure/app-service/overview-managed-identity>

- Use RBAC to enforce authorization to Azure Functions according to business needs to avoid unauthorized access, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-functions/security-concepts?tabs=v4#permissions>

- Use **Azure Key Vault** for the storage, retrieval, and rotation of secrets to avoid unauthorized access to static credentials. Azure allows you to integrate Azure Functions with Azure Key Vault using key vault references, as explained here:

<https://learn.microsoft.com/en-us/azure/app-service/app-service-key-vault-references>

Google Cloud Run functions

Google Cloud IAM is the supported service for managing permissions to your Google Cloud Run functions (as explained in *Chapter 9, Effective Strategies for Implementing IAM Solutions*).

Read information about IAM for Google Cloud Run functions, which can be found at this link: <https://cloud.google.com/functions/docs/securing#identity>

Use the following best practices for authentication and authorization in Google Cloud Run functions:

- Create a unique identity (service account) for each newly created Google Cloud Run function with minimal permissions using the Google Cloud IAM service to avoid unauthorized access by Cloud Run functions to GCP resources, as explained here:

<https://cloud.google.com/functions/docs/securing/function-identity>

- Use IAM roles to grant permissions to access Google Cloud Run functions and maintain them, following the principle of least privilege, to avoid unauthorized access to Cloud Run functions. For more information, see the following:

<https://cloud.google.com/functions/docs/securing/managing-access-iam>

- Use **Google Secret Manager** for the storage, retrieval, and rotation of secrets to avoid unauthorized access to static credentials. GCP allows you to integrate Google Cloud Run functions with Google Secret Manager, as explained at the following link:

<https://cloud.google.com/functions/docs/configuring/secrets>

In this section, we reviewed security best practices for IAM in AWS Lambda, Azure Functions, and Google Cloud Run functions. Next, we will look at the best practices for securing network access in FaaS.

Best practices for data protection in FaaS

Functions can be used internally inside a customer's environment (for example, as part of event-driven architecture between other components such as APIs) or for serving external customers, which requires network access protection.

Here are the high-level best practices for securing network access in FaaS:

- Protect access to internal functions using network access control lists to limit the access to functions or the access to other cloud resources by the function
- Implement a zero-trust model where every access request is authenticated, authorized, and encrypted, regardless of the source
- Use network segmentation to isolate different parts of your network, reducing the risk of lateral movement in case of a breach

- Use private endpoints to connect functions to other cloud services securely, avoiding exposure to the public internet
- Protect access to publicly exposed functions using API gateways, which adds authentication and network throttling mechanisms
- Implement rate limiting to protect your functions from being overwhelmed by too many requests in a brief period
- Protect access to publicly exposed functions using WAFs and **distributed denial-of-service (DDoS)** protection mechanisms to protect the functions against both application-layer-based and network-layer-based attacks
- Enforce traffic encryption using the **Transport Layer Security (TLS)** protocol for any access to/from a function to keep data confidentiality

Read on to understand the best practices for securing network access in AWS Lambda, Azure Functions, and Google Cloud Run functions.

AWS Lambda

AWS Lambda can be deployed either as an external resource outside your VPC, to access publicly facing resources, or inside your VPC, to access resources inside it (as explained in *Chapter 6, Securing Networking Services – Part 1*). For that reason, it is important to plan before deploying each Lambda function.

Read thorough documentation on network access in AWS Lambda, which can be found at the following links:

- <https://docs.aws.amazon.com/whitepapers/latest/security-overview-aws-lambda/lambda-isolation-technologies.html>
- <https://docs.aws.amazon.com/lambda/latest/dg/security-dataprotection.html>
- <https://docs.aws.amazon.com/lambda/latest/dg/foundation-networking.html>
- <https://docs.aws.amazon.com/lambda/latest/operatorguide/networking-vpc.html>

The best practices for network access in AWS Lambda are as follows:

- Use the Lambda interface endpoint to keep the traffic between the **Amazon Virtual Private Cloud (Amazon VPC)** and the Lambda functions private (i.e., within the AWS backbone), as explained here:

<https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc-endpoints.html>

- When a Lambda function is deployed in a private subnet inside a VPC, use **NAT Gateway** to allow Lambda functions access to outbound (IPv4) resources from private subnets. For more information, check out the following:

`https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc-internet.html`

- When a Lambda function is deployed inside a VPC, use an **egress-only internet gateway** to allow Lambda functions access to outbound (IPv6) resources, as explained here:

`https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc-internet.html`

- Use **Amazon API Gateway** for exposing Lambda functions to add a layer of authorization and network throttling mechanisms, as explained here:

`https://docs.aws.amazon.com/lambda/latest/dg/services-apigateway-tutorial.html`

- Use **AWS WAF** to protect Lambda URLs behind Amazon API Gateway to decrease the attack surface against the Lambda function, as explained here:

`https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-aws-waf.html`

Azure Functions

Azure Functions can access resources in your Azure subscription – for that reason, it is important to plan before deploying each Azure function that is connected to an Azure VNet (as explained in *Chapter 6, Securing Networking Services – Part 1*).

Read thorough documentation on network access in Azure Functions, which can be found in the following links:

- `https://learn.microsoft.com/en-us/azure/azure-functions/functions-networking-options`
- `https://learn.microsoft.com/en-us/azure/azure-functions/functions-networking-faq`

The following are the best practices for network access in Azure Functions:

- For production Azure Functions, select the premium hosting plan, which allows private network connectivity, as explained here:

`https://learn.microsoft.com/en-us/azure/azure-functions/functions-premium-plan`

- For production, internet-facing Azure Functions, select a dedicated hosting plan, which deploys Azure Functions inside an isolated environment called **App Service Environment (ASE)**, as documented here:

- <https://learn.microsoft.com/en-us/azure/azure-functions/dedicated-plan>
- <https://learn.microsoft.com/en-us/azure/app-service/environment/overview>

- To allow Azure Functions access to resources within a VNet and avoid unauthorized access to Azure resources, integrate Azure Functions with Azure VNet:

<https://learn.microsoft.com/en-us/azure/virtual-network/virtual-network-service-endpoints-overview>

- To keep the traffic between the Azure VNet and Azure Functions (i.e., within Azure backbone), create private endpoints, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-functions/functions-create-vnet>

- Use Azure **network security groups (NSGs)** to restrict inbound traffic to an Azure function:

<https://learn.microsoft.com/en-us/azure/azure-functions/functions-networking-options#network-security-groups>

- Use an **Azure virtual network NAT gateway** to allow Azure functions access to outbound resources from private subnets, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-functions/functions-how-to-use-nat-gateway>

- Use **Azure API Management** to authenticate incoming API requests to Azure Functions. See the following:

<https://learn.microsoft.com/en-us/azure/api-management/api-management-policies#authentication-policies>

- Enforce traffic encryption using the TLS protocol to protect traffic confidentiality, as explained here:

<https://learn.microsoft.com/en-us/azure/app-service/configure-ssl-certificate>

- In case you are planning to deploy Azure Functions using the **File Transfer Protocol (FTP)**, enforce the use of secure protocols (FTPS), explained as follows:

<https://learn.microsoft.com/en-us/azure/app-service/deploy-ftp#enforce-ftp>

- Create a separate Azure storage account for any newly created Azure function to avoid unauthorized access between different Azure Functions. See the following:

<https://learn.microsoft.com/en-us/azure/azure-functions/configure-networking-how-to#restrict-your-storage-account-to-a-virtual-network>

- To protect inbound traffic to Azure Functions deployed in an ASE mode from the public internet, use the **Web Application Firewall on Application Gateway**, as explained here:

<https://learn.microsoft.com/en-us/azure/app-service/environment/integrate-with-application-gateway>

Google Cloud Run functions

Google Cloud Run functions can access resources in your GCP project – for that reason, it is important to plan before deploying each Google Cloud Run function that is connected to a Google Cloud VPC (as explained in *Chapter 6, Securing Networking Services – Part 1*).

Read thorough documentation on network access in Google Cloud Run functions, which can be found here: <https://cloud.google.com/functions/docs/networking/network-settings>

The best practices for network access in Google Cloud Run functions are as follows:

- To restrict access to Google Cloud Run functions from trusted sources and reduce the risk of data exfiltration, use **VPC Service Controls**, as explained at the following link:

<https://cloud.google.com/functions/docs/securing/using-vpc-service-controls>

- In case your Cloud Run function requires access to resources inside your VPC or you need to control egress traffic from the Cloud Run function located in private subnets, use the **Serverless VPC Access** connector, as explained at the following link:

<https://cloud.google.com/functions/docs/networking/connecting-vpc>

Now let's review the best practices for auditing and monitoring in FaaS.

Best practices for auditing and security monitoring in FaaS

Auditing is a crucial part of data protection. There are many aspects to review when talking about auditing and monitoring in FaaS, including the following, among others:

- **Function activity logs**, such as errors while running a function (for troubleshooting purposes)

- **FaaS admin activity**, such as an upload of a new version of code to a function
- **FaaS access logs to other cloud resources**, such as API activity that a function initiated to cloud resources

Adopt the following best practices for auditing and monitoring in FaaS:

- Although admin activity is automatically audited by the CSPs, its log history is limited. As a best practice, send all audit and monitoring logs to a central storage for further analysis and incident response and to comply with various regulations.
- Implement comprehensive logging and monitoring to detect and respond to suspicious activities. Use tools provided by your cloud provider to monitor function execution and performance.
- Integrate your logging and monitoring systems with a **security information and event management (SIEM)** solution for centralized analysis and correlation of security events.
- Define and implement log retention policies to ensure that logs are kept for an appropriate period, balancing between compliance requirements and storage costs.
- Regularly verify the integrity of your logs to ensure they have not been tampered with. This can be done using checksums or digital signatures.
- Set up automated alerts for specific events or anomalies in your logs. This ensures that critical issues are promptly addressed.
- Add a threat detection solution to review the logs and generate prioritized follow-up actions.
- Implement real-time monitoring to detect and respond to issues as they occur. This can help in quickly identifying and mitigating potential security threats.
- Monitor user activities, especially activities of those with elevated privileges, to detect any unauthorized or suspicious actions.
- Regularly perform compliance audits to ensure that your FaaS environment adheres to relevant regulations and standards.

Let us dive into how these practices figure in specific platforms.

AWS Lambda

As with any other managed service, AWS allows you to enable logging and auditing using two built-in services:

- **Amazon CloudWatch:** A service that allows you to log Lambda activities and raise an alarm according to predefined thresholds (for example, number of invocations, percentage of errors, etc.)
- **AWS CloudTrail:** A service that allows you to monitor API activities (basically, any action performed by the Lambda function)

The best practices are as follows:

- Amazon CloudWatch automatically collects Lambda function's logs. For troubleshooting purposes, you can configure advanced logging, as explained here:

<https://docs.aws.amazon.com/lambda/latest/dg/monitoring-cloudwatchlogs.html#monitoring-cloudwatchlogs-advanced>

- Enable **Amazon CloudWatch Lambda Insights** to collect information about the Lambda activity for further investigation and troubleshooting. For more information, see the following:

<https://docs.aws.amazon.com/lambda/latest/dg/monitoring-insights.html>

- AWS CloudTrail is enabled by default for AWS Lambda. To keep Lambda API activity for longer than 90 days for further investigation and analysis, send CloudTrail logs to an Amazon S3 bucket, as explained here:

<https://docs.aws.amazon.com/lambda/latest/dg/logging-using-cloudtrail.html>

- Enable **Lambda Protection in Amazon GuardDuty** to detect security threats in Lambda network activity, as documented here:

<https://docs.aws.amazon.com/guardduty/latest/ug/lambda-protection.html>

Azure Functions

As with any other managed service, Azure allows you to enable logging and auditing using the Azure Monitor service.

Read thorough documentation of Azure Functions and identity management, which can be found here: <https://learn.microsoft.com/en-us/azure/azure-functions/monitor-functions>

The best practices are as follows:

- Enable diagnostic settings for Azure Functions for further investigation and analysis and send all logs to a **Log Analytics workspace**, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/essentials/create-diagnostic-settings>

- For production, internet-facing Azure Functions deployed inside an ASE, enable **Microsoft Defender for App Service** to detect web application attacks against your application, as documented here:

<https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-app-service-introduction>

Google Cloud Run functions

As with any other managed service, Google allows you to enable logging and auditing using the Google Cloud Audit Logs service.

Read thorough documentation of Google Cloud Run functions and identity management, which can be found at the following links:

- <https://cloud.google.com/functions/docs/monitoring>
- <https://cloud.google.com/functions/docs/monitoring/audit-logging>

The best practices are as follows:

- For sensitive/critical environments, enable data access audit logs (actions such as read configuration, create/modify/read user-provided resource data) for further investigation and analysis, as explained here:

<https://cloud.google.com/logging/docs/audit/configure-data-access>

Note

Data access audit logs increase storage costs. Review the business requirements to enable them.

In this section, we reviewed security-related best practices for auditing, monitoring, and threat detection across various managed services. Next, let's learn about compliance in FaaS.

Best practices for compliance in FaaS

Security configuration and compliance are crucial factors from a security point of view – both for working according to industry best practices and for complying with various regulations.

As a customer, you cannot control the underlying infrastructure – as a result, you must invest in secure coding to avoid attackers breaking into your application and causing harm that FaaS cannot protect.

At the time this book was written, only AWS offered the ability to configure compliance rules for running functions.

AWS Lambda

AWS Lambda is mainly code running inside a closed, managed environment.

The best practices are as follows:

- Use **AWS Config** for configuration compliance checks in Lambda functions (such as allowed actions, hardware resource limitation, etc.), as explained here:

`https://docs.aws.amazon.com/lambda/latest/dg/governance-config.html`

- Use **Amazon Inspector** to scan for vulnerabilities in code and packages inside Lambda functions, as shown here:

`https://docs.aws.amazon.com/inspector/latest/user/scanning-lambda.html`

- Use **AWS Signer** to sign your Lambda function code and make sure you only run signed code, as explained at the following link:

`https://docs.aws.amazon.com/lambda/latest/dg/governance-code-signing.html`

In this section, we learned how to secure FaaS based on AWS, Azure, and GCP – from IAM to network access, auditing, and finally, monitoring.

Summary

In this chapter, we focused on FaaS in AWS, Azure, and GCP.

In each section, we reviewed how to manage access (for authentication and authorization), how to control network access (from access controls to network encryption), how to configure auditing and logging, and finally, how to configure compliance (for services that support those capabilities).

In the next chapter, we will review storage services in the cloud.

Securing Storage Services

In the previous three chapters, we covered computing services. After that, the second most common resource when designing applications is storage – from **object storage** to **block storage** (which is also known as *instance-attached storage*), **file storage** to **Container Storage Interface (CSI)**.

We use storage services to store our data. Most applications require persistent storage for storing and retrieving data – from banks holding customers' financial records, e-commerce sites containing a product catalog, healthcare organizations storing patient data, and more.

The following is a list of common threats that might impact our data because it has been stored in the cloud:

- Unauthorized access
- Data leakage
- Data exfiltration
- Data loss

This chapter will cover all types of storage services and provide you with best practices on how to securely connect and use each of them.

In this chapter, we will cover the following topics:

- Securing object storage
- Securing block storage
- Securing file storage
- Securing the Container Storage Interface

Securing object storage

Each cloud provider has its own implementation of object storage, but at the end of the day, the basic idea is the same:

- Files (or objects) are stored inside buckets (these are logical concepts such as directories or logical containers)
- Access to files in object storage is either through HTTP(S), web command-line tools, or programmatically using SDKs

In the following diagram, we see how a virtual machine (VM) accesses object storage over the network:

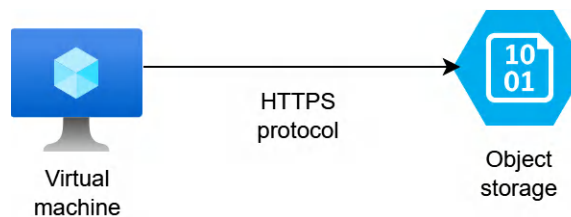


Figure 5.1 – VM accessing object storage

Object storage is not meant to store operating systems or databases (please refer to the *Securing block storage* section).

General best practices for securing object storage

Regardless of the cloud provider you are using when using object storage, you should adopt the following best practices:

- Use **access control lists (ACLs)** and **identity and access management (IAM)** to restrict access from the cloud environment to the storage service
- Use encryption at both transit and rest to ensure data confidentiality
- Use bucket policies to define permissions and access controls at the bucket level
- Enable versioning to keep multiple versions of an object in the same bucket
- Implement lifecycle policies to manage storage costs and data retention
- Classify data based on its sensitivity and apply appropriate security measures
- Ensure that public access to buckets and objects is blocked unless explicitly required
- Use secure APIs and manage API keys and credentials securely

- Enable auditing to have a log of who has access to the data and what actions were performed on the data (for instance, uploads, downloads, updates, deletions, and more)
- Create backups or snapshots to restore deleted data or return to the previous version of the data
- Enable the object storage immutability option to protect data in case of ransomware

Amazon S3

Amazon Simple Storage Service (Amazon S3) is the Amazon object storage service.

Read thorough documentation on security in Amazon S3, which can be found at the following links:

- <https://docs.aws.amazon.com/AmazonS3/latest/userguide/security.html>
- <https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-best-practices.html>

Azure Blob Storage

Azure Blob Storage is the Azure object storage service.

Read thorough documentation on security in Azure Blob Storage, which can be found at the following links:

- <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/storage-security-baseline>
- <https://learn.microsoft.com/en-us/azure/storage/blobs/security-recommendations>

Google Cloud Storage

Google Cloud Storage is the GCP object storage service.

Read thorough documentation on security in Google Cloud Storage, which can be found at the following links:

- <https://cloud.google.com/storage/docs/access-control>
- <https://cloud.google.com/storage/docs/best-practices>

Next, we are going to examine the best practices for securing object storage services from AWS, Azure, and GCP.

For more information, please refer to the following resource:

<https://aws.amazon.com/what-is/object-storage>

Best practices for IAM in object storage

IAM in object storage involves controlling which identities (humans or applications) can access files (objects) inside the managed storage.

Note

Unless you have a specific business requirement to expose object storage to the public internet, always enforce authentication and authorization mechanisms to restrict access to the object storage.

Here are the general best practices for securing IAM in object storage:

- Enforce authorization for any access request to an object storage and follow the principle of least privilege to prevent unauthorized access
- Avoid using static credentials to grant permanent access to object storage – use roles instead, to prevent credentials compromise
- Use conditions inside access policies to limit access to specific object storage according to business needs, to limit access to resources
- Use identity federation to manage user identities across multiple services and platforms
- Implement **multi-factor authentication (MFA)** for any human (interactive) login accessing the management console and other critical resources
- Use **role-based access control (RBAC)** to assign permissions based on roles rather than individual users
- Use temporary credentials for accessing object storage
- Regularly review and update IAM policies and permissions

Now, let's see the provisions for these best practices across the major cloud platforms.

Amazon S3

AWS IAM is a supported service for managing permissions to access objects inside S3 buckets. Permissions are managed using IAM policies – a JSON format file for configuring permissions (allow or deny) and conditions (such as allowing access to objects only for end users who authenticated using MFA, from a specific IP range, or within a specific time of the day/week).

AWS supports the following methods to access S3 bucket permissions:

- **IAM policies:** This allows you to set permissions for what actions are allowed or denied from an identity (for instance, a user, a group, or a role)

- **Bucket policies:** This allows you to set permissions at the S3 bucket level – it applies to all objects inside a bucket
- **S3 access points:** This gives you the ability to grant access to S3 buckets to a specific group of users or applications

Additionally, AWS controls permissions for identities (regardless of resources such as S3) on an AWS organizational basis. These are also called **service control policies (SCPs)**.

The effective permissions of an S3 bucket are the sum of SCPs with identity permissions (IAM policies), total resource permissions (bucket policies), and an AWS KMS policy (such as allowing access to an encrypted object), assuming the user was not denied in any of the previous criteria.

Read thorough documentation on IAM in Amazon S3, which can be found at the following link:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-management.html>

Use the following best practices for authentication and authorization in Amazon S3:

- Use identity-based policies to grant minimal access to objects inside S3 buckets, as explained here:
 - https://docs.aws.amazon.com/AmazonS3/latest/userguide/security_iam_id-based-policy-examples.html
 - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-iam.html>

- Block all public access to existing S3 buckets and make sure all newly created S3 buckets are configured as private, using bucket policies to prevent unauthorized access, as explained here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html>

- Create an IAM group, add users to it, and grant the required permissions on the target S3 bucket to the target IAM group, as explained here:

https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-iam.html#security_iam_authentication

- Restrict access for IAM users/groups to a specific S3 bucket rather than using wildcard permissions for all S3 buckets in the AWS account to prevent unauthorized access to resources, as explained here:

https://docs.aws.amazon.com/AmazonS3/latest/userguide/security_iam_service-with-iam.html#security_iam_service-with-iam-id-based-policies-actions

- Remove default bucket owner access permissions to S3 buckets and disable ACLs for S3 buckets to remove unnecessary permissions:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/about-object-ownership.html>

- Use **Amazon S3 access points** to enforce permissions and network controls for S3 object operations and prevent unauthorized access, as explained here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-points.html>

- Enable **MFA delete** for S3 buckets to prevent the accidental deletion of objects from a bucket, as explained here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/MultiFactorAuthenticationDelete.html>

Azure Blob Storage

Azure controls authorization to Blob Storage using **Microsoft Entra ID**. Permissions are managed using Azure RBAC.

Read thorough documentation on IAM in Azure Blob Storage, which can be found at the following link: <https://learn.microsoft.com/en-us/azure/storage/common/authorize-data-access>

Use the following best practices for authentication and authorization in Azure Blob Storage:

- Create a **Microsoft Entra ID** group, add users to the Entra ID group, and then grant the required permissions on the target Blob storage to the target Entra ID group. Assign an Azure role to gain access to Azure Blob Storage, as explained here:

<https://learn.microsoft.com/en-us/azure/storage/blobs/assign-azure-role-data-access>

- Use managed identities instead of a **shared access signature (SAS)** to grant applications/service accounts access to Azure Blob Storage to prevent credential compromise, as explained here:

<https://learn.microsoft.com/en-us/entra/identity-platform/multi-service-web-app-access-storage>

- When granting access to Azure Blob Storage, follow the principle of least privilege to prevent excessive permissions, as explained here:

<https://learn.microsoft.com/en-us/azure/storage/blobs/authorize-access-azure-active-directory>

- Disable Shared Key authorization for all storage accounts to prevent credential compromise, as explained here:

<https://learn.microsoft.com/en-us/azure/storage/common/shared-key-authorization-prevent>

Google Cloud Storage

Google Cloud IAM is a supported service for managing permissions to Google Cloud Storage.

Access can be controlled at the entire bucket level (including all objects inside this bucket) or at a specific object level (for example, suppose you would like to share a specific file with several of your colleagues).

GCP supports the following methods to access a cloud storage bucket:

- **Uniform bucket-level access:** This method sets permissions based on the Google Cloud IAM role (that is, user, group, domain, or public)
- **Fine-grained:** This method sets permissions based on a combination of both Google Cloud IAM and an ACL – it applies to either the entire bucket level or to a specific object

For authentication and authorization in Google Cloud Storage, use IAM policies to grant permissions to a cloud storage bucket and prevent unauthorized access, as documented here: <https://cloud.google.com/storage/docs/access-control/using-iam-permissions>

Read thorough documentation on IAM in Google Cloud Storage can be found at the following links:

- <https://cloud.google.com/storage/docs/authentication>
- <https://cloud.google.com/storage/docs/access-control/iam>

In this section, we have shared best practices for securing identity and access management in object storage services from AWS, Azure, and GCP.

Next, we will review best practices for data protection in object storage services.

Best practices for data protection in object storage

Data protection involves several controls such as protecting network access to the object storage, encrypting data in transit and at rest, object immutability, and more.

Note

Unless you have a specific business requirement to expose object storage to the public internet, always configure private buckets and enforce encryption to minimize the attack surface on object storage.

Here are the general best practices for data protection in object storage:

- Keep all object storage buckets private
- Enforce encryption in traffic using the TLS protocol and at rest for all object storage buckets
- Implement robust access control mechanisms, such as IAM policies and bucket policies
- Enable versioning to keep multiple versions of an object in the same bucket
- Implement lifecycle policies to manage storage costs and data retention
- Classify data based on its sensitivity and apply appropriate security measures
- Set up monitoring and alerts to detect unusual access patterns or potential security incidents
- Conduct regular security audits and vulnerability assessments
- Regularly back up your data and test your recovery procedures
- Implement a Zero Trust model where every access request is authenticated, authorized, and encrypted
- For regulated environments, enable object immutability to keep objects from accidental deletion

Let's see the provisions for data protection in object storage across the major cloud platforms next.

Amazon S3

Because Amazon S3 is a managed service, it is located outside the customer's **virtual private cloud (VPC)**. It is important to protect access to the Amazon S3 service.

Use the following best practices for data protection in Amazon S3:

- To secure access from your VPC to the Amazon S3, use **AWS PrivateLink**. This keeps traffic internally inside the AWS backbone through a secure channel, using the interface VPC endpoint, as explained here:

`https://docs.aws.amazon.com/AmazonS3/latest/userguide/privatelink-interface-endpoints.html`

- Use bucket policies to enforce the use of transport encryption (HTTPS only) and maintain data confidentiality, as explained here:

`https://docs.aws.amazon.com/IAM/latest/UserGuide/reference\_policies\_elements\_condition\_operators.html#Conditions\_Boolean`

- Use bucket policies to enforce the use of TLS version 1.2, as explained here:
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/amazon-s3-policy-keys.html#example-object-tls-version>
- For sensitive environments (which contain **Personally Identifiable Information (PII)**, credit card details, healthcare data, and more), encrypt data at rest using SSE-C, which encrypts data using a customer-managed key (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/ServerSideEncryptionCustomerKeys.html>
- Enable object versioning to prevent accidental deletion and to protect against ransomware, as explained here:
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html>
- For data that you need to retain for long periods due to regulatory requirements, use **S3 Object Lock** in compliance mode to protect the data from accidental deletion, explained as follows:
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>
- Consider using pre-signed URLs for scenarios where you need to allow external user access (with specific permissions such as file download) to an S3 bucket for a short period, as explained here:
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html>

Azure Blob Storage

Because Azure Blob Storage is a managed service; it is located outside the customer's **virtual network (VNet)**. It is important to protect access to the Azure Blob Storage service.

Use the following best practices for data protection in Azure Blob Storage:

- To secure access from your VNet to Azure Blob Storage, use an Azure private endpoint, which prevents sending network traffic outside your VNet through a secure channel, as explained here:
<https://learn.microsoft.com/en-us/azure/storage/common/storage-private-endpoints>
- Enforce the use of transport encryption (HTTPS only) for Azure Blob Storage, as explained here:
<https://learn.microsoft.com/en-us/azure/storage/common/storage-require-secure-transfer>

- Enforce a minimum of TLS version 1.2 for Azure Blob Storage, as explained here:
<https://learn.microsoft.com/en-us/azure/storage/common/transport-layer-security-configure-minimum-version>
- For sensitive environments (for example, which contain PII, credit card details, healthcare data, and more), encrypt data at rest using **customer-managed keys (CMKs)** stored inside Azure Key Vault (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:
<https://learn.microsoft.com/en-us/azure/storage/common/customer-managed-keys-overview>
- Enable Blob Storage versioning to prevent accidental deletion and to protect against ransomware, as explained here:
<https://learn.microsoft.com/en-us/azure/storage/blobs/versioning-enable>
- Deny default network access to the Azure storage account and only allow access from predefined conditions such as the setting up of IP addresses, as explained here:
<https://learn.microsoft.com/en-us/azure/storage/common/storage-network-security>
- To prevent the accidental deletion of objects from Azure Blob Storage, enable soft delete for blobs (which can be combined with Blob Storage versioning to protect objects from accidental deletion), as explained here:
<https://learn.microsoft.com/en-us/azure/storage/blobs/soft-delete-blob-enable>
- To prevent the accidental deletion of objects from Azure Blob Storage at the container level, enable soft delete for containers, as explained here:
<https://learn.microsoft.com/en-us/azure/storage/blobs/soft-delete-container-enable>
- For data that you need to retain for long periods due to regulatory requirements, use an immutable Blob Storage lock to protect the data from accidental deletion, as explained here:
<https://learn.microsoft.com/en-us/azure/storage/blobs/immutable-storage-overview>
- Disable object replication across Microsoft Entra tenants to prevent data exfiltration, as explained here:
<https://learn.microsoft.com/en-us/azure/storage/blobs/object-replication-prevent-cross-tenant-policies>

Google Cloud Storage

Because Google Cloud Storage is a managed service, it is located outside the customer's VPC. It is important to protect access to Google Cloud Storage.

Use the following best practices for data protection in Google Cloud Storage:

- To restrict access to Google Cloud Storage to trusted sources and reduce the risk of data exfiltration, use **VPC Service Controls**, as explained here:
 - <https://cloud.google.com/vpc-service-controls/docs/create-service-perimeters>
 - <https://cloud.google.com/vpc-service-controls/docs/manage-service-perimeters>

- To keep traffic from your VPC to Google Cloud Storage API private, use **Private Service Connect**, as documented here:

<https://cloud.google.com/vpc/docs/private-service-connect>

- For sensitive environments (which contain PII, credit card information, healthcare data, and more), encrypt cloud storage buckets using a customer-managed key inside Google Cloud KMS (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as explained here:

<https://cloud.google.com/storage/docs/encryption/using-customer-managed-keys>

- Enable object versioning to prevent accidental deletion and to protect against ransomware, as explained here:

<https://cloud.google.com/storage/docs/using-object-versioning>

- For data that you need to retain for long periods due to regulatory requirements, use the **Bucket Lock** feature to protect the data from accidental deletion, as explained here:

<https://cloud.google.com/storage/docs/bucket-lock>

- Consider using pre-signed URLs for scenarios where you need to allow external user access (with specific permissions such as file download) to a Cloud Storage bucket for a short period, as explained here:

<https://cloud.google.com/storage/docs/access-control/signed-urls>

In this section, we have shared best practices for data protection in object storage services from AWS, Azure, and GCP. Next, we will review best practices for auditing and monitoring in object storage services.

Best practices for auditing and security monitoring in object storage

Auditing is a crucial part of data protection.

There are many aspects to review when talking about auditing and monitoring in object storage, including the following:

- Users' activity logs (such as reading/writing/updating/deleting an object)
- Object storage admin activity (such as creating/deleting a bucket)

Follow these best practices for auditing and monitoring in object storage:

- Set up monitoring and alerts to detect unusual access patterns or potential security incidents.
- Conduct regular security reviews and audits of your object storage configurations.
- Although admin activity is automatically audited by the CSPs, its log history is limited. As best practice, send all audit and monitoring logs to a central storage for further analysis and incident response and to comply with various regulations.
- Implement real-time monitoring to detect and respond to issues as they occur.
- Define and implement log retention policies to balance compliance requirements and storage costs.
- Regularly verify the integrity of your logs using checksums or digital signatures.
- Monitor user activities, especially those with elevated privileges, to detect any unauthorized or suspicious actions.
- Integrate your logging and monitoring systems with a SIEM solution.
- Use machine learning or other advanced techniques to detect anomalies in function behavior or access patterns.
- Add a threat detection solution to review the logs and generate prioritized follow-up actions.

Next, we will review how AWS, Azure, and GCP implement auditing and monitoring in object storage.

Amazon S3

As with any other managed service, AWS allows you to enable logging and auditing using two built-in services:

- **Amazon CloudWatch:** This is a service that allows you to log object storage activities and raise the alarm according to predefined activities (such as excessive delete actions)
- **AWS CloudTrail:** This is a service that allows you to monitor API activities (essentially, any action performed on Amazon S3)

Read thorough documentation on auditing and monitoring in Amazon S3, which can be found at the following links:

- <https://docs.aws.amazon.com/AmazonS3/latest/userguide/monitoring-data-security.html>
- <https://docs.aws.amazon.com/AmazonS3/latest/userguide/monitoring-overview.html>

Use the following best practices for auditing and monitoring in Amazon S3:

- AWS CloudTrail is enabled by default for Amazon S3. CloudTrail logs provide a detailed record of API calls made by users and services in your AWS account (such as creating/deleting/listing buckets). To keep S3 API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/enable-cloudtrail-logging-for-s3.html>

- Enable S3 server access logs to record all access activities as complementary to AWS CloudTrail API-based logging (for future forensics). S3 server access logs provide detailed records of requests made to your S3 buckets (such as request type, requester, bucket name, object key, response status, etc.). Enable S3 server access logs, as explained here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/ServerLogs.html>

Note

S3 server access logs increase storage costs. Review the business requirements to enable them.

- Use Amazon CloudWatch alarms to generate notifications about abnormal activities (such as excessive upload or download requests, excessive object delete requests, etc.), as documented here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/cloudwatch-monitoring.html>

- Use the bucket ACL's **write** permissions for the Amazon S3 log delivery group to allow this group the ability to write access logs (for further analysis), as explained here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/enable-server-access-logging.html#grant-log-delivery-permissions-general>

- Use **Access Analyzer for S3** to locate S3 buckets with public access or S3 buckets that have access from external AWS accounts, as explained here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-analyzer.html>

- Use **Amazon S3 Protection** in **Amazon GuardDuty** to detect threats in Amazon S3 resources, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/s3-protection.html>

- For S3 buckets containing critical business information, use the S3 object integrity check capability to evaluate object upload/download activities against an object's checksum to make sure an object's integrity remains intact, as explained here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/checking-object-integrity.html>

- Use **Amazon S3 Inventory** to monitor the status of S3 bucket replication (such as encryption on both the source and destination buckets), as explained here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-inventory.html>

- Use **AWS Config** to monitor for configuration changes in S3 buckets (such as allowing public access, allowing unencrypted object upload to an S3 bucket, etc.), as explained here:

<https://aws.amazon.com/blogs/security/how-to-use-aws-config-to-monitor-for-and-respond-to-amazon-s3-buckets-allowing-public-access/>

- Use **Amazon Macie** to scan S3 buckets and search for sensitive data (such as PII, financial data, credentials, etc.), as explained here:

<https://docs.aws.amazon.com/macie/latest/user/monitoring-s3-how-it-works.html>

Azure Blob Storage

Azure allows you to monitor blob storage using the following services:

- **Azure Monitor:** This service logs access and audit events from Azure Blob Storage
- **Microsoft Defender for Storage:** This service (part of Microsoft Defender for Cloud), allows you to detect potential threats in the Azure Blob Storage configuration

Read thorough documentation on auditing and monitoring in Azure Blob Storage, which can be found at the following link:

<https://learn.microsoft.com/en-us/azure/storage/blobs/monitor-blob-storage>

Use the following best practices for auditing and monitoring in Azure Blob Storage:

- Enable diagnostic settings for Azure Blob Storage and send all logs to a Log Analytics workspace, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/essentials/create-diagnostic-settings>

- Enable log alerts using the Azure Monitor service to track access to Azure Blob Storage and raise alerts (such as multiple failed access attempts to Blob Storage in a short period), as explained here:

- <https://learn.microsoft.com/en-us/azure/storage/blobs/blob-storage-monitoring-scenarios>

- <https://learn.microsoft.com/en-us/azure/azure-monitor/alerts/alerts-unified-log>

- Log anonymous successful access attempts to locate an unauthorized access attempt to Azure Blob Storage, as documented here:

<https://learn.microsoft.com/en-us/rest/api/storageservices/storage-analytics-logged-operations-and-status-messages>

- Use **Azure Storage Analytics logging** to monitor for excessive successful or failed login attempts to access Azure Blob Storage, as explained here:

<https://learn.microsoft.com/en-us/azure/storage/common/storage-analytics-logging>

- Enabled **Microsoft Defender for Storage** to receive security alerts about potential threats in Blob Storage resources, as explained here:

- <https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-storage-introduction>

- <https://learn.microsoft.com/en-us/azure/defender-for-cloud/tutorial-enable-storage-plan>

- Use **Microsoft Purview** to scan Azure Blob Storage and search for sensitive data (such as PII, financial data, credentials, etc.), as explained here:

<https://learn.microsoft.com/en-us/purview/register-scan-azure-blob-storage-source>

Google Cloud Storage

As with any other managed service, Google allows you to enable auditing and security monitoring using the Google Cloud Logging service – a service that allows you to audit Cloud Storage-related activities.

Read thorough documentation on auditing and monitoring in Google Cloud Storage, which can be found at the following links:

- <https://cloud.google.com/storage/docs/audit-logging>
- <https://cloud.google.com/storage/docs/access-logs>

For the auditing and monitoring of Google Cloud Storage, use the following best practices:

- For sensitive/critical environments, enable data access audit logs (actions such as read configuration, create/modify/read user-provided resource data, etc.), as explained here:

<https://cloud.google.com/logging/docs/audit/configure-data-access>

Note

Data access audit logs increase storage costs. Review the business requirements to enable them.

- Use **Google Sensitive Data Protection** to scan Google Cloud Storage and search for sensitive data (such as PII, financial data, credentials, etc.), as explained here:

<https://cloud.google.com/sensitive-data-protection/docs/profile-org-folder-cloud-storage>

In this section, we learned how to secure object storage based on the AWS, Azure, and GCP infrastructure. This included IAM, data protection, logging and auditing. In the next section, we will explore block storage and how to secure it.

Securing block storage

Block storage is a storage scheme such as the on-premises **storage area network (SAN)**.

It allows you to mount a volume (disk), format it to a common filesystem (such as NTFS for Windows or Ext4 for Linux), and store various files, databases, or entire operating systems.

In the following diagram, we can see how a VM is mounting block storage directly from within the operating system:

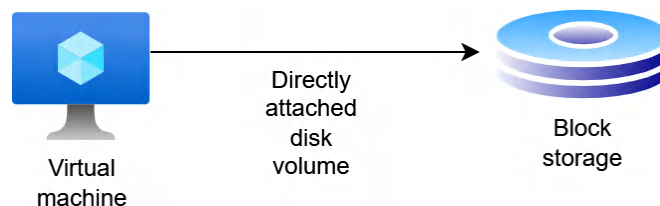


Figure 5.2 – Attaching block storage

General best practices for securing block storage

Regardless of the cloud provider you are using when using block storage, you should follow these best practices:

- Use IAM services to control who can attach, detach, or create a snapshot for block storage volumes to minimize the risk of data exfiltration
- Enforce encryption at rest on all block storage volumes and snapshots during creation to maintain data confidentiality
- Use ACLs to define who can access specific volumes and what actions they can perform
- Implement network security measures such as VPC configurations, security groups, and firewalls
- Conduct regular security audits and vulnerability assessments
- Enable detailed monitoring and logging to track access and changes to block storage volumes
- Regularly verify the integrity of your data using checksums or other methods
- Use automated backup solutions to ensure regular and consistent backups
- Develop and maintain a disaster recovery plan
- Create backups or snapshots to restore deleted data or return to the previous version of the data

Next, we are going to examine what the best practices are for securing block storage from AWS, Azure, and GCP.

For more information, please refer to the following resource:

<https://aws.amazon.com/what-is/block-storage>

Best practices for securing Amazon Elastic Block Store

Amazon Elastic Block Store (Amazon EBS) is the AWS block storage service.

It is common, when working with EC2 instances, to attach an additional volume to store your data (separately from the operating system volume). This is also known as block storage.

Amazon EBS can be attached to a single EC2 instance and can be accessed from within the operating system. However, EBS volumes are not limited to EC2 instances; they can be used as persistent storage for Amazon ECS or Amazon EKS (using a CSI driver).

The traffic between your EC2 instance and your attached EBS volume is encrypted at transit (and is automatically configured and controlled by AWS).

The following command uses the AWS CLI tool to create an encrypted EBS volume in a specific AWS Availability Zone:

```
aws ec2 create-volume \  
  --size 80 \  
  --encrypted \  
  --availability-zone <AWS_AZ_code>
```

The following command uses the AWS CLI tool to enable EBS encryption by default in a specific AWS Region:

```
aws ec2 enable-ebs-encryption-by-default --region <Region_Code>
```

Read thorough documentation on security in Amazon EBS can be found at the following link:

<https://docs.aws.amazon.com/ebs/latest/userguide/security.html>

Use the following best practices for data protection in Amazon EBS:

- Encrypt data at rest on all EBS volumes and snapshots of the EC2 instance during creation, as documented here:

<https://docs.aws.amazon.com/ebs/latest/userguide/ebs-encryption.html>

- For sensitive environments (which contain PII, credit card details, healthcare data, and more), encrypt data at rest on all EBS volumes and snapshots of the EC2 instance during creation using a CMK (as explained in *Chapter 11, Applying Encryption in Cloud Services*) as documented here:

<https://docs.aws.amazon.com/mgn/latest/ug/ebs-encryption-kms.html>

- Encrypt data at rest on all EBS volumes and snapshots of an Amazon ECS task during creation to maintain data confidentiality, as explained here:

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ebs-kms-encryption.html>

- Use AWS IAM identity-based policies to control who can attach, detach, or create a snapshot for EBS volumes to minimize the risk of data exfiltration, as explained here:

https://docs.aws.amazon.com/ebs/latest/userguide/security_iam_service-with-iam.html

- Enable block public access to snapshots to prevent data exfiltration, as explained here:

<https://docs.aws.amazon.com/ebs/latest/userguide/block-public-access-snapshots.html>

- Use **Amazon GuardDuty Malware Protection** to detect malware on EBS volumes, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/gdu-malpro-supported-volumes.html>

- Set names and descriptions for EBS volumes to better understand which EBS volume belongs to which EC2 instance or an ECS task.
- Use tagging (that is, *labeling*) for EBS volumes to allow a better understanding of which EBS volume belongs to which EC2 instance or an ECS task.

Best practices for securing Azure-managed disks

Azure-managed disks are Azure-managed block-level storage.

It is common, when working with VMs, to attach an additional volume to store your data (that is, separately from the operating system volume). This is also known as block storage.

Azure-managed disks are not limited to Azure VMs; they can also be used as persistent storage for **Azure Kubernetes Service (AKS)**.

The following command uses the Azure CLI tool to encrypt a specific VM in a specific resource group using a unique customer *key vault*:

```
az vm encryption enable -g MyResourceGroup --name MyVM --disk-encryption-keyvault myKV
```

The following command uses the Azure CLI tool to show the encryption status of a specific VM in a specific resource group:

```
az vm encryption show --name "myVM" -g "MyResourceGroup"
```

Read thorough documentation on security in Azure-managed disks, which can be found at the following link:

<https://learn.microsoft.com/en-us/azure/virtual-machines/managed-disks-overview#security>

Use the following best practices for data protection in Azure-managed disks:

- Encrypt data at rest on all Azure VM disk volumes during creation to maintain data confidentiality, as documented here:

<https://learn.microsoft.com/en-us/azure/virtual-machines/disk-encryption>

- For sensitive environments (which contain PII, credit card details, healthcare data, and more), encrypt data at rest on all Azure VM disk volumes during creation using CMKs (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:

<https://learn.microsoft.com/en-us/azure/virtual-machines/disks-enable-customer-managed-keys-portal>

- Use Azure RBAC to control who can import or export Azure-managed disk snapshots to minimize the risk of data exfiltration, as explained here:

<https://learn.microsoft.com/en-us/azure/virtual-machines/disks-restrict-import-export-overview>

- Set names for the Azure disk volumes to allow a better understanding of which disk volume belongs to which VM
- Use tagging (that is, *labeling*) for disk volumes to allow a better understanding of which disk volume belongs to which VM

Best practices for securing Google's Persistent Disk

Google's Persistent Disk is a part of the GCP block storage.

Persistent Disk is not limited to **Google Compute Engine (GCE)**; it can also be used as persistent storage for **Google Kubernetes Engine (GKE)**.

The following command uses the Google Cloud SDK to encrypt a persistent disk in a specific GCP project in a specific region, using a specific encryption key:

```
gcloud compute disks \
  create encrypted-disk \
  --kms-key \ projects/[KMS_PROJECT_ID]/locations/[REGION]/keyRings/
[KEY_RING]/cryptoKeys/[KEY]
```

Use the following best practices for data protection in persistent disks:

- Encrypt data at rest on all persistent disks during creation, as documented here:
- <https://cloud.google.com/compute/docs/disks/disk-encryption>
- For sensitive environments (which contain PII, credit card details, healthcare data, and more), encrypt data at rest on all Persistent Disks during creation using **customer-managed encryption keys (CMEKs)** (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:

<https://cloud.google.com/compute/docs/disks/customer-managed-encryption>

- Use IAM roles to control who can list, delete, or share a snapshot to minimize the risk of data exfiltration, as explained here:

<https://cloud.google.com/compute/docs/disks/manage-snapshots>

- Set names for Google's persistent disks to allow you to have a better understanding of which persistent disk belongs to which machine instance
- Use tagging (that is, *labeling*) for persistent disks or snapshots to allow you to have a better understanding of which disk or snapshot belongs to which machine instance

In this section, we learned how to secure block storage. Since block storage volumes are part of the common compute services (such as Amazon EC2, Azure VM, GCE, and more), the only way to protect block storage volumes (and their snapshots) is to encrypt them before using them.

Access to block storage volumes is done from within the guest operating system and auditing is part of the capabilities of guest operating systems.

Securing file storage

File storage is a piece of storage such as the on-premises **network-attached storage (NAS)**.

Each cloud provider has its own implementation of file storage, but at the end of the day, the basic idea is the same:

- They offer support for common file-sharing protocols (such as NFS and SMB/CIFS)
- They can mount a volume from a managed file service into an operating system to store and retrieve files for multiple VMs in parallel
- They can control access permissions to the remote filesystem
- They enable automatic filesystem growth

In the following diagram, we can see how a VM is connecting to a remote file storage, using the NFS protocol for Linux VMs:

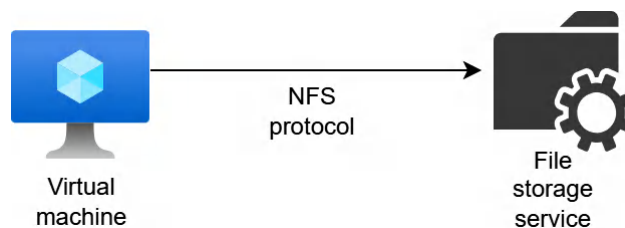


Figure 5.3 – VM connecting to an NFS storage

General best practices for securing file storage

Regardless of the cloud provider you are using when using file storage, you should follow these best practices:

- Use IAM services to control who can access file storage
- Enforce encryption at rest on all file storage volumes during creation
- Ensure that data is encrypted in transit to protect it from interception during transfer
- Implement MFA for any human (interactive) login accessing the management console and other critical resources
- Use ACLs to define who can access specific files and what actions they can perform
- Conduct regular security audits and vulnerability assessments
- Enable detailed monitoring and logging to track access and changes to file storage
- Regularly verify the integrity of your data using checksums or other methods
- Use automated backup solutions to ensure regular and consistent backups
- Develop and maintain a disaster recovery plan
- Implement network security measures such as VPC configurations, security groups, and firewalls
- Avoid exposing file storage to the public internet

Next, we are going to examine the best practices for securing file storage services from AWS, Azure, and GCP.

For more information, please refer to the following resource:

<https://aws.amazon.com/what-is/nas>

Amazon Elastic File System

Amazon Elastic File System (Amazon EFS) is the Amazon file storage service based on the NFS protocol, mostly used by Linux servers.

Amazon EFS is not limited to EC2 instances; it can be used as persistent storage for Amazon ECS, Amazon EKS (using a CSI driver), and AWS Lambda.

Thorough documentation on security in Amazon EFS can be found at the following link: <https://docs.aws.amazon.com/efs/latest/ug/security-considerations.html>.

Amazon FSx for Windows File Server

Amazon FSx for Windows File Server is the Amazon file storage service based on the CIFS/SMB protocol, mostly used by Windows servers.

Amazon FSx for Windows File Server is not limited to EC2 instances; it can be used as persistent storage for Amazon ECS, Amazon EKS (using a CSI driver), and AWS Lambda.

Thorough documentation on security in Amazon FSx for Windows File Server can be found at the following link: <https://docs.aws.amazon.com/fsx/latest/WindowsGuide/security.html>.

Azure Files

Azure Files is the Azure file storage based on both NFS and CIFS/SMB protocols, serving both Windows and Linux servers.

Azure Files is not limited to Azure VMs; it can also be used as persistent storage for AKS, Azure Container Apps, and Azure Functions.

Google Filestore

Google Filestore is a GCP file storage service based on the NFS protocol, serving Linux servers.

Google Filestore is not limited to GCE; it can also be used as persistent storage for GKE and Google Cloud Run.

Thorough documentation on security in Google Filestore can be found at the following link: <https://cloud.google.com/filestore/docs/overview#data-protection>.

Best practices for IAM in file storage

IAM in file storage involves controlling which identities (humans or applications) can access files inside a managed file storage.

General best practices for securing IAM in file storage are as follows:

- Enforce authorization for any access request to file storage and follow the principle of least privilege
- Use conditions inside access policies to limit access to specific file storage according to business needs
- Implement MFA for any human (interactive) login accessing the management console and other critical resources

- Use RBAC to assign permissions based on roles rather than individual users
- Enable detailed logging and monitoring of all access and actions performed within your file storage environment
- Use temporary credentials for accessing file storage
- Ensure that data is encrypted both at rest and in transit
- Regularly review and update IAM policies and permissions
- Use identity federation to manage user identities across multiple services and platforms

Amazon EFS

AWS IAM is a supported service in which you manage permissions to access Amazon EFS.

Use the following best practices for authentication and authorization in Amazon EFS:

- Use identity-based policies to enforce access permissions for humans or applications access to Amazon EFS, following the principle of least privileges, as explained at the following links:
 - https://docs.aws.amazon.com/efs/latest/ug/security_iam_id-based-policy-examples.html
 - <https://docs.aws.amazon.com/efs/latest/ug/iam-access-control-nfs-efs.html>
- Combine AWS IAM policies with Amazon EFS access points to enforce access permissions to Amazon EFS, as explained here:
<https://docs.aws.amazon.com/efs/latest/ug/efs-access-points.html>
- Create an IAM group, add users to the IAM group, and then grant the required permissions on the target Amazon EFS to the target IAM group, as explained here:
https://docs.aws.amazon.com/efs/latest/ug/security-iam.html#security_iam_authentication
- Use IAM policies to grant the minimal required permissions to create EFS volumes or access and use Amazon EFS, as explained here:
<https://docs.aws.amazon.com/efs/latest/ug/security-iam-awsmanpol.html>
- When using IAM policies, specify conditions and what actions an end user can take (along with the mentioned condition) on the target filesystem to limit access to resources, as explained here:
https://docs.aws.amazon.com/service-authorization/latest/reference/list_amazonelasticfilesystem.html

- To implement **attribute-based access control (ABAC)**, use tags to control access to Amazon EFS, as explained here:

<https://docs.aws.amazon.com/efs/latest/ug/using-tags-efs.html>

Amazon FSx for Windows File Server

FSx for Windows File Server supports the following identity management services:

- **Microsoft Active Directory (AD)**: Used to authenticate and authorize users and groups from an Active Directory (such as any Windows-based file storage on-premises)
- **AWS IAM**: Used to authenticate and authorize AWS cloud-based identities (users, groups, and EC2 instances)

Use the following best practices for authentication and authorization in FSx for Windows File Server:

- For organizations using Microsoft AD, where all identities (user and groups) are managed in AD and all machines (such as EC2 instances) are domain joined, integrate FSx for Windows File Server with an AD domain and enforce access permissions in the same way it was done on-premises, as explained here:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/aws-ad-integration-fsxW.html>

- For cloud-native authentication, use identity-based policies to enforce access permissions for humans or applications access to FSx for Windows File Server, following the principle of least privileges, as explained here:

https://docs.aws.amazon.com/fsx/latest/WindowsGuide/security_iam_id-based-policy-examples.html

- When using AWS IAM, use IAM policies to grant the minimal required permissions to use FSx for Windows File Server, as explained here:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/security-iam-awsmanpol.html>

- To implement ABAC, use tags to control access to FSx for Windows File Server, as explained here:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/using-tags-fsx.html>

Azure Files

Azure supports the following authentication and authorization mechanisms to control access to Azure Files:

- **Microsoft Entra ID:** This is the built-in mechanism to authenticate and authorize Azure cloud-based identities
- **Microsoft Entra Domain Services:** This is a managed domain service in Azure, like the on-premises AD, which supports domain join machines, group policies, LDAP, and Kerberos authentication
- **On-premises Active Directory Domain Services (AD DS):** This is the identity management mechanism for organizations who still manage their resources on-premises and supports domain join machines, group policies, LDAP, and Kerberos authentication

Read thorough documentation on IAM in Azure Files, which can be found at the following link:

<https://learn.microsoft.com/en-us/azure/storage/files/storage-files-active-directory-overview>

Use the following best practices for authentication and authorization in Azure Files:

- To allow identities to access Azure Files using **Microsoft Entra Domain Services** and prevent unauthorized access, follow the instructions here:

<https://learn.microsoft.com/en-us/azure/storage/files/storage-files-identity-auth-domain-services-enable>

- To allow identities from an on-premises environment access to Azure Files (using the SMB/CIFS protocol) in a hybrid architecture, enable AD DS authentication, as explained at the following links:

- <https://learn.microsoft.com/en-us/azure/storage/files/storage-files-identity-ad-ds-enable>

- <https://learn.microsoft.com/en-us/azure/storage/files/storage-files-identity-ad-ds-configure-permissions>

- <https://learn.microsoft.com/en-us/azure/storage/files/storage-files-identity-auth-hybrid-identities-enable>

- <https://learn.microsoft.com/en-us/azure/storage/files/storage-files-identity-auth-linux-kerberos-enable>

- For cloud-native authorization, use Azure RBAC to follow the principle of least privilege, as explained here:

<https://learn.microsoft.com/en-us/azure/storage/files/storage-files-identity-ad-ds-configure-permissions#azure-rbac-permissions>

Google Filestore

Google Cloud IAM is a supported service in which to manage permissions to access Google Filestore.

Use IAM roles to configure minimal permissions to any Google Filestore instance, as explained here: <https://cloud.google.com/filestore/docs/iam>.

In this section, we discussed IAM in file storage services. Next, we will discuss data protection mechanisms in file storage services.

Best practices for data protection in file storage

Data protection involves several controls such as protecting network access to the file storage, encrypting data in transit and at rest, and more.

General best practices for data protection in file storage are as follows:

- Keep all file storage private
- Enforce encryption in traffic and at rest for all file storage volumes
- Implement robust access control mechanisms, such as IAM policies and ACLs
- Regularly verify the integrity of your data using checksums or other methods
- Set up monitoring and alerts to detect unusual access patterns or potential security incidents
- Conduct regular security audits and vulnerability assessments
- Use automated backup solutions to ensure regular and consistent backups
- Develop and maintain a disaster recovery plan
- Classify data based on its sensitivity and apply appropriate security measures
- Use backup or snapshots to allow file restoration in case of accidental deletion or to return to a previous file version

Amazon EFS

Because Amazon EFS is a managed service, it is located outside the customer's VPC. It is important to protect access to the Amazon EFS service.

Use the following best practices for data protection in Amazon EFS:

- Enforce encryption in transit using the TLS protocol, as explained here:
<https://docs.aws.amazon.com/efs/latest/ug/encryption.html>
- Encrypt data at rest using AWS-managed keys (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:
<https://docs.aws.amazon.com/efs/latest/ug/encryption-at-rest.html>

- For sensitive environments (which contain PII, credit card details, healthcare data, and more), encrypt data at rest using symmetric CMKs (as explained in *Chapter 11*), as documented here:

<https://docs.aws.amazon.com/efs/latest/ug/encryption-at-rest.html#howencrypt>

- Use an IAM policy to enforce encryption at rest for Amazon EFS filesystems. You can do this by setting the value of `elasticfilesystem:Encrypted` to `True` inside the IAM policy condition, as explained here:

https://docs.aws.amazon.com/efs/latest/ug/security_iam_id-based-policy-examples.html#using-iam-to-enforce-encryption-at-rest

- Use VPC security groups to control the access between your Amazon EC2 machines and the Amazon EFS mount volumes and prevent unauthorized access, as explained here:

<https://docs.aws.amazon.com/efs/latest/ug/network-access.html>

- To secure access from your VPC to the Amazon EFS, use AWS PrivateLink. This keeps traffic internally inside the AWS backbone through a secure channel, using the interface VPC endpoint, as explained here:

<https://docs.aws.amazon.com/efs/latest/ug/efs-vpc-endpoints.html>

- Use Amazon EFS access points to enforce permissions and network controls for EFS operations and prevent unauthorized access, as explained here:

<https://docs.aws.amazon.com/efs/latest/ug/efs-access-points.html>

- Use **AWS Backup** to back up and restore files from Amazon EFS for data resiliency, as explained here:

<https://docs.aws.amazon.com/efs/latest/ug/awsbackup.html>

Amazon FSx for Windows File Server

FSx for Windows File Server may contain critical business information and, for that reason, it is important to protect access to the Amazon FSx for Windows File Server.

Use the following best practices for data protection in Amazon FSx for Windows File Server:

- Enforce encryption in transit using the SMB encryption, as explained here:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/encryption-in-transit.html>

- Encrypt data at rest using AWS-managed keys (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/encryption-at-rest.html>

- For sensitive environments (which contain PII, credit card details, healthcare data, and more), encrypt data at rest using CMKs (as explained in *Chapter 11*), as explained here:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/encryption-at-rest.html#EFSKMS>

- Use VPC security groups to control the access between your Amazon EC2 machines and the Amazon FSx for Windows File Server shared volumes and prevent unauthorized access, as explained here:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/limit-access-security-groups.html>

- To secure access from your VPC to the Amazon FSx for Windows File Server, use AWS PrivateLink. This keeps traffic internally inside the AWS backbone, through a secure channel, using the interface VPC endpoint, as explained here:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/fsx-vpc-endpoints.html>

- Use **AWS Backup** to back up and restore files from Amazon FSx for Windows File Server and for data resiliency, as explained here:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/using-backups.html>

Azure Files

Because Azure Files is a managed service, it is located outside the customer's VNet. It is important to protect access to the Azure Files service.

Read thorough documentation on network protection in Azure Files, which can be found at the following link:

<https://learn.microsoft.com/en-us/azure/storage/files/storage-files-networking-overview>

Use the following best practices for data protection in Azure Files:

- Enforce encryption in transit using TLS 1.2, as explained here:

<https://learn.microsoft.com/en-us/azure/storage/common/transport-layer-security-configure-minimum-version>

- Encrypt data at rest using Microsoft-managed keys (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:

<https://learn.microsoft.com/en-us/azure/storage/common/storage-service-encryption>

- For sensitive environments (which contain PII, credit card details, healthcare data, and more), encrypt data at rest using CMKs stored inside Azure Key Vault (as explained in *Chapter 11*), as documented here:

<https://learn.microsoft.com/en-us/azure/storage/common/customer-managed-keys-overview>

- To secure access from your VNet to the Azure Files, use an Azure private endpoint, which prevents sending network traffic outside your VNet through a secure channel and prevents unauthorized access, as explained here:

<https://learn.microsoft.com/en-us/azure/storage/files/storage-files-networking-endpoints>

- To prevent the accidental deletion of files from Azure Files (SMB only), enable soft delete for Azure file shares, as explained here:

<https://learn.microsoft.com/en-us/azure/storage/files/storage-files-enable-soft-delete>

- Use share snapshots to allow the restoration of files from Azure Files, as explained here:

<https://learn.microsoft.com/en-us/azure/storage/files/storage-snapshots-files>

Google Filestore

Because Google Filestore is a managed service, it is located outside the customer's VPC. It is important to protect access to Google Filestore.

Use the following best practices for data protection in Google Filestore:

- For sensitive environments (which contain PII, credit card information, healthcare data, and more), encrypt Google Filestore volumes using a CMEK inside Google Cloud KMS (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as explained here:

<https://cloud.google.com/filestore/docs/cmek>

- Use VPC firewall rules to control access to Google Filestore, as explained here:

<https://cloud.google.com/filestore/docs/configuring-firewall>

- Use IP-based access control to restrict access to Google Filestore and prevent unauthorized access, as explained at the following links:

- https://cloud.google.com/filestore/docs/creating-instances#configure_ip-based_access_control

- <https://cloud.google.com/filestore/docs/network-ip-requirements>

- To restrict access to Google Filestore from trusted sources and reduce the risk of data exfiltration, use VPC Service Controls, as explained here:

<https://cloud.google.com/filestore/docs/vpc-sc>

- Use **Filestore backup** to back up and restore files from Google Filestore, as explained here:

<https://cloud.google.com/filestore/docs/backups>

In this section, we discussed data protection in file storage services. Next, we will discuss auditing and monitoring in file storage services.

Best practices for auditing and security monitoring in file storage

Auditing is a crucial part of data protection.

There are many aspects to review when talking about auditing and monitoring in file storage, including the following:

- Users' activity logs
- File storage admin activity

Follow these best practices for auditing and monitoring in file storage:

- Although admin activity is automatically audited by the CSPs, its log history is limited. As best practice, send all audit and monitoring logs to a central storage for further analysis and incident response and to comply with various regulations.
- Implement real-time monitoring to detect and respond to issues as they occur.
- Set up automated alerts for specific events or anomalies in your logs.
- Regularly perform compliance audits to ensure adherence to relevant regulations and standards.
- Define and implement log retention policies to balance compliance requirements and storage costs.
- Regularly verify the integrity of your logs using checksums or digital signatures.
- Monitor user activities, especially those with elevated privileges, to detect any unauthorized or suspicious actions.
- Integrate your logging and monitoring systems with a SIEM solution.
- Add a threat detection solution to review the logs and generate prioritized follow-up actions.
- Use machine learning or other advanced techniques to detect anomalies in function behavior or access patterns.

Amazon EFS

As with any other managed service, AWS allows you to enable logging and auditing using two built-in services:

- **Amazon CloudWatch:** This is a service that allows you to log access activities and raise the alarm according to predefined activities (such as excessive delete actions).
- **AWS CloudTrail:** This is a service that allows you to monitor API activities (essentially, any action performed on Amazon EFS).

Use the following best practices for auditing and monitoring in Amazon EFS:

- Thorough documentation on auditing and monitoring in Amazon EFS can be found in the following link:

`https://docs.aws.amazon.com/efs/latest/ug/monitoring\_overview.html`

- AWS CloudTrail is enabled by default for Amazon EFS. To keep EFS API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here:

`https://docs.aws.amazon.com/efs/latest/ug/logging-using-cloudtrail.html`

- Use Amazon CloudWatch alarms to generate notifications about abnormal activities (such as excessive upload or download requests, excessive object delete requests, etc.), as documented here:

`https://docs.aws.amazon.com/efs/latest/ug/creating\_alarms.html`

Amazon FSx for Windows File Server

As with any other managed service, AWS allows you to enable logging and auditing using two built-in services:

- **Amazon CloudWatch:** This is a service that allows you to log access activities and raise the alarm according to predefined activities (such as excessive delete actions).
- **AWS CloudTrail:** This is a service that allows you to monitor API activities (essentially, any action performed on Amazon FSx for Windows File Server).

Read thorough documentation on auditing and monitoring in Amazon FSx for Windows File Server, which can be found at the following link:

`https://docs.aws.amazon.com/fsx/latest/WindowsGuide/monitoring\_overview.html`

Use the following best practices for auditing and monitoring in Amazon FSx for Windows File Server:

- AWS CloudTrail is enabled by default for Amazon FSx for Windows File Server. To keep FSx for Windows File Server API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here:

<https://docs.aws.amazon.com/fsx/latest/WindowsGuide/logging-using-cloudtrail.html>

- Use Amazon CloudWatch alarms to generate notifications about abnormal activities (such as excessive upload or download requests, excessive object delete requests, etc.), as documented here:

https://docs.aws.amazon.com/fsx/latest/WindowsGuide/creating_alarms.html

Azure Files

Auditing is a crucial part of data protection.

Azure allows you to monitor Azure Files using the following services:

- **Azure Monitor:** This logs access and audit events from Azure Files
- **Microsoft Defender for Storage:** This service (part of Microsoft Defender for Cloud), allows you to detect potential threats in Azure Files

Read thorough documentation on monitoring in Azure Files, which can be found at the following link:

<https://learn.microsoft.com/en-us/azure/storage/files/storage-files-monitoring>

Use the following best practices for auditing and monitoring in Azure Files:

- Enable diagnostic settings for Azure Files, and send all logs to a Log Analytics workspace, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/essentials/create-diagnostic-settings>

- Enable log alerts using the Azure Monitor service to track access to Azure Files and raise alerts (such as multiple failed access attempts to Azure Files in a short period), as explained here:

<https://learn.microsoft.com/en-us/azure/storage/files/files-monitoring-alerts>

- Enable Microsoft Defender for Storage to receive security alerts about potential threats in Azure Files, as explained here:

<https://learn.microsoft.com/en-us/azure/defender-for-cloud/tutorial-enable-storage-plan>

- Use Microsoft Purview to scan Azure Files and search for sensitive data (such as PII, financial data, credentials, etc.), as explained here:

<https://learn.microsoft.com/en-us/purview/register-scan-azure-files-storage-source>

Google Filestore

As with any other managed service, Google allows you to enable logging and auditing using **Google Cloud Audit Logs** – a service that allows you to audit Cloud Filestore activities.

For auditing and monitoring in Google Filestore, use the following best practices:

- Use **Google Cloud Monitoring** to monitor the activity of Google Filestore, as explained here:

<https://cloud.google.com/filestore/docs/monitoring-instances>

- For sensitive/critical environments, enable data access audit logs (actions such as read configuration, create/modify/read user-provided resource data, etc.), as explained here:

<https://cloud.google.com/filestore/docs/audit-logging>

Note

Data access audit logs increase storage costs. Review the business requirements to enable them.

In this section, we learned how to secure the file storage service based on the AWS, Azure, and GCP infrastructure. This included IAM, data protection, logging and auditing. In the next section, we will review the CSI driver and how to secure it.

Securing the Container Storage Interface

A CSI is a standard driver for connecting container orchestration systems such as Kubernetes to block and file storage from various cloud providers.

A CSI driver can be used as a persistent storage and as a mount point for storing secrets (as explained in *Chapter 11, Applying Encryption in Cloud Services*).

General best practices for securing a CSI driver

Regardless of the cloud provider you are using, when using CSI driver as a storage solution for Kubernetes, you should follow these best practices:

- A CSI driver is simply a mount point to a persistent store. Follow the best practices mentioned in the previous sections to secure block, file, or object storage that uses a CSI driver.
- Ensure that data is encrypted in transit to protect it from interception during transfer.

- Enforce encryption at rest on all CSI-based storage volumes during creation.
- Always use the latest CSI driver version for your chosen storage type.
- Regularly update and patch your CSI drivers and Kubernetes clusters.
- Implement robust access control mechanisms, such as IAM policies and RBAC.
- Enable detailed monitoring and logging to track access and changes to CSI-based storage.
- Implement automated backup solutions and test your recovery procedures.
- Use network security measures such as VPC configurations, security groups, and firewalls.
- Regularly perform compliance audits to ensure adherence to relevant regulations and standards.

In the following diagram, we can see how a Pod in a Kubernetes cluster is using a CSI driver to connect to NFS storage:

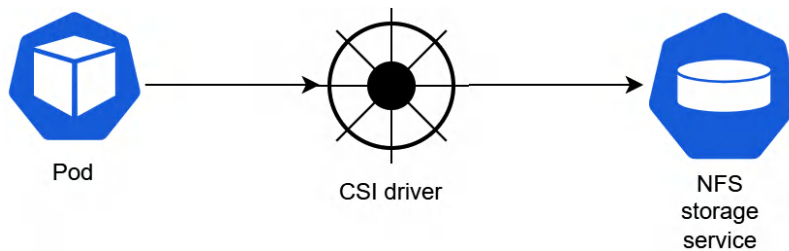


Figure 5.4 – Pod connecting to NFS storage using a CSI driver

Next, we are going to examine what the best practices are for securing CSI-based storage from AWS, Azure, and GCP.

For more information, please refer to the following resources:

- *Kubernetes Container Storage Interface (CSI) Documentation:*
<https://kubernetes-csi.github.io/docs/introduction.html>
- *Kubernetes Secrets Store CSI Driver:*
<https://secrets-store-csi-driver.sigs.k8s.io/introduction>

Securing Amazon EKS and the CSI driver

Amazon EKS has a CSI driver for the following storage types:

- **Block storage:** Amazon EBS
- **Managed NFS:** Amazon EFS
- **Parallel filesystem (for HPC workloads):** Amazon FSx for Lustre

Use the following best practices for securing a CSI driver in Amazon EKS:

- Deploy a CSI driver (for EBS, EFS, or FSx for Lustre), as explained at the following links:
 - <https://github.com/kubernetes-sigs/aws-ebs-csi-driver>
 - <https://github.com/kubernetes-sigs/aws-efs-csi-driver>
 - <https://github.com/kubernetes-sigs/aws-fsx-csi-driver>
- For mounting an EBS volume, use the Amazon EBS CSI plugin and create an IAM role to allow access to the EBS volume, as explained here:
<https://docs.aws.amazon.com/eks/latest/userguide/csi-iam-role.html>
- Create an AWS IAM **OpenID Connect (OIDC)** provider to grant access from the EKS cluster to the CSI storage, as explained here:
<https://docs.aws.amazon.com/eks/latest/userguide/enable-iam-roles-for-service-accounts.html>
- Encrypt data at rest using AWS-managed keys (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:
<https://aws.github.io/aws-eks-best-practices/security/docs/data/>
- For sensitive environments (which contain PII, credit card details, healthcare data, and more), encrypt data at rest using CMKs and rotate them periodically (as explained in *Chapter 11*), as documented here:
<https://aws.github.io/aws-eks-best-practices/security/docs/data/#rotate-your-cmks-periodically>
- To use AWS Secrets Manager in Amazon EKS using a CSI driver, follow these instructions:
https://docs.aws.amazon.com/secretsmanager/latest/userguide/integrating_csi_driver.html

Securing AKS and the CSI driver

AKS has a CSI driver for the following storage types:

- **Block storage:** Azure Disk Storage
- **Managed SMB and NFS:** Azure Files
- **Object storage:** Azure Blob Storage

Use the following best practices for securing a CSI driver in AKS:

- Deploy a CSI driver (for Azure Disk Storage, Azure Files, or Azure Blob Storage), as explained at the following links:

- <https://learn.microsoft.com/en-us/azure/aks/csi-storage-drivers>
- <https://learn.microsoft.com/en-us/azure/aks/azure-disk-csi>
- <https://learn.microsoft.com/en-us/azure/aks/azure-files-csi>
- <https://learn.microsoft.com/en-us/azure/aks/azure-blob-csi>

- To secure access from your AKS cluster and the Azure Files CSI driver, use an Azure private endpoint, which prevents sending network traffic outside your VNet through a secure channel, as explained here:

<https://learn.microsoft.com/en-us/azure/aks/azure-files-csi#azure-files-csi-driver-new-features>

- For sensitive environments (which contain PII, credit card details, healthcare data, and more), encrypt data at rest using CMKs stored inside Azure Key Vault (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:

<https://learn.microsoft.com/en-us/azure/aks/azure-disk-customer-managed-keys>

- To use an Azure Key Vault provider in AKS for a Secrets Store CSI driver, follow the instructions here:

<https://learn.microsoft.com/en-us/azure/aks/csi-secrets-store-driver>

Securing GKE and the CSI driver

GKE has a CSI driver for the following storage types:

- **Block storage:** GCE Persistent Disk
- **Object storage:** Google Cloud Storage
- **Managed NFS:** Google Filestore

Here is a list of best practices to follow:

- Deploy a CSI driver (for Google Persistent Disk, Google Cloud Storage, or Google Filestore), as explained at the following links:
 - <https://cloud.google.com/kubernetes-engine/docs/how-to/persistent-volumes/gce-pd-csi-driver>
 - <https://cloud.google.com/kubernetes-engine/docs/how-to/persistent-volumes/cloud-storage-fuse-csi-driver>
 - <https://cloud.google.com/kubernetes-engine/docs/how-to/persistent-volumes/filestore-csi-driver>
- For sensitive environments (which contain PII, credit card details, healthcare data, and more), encrypt data at rest on all Google Persistent Disk or Google Filestore volumes during creation using CMEKs (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:

<https://cloud.google.com/kubernetes-engine/docs/how-to/using-cmek>

- Use **Cloud IAM** roles to restrict access from your GKE cluster to Google Cloud Storage, as documented here:

<https://cloud.google.com/storage/docs/access-control/iam-roles>

- To use **Google Secret Manager** in GKE using a CSI driver, follow the instructions here:

<https://cloud.google.com/secret-manager/docs/secret-manager-managed-csi-component>

In this section, we learned how to secure CSI-based storage based on the AWS, Azure, and GCP infrastructures – from deployment and encryption at rest to secret management.

Summary

In this chapter, we focused on the various storage services in AWS, Azure, and GCP. We covered object storage, block storage, file storage, and, finally, container storage.

In each section, we learned how to manage identity management (for authentication and authorization), how to control network access (from access controls to network encryption), and how to configure auditing and logging.

In the next chapter, we will review the various network services in the cloud (including virtual networking, security groups and ACLs, DNS, and VPN).

6

Securing Networking Services – Part 1

In previous chapters, we covered cloud fundamental services (compute and storage).

Now, it is time to talk about securing *networking services*. We will discuss virtual networking services, **Domain Name System (DNS)** services, **virtual private networks (VPNs)**, and **Zero Trust** services.

This chapter will cover various networking services and provide you with the best practices on how to securely connect to and use each of them.

In this chapter, we will cover the following topics:

- Securing virtual networking
- Securing DNS services
- Securing VPN services
- Securing Zero Trust services

Securing virtual networking

Each cloud provider has its own implementation of virtual networking.

According to the **shared responsibility model**, responsibility over the network is split between the cloud provider and the customer. The physical network layer (that is, access between the physical network equipment and physical host servers and storage inside the cloud provider's data centers) is the cloud provider's responsibility.

Virtual networking (such as **Amazon VPC**, **Azure VNet**, or **Google Cloud Platform VPC (GCP VPC)**), is a network layer that is the responsibility of the customers. This layer enables access between virtual servers, managed storage services, managed databases, and more.

Traditional on-premises networking deals with the physical connections between devices in a system: for example, concepts such as **virtual local area networks (VLANs)** or subnetting to split a network (with the devices connected to a network) and create network security barriers.

In the cloud, a network is software-based (this is also known as **software-defined networking (SDN)**). SDN allows us to achieve a similar concept of the traditional network segmentation as we used to have in the on-premises data centers but in cloud environments. One of the capabilities of SDN is called micro-segmentation. Micro-segmentation allows you to configure *allow* or *deny* access control rules between two instances even if they are located on the same subnet. You will also be able to control access to a resource such as an API.

A virtual network is a network area inside your cloud environment where most of the common cloud resources reside, such as virtual servers, managed databases, and so on. These resources are split into multiple network segments called *subnets*.

There can be one or more virtual networks in each customer's cloud environment, but at the end of the day, the basic idea is the same. The following are some important points regarding virtual networks:

- Access to subnets is controlled by access controls, such as Layer 4 firewalls
- Subnets can be *private* (no direct access from the internet) or *public* (access from the internet is allowed)
- Virtual networks can be connected via peer connections
- Access to virtual networks should be restricted to authorized traffic (from well-known or customer-legitimate IP ranges)

Here are some recommendations to keep resources within a virtual network private:

- Use an access control list (the equivalent of a Layer 4 firewall)
- Use a WAF to protect publicly exposed web resources from Layer 7 attacks
- Use a public-facing load balancer for high availability and to block direct access to resources that serve public customers (such as web servers or API endpoints)
- Use a **content delivery network (CDN)** for caching public content instead of exposing storage services to the internet

The following are some of the consequences of failure to protect resources inside a virtual network:

- Unauthorized network access
- Data breach
- Data leakage
- Resource inaccessibility (due to denial-of-service attacks)

Network access controls work in one of the following ways:

- **Stateless:** The receiver (for example, a firewall) does not maintain the previous connection's state (of the client). Suitable for scenarios that require high network speed. It requires the configuration of both inbound and outbound network rules.
- **Stateful:** The receiver (for example, a firewall) maintains the entire connection's state (of the client). Suitable for scenarios where we need to maintain information about connections between a client and a server (for example, to be able to block suspicious traffic). It requires the configuration of only inbound network rules.

Since each cloud provider has its own network access control mechanism, the following table compares the differences between access control mechanisms:

	AWS network ACLs	AWS security groups	Azure network security groups (NSGs)	GCP VPC firewall rules
Stateless/Stateful	Stateless	Stateful	Stateful	Stateful
Location	Subnet level	Instance level	VM, subnet, tagging level	Instance level

Table 6.1 – Access control mechanisms

Amazon Virtual Private Cloud

Amazon Virtual Private Cloud (Amazon VPC) is the Amazon virtual networking service.

Read thorough documentation on security in Amazon VPC, which can be found at the following link: <https://docs.aws.amazon.com/vpc/latest/userguide/security.html>

In the following diagram, we can see a sample Amazon VPC. Customers originate from the public internet and pass through an internet gateway into the VPC. Traffic is sent to a public subnet containing a front web server, and from the web server, the traffic goes through a security group to a private subnet containing a database server.

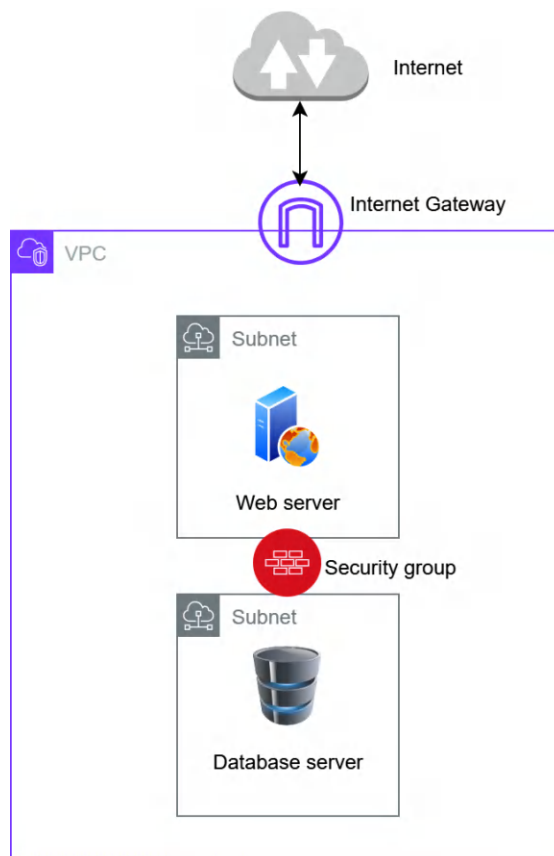


Figure 6.1 – A sample Amazon VPC architecture

Azure VNet

Azure has a concept of a logical container called a **resource group**.

Azure Virtual Network (Azure VNet) is the Azure virtual networking service deployed inside a resource group.

Thorough documentation on security in Azure VNet can be found at the following links:

- <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/virtual-network-security-baseline>
- <https://learn.microsoft.com/en-us/azure/virtual-network/concepts-and-best-practices>

In the following diagram, we can see a sample Azure VNet. Customers originate from the public internet and pass through an NSG into the VNet. Traffic is sent to a public subnet containing Azure App Service deployed with a public-facing application.

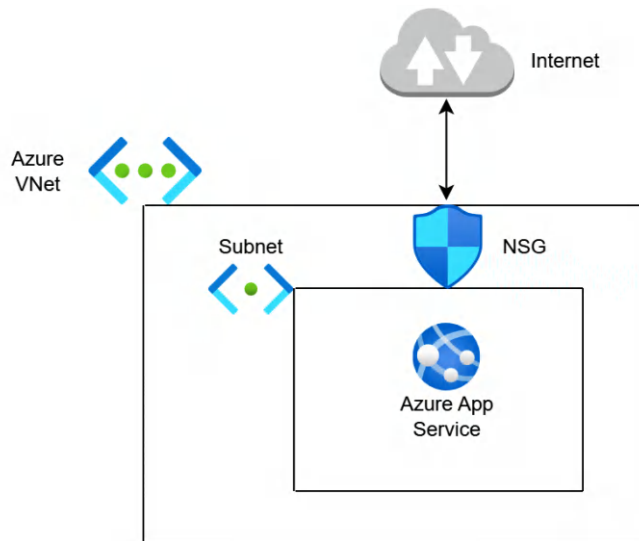


Figure 6.2 – A sample Azure VNet architecture

Google Cloud VPC

Google Cloud VPC is the Google virtual networking service.

Thorough documentation about Google Cloud VPC can be found at the following link:

<https://cloud.google.com/vpc/docs/overview>

In the following diagram, we can see a sample Google Cloud VPC. Customers originate from the public internet and pass through a VPC firewall rule. Traffic is sent to a public subnet containing an application deployed on top of a GKE cluster, and from the GKE cluster, the traffic goes through another firewall rule to a private subnet containing a database server.

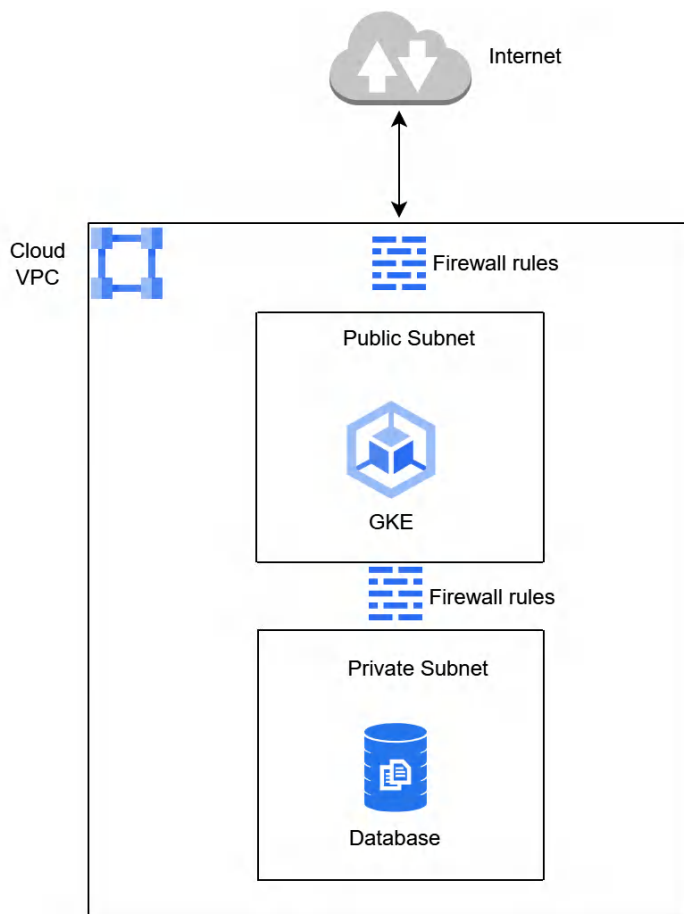


Figure 6.3 – A sample Google Cloud VPC architecture

General best practices for securing virtual networks

To protect virtual networks (both inbound and outbound), use the following best practices:

- Create subnets according to the resource function (for example, public subnets for web servers, private subnets for database servers, and so on). This will allow network segmentation and will limit the blast radius in case of unauthorized access to cloud resources.
- For remote access protocols (SSH/RDP), limit the source IP address (or **Classless Inter-Domain Routing (CIDR)**) to well-known sources (preferred from internal IPs).
- For file sharing protocols (CIFS/SMB/FTP), limit the source IP address (or CIDR) to well-known sources (preferred from internal IPs).

Note

Today, each of the hyperscale cloud providers offers secure methods to remotely connect and manage resources privately (i.e., not passing over the public internet). Services such as AWS PrivateLink, Azure PrivateLink, and Google Private Service Connect assign a private IP address to a managed service that is deployed outside a customer's virtual network (such as object storage, managed database, etc.) from a customer's private IP range and enforce all access internally from the customer's subnets.

- Use network access control lists to control access between public resources (such as load balancers or public-facing web servers) and private resources (such as databases) and limit access to the minimum required ports/protocols.
- To allow access to the internet from private subnet resources, use a NAT gateway, which will allow outbound access and hide internal resources' IP addresses.
- To allow secure access to a cloud virtual environment from the on-premises data center over the public internet, configure a site-to-site VPN (as explained later in this chapter – in the *Securing VPN services* section).
- Set names and descriptions for the network access control lists to allow a better understanding of any ACL's purpose for anyone who deploys and maintains the cloud environments.
- Use *tagging* (also known as *labeling*) for network access control lists to allow a better understanding of which ACL belongs to which resource.

Next, we are going to see what the best practices are for securing virtual networking services from Amazon, Azure, and GCP.

Amazon VPC

AWS supports the following mechanisms to protect access to resources inside a VPC:

- **Network ACL:** A stateless mechanism for protecting access to/from resources at a subnet and an entire Amazon VPC level. By default, an Amazon VPC has a default NACL, which allows inbound/outbound traffic. You need to configure both inbound and outbound rules. Network ACLs support both *allow* and *deny* rules.
- **Security group:** A stateful mechanism for protecting access to resources at an Elastic Network Interface (ENI) level. You need to configure inbound rules. Security groups support only *allow* rules.

Note

Network access to a resource is granted by a combination of the network ACLs on a subnet level with an aggregation of all the security groups. This is effective on a resource such as a virtual machine (in case several security groups allow different access to a resource such as a virtual machine).

Use the following best practices for securing network access in Amazon VPC:

- When creating custom network ACLs, create a final deny rule for both inbound and outbound traffic for better protection from unauthorized access, as explained here:

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html>

- Use security groups to control access between public resources (such as load balancers or public-facing web servers) and private resources (such as databases) and limit access to the minimum required ports/protocols, as explained here:

<https://docs.aws.amazon.com/vpc/latest/userguide/security-group-rules.html>

- For large-scale environments with multiple AWS accounts, use **AWS Firewall Manager** to centrally create and enforce VPC security groups for configuring who has access to resources inside the VPC, as explained here:

- <https://docs.aws.amazon.com/waf/latest/developerguide/getting-started-fms-security-group.html>

- <https://docs.aws.amazon.com/vpc/latest/userguide/network-firewall.html>

- For secure access from resources inside your VPC to **AWS Managed Services** (such as AWS S3, Amazon RDS, and more) and to keep traffic inside the AWS backbone, use **AWS PrivateLink** and configure your VPC security groups to allow traffic from your VPC to AWS managed services, as explained here:

<https://docs.aws.amazon.com/vpc/latest/privatelink/create-interface-endpoint.html>

- Allow outbound access from internal resources inside private subnets to destinations on the internet:

- **Based on the IPv4 protocol:** Use NAT gateways or any self-hosted NAT proxy, as explained here: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

- **Based on the IPv6 protocol:** Use an *egress-only* internet gateway, as explained here: <https://docs.aws.amazon.com/vpc/latest/userguide/egress-only-internet-gateway.html>

- For a simple connection between two VPCs, configure a VPC peering connection, as explained here:

<https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html>

- In case you need to configure connectivity between multiple VPCs, including connectivity to VPN from on-premises, use **AWS Transit Gateway**, as documented here:

<https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html>

Azure VNet

Azure supports the following mechanisms to protect access to resources inside a VNet:

- **NSG**: A stateful mechanism for protecting access to resources at a Network Interface (NIC) or a subnet level or based on service tagging. You need to configure both inbound and outbound rules for VNet NSGs.
- **Application security group (ASG)**: A stateless mechanism for protecting access to resources at a network interface level. An ASG is an object reference within an NSG. You need to configure both inbound and outbound rules for VNet ASGs.

Use the following best practices for securing network access in Azure VNet:

- Use NSGs to control access between public resources (such as load balancers or public-facing web servers) and private resources (such as databases) and limit access to the minimum required ports/protocols, as explained here:
 - <https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview>
 - <https://learn.microsoft.com/en-us/azure/virtual-network/tutorial-restrict-network-access-to-resources>
- Use ASGs to group resources and define access rules at the application layer (for example, rules for allowing inbound HTTP access), as explained here:

<https://learn.microsoft.com/en-us/azure/virtual-network/application-security-groups>

- To overcome the default unrestricted outbound traffic, create a **user-defined route (UDR)** and forward all outbound traffic (to destination 0.0.0.0/0) through an Azure firewall, as explained here:

<https://learn.microsoft.com/en-us/azure/firewall/tutorial-firewall-deploy-portal#create-a-default-route>

- Use Azure network service tags to configure rules for predefined services (for example, **Azure Load Balancer**), instead of using IP addresses, as explained here:

<https://learn.microsoft.com/en-us/azure/virtual-network/service-tags-overview>

Note

Azure network service tags are not considered a security boundary and should be used as a routing mechanism within NSGs.

- For large-scale environments with multiple Azure subscriptions, use Azure Firewall to centrally create, enforce, and log network policies across multiple subscriptions, as explained here:

<https://learn.microsoft.com/en-us/azure/firewall/overview>

- For sensitive environments (for example, ones that contain PII, credit card details, healthcare data, and more), enable virtual network encryption (i.e., encrypt/decrypt traffic between Azure VMs by creating a DTLS tunnel), as explained here:

<https://learn.microsoft.com/en-us/azure/virtual-network/how-to-create-encryption-portal>

- For secure access from resources inside your VNet to Azure-managed services (such as Azure Blob storage, Azure SQL, and more) and to keep traffic inside the Azure backbone, use **Azure Private Link** and configure your VNet NSGs to allow traffic from your VNet to Azure managed services, as explained here:

<https://learn.microsoft.com/en-us/azure/private-link/private-link-overview>

- To allow outbound access from internal resources inside private subnets to destinations on the internet (based on the IPv4 protocol), use **Azure VNet NAT** (or use NAT Gateway), as explained here:

<https://learn.microsoft.com/en-us/azure/nat-gateway/nat-overview>

- For scenarios where VMs behind Azure Load Balancer require access to destinations on the internet, use **source network address translation (SNAT)**, as explained here:

<https://learn.microsoft.com/en-us/azure/load-balancer/load-balancer-outbound-connections>

- For setting a connection between two VNets, configure a VNet peering connection, as explained here:

<https://learn.microsoft.com/en-us/azure/virtual-network/virtual-network-peering-overview>

Google Cloud VPC

To protect resources inside a VPC, Google supports **VPC firewall rules** – a stateful mechanism for protecting access to resources. You need to configure either inbound or outbound rules, and for each rule, you can configure either an action of *allow* or *deny*.

Use the following best practices for securing network access in Google Cloud VPC:

- Use VPC firewall rules to control access between public resources (such as load balancers or public-facing web servers) and private resources (such as databases) and limit access to the minimum required ports/protocols, as explained here:

<https://cloud.google.com/firewall/docs/using-firewalls>

- Use network tags to configure rules to groups of resources (such as a group of compute engine instances) instead of using IP addresses for easier management of network access rules over resources, as explained here:

- <https://cloud.google.com/firewall/docs/tags-firewalls-overview>

- <https://cloud.google.com/vpc/docs/add-remove-network-tags>

- For large-scale environments with multiple GCP projects, use Google **Cloud Next Generation Firewall (Cloud NGFW)** to centrally implement firewall policies and micro-segmentation across VPCs, as explained here:

<https://cloud.google.com/firewall/docs/about-firewalls>

- To restrict access to Google services to trusted sources and reduce the risk of data exfiltration, use **VPC Service Controls**, as explained here:

<https://cloud.google.com/vpc-service-controls/docs/create-service-perimeters>

- To allow access to managed services from within your VPC, configure **Private Service Connect (PSC)**, as explained here:

<https://cloud.google.com/vpc/docs/private-service-connect>

- To allow access to Google APIs and services from your VPC or private IPs on-premises, use Private Google Access with VPC Service Controls, as explained here:

<https://cloud.google.com/vpc-service-controls/docs/private-connectivity>

- To allow outbound access from internal resources inside private subnets to destinations on the internet (based on the IPv4 protocol), use **Google Cloud NAT**, as explained here:

<https://cloud.google.com/nat/docs/set-up-manage-network-address-translation>

- For setting a connection between two VPCs, configure a VPC network peering connection, as explained here:

<https://cloud.google.com/vpc/docs/using-vpc-peering>

Best practices for monitoring virtual networks

There are many aspects to review when talking about monitoring virtual networks, including the following, among others:

- Audit API activity, such as a privileged action for making changes to a network configuration
- Collecting network-related metrics, such as the number of resources inside a virtual network
- Collecting network traffic activity using flow logs, such as the number of incoming or outgoing bytes, the number of accepted or denied network requests, and so on
- Packet capturing of network activity, such as a network capture in on-premises networks

Follow the best practices here for monitoring virtual networks:

- Enable flow logs to collect and further analyze allowed and denied traffic activity

Note

For large-scale production environments, enable flow logs only for short periods for troubleshooting purposes only due to high storage costs and large amounts of data generated by flow logs.

- If you need to troubleshoot network issues by capturing network traffic without interrupting production systems, enable packet capture to copy live network traffic from a network interface of a VM or a network load balancer.

Note

Some packet capture services offered by the CSPs have a performance impact on VMs. Consider the business requirements before enabling packet capture and enable it for short periods for troubleshooting purposes only.

Amazon VPC

AWS allows you to monitor Amazon VPC using the following built-in services:

- **AWS CloudTrail:** Audit all API calls from Amazon VPC, such as creating a new subnet, creating a new VPC, and so on.
- **Amazon CloudWatch:** This is a service that allows you to monitor your VPC components, such as ingress/egress traffic volumes.
- **VPC Flow Logs:** This is an Amazon VPC feature that allows you to log network activity inside your VPC, for example, traffic metadata such as source, destination, port, timestamp, and size, and *allow* and *deny* network events. This is useful for both troubleshooting and investigating security-related events.

- **Traffic Mirroring:** An Amazon VPC feature that allows you to copy (i.e., mirror) real-time network traffic from a VM network interface, network load balancers, or global load balancers, and capture the traffic to a network interface of another instance.

Read thorough documentation on monitoring in Amazon VPC, which can be found at the following link:

<https://docs.aws.amazon.com/vpc/latest/userguide/monitoring.html>

Use the following best practices for monitoring Amazon VPC:

- Enable CloudWatch Logs to monitor your VPC components' activity and the traffic between your VPC resources and the VPC endpoint (AWS Managed Services), as explained here:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/cloudwatch-logs-and-interface-VPC.html>

- Use AWS CloudTrail to monitor VPC configuration, as explained in the documentation here:

- <https://docs.aws.amazon.com/AWSEC2/latest/APIReference/using-cloudtrail.html>
- <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-and-interface-VPC.html>

- Enable VPC Flow Logs to log and further analyze allowed and denied traffic activity, as documented here:

<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

- If you need to troubleshoot network issues by capturing network traffic without interrupting production systems, use Traffic Mirroring in Amazon VPC to copy live network traffic from a network interface of an Amazon EC2 instance or a network load balancer to an out-of-band security appliance, as documented here:

<https://docs.aws.amazon.com/vpc/latest/mirroring/traffic-mirroring-how-it-works.html>

- To identify unintended network traffic to your VPC, use Network Access Analyzer, as explained here:

<https://docs.aws.amazon.com/vpc/latest/network-access-analyzer/how-network-access-analyzer-works.html>

- Use Amazon GuardDuty to detect anomalous network behavior, such as interaction with **command and control (C&C)** networks, malicious IP addresses, and more, from VPC Flow Logs, as explained here:

https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_data-sources.html#guardduty_vpc

Azure VNet

Azure allows you to monitor Azure VNet using the following built-in services:

- **Azure Monitor:** Monitors network activity metrics and activity logs such as changes to network configuration
- **Azure Network Watcher:** Monitors IaaS activity such as Azure VMs, VNets, application gateways, load balancers, and so on, collects NSG flow logs, and captures network packet activity

Use the following best practices for monitoring Azure VNet:

- Enable diagnostic settings for Azure VNet, and send all logs to a Log Analytics workspace, as explained here:

`https://learn.microsoft.com/en-us/azure/virtual-network/monitor-virtual-network`

- Enable **virtual network flow logs** to log and further analyze allowed and denied traffic activity, as documented here:

`https://learn.microsoft.com/en-us/azure/network-watcher/vnet-flow-logs-overview`

- If you need to troubleshoot network issues by capturing network traffic, use the Azure Network Watcher packet capture extension to copy live network traffic from a virtual machine to an Azure storage account, as documented here:

`https://learn.microsoft.com/en-us/azure/network-watcher/packet-capture-vm-portal`

- Use Azure Traffic Analytics to locate security threats such as open ports, application outbound internet traffic, and so on, as documented here:

`https://learn.microsoft.com/en-us/azure/network-watcher/traffic-analytics`

- To get insights into your VNet and receive hardening recommendations of resources inside your account, enable Microsoft Defender for Cloud, as documented here:

`https://learn.microsoft.com/en-us/azure/defender-for-cloud/protect-network-resources`

- To audit changes in a large-scale environment, use Azure Virtual Network Manager, as documented here:

`https://learn.microsoft.com/en-us/azure/virtual-network-manager/concept-event-logs`

Google Cloud VPC

Google allows you to monitor Google Cloud VPC using the following built-in services:

- **Cloud Logging:** This is a service that allows you to collect logs from Google Cloud services (such as firewall rules logging)
- **Cloud Audit Logs:** This is a service that allows you to record audit logs for changes in your VPC components
- **VPC Flow Logs:** This is an Amazon VPS feature that allows you to log network activity inside your VPC (for example, *allow* and *deny* network events)
- **Packet Mirroring:** This is a Google Cloud VPC feature that allows you to copy (i.e., mirror) real-time network traffic from a VM instance and capture the traffic to another instance – part of an instance group

Use the following best practices for monitoring Google Cloud VPC:

- Use Google Cloud Logging to enable Firewall Rules Logging and analyze the effect of VPC firewall rules (i.e., who has access to which resource), as explained here:

<https://cloud.google.com/firewall/docs/using-firewall-rules-logging>

- For sensitive/critical environments, enable data access audit logs in Google Cloud audit logs (actions such as firewall get/list, networks get/list, etc.), as explained here:

<https://cloud.google.com/vpc/docs/audit-logging>

Note

Data access audit logs increase storage costs. Review the business requirements to enable them.

- Enable VPC Flow Logs to log and further analyze allowed and denied traffic activity, as explained here:

<https://cloud.google.com/vpc/docs/using-flow-logs>

- If you need to troubleshoot network issues by capturing network traffic, use Packet Mirroring to copy live network traffic from a compute engine VM instance to an instance group behind an internal load balancer, as explained here:

<https://cloud.google.com/vpc/docs/using-packet-mirroring>

- Use **Google Security Command Center** to review VPC logs and detect misconfigurations and vulnerabilities (such as resources with a public IP, directly exposed from the internet), as documented here:

<https://cloud.google.com/security-command-center/docs/concepts-vulnerabilities-findings>

In this section, we learned how to secure virtual networks based on the AWS, Azure, and GCP infrastructure. This included network access, audit, monitoring, and capturing network traffic.

Next, we are going to review how to secure DNS services.

Securing DNS services

Each cloud provider has its own implementation of managed DNS services – these include services for translating hostnames into IP addresses, different types of DNS records services (such as **Alias**, **CNAME**, and more), resolving hostnames to load balance IP, and more.

Common security risks related to DNS services are as follows:

- DNS spoofing, where an attacker redirects clients to a malicious IP address instead of resolving the correct DNS record
- **Distributed denial-of-service (DDoS)** attack, where an attack consumes all bandwidth to the DNS server, making it inaccessible to customers' requests
- DNS tunneling attack, where an attacker encodes data within DNS queries to exfiltrate information from the organization or establish a C&C channel to allow external access to the organization

Regardless of the cloud provider you are using for DNS, you should follow the best practices here:

- Enforce authorization for any request to make changes in a DNS configuration (such as creating a new zone, updating DNS records, etc.), and follow the principle of least privilege
- When supported, enable **Domain Name System Security Extensions (DNSSEC)** signing on any public hosted zone to protect against DNS spoofing attacks
- Use a DDoS protection service to mitigate volumetric attacks against public DNS services
- Use private hosted zones to manage DNS records for internal resources such as resources within your virtual network or on-premises resources
- Remove unassigned DNS records from your hosted zones' records of resources such as IP addresses that are connected to a resource that was removed
- Enable auditing on the DNS service to allow further investigation and analysis of DNS activity and suspicious DNS traffic to malicious destinations
- Add a threat detection solution to review the logs and generate prioritized follow-up actions

Securing Amazon Route 53

Amazon Route 53 is the Amazon-managed DNS service, supporting the following capabilities:

- Hosting DNS zones for managing DNS records (for both public and private resources)
- DNS resolver (for both public and private resources)
- DNS registrar – manages the reservations of internet domain names
- Supports DNSSEC signing for all public hosted zones

Read thorough documentation on security in Amazon Route 53, which can be found at the link here: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/security.html>

Use the following best practices for securing Amazon Route 53:

- Use IAM policies for Route 53 to control who has access to make changes to DNS zones and records, following the principle of least privilege, as explained here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/access-control-managing-permissions.html>

- Enable DNSSEC signing for public DNS zones to protect against DNS spoofing and cache poisoning attacks, as explained here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/domain-configure-dnssec.html>

- When using DNSSEC for signing public DNS zones, use customer-managed keys, as documented here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-configuring-dnssec-cmk-requirements.html>

- Use private hosted zones to manage DNS records for internal resources (such as resources located inside private subnets), as explained here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/hosted-zone-private-creating.html>

- Enable privacy protection for any domain you purchased using Route 53 to protect the privacy of domain owners' contact information, as explained here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/domain-privacy-protection.html>

- Enable the Route 53 domain transfer lock to prevent your domains from being transferred to another registrar, as explained here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/domain-lock.html>

- Use Route 53 Resolver DNS Firewall to block DNS-level threats originating from your VPC and prevent data exfiltration, as explained here:

- <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-dns-firewall-getting-started.html>

- <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-dns-firewall-vpc-protections.html>

- Enable public DNS query logging to be able to analyze which public DNS queries were submitted to Route 53 about your domains, as explained here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/query-logs.html>

Note

DNS query logs increase storage costs. Review the business requirements to enable them.

- Enable **Resolver query logging** to be able to analyze information such as the Route 53 Resolver DNS Firewall block rules, as explained here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-query-logs.html>

- AWS CloudTrail is enabled by default for Amazon Route 53. To keep Route 53 API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/logging-using-cloudtrail.html>

- Use Amazon CloudWatch to monitor public hosted zone activity and collect metrics for further analysis, as explained here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/monitoring-hosted-zones-with-cloudwatch.html>

- Use Amazon CloudWatch to monitor Route 53 Resolver endpoints and collect metrics for further analysis, as explained here:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/monitoring-resolver-with-cloudwatch.html>

- Enable Amazon GuardDuty to analyze DNS logs and raise alerts about suspicious activity, such as C&C activity, crypto mining, and more, as explained here:

https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_data-sources.html#guardduty_dns

Securing Azure DNS

Azure DNS is the Azure-managed DNS service, supporting the following capabilities:

- Hosting DNS zones for managing DNS records (for both public and private resources)
- DNS resolver (for both public and private resources)
- Supports DNSSEC signing for all public hosted zones

Read thorough documentation on security in Azure DNS, which can be found at the link here: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/azure-dns-security-baseline>

Use the following best practices for securing Azure DNS:

- Use **Microsoft Entra RBAC** to control who has access to make changes to DNS zones and records, following the principle of least privilege, as explained here:
 - <https://learn.microsoft.com/en-us/azure/dns/dns-protect-zones-recordsets>
 - <https://learn.microsoft.com/en-us/azure/dns/dns-protect-private-zones-recordsets>
- Use Private DNS to manage DNS records for internal resources (such as resources located inside private subnets), as explained here:

<https://learn.microsoft.com/en-us/azure/dns/private-dns-privatednszone>
- Use Private Resolver to enable DNS queries from on-premise environments, as explained here:

<https://learn.microsoft.com/en-us/azure/dns/private-resolver-hybrid-dns>
- Enable DNSSEC signing for public DNS zones to protect against DNS spoofing and cache poisoning attacks, as explained here: <https://learn.microsoft.com/en-us/azure/dns/dnssec>

- Use Azure DNS metrics (currently only supporting public hosted DNS zones) to monitor Azure DNS activity (such as query volume, record set count, and record set capacity utilization) and generate alerts once a threshold passes a predefined value, as explained here:

<https://learn.microsoft.com/en-us/azure/dns/dns-alerts-metrics>

- Use **Microsoft Defender for DNS** to detect and send alerts about suspicious DNS-related activities, as explained here:

<https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-dns-introduction>

Securing Google Cloud DNS

Google Cloud DNS is the Google-managed DNS service, supporting the following capabilities:

- Hosting DNS zones for managing DNS records (for both public and private resources)
- DNS resolver (for both public and private resources)
- Supports DNSSEC signing for all public hosted zones

Use the following best practices for securing Google Cloud DNS:

- General best practices for Google Cloud DNS can be found at the link here:

<https://cloud.google.com/dns/docs/best-practices>

- Use IAM policies for Google Cloud DNS to control who has access to make changes to DNS zones and records, following the principle of least privilege, as explained here:

<https://cloud.google.com/dns/docs/access-control>

- Enable DNSSEC signing on any public hosted zone to protect against DNS spoofing attacks, as explained here:

<https://cloud.google.com/dns/docs/dnssec-config>

- Use Google Cloud DNS private zones to manage DNS records for internal resources (such as resources located inside private subnets), as explained here:

<https://cloud.google.com/dns/docs/zones#create-private-zone>

- To restrict access to Google Cloud DNS to trusted sources and reduce the risk of data exfiltration, use VPC Service Controls, as mentioned here:

<https://cloud.google.com/dns/docs/vpc-sc-support>

- For sensitive/critical environments, enable data access audit logs (actions such as get/list DNS changes, get/list DNS policies, etc.), as explained here:

<https://cloud.google.com/dns/docs/audit-logging>

Note

Data access audit logs increase storage costs. Review the business requirements to enable them.

- Enable Cloud DNS logs to track queries within your VPC and to resources in your public DNS zones from external entities, as explained here:

<https://cloud.google.com/dns/docs/monitoring>

Note

Cloud DNS logs increase storage costs. Review the business requirements to enable them.

- Use **Google Security Command Center** to detect and send alerts about suspicious DNS-related activities (such as crypto mining and malware), as mentioned here:

<https://cloud.google.com/security-command-center/docs/concepts-security-sources>

In this section, we learned how to secure managed DNS services based on the AWS, Azure, and GCP infrastructure. This included IAM, enabling DNSSEC, auditing, monitoring, and threat detection.

Next, we are going to review how to secure VPN services.

Securing VPN services

Each cloud provider has its own implementation of a managed VPN service. VPNs allow network-based access to private resources over untrusted networks.

There are two types of VPN services:

- **Site-to-Site VPN:** Allows a permanent connection (or secure tunnel) between the on-premises network and the cloud environment (or between different cloud environments). The connection between the clients (from on-premises, for example) to resources in the cloud environment is transparent and there is no additional configuration required.
- **Point-to-Site VPN (client VPN):** Allows ad hoc connection between clients over the public internet and the cloud environment.

In the following diagram, we can see a common Site-to-Site VPN architecture. The on-premises data center connects a customer VPN endpoint, through an IPsec tunnel, to a VPN gateway in the cloud environment.

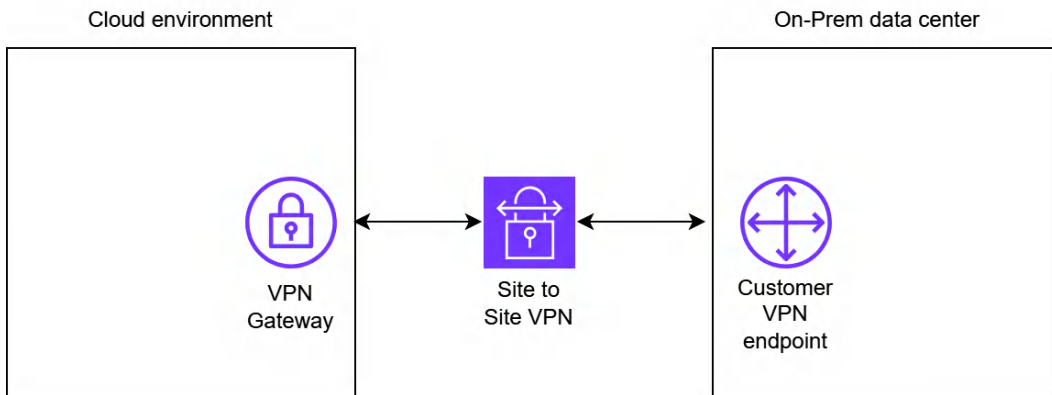


Figure 6.4 – A Site-to-Site VPN architecture

Regardless of the cloud provider you are using for VPN services, you should follow the best practices here:

- Restrict access to resources inside your cloud environment using access control lists at the network layer
- Enforce traffic encryption using the latest supported cipher suites
- For a Site-to-Site VPN, it is preferred to encrypt the VPN tunnel using certificates (for the most secure tunnel encryption)
- For a client VPN, enforce access permissions using an IAM service and follow the principle of least privilege
- For a client VPN, enforce the use of **multi-factor authentication (MFA)**
- Enable auditing to detect anomalous behavior such as a high amount of inbound/outbound traffic from a specific IP or a client accessing multiple resources inside the cloud environment

Next, we will review how Site-to-Site VPN services and client VPN services can allow connectivity to your cloud environment.

Securing AWS Site-to-Site VPN

AWS Site-to-Site VPN is a managed service that allows you to connect your corporate network to the AWS environment in a secure IPsec channel.

AWS Site-to-Site VPN has the following components:

- **Virtual private gateway:** A VPN endpoint on the AWS side, receiving connections from the customer's VPN tunnels and connected to a single VPC
- **Customer gateway (CGW):** A resource in AWS that represents the customer's VPN device (also known as the CGW device) on-premises

Read thorough documentation on security in AWS Site-to-Site VPN can be found at the link here: <https://docs.aws.amazon.com/vpn/latest/s2svpn/security.html>

Use the following best practices for securing AWS Site-to-Site VPN:

- Encrypt traffic over the VPN tunnel using the AES256-GCM-16 algorithm and SHA2-256 (at a minimum) as a hashing algorithm, as documented here:

<https://docs.aws.amazon.com/vpn/latest/s2svpn/your-cgw.html>

- For non-sensitive environments, use pre-shared keys to authenticate to the Site-to-Site VPN tunnel, as explained here:

<https://docs.aws.amazon.com/vpn/latest/s2svpn/vpn-tunnel-authentication-options.html#pre-shared-keys>

- For highly sensitive environments, use a private certificate from the **AWS Certificate Manager (ACM)** private **certificate authority (CA)** service, as explained here:

<https://docs.aws.amazon.com/vpn/latest/s2svpn/vpn-tunnel-authentication-options.html#certificate>

- It is recommended to schedule a maintenance window and rotate the pre-shared keys or the certificate for the AWS Site-to-Site VPN connection once a year, to avoid potential compromise, as explained here:

<https://docs.aws.amazon.com/vpn/latest/s2svpn/CompromisedCredentials.html>

- Restrict access to AWS resources inside your AWS environment using Amazon VPC security groups, as explained here:

<https://docs.aws.amazon.com/vpn/latest/s2svpn/SetUpVPNConnections.html#vpn-configure-security-groups>

- Use AWS IAM identity-based policies to control who can create or modify Site-to-Site VPN resources, as explained here:

https://docs.aws.amazon.com/vpn/latest/s2svpn/security_iam_id-based-policy-examples.html

- Enable Site-to-Site VPN logs to allow further analysis of the VPN activity (such as errors of negotiation messages), as explained here:

<https://docs.aws.amazon.com/vpn/latest/s2svpn/monitoring-logs.html#enable-logs>

- Use Amazon CloudWatch alarms to generate alerts via Amazon SNS when certain metrics pass a predefined value (for example, when the amount of traffic in bytes is above a predefined threshold), as explained here:

<https://docs.aws.amazon.com/vpn/latest/s2svpn/monitoring-cloudwatch-vpn.html#creating-alarms-vpn>

- AWS CloudTrail is enabled by default for AWS Site-to-Site VPN. To keep AWS VPN API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as mentioned here:

<https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-working-with-log-files.html>

Securing AWS Client VPN

AWS Client VPN allows you to connect to the AWS environment from anywhere on the internet using an **OpenVPN** client in a secure TLS channel.

AWS Client VPN has the following components:

- **Client VPN endpoint:** A package for installing a VPN client (using either an AWS-provided client or an OpenVPN client)
- **Target network:** A network (such as a subnet) inside your VPC, containing AWS resources

Read thorough documentation on security in AWS Client VPN, which can be found at the link here: <https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/security.html>

Use the following best practices for securing AWS Client VPN:

- To control access of VPN clients to specific subnets within your VPC, configure target networks and associate them with Client VPN endpoints, as explained here:

<https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/cvpn-working-target.html>

- Restrict access to AWS resources inside your AWS environment using Amazon VPC security groups, as explained here:

- <https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/client-authorization.html#security-groups>

- <https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/scenario-restrict.html>

- For granular control of each client accessing specific AWS resources, use authorization rules, as explained here:

<https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/cvpn-working-rules.html>

- Use AWS IAM identity-based policies to control who can create or modify Client VPN resources, as explained here:

https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/security_iam_id-based-policy-examples.html

- If you are managing your user identities with **AWS Directory Service**, use AWS Client VPN **Active Directory** authentication, as mentioned here:

<https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/ad.html>

- When authenticating Client VPN against AWS Directory Service, enforce the use of MFA, as explained here:

- https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_mfa.html

- https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ad_connector_mfa.html

- Organizations in a hybrid environment (for example, AWS and Azure), can use **single sign-on (SSO)** SAML 2.0-based federated authentication to authenticate their VPN clients using a compatible SAML 2.0 identity provider, as documented here:

- <https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/federated-authentication.html>

- <https://learn.microsoft.com/en-us/entra/identity/saas-apps/aws-clientvpn-tutorial>

- For the most secure access, use AWS Client VPN **certificate-based** authentication (also known as mutual authentication) instead of pre-shared keys (i.e., a static key that is rarely replaced and can be compromised), using the ACM service, as explained here:

<https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/mutual.html>

- If you are using AWS Client VPN certificate-based authentication, use client certificate revocation lists to revoke access to employees who have left the organization or do not need access through the VPN, as explained here:

<https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/cvpn-working-certificates.html>

- Use Amazon CloudWatch Logs to collect AWS Client VPN connection logging information (such as connection attempt status, client IP, device IP, etc.) for troubleshooting purposes, as documented here:
 - <https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/connection-logging.html>
 - <https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/cvpn-working-with-connection-logs.html>
- Use Amazon CloudWatch to collect metrics about AWS Client VPN for further analysis, as mentioned here:
<https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/monitoring-cloudwatch.html>
- AWS CloudTrail is enabled by default for AWS Client VPN. To keep AWS VPN API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as mentioned here:
<https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/monitoring-cloudtrail.html>

Securing Azure VPN Gateway (Site-to-Site)

Azure VPN Gateway (*Site-to-Site*) is a managed service that allows you to connect your corporate network to the Azure environment in a secure channel.

Azure VPN Gateway has the following components:

- **VPN Gateway:** A VPN endpoint on the Azure side, receiving connections from customer's VPN tunnels
- **VPN device:** A customer's on-premises VPN device

Read thorough documentation on security in Azure VPN Gateway can be found at the link here: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/vpn-gateway-security-baseline>

Use the following best practices for securing Azure VPN Gateway:

- Encrypt traffic over the VPN tunnel using the GCM/AES256 algorithm and SHA256 as a hashing algorithm, as documented here:
<https://learn.microsoft.com/en-us/azure/vpn-gateway/vpn-gateway-about-vpn-devices#ipsec>

- Enable diagnostic settings for Azure VPN Gateway and send all logs to a Log Analytics workspace, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/essentials/diagnostic-settings>

- Use Azure Monitor to monitor the VPN tunnels (for example, it could send alerts when the amount of traffic in bytes is above a predefined threshold), as mentioned here:

<https://learn.microsoft.com/en-us/azure/vpn-gateway/vpn-gateway-howto-setup-alerts-virtual-network-gateway-metric>

- If you need to troubleshoot VPN-related network issues by capturing network traffic, use the VPN Gateway Packet Capture, as documented here:

<https://learn.microsoft.com/en-us/azure/vpn-gateway/packet-capture>

Securing Azure VPN Point-to-Site

Azure VPN Point-to-Site (P2S) allows you to connect from anywhere on the internet to the Azure environment in a secure channel, using an OpenVPN client, **Secure Socket Tunneling Protocol (SSTP)**, or **Internet Key Exchange version 2 (IKEv2)** VPN client.

Azure VPN P2S has the following components:

- **VPN gateway:** A VPN endpoint on the Azure side, receiving connections from customer's VPN clients
- **VPN client:** A package installed on the customer's desktop or mobile device (using either an Azure-provided client or an OpenVPN client)

Use the following best practices for securing Azure VPN P2S:

- Encrypt traffic over the VPN tunnel using the GCMAES256 algorithm and SHA256 as a hashing algorithm, as documented here:

<https://learn.microsoft.com/en-us/azure/vpn-gateway/point-to-site-about#what-ikeipsec-policies-are-configured-on-vpn-gateways-for-p2s>

- For organizations managing their identities in **Microsoft Entra ID**, configure Azure VPN P2S to authenticate using Microsoft Entra authentication, as explained here:

<https://learn.microsoft.com/en-us/azure/vpn-gateway/openvpn-azure-ad-tenant>

Note

Microsoft Entra authentication (using OpenVPN) is suitable only for Windows, Linux, and macOS clients.

- When using Microsoft Entra authentication, assign users to groups and restrict the access of end users to specific applications, as explained here:

<https://learn.microsoft.com/en-us/azure/vpn-gateway/openvpn-azure-ad-tenant-multi-app#assign-users-to-applications>

- When using Microsoft Entra authentication, enforce the use of MFA for all VPN users, as explained here:

<https://learn.microsoft.com/en-us/azure/vpn-gateway/openvpn-azure-ad-mfa>

- An alternative to using Microsoft Entra authentication is to use certificate authentication. It is suitable for Windows, macOS/iOS, Linux, and Android clients. Follow the instructions here to configure certificate authentication:

<https://learn.microsoft.com/en-us/azure/vpn-gateway/vpn-gateway-howto-point-to-site-resource-manager-portal>

Securing Google Cloud VPN

Google Cloud VPN is a managed service that allows you to connect your corporate network to the GCP environment in a secure channel (using an IPSec tunnel).

Google Cloud VPN has the following components:

- **Cloud VPN gateway:** A VPN endpoint on the GCP side, receiving connections from a peer VPN gateway
- **Peer VPN gateway:** A gateway connected to a Cloud VPN gateway such as another Cloud VPN gateway, a VPN gateway from AWS or Azure, or an on-premises VPN device

Read thorough documentation on security in Google Cloud VPN can be found at the link here: <https://cloud.google.com/network-connectivity/docs/vpn/concepts/best-practices#security>

Use the following best practices for securing Google Cloud VPN:

- Restrict access to GCP resources inside your GCP environment using VPC firewall rules, as explained here:

<https://cloud.google.com/network-connectivity/docs/vpn/how-to/configuring-firewall-rules>

- Restrict the IP addresses of the peer VPN gateway to prevent unauthorized VPN tunnels, as explained here:

<https://cloud.google.com/network-connectivity/docs/vpn/how-to/restrict-peer-ip-addresses>

- Encrypt traffic over the VPN tunnel using the AES-GCM-16-256 algorithm and SHA2-256 (at a minimum) as a hashing algorithm, as documented here:

<https://cloud.google.com/network-connectivity/docs/vpn/concepts/supported-ike-ciphers>

- Since Google Cloud VPN encrypts VPN tunnel based on a pre-shared key, it is recommended to generate a strong (32-character) pre-shared key and rotate it at least every 12 months, as explained here:

<https://cloud.google.com/network-connectivity/docs/vpn/how-to/generating-pre-shared-key>

- Use Google Cloud Logging to monitor activity on the Google Cloud VPN service, and define alerts based on predefined thresholds (such as connectivity issues, tunnel down, etc.), as mentioned here:

- <https://cloud.google.com/network-connectivity/docs/vpn/how-to/viewing-logs-metrics>

- <https://cloud.google.com/network-connectivity/docs/vpn/support/troubleshooting>

In this section, we learned how to secure managed VPN services (both site-to-site and client-based) based on the AWS, Azure, and GCP infrastructure. This included IAM, encryption, ACLs, and monitoring.

Securing Zero Trust services

In the previous section, we reviewed VPN services for allowing remote connectivity to cloud environments.

In the past few years, a new approach has emerged allowing remote connectivity, called Zero Trust.

Zero Trust is a modern security approach that assumes no user, device, or network should be automatically trusted, regardless of their location or previous access privileges.

Key aspects of Zero Trust include the following:

- Continuous identity verification
- Least-privilege access
- Micro-segmentation
- Continuous monitoring and logging
- Focus on protecting what matters most rather than just defending against threats

The main differences between client-side VPN and the Zero Trust approach are outlined here:

- Zero Trust assumes no implicit trust, regardless of network location
- In Zero Trust, access is based on user identity, device health, and context
- Zero Trust can provide direct access to resources, potentially reducing latency
- Zero Trust is designed to scale easily for distributed workforces
- Zero Trust can provide consistent access policies across multiple cloud environments

Securing AWS Verified Access

AWS Verified Access is a service that provides secure access to corporate applications without requiring a VPN. It validates every application request before granting access, following Zero Trust principles, and it allows you to set detailed access policies for your applications based on factors such as user identity, device security status, and location.

AWS Verified Access has the following components:

- **Verified Access instances:** An instance assesses application requests and only permits access when security criteria are satisfied.
- **Verified Access endpoints:** Each endpoint stands for an application. You can set up either a load balancer endpoint or a network interface endpoint.
- **Verified Access group:** A group of Verified Access endpoints. It is recommended to organize the endpoints for applications with similar security needs to make managing policies easier.
- **Access policies:** A set of rules created by users that decides whether to grant or deny access to an application.
- **Trust providers:** A trusted provider is a service that handles user identities or checks device security. AWS Verified Access can work with trust providers from AWS or other companies.

In *Figure 6.5*, we can see an AWS Verified Access connection workflow:

1. Users initiate requests to access an application.
2. The service then assesses these requests by comparing them against two key elements:
 - I. The access policy set for the user's group
 - II. Any specific policies defined for the application's endpoint
3. If the request meets all the policy requirements, Verified Access grants permission.
4. Once approved, the user's request is securely routed to the target application via the designated endpoint.

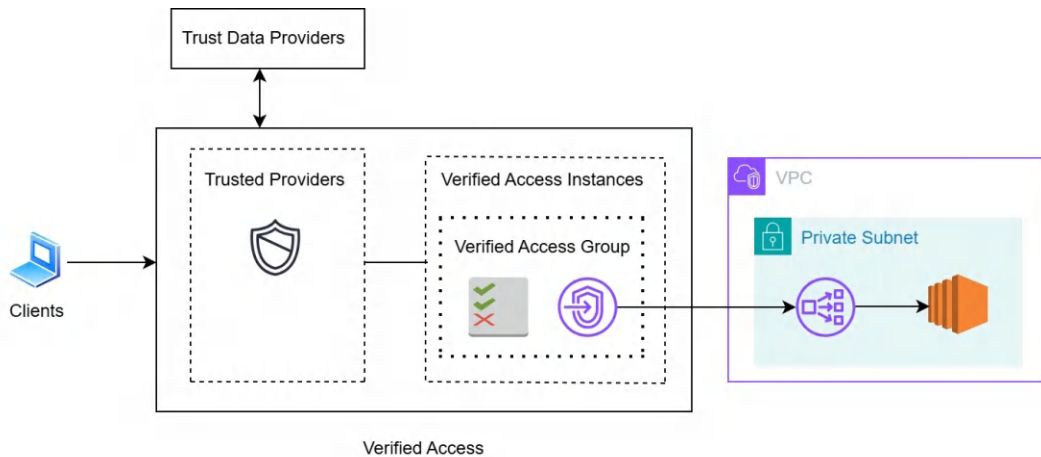


Figure 6.5 – AWS Verified Access connection workflow

Thorough documentation on security in AWS Verified Access can be found at the link here: <https://docs.aws.amazon.com/verified-access/latest/ug/security.html>.

Use the following best practices for securing AWS Verified Access:

- Use AWS **Identity and Access Management (IAM)** identity-based policies to control who can create or modify AWS Verified Access resources, as explained here: https://docs.aws.amazon.com/verified-access/latest/ug/security_iam_id-based-policy-examples.html.
- Use **AWS IAM Identity Center** as a trusted provider for centrally allowing identities access through AWS Verified Access (that is, as an **identity provider (IdP)**, not as an authorization mechanism), as explained here: <https://docs.aws.amazon.com/verified-access/latest/ug/user-trust.html>.
- Configure Verified Access policies (written in Cedar language) to restrict access to AWS resources (such as group membership, source IP, traffic originating from Verified Access endpoint, and more), as explained here:
 - <https://docs.aws.amazon.com/verified-access/latest/ug/auth-policies.html>
 - <https://docs.aws.amazon.com/verified-access/latest/ug/configure-endpoint-security-group.html>.
- For sensitive/critical environments (for example, which contain **personally identifiable information (PII)**, credit card details, healthcare data, and more), encrypt data at rest using customer-managed keys (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here: <https://docs.aws.amazon.com/verified-access/latest/ug/encryption-at-rest.html>.

- **AWS CloudTrail** is enabled by default for AWS Verified Access. To keep Verified Access API activity for longer than 90 days, send CloudTrail logs to an Amazon **Simple Storage Service (S3)** bucket, as explained here: <https://docs.aws.amazon.com/verified-access/latest/ug/logging-using-cloudtrail.html>.

Securing Global Secure Access

Microsoft Global Secure Access is a modern security solution that aligns with the Zero Trust security model. Global Secure Access helps organizations implement Zero Trust principles by providing a secure, flexible, and user-friendly way to access resources, whether they are in the cloud or on-premises, from any location or device.

It consists of two main components:

- **Microsoft Entra Internet Access:** This protects users when they access public websites, cloud apps, and Microsoft 365
- **Microsoft Entra Private Access:** This provides secure access to private company resources, replacing traditional VPNs

For the context of this chapter, we will review the capabilities of Microsoft Entra Private Access.

Microsoft Entra Private Access has the following components:

- **Global Secure Access client:** This is a client-side component that needs to be installed on end-user devices.
- **Network connectors:** These are server-side components that connect on-premises networks to the Global Secure Access endpoint. They enable the tunneling of traffic from remote networks.
- **Quick Access:** This is a method to secure a main group of internal resources by creating a list of domain names (**fully qualified domain names** or **FQDNs**) and IP addresses.
- **Conditional Access integration:** Entra Private Access integrates with Microsoft Entra Conditional Access, allowing you to enforce adaptive access policies, including **multi-factor authentication (MFA)**, location-based security, and least-privilege access.
- **Enterprise applications:** These are created in Microsoft Entra to act as gateways to private resources. Each application has its connector to link with internal resources.

In *Figure 6.6*, we can see an Entra Private Access connection workflow:

- A Global Secure Access client initiates a connection to Entra Private Access to be able to access a web application
- Entra Private Access validates that an enterprise app is configured for the target request and that there is a Conditional Access policy (for example, a client member in an Entra ID group)

- In case the request is authorized, the connection is sent to a network connector with access rules for the web app
- The client was successfully able to access the target web app

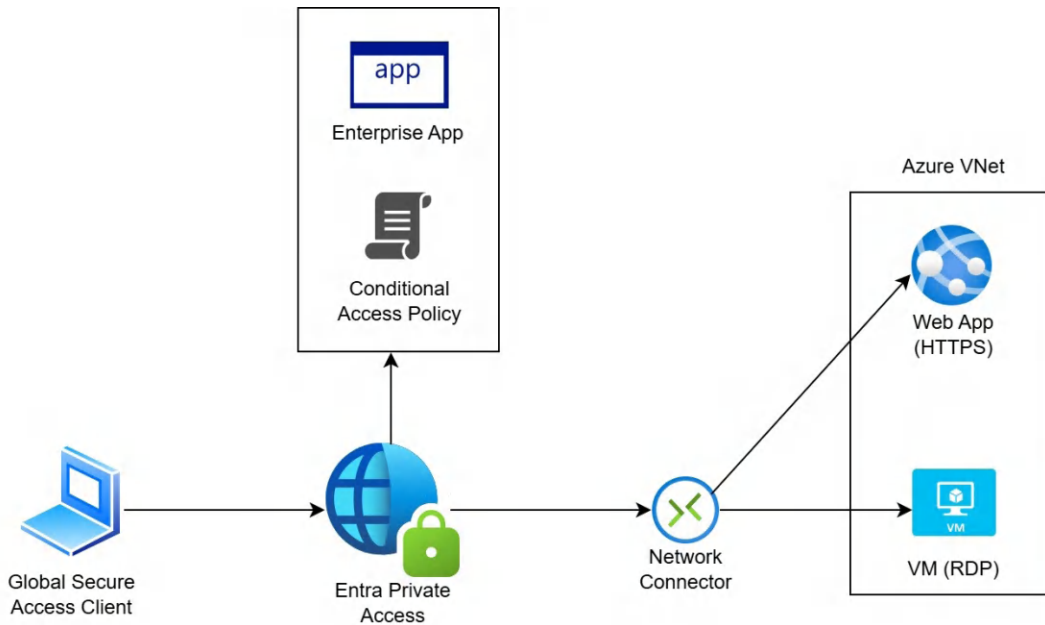


Figure 6.6 – Microsoft Entra Private Access connection workflow

Use the following best practices for securing Entra Private Access:

- Use **Microsoft Entra Conditional Access** policies to configure rules for which identities or devices can access Azure resources using Entra Private Access and in what terms (such as group membership, enforcement of MFA, device compliant with company policy, and so on), as explained here:
 - <https://learn.microsoft.com/en-us/entra/global-secure-access/how-to-target-resource-private-access-apps>
 - <https://learn.microsoft.com/en-us/entra/global-secure-access/how-to-compliant-network>
- To allow organizational users access to external applications on a partner's network, use tenant restrictions v2 to limit access according to a business use case, as explained here: <https://learn.microsoft.com/en-us/entra/global-secure-access/how-to-universal-tenant-restrictions>

- Configure a Private Access traffic forwarding profile to allow secure access to private resources without using a VPN, as explained here: <https://learn.microsoft.com/en-us/entra/global-secure-access/how-to-manage-private-access-profile>
- For easier management, assign users to groups and configure traffic forwarding profiles for groups according to business needs, as explained here: <https://learn.microsoft.com/en-us/entra/global-secure-access/how-to-manage-users-groups-assignment>
- To avoid overpermissions and unauthorized access, configure per-app access, as explained here: <https://learn.microsoft.com/en-us/entra/global-secure-access/how-to-configure-per-app-access>
- Use Microsoft Entra **Privileged Identity Management (PIM)** to restrict any privileged access attempt for **just-in-time (JIT)** access using Entra Private Access, as explained here: <https://learn.microsoft.com/en-us/entra/global-secure-access/how-to-configure-global-access-with-pim>
- Enable diagnostic settings for Global Secure Access for further investigation and analysis, and send all logs to a Log Analytics workspace, as explained here: <https://learn.microsoft.com/en-us/entra/global-secure-access/how-to-view-enriched-logs>

Securing BeyondCorp

BeyondCorp Enterprise (part of **Chrome Enterprise Premium**) is Google Cloud's Zero Trust solution. It is built on the *never trust, always verify* model, and it authenticates and authorizes every user, device, and application before granting access to resources, regardless of their location.

BeyondCorp Enterprise has the following components:

- **Google global frontend/Google Identity-Aware Proxy (IAP):** Serves as the frontend infrastructure for handling access requests
- **Google Cloud IAM/single sign-on (SSO) system:** Provides centralized user authentication and generates short-lived tokens for resource access
- **Device identity/endpoint verification:** Uses device certificates to uniquely identify each managed device (based on Google Chrome extension)
- **Access Context Manager rules engine:** This acts as the central decision-making component, determining whether to grant or deny access based on various factors
- **Policy enforcement:** Implements and enforces access control policies based on user identity, device state, and other contextual factors

In *Figure 6.7*, we can see a BeyondCorp connection workflow:

- A user is sending a request to the Google global frontend to be able to access a VM in a private subnet
- The global frontend evaluates both user identity (using Google Cloud IAM) and device identity (using certificate-based access) to make sure the request is originating from a trusted user/device
- The request is sent to a rules engine to verify that the user has permission to access the VM
- Based on the policy assigned to the specific request (user identity, device state, permissions to access the VM, and so on), the request is sent to the remote VM (inside the private subnet), and the user can access the VM

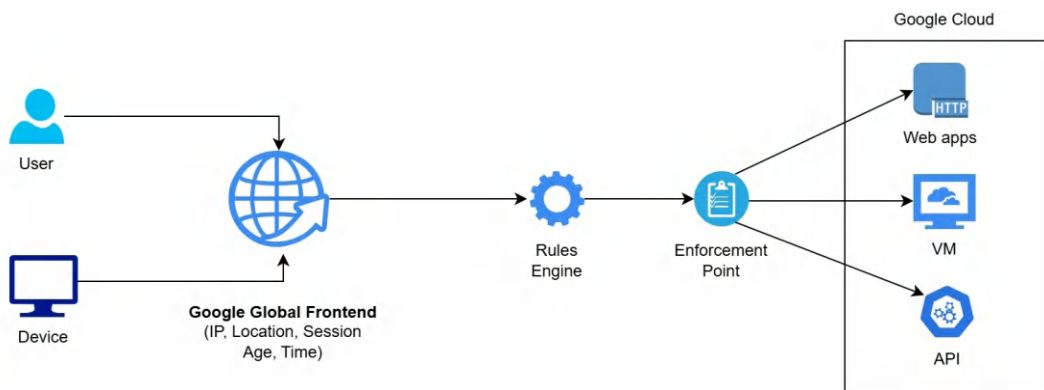


Figure 6.7 – BeyondCorp connection workflow

Use the following best practices for securing BeyondCorp:

- Use IAP to configure secure access to resources (such as **Google App Engine (GAE)** apps, **Google Compute Engine (GCE)** apps, **Google Kubernetes Engine (GKE)** apps, and VMs) located in private subnets, as explained here:
 - <https://cloud.google.com/beyondcorp-enterprise/docs/securing-app-engine>
 - <https://cloud.google.com/beyondcorp-enterprise/docs/securing-compute-engine>
 - <https://cloud.google.com/beyondcorp-enterprise/docs/securing-kubernetes-engine>
 - <https://cloud.google.com/beyondcorp-enterprise/docs/securing-virtual-machines>

- Use the Chrome Enterprise Premium app connector to secure access to non-Google Cloud applications (provides identity and context-aware to HTTPS-based applications), as explained here: <https://cloud.google.com/beyondcorp-enterprise/docs/enable-app-connector>
- Use **Access Context Manager** combined with **VPC Service Controls** to create an access policy and allow fine-grained access to resources for devices within the Google organization or from partner organizations, as explained here: <https://cloud.google.com/access-context-manager/docs/create-access-policy>
- To allow endpoint (device) verification, configure certificate-based access, which adds an authorization factor for devices to be able to access private resources and APIs (in GCP or on-prem), as explained here:
 - <https://cloud.google.com/beyondcorp-enterprise/docs/set-up-cba>
 - <https://cloud.google.com/beyondcorp-enterprise/docs/enable-cba-user-groups>
 - <https://cloud.google.com/beyondcorp-enterprise/docs/enable-cba-vpcsc>
- To add a layer of protection to devices (that is, Chrome users), configure **Chrome Enterprise connector** policies for Chrome Enterprise Premium (such as DLP policies and safe browsing), as explained here:
 - <https://support.google.com/a/answer/10104463>
 - <https://support.google.com/chrome/a/answer/10106035>

In this section, we learned how to secure Zero Trust services based on the AWS, Azure, and GCP infrastructure. This included authentication and authorization processes.

Summary

In this chapter, we focused on the various network services offered by AWS, Azure, and Google. This included virtual networking, DNS, VPN services, and Zero Trust services. In each section, we reviewed the best practices for configuration, authentication, authorization, monitoring, and auditing. Managing these services allows us to control who has access to them and to monitor their activity.

In the next chapter, we will review additional networking services (including **distributed denial-of-service (DDoS)** protection and **web application firewall (WAF)** services) and the best practices for configuring and securing them.

Securing Networking Services – Part 2

In the previous chapters, we covered securing computing and storage services and networking, such as virtual networks and DNS and VPN services, respectively.

In this chapter, we will focus on *networking services* and learn about DDoS protection services and WAFs.

A WAF and DDoS protection are both necessary for a strong defense against different types of attacks. While a WAF can help mitigate application-layer attacks and some smaller-scale DDoS attacks, a dedicated DDoS protection service is essential for mitigating large-scale and network-layer DDoS attacks. Together, they provide comprehensive security for web applications.

This chapter will cover various networking services and provide you with best practices on how to securely connect to and use each of them.

In this chapter, we will cover the following topics:

- Using DDoS protection services
- Using WAF services

Using DDoS protection services

DoS stands for **Denial of Service**, and **DDoS** stands for **Distributed Denial of Service**. Each cloud provider has its implementation of a managed DDoS protection service. Because cloud providers have a large bandwidth (compared to most customers or even internet service providers), they can offer (as a paid service) mechanisms to protect customers' environments from DDoS attacks, by spreading the traffic across its global infrastructure and its multiple data centers. Due to the large network capacity, CSPs can filter much larger amounts of data compared to on-premises infrastructure. CSPs often provide automated scaling tools that can increase the bandwidth and resource allocation temporarily, allowing services to remain operational during an attack. CSPs have dedicated teams and automated systems to monitor unusual traffic patterns. Because of their vast networks, they can detect DDoS attacks quickly, as they begin.

The types of DoS and DDoS attacks are as follows:

- **Volumetric attacks:** Attacks at Layer 3 (network) of the OSI model to saturate the bandwidth of the target network
- **Protocol attacks:** Attacks at Layer 4 (transport) of the OSI model to abuse a protocol of the target server to deny legitimate customers traffic (vulnerabilities such as a TCP syn flood)
- **Application layer attacks:** Attacks at Layer 7 (application) of the OSI model to deny legitimate customers traffic by flooding the system with multiple valid queries (such as web request floods)

In the following example, we see a common DDoS attack, where an attacker is sending commands to a command-and-control server, which uses multiple bots to send malicious requests to consume a victim's cloud environment bandwidth causing a volumetric attack.

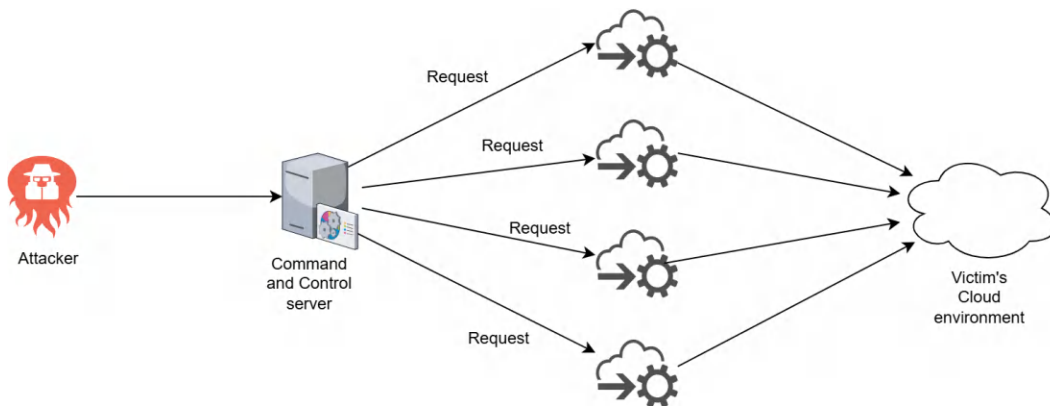


Figure 7.1 – A sample volumetric attack

The following services help to mitigate DDoS attacks:

- DDoS protection services (discussed in this section)
- Auto-scaling groups combined with load-balancing services
- WAF services (discussed later in this chapter)

Sidenote about DDoS and auto-scaling groups

When designing highly available cloud architecture, the common best practice is to deploy resources (such as VMs, Kubernetes clusters, databases, and so on) behind a managed load-balancer service, in an auto-scaling group. The auto-scaling group allows customers to increase the number of running compute nodes to be able to handle the spike in traffic. Well-architected workloads will be able to increase their capacity resources (VMs, CPU, memory), but the huge demand for additional resources comes with an inflated cost, also known as *Denial of Wallet*, where a customer is required to pay a lot

of money for the extra capacity. It is common for CSPs to offer a costly pricing tier where, in case of a DDoS attack, the CSP will monitor the spike in resource usage and offer customers a cost protection (wherein the CSP caps or absorbs the extra costs) to decrease the risk of monthly bills for the extra resources during the DDoS attack.

These are general best practices for managed DDoS protection services:

- Enable DDoS protection for any publicly exposed production environment
- Use monitoring services to review DDoS-related activities and investigate and configure blocking or filtering mechanisms to prevent future attacks against your environments

In this section, we will focus on DDoS protection services.

Using AWS Shield

AWS Shield is an Amazon-managed DDoS protection service.

It comes in two price models:

- **AWS Shield Standard:** This is the default service provided to all customers for free and protects against attacks on Layers 3 and 4 of the OSI model.
- **AWS Shield Advanced:** This offers protection for attacks on Layer 3 (network layer), Layer 4 (transport layer), and Layer 7 (application layer) DDoS protection with additional protection for AWS services, such as DNS (Route 53), CDN (CloudFront), **Elastic Load Balancing (ELB)**, and virtual machine (EC2) services. This price tier also offers support from the AWS DDoS response team.

AWS Shield Advanced can protect Amazon CloudFront distributions, Amazon Route 53 hosted zones, AWS Global Accelerator, Amazon EC2 instances, network load balancers, and application load balancers.

Read thorough documentation on security in AWS Shield, which can be found in the links here:

- <https://docs.aws.amazon.com/waf/latest/developerguide/shd-security.html>
- <https://docs.aws.amazon.com/whitepapers/latest/aws-best-practices-ddos-resiliency/introduction-denial-of-service-attacks.html>

Use the following best practices for configuring AWS Shield:

- AWS Shield Standard is enabled by default and defends against common network or transport layer attacks against any publicly exposed websites or applications, as documented here:

https://aws.amazon.com/shield/features/#AWS_Shield_Standard

- For large-scale publicly exposed production environments, configure AWS Shield Advanced protection groups to protect your resources, as documented here:
 - <https://docs.aws.amazon.com/waf/latest/developerguide/ddos-manage-protected-resources.html>
 - <https://docs.aws.amazon.com/waf/latest/developerguide/manage-protection-group.html>
- For best protection against application layer attacks against websites, combine AWS Shield Advanced with AWS WAF, and configure web ACL, as explained here:
 - <https://docs.aws.amazon.com/waf/latest/developerguide/ddos-app-layer-web-ACL-and-rbr.html>
 - <https://docs.aws.amazon.com/waf/latest/developerguide/ddos-automatic-app-layer-response-rg.html>
- Associate health checks with protected resources, as explained here:
<https://docs.aws.amazon.com/waf/latest/developerguide/manage-health-check-associations.html>
- Use Amazon CloudWatch to collect metrics about AWS Shield Advanced for further analysis, as mentioned here:
<https://docs.aws.amazon.com/waf/latest/developerguide/shield-metrics.html>
- AWS CloudTrail is enabled by default for Amazon Shield. To keep AWS Shield API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here:
 - <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-create-a-trail-using-the-console-first-time.html>
 - <https://docs.aws.amazon.com/waf/latest/developerguide/shield-info-in-cloudtrail.html>
- Use the global threat dashboard to view AWS Shield activity for your entire AWS organization or a specific AWS account, as explained here:
 - <https://docs.aws.amazon.com/waf/latest/developerguide/ddos-viewing-events.html>
 - <https://docs.aws.amazon.com/waf/latest/developerguide/ddos-standard-event-visibility.html>

- Integrate AWS Shield Advanced with **AWS Security Hub** to gain central visibility into DDoS events, as explained here:

<https://aws.amazon.com/blogs/security/set-up-centralized-monitoring-for-ddos-events-and-auto-remediate-noncompliant-resources/>

- To conduct DDoS simulation testing on your AWS accounts, follow the documentation here:

<https://aws.amazon.com/blogs/security/understanding-ddos-simulation-testing-at-aws/>

Using Azure DDoS Protection

Azure DDoS Protection is an Azure-managed DDoS protection service.

It comes in two price models:

- **DDoS Network Protection:** It provides DDoS protection against attacks on Layers 3 and 4, integrated into Azure VNet. This price tier also offers support from the Azure DDoS response team.
- **DDoS IP Protection:** This provides Layer 3/4 (network and transport layers) DDoS protection, especially suited for protecting publicly exposed IP addresses.

The main difference between the pricing models is DDoS Network Protection adds DDoS rapid response support, and in case of a DDoS attack, which would cause a spike in resource usage and costs, Azure offers cost protection to avoid paying for the extra resources that were created (using mechanisms such as Azure Load Balancer) during the attack.

Azure DDoS Protection can protect Azure VMs, Azure Application Gateway, Azure API Management, Azure Bastion, external Azure VNets, Azure Firewall, Azure Load Balancer, and Azure VPN Gateway.

Read thorough documentation on security in Azure DDoS Protection, which can be found at the link here: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/azure-ddos-protection-security-baseline>

Use the following best practices for configuring Azure DDoS Protection:

- Create a DDoS protection plan for your public VNets as explained here:

<https://learn.microsoft.com/en-us/azure/ddos-protection/manage-ddos-protection>

- Enable DDoS protection for your publicly exposed IP addresses, as explained here:

<https://learn.microsoft.com/en-us/azure/ddos-protection/manage-ddos-ip-protection-portal>

- For best protection against application layer attacks (Layer 7) against websites in a single region, combine Azure DDoS Protection with Azure Application Gateway with WAF v2 SKU, as explained here:

<https://learn.microsoft.com/en-us/azure/web-application-firewall/ag/application-gateway-web-application-firewall-portal>

- For best protection against application layer attacks (Layer 7) against large-scale/multi-region websites, combine Azure DDoS Protection with Azure Front Door, as explained here:

<https://learn.microsoft.com/en-us/azure/web-application-firewall/afds/waf-front-door-create-portal>

- Enable diagnostic settings for Azure DDoS Protection and send all logs to a Log Analytics workspace, as explained here:

<https://learn.microsoft.com/en-us/azure/ddos-protection/diagnostic-logging>

- Use **Azure Monitor** to collect metrics about Azure DDoS Protection for further analysis, as mentioned here:

- <https://learn.microsoft.com/en-us/azure/ddos-protection/alerts>
- <https://learn.microsoft.com/en-us/azure/ddos-protection/telemetry>
- <https://learn.microsoft.com/en-us/azure/ddos-protection/monitor-ddos-protection-reference>

- Use Microsoft Defender for Cloud to view DDoS attack-related alerts from your Azure organization, as explained here:

<https://learn.microsoft.com/en-us/azure/ddos-protection/ddos-view-alerts-defender-for-cloud>

- Integrate Azure DDoS Protection with **Microsoft Sentinel** to be able to investigate DDoS-related activities, as mentioned here:

<https://learn.microsoft.com/en-us/azure/sentinel/data-connectors/azure-ddos-protection>

- To conduct DDoS simulation testing on your Azure subscriptions, follow the documentation here:

<https://learn.microsoft.com/en-us/azure/ddos-protection/test-through-simulations>

Using Google Cloud Armor (for DDoS protection)

Google Cloud Armor is a Google-managed WAF and DDoS protection service.

It comes in two price models:

- **Standard network DDoS Protection:** This is the default free Layers 3/4 (network and transport layers) DDoS protection provided for all customers for free. This license also includes preconfigured WAF rules for OWASP Top 10 protection.
- **Advanced network DDoS Protection:** This offers the same capabilities as the standard network DDoS Protection, but adds monitoring, alerting, and protection on a per-region basis. This license also includes Layer 7 protection, including WAF rules, policies, and requests. This price tier also offers support from the Google DDoS response team. This option requires a Cloud Armor Enterprise license.

Using Google Cloud Armor as DDoS protection can protect external load-balancers, protocol forwarding, and VMs with public IP addresses.

In this section, we will be focusing on Google Cloud Armor as a DDoS protection service.

Read thorough documentation on security in Google Cloud Armor and DDoS protection, which can be found at the link here: <https://cloud.google.com/armor/docs/advanced-network-ddos>

Use the following best practices for configuring Google Cloud Armor as a DDoS protection service:

- For production environments, activate advanced network DDoS Protection for each region with exposed workloads, as explained here:

`https://cloud.google.com/armor/docs/advanced-network-ddos#activate-advanced-ddos-protection`

- Configure security policies to filter incoming traffic to load-balancers, protocol forwarding, and VMs with public IP addresses, as explained here:

`https://cloud.google.com/armor/docs/configure-security-policies`

- Configure network edge security policies to block traffic to your publicly exposed environments at the edge of Google's network, as explained here:

`https://cloud.google.com/armor/docs/network-edge-policies`

- Configure Adaptive Protection to gain protection against Layer 7 attacks, as explained here:

`https://cloud.google.com/armor/docs/configure-adaptive-protection`

- Use Google Cloud Logging to monitor DDoS-related activity, as mentioned here:
<https://cloud.google.com/armor/docs/ddos-visibility>
- Use Google Cloud Monitoring to monitor for network-policy-related activity, as mentioned here:
<https://cloud.google.com/armor/docs/monitoring>

In this section, we learned about the DDoS protection services from AWS, Azure, and Google. We also discussed their pricing models and the advanced reporting and alerting capabilities provided by the more advanced DDoS protection services.

Next, we are going to review managed WAF services.

Using WAF services

WAF stands for **Web Application Firewall**. Each cloud provider has its implementation of a managed WAF service – that is, an application-layer firewall with capabilities to detect and mitigate common HTTP/HTTPS-based attacks against your publicly exposed web applications. Examples of application-layer attacks include injection attacks, cross-site scripting, OWASP Top 10 (injecting malicious code for the purpose of impacting data confidentiality or integrity), and various malicious bot activities (such as scrapers, scanners, and crawlers, which take network bandwidth with non-legitimate traffic).

In the following example, we see a common web application attack, where an attacker is sending a malicious script to a victim's publicly facing web server deployed in a cloud environment to exploit a vulnerability with the intent to exfiltrate customer data.

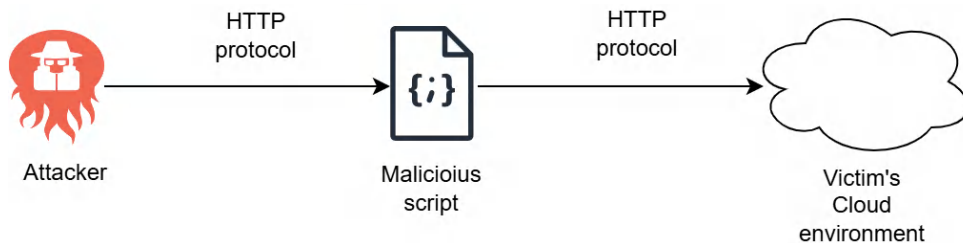


Figure 7.2 – A sample web application attack

Here are general best practices for managed WAF services:

- Create policies to prevent web-application-related attacks
- Create geo-location rules to restrict access by country
- Use bot detection mechanisms to restrict access from non-legitimate traffic
- Use monitoring services to review WAF-related activities and investigate and configure blocking or filtering mechanisms to prevent future attacks against your environments

In this section, we will focus on managed WAF services.

Using AWS WAF

AWS WAF is an AWS-managed web application firewall service.

AWS WAF offers protection against the following types of attacks:

- Layer 7 DDoS attacks (when combined with AWS Shield)
- Common web application attacks
- **Bots** (non-human generated traffic)

AWS WAF offers the following protection mechanisms:

- **Web ACLs:** These help protect web applications by filtering traffic based on a set of user-defined or pre-configured rules (such as IP Address, geo-location, or string match). These rules inspect traffic patterns, allowing or blocking requests to mitigate attacks such as DDoS, SQL injection, and XSS while allowing legitimate traffic to pass.
- **Rules:** These act as security guards that control what kind of web traffic can reach your website or web app. Here are some rule examples: “Block traffic from this specific IP address,” “Allow requests only if they don’t contain dangerous patterns like SQL injections,” or “Block users if they make too many requests in a short time (rate limiting).”
- **Managed rules:** Managed rules protect web apps from common threats (such as SQL injection attacks, **Cross-Site Scripting (XSS)**, and others).
- **Fraud control:** This helps reduce the impact of fraud on an application’s performance, while also lowering operational and infrastructure costs. This is useful to prevent unauthorized login attempts and the creation of fraudulent or fake accounts.
- **Bot control:** This provides visibility and control over common, widespread bot traffic, which can consume excessive resources, distort metrics, cause downtime, or engage in other unwanted activities.

AWS WAF can protect Amazon CloudFront distribution, Amazon API Gateway, Application Load Balancer, AWS AppSync GraphQL API, Amazon Cognito, AWS App Runner, and AWS Verified Access instances.

Read thorough documentation on security in AWS WAF, which can be found at the links here:

- <https://docs.aws.amazon.com/waf/latest/developerguide/security.html>
- <https://docs.aws.amazon.com/whitepapers/latest/guidelines-for-implementing-aws-waf/guidelines-for-implementing-aws-waf.html>

Use the following best practices for securing AWS WAF:

- Create web ACLs for protecting publicly exposed resources, based on criteria such as origin IP address or country, string match or regular expression, size or request, or detection of malicious SQL code or scripting, as explained here:
 - <https://docs.aws.amazon.com/waf/latest/developerguide/waf-rules.html>
 - <https://docs.aws.amazon.com/waf/latest/developerguide/web-acl-creating.html>
- Add web ACLs to rule groups, and assign each rule group a number to be processed in priority order (from a lower to a higher number), as explained here:
 - <https://docs.aws.amazon.com/waf/latest/developerguide/waf-rule-group-creating.html>
 - <https://docs.aws.amazon.com/waf/latest/developerguide/web-acl-processing-order.html>
- For protection against common web-related attacks, use AWS-managed rules (based on the OWASP Top 10 – most common web application vulnerabilities), as documented here:
<https://docs.aws.amazon.com/waf/latest/developerguide/aws-managed-rule-groups-list.html>
- Before implementing AWS-managed rules in production, evaluate the rules in a test environment (to make sure they will not break applications due to restrictions), as explained here:
<https://docs.aws.amazon.com/waf/latest/developerguide/web-acl-testing-enable-production.html>
- In case you wish to create geo-restrictions for customers from specific countries, configure a geo match statement in AWS WAF, as explained here:
<https://docs.aws.amazon.com/waf/latest/developerguide/waf-rule-statement-type-geo-match.html>
- To protect against account creation fraudulent activity on websites that allow user registration, use the AWS WAF Fraud Control **account creation fraud prevention (ACFP)** feature, as explained here:
<https://docs.aws.amazon.com/waf/latest/developerguide/waf-acfp-rg-using.html>

- To protect against account takeover activity on websites that allow user registration, use the AWS WAF Fraud Control **account takeover prevention (ATP)** feature, as explained here:

<https://docs.aws.amazon.com/waf/latest/developerguide/waf-atp-rg-using.html>

- To protect against malicious bot activity against your websites, use the AWS WAF Bot Control, as explained here:

<https://docs.aws.amazon.com/waf/latest/developerguide/waf-bot-control-rg-using.html>

- Use Amazon CloudWatch to collect metrics about AWS WAF for further analysis, as mentioned here:

<https://docs.aws.amazon.com/waf/latest/developerguide/waf-metrics.html>

- AWS CloudTrail is enabled by default for AWS WAF. To keep AWS WAF API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here:

- <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-create-a-trail-using-the-console-first-time.html>

- <https://docs.aws.amazon.com/waf/latest/developerguide/understanding-waf-entries.html>

- Enable logging of AWS WAF web ACL traffic, for further investigation of application layer attacks, as explained here:

<https://docs.aws.amazon.com/waf/latest/developerguide/logging-management.html>

Note

To protect sensitive information within AWS WAF logs, encrypt the WAF logs, or use the `RedactedFields` option in the logging configuration, as explained in the documentation:https://docs.aws.amazon.com/waf/latest/APIReference/API_LoggingConfiguration.html

- Use Web ACL traffic overview dashboards to review a near-real-time summary of AWS WAF activity, as explained here:

<https://docs.aws.amazon.com/waf/latest/developerguide/web-acl-dashboards.html>

Using Azure Application Gateway WAF v2

Azure Application Gateway is a Layer 7 load-balancer. To protect publicly exposed workloads against web application-related attacks (inbound HTTP/HTTPS traffic) in a specific Azure region, deploy Azure Application Gateway with a WAF v2 license.

Azure Application Gateway has many capabilities relating to load-balancing, but for this section, we will be focusing on its WAF capabilities. Azure Application Gateway with WAF v2 can protect publicly exposed resources such as Azure VMs, **Azure Virtual Machine Scale Sets (VMSS)**, **Azure Kubernetes Service (AKS)**, Azure App Service, and on-premise or external application servers.

Read thorough documentation on security in Azure Application Gateway, which can be found at the link here: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/application-gateway-security-baseline>

Use the following best practices for securing Azure Application Gateway:

- Create a WAF policy for protecting publicly exposed resources, as explained here:
<https://learn.microsoft.com/en-us/azure/web-application-firewall/ag/create-waf-policy-ag>
- To protect against malicious bot activity against your websites, configure bot protection, as explained here:
<https://learn.microsoft.com/en-us/azure/web-application-firewall/ag/bot-protection>
- Mask sensitive data inside WAF logs using a WAF policy, as explained here:
<https://learn.microsoft.com/en-us/azure/web-application-firewall/ag/waf-sensitive-data-protection-configure>
- Enforce the use of strong transport protocols and cipher suites using a TLS policy, as explained here:
<https://learn.microsoft.com/en-us/azure/application-gateway/application-gateway-ssl-policy-overview>
- Perform TLS termination to examine encrypted traffic to your publicly exposed workloads, as mentioned here:
<https://learn.microsoft.com/en-us/azure/application-gateway/ssl-overview>
- When performing TLS termination, use Azure Key Vault certificates, as explained here:
<https://learn.microsoft.com/en-us/azure/application-gateway/key-vault-certs>

- For secure access between Azure Application Gateway and your internal Azure resources, and to keep traffic inside the Azure backbone, use Azure Private Link, as explained here:

<https://learn.microsoft.com/en-us/azure/application-gateway/private-link-configure>

- Use Azure NSG to restrict access to resources behind Azure Application Gateway, as explained here:

<https://learn.microsoft.com/en-us/azure/application-gateway/configuration-infrastructure#network-security-groups>

- Enable diagnostic settings for Azure Application Gateway and send all logs to a Log Analytics workspace, as explained here:

<https://learn.microsoft.com/en-us/azure/application-gateway/application-gateway-diagnostics>

- Use Azure Monitor to collect metrics about WAF-related activity for further analysis, as mentioned here:

<https://learn.microsoft.com/en-us/azure/application-gateway/monitor-application-gateway-reference#application-gateway-v1-metrics>

- Use Microsoft Defender for Cloud to view WAF-related alerts from your Azure organization, as explained here:

<https://learn.microsoft.com/en-us/azure/defender-for-cloud/other-threat-protections#display-azure-waf-alerts-in-defender-for-cloud>

- Use Microsoft Sentinel to be able to investigate WAF-related activities, as explained here:

<https://learn.microsoft.com/en-us/azure/web-application-firewall/waf-new-threat-detection>

Using Azure Front Door

Azure Front Door is a Layer 7 WAF and CDN service. To protect publicly exposed workloads against web application-related attacks (inbound HTTP/HTTPS traffic) in a large-scale global architecture, deploy Azure Front Door. Azure Front Door has many capabilities, but for this section, we will be focusing on its WAF capabilities.

Azure Front Door can protect publicly exposed resources such as Azure Blob, Azure App Service, Azure API Management, Azure Application Gateway, public IP addresses, Azure Container Apps, Azure Container Instances, and Azure AKS.

Read thorough documentation on security in Azure Front Door, which can be found at the link here: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/azure-front-door-security-baseline>

Use the following best practices for securing Azure Front Door:

- Create a WAF policy for protecting publicly exposed resources, as explained here:
<https://learn.microsoft.com/en-us/azure/web-application-firewall/aafs/waf-front-door-create-portal>
- In case you wish to create geo-restrictions for customers from specific countries, configure geo-filtering in Azure Front Door, as explained here:
<https://learn.microsoft.com/en-us/azure/web-application-firewall/aafs/waf-front-door-tutorial-geo-filtering>
- Use IP restriction to restrict access to resources behind Azure Front Door from specific IP addresses, as explained here:
<https://learn.microsoft.com/en-us/azure/web-application-firewall/aafs/waf-front-door-configure-ip-restriction>
- To protect against malicious bot activity against your websites, configure bot protection, as explained here:
<https://learn.microsoft.com/en-us/azure/web-application-firewall/aafs/waf-front-door-policy-configure-bot-protection?pivots=portal>
- Perform TLS termination to examine encrypted traffic to your publicly exposed workloads, as mentioned here:
<https://learn.microsoft.com/en-us/azure/frontdoor/end-to-end-tls>
- For secure access between Azure Front Door and your internal Azure resources and to keep traffic inside the Azure backbone, use Azure Private Link, as explained here:
<https://learn.microsoft.com/en-us/azure/frontdoor/private-link>
- To allow Azure Front Door access to Azure resources, configure managed identity inside Microsoft Entra ID, as explained here:
<https://learn.microsoft.com/en-us/azure/frontdoor/managed-identity>
- Enable diagnostic settings for Azure Front Door and send all logs to a Log Analytics workspace, as explained here:
<https://learn.microsoft.com/en-us/azure/frontdoor/standard-premium/how-to-logs>

- Use Azure Monitor to view real-time WAF-related activity for further analysis, as mentioned here:
<https://learn.microsoft.com/en-us/azure/frontdoor/standard-premium/how-to-monitor-metrics>
- Mask sensitive data inside the Azure Front Door logs, using log scrubbing, as explained here:
<https://learn.microsoft.com/en-us/azure/frontdoor/standard-premium/how-to-protect-sensitive-data>
- Use Microsoft Sentinel to be able to investigate WAF-related activities, as explained here:
<https://learn.microsoft.com/en-us/azure/web-application-firewall/waf-new-threat-detection>

Using Google Cloud Armor (as a managed WAF)

Google Cloud Armor is a Google-managed WAF and DDoS protection service.

It comes in two price models:

- **Google Cloud Armor Standard:** This is a Layer 7 web application firewall. This license also includes preconfigured WAF rules for OWASP Top 10 protection.
- **Google Cloud Armor Enterprise:** This is a Layer 7 web application firewall. This license also includes WAF rules, policies, requests, and bot management.

Using Google Cloud Armor as a WAF service can protect external load-balancers, protocol forwarding, and VMs with public IP addresses. In this section, we will be focusing on Google Cloud Armor as a managed WAF service.

Use the following best practices for securing Google Cloud Armor as a managed WAF service:

- Configure security policies to filter incoming traffic to load-balancers, protocol forwarding, and VMs with public IP addresses, as explained here:
<https://cloud.google.com/armor/docs/configure-security-policies>
- For protection against common web-related attacks, use Google Cloud Armor preconfigured WAF rules, as documented here:
 - <https://cloud.google.com/armor/docs/waf-rules>
 - <https://cloud.google.com/armor/docs/rule-tuning>
- To create fine-tuned WAF rules to match your websites, configure custom rules, as explained here:
<https://cloud.google.com/armor/docs/rules-language-reference>

- Use IP addresses with whitelists to restrict access to resources behind Google Cloud Armor from specific IP addresses, as explained here:

`https://cloud.google.com/armor/docs/common-use-cases#limit-access-to-https`

- To protect against malicious bot activity against your websites, use Google Cloud Armor bot management and reCAPTCHA, as explained here:

- `https://cloud.google.com/armor/docs/bot-management`

- `https://cloud.google.com/recaptcha-enterprise/docs/waf-overview`

- For troubleshooting Google Cloud Armor WAF activity, enable verbose logging, as explained here:

`https://cloud.google.com/armor/docs/verbose-logging`

In this section, we have learned how to use managed WAF services from AWS, Azure, and GCP, covering everything from rule configuration to monitoring.

Summary

In this chapter, we have focused on the various network services offered by AWS, Azure, and Google. These include DDoS protection and managed WAF services. In each section, we reviewed the best practices for configuring protection rules, monitoring, and threat detection.

In the next chapter, we will review how to embed security in generative AI-managed services.

Securing Generative AI Services

In previous chapters, we covered fundamental cloud services, from computing to storage and networking.

In this chapter, we will cover an emerging topic, **generative AI** (also known as **GenAI**), and we will learn how to use the fundamental cloud services to secure GenAI workloads. We will not be focusing on AI or machine learning in general, but on how to secure the infrastructure for running GenAI models.

We will also learn about services from AWS, Azure, and GCP, for running GenAI models and provide you with best practices of how to securely deploy and manage each of them.

In this chapter, we will cover the following topics:

- Introduction to GenAI
- Introduction to managed GenAI services
- Best practices for identity and access management in GenAI services
- Best practices for data protection in GenAI services
- Best practices for auditing and monitoring in GenAI services

Introduction to GenAI

Before we begin the discussion about security in GenAI services, let's review the terminology in this domain.

GenAI is a kind of AI that can create new content and ideas. This includes things such as conversations, stories, images, videos, and music.

Foundation models

Foundation models (FMs) are machine learning models that learn from a wide variety of general and unlabeled data. They use the patterns and connections they've discovered to guess what comes next in a sequence.

Large language models

Large language models (LLMs) are a type of foundation model that focuses on tasks related to language. They are designed to handle activities such as summarizing text, generating new text, classifying information, having open-ended conversations, and extracting information.

The following diagram shows the correlation between GenAI, FMs, and LLMs:

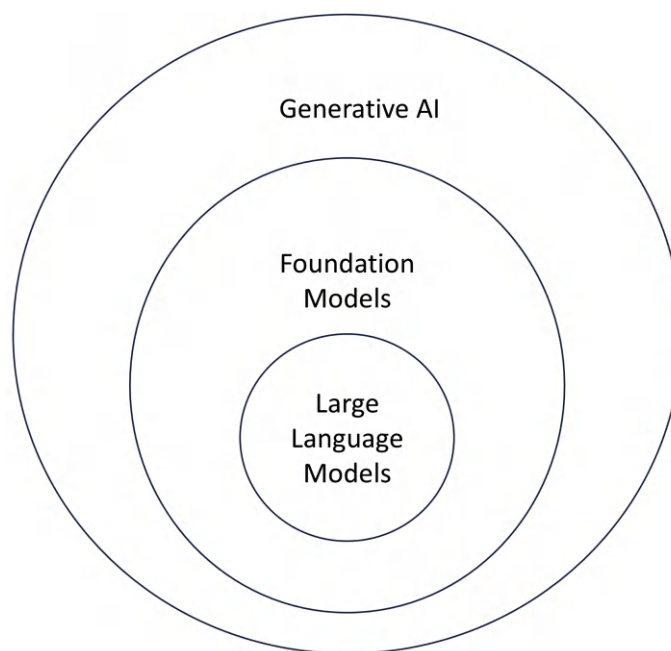


Figure 8.1 – GenAI, FMs, and LLMs

Prompt

A prompt is input supplied by a user (such as a data scientist) to a GenAI model/application to get a response or output from the model.

For example, you could send a query to an API endpoint of a GenAI service to return insights about sales data from the past quarter.

Response

A response is output sent from a GenAI model/application to a user.

Example of a GenAI workflow

The following is an example workflow for connecting and using GenAI services, based on an AWS infrastructure (Azure and GCP might use a similar architecture). A user sends a request, such as “provide a summary of document X,” the GenAI application searches for the document in its database, and, using the LLM module, generates a summary, which is returned to the user:

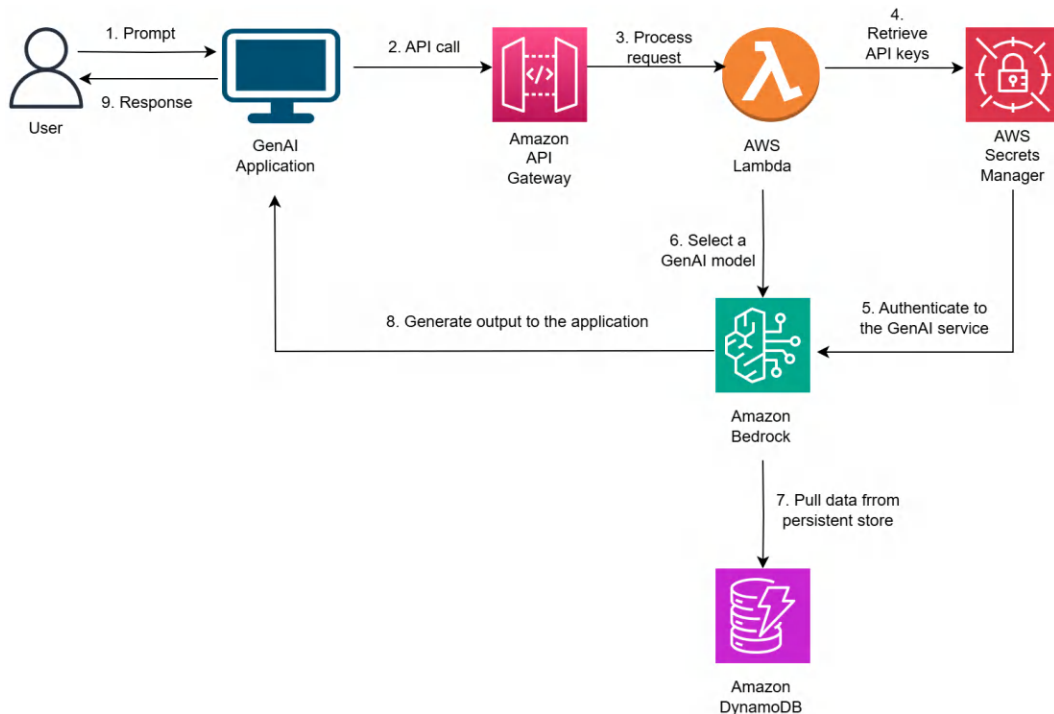


Figure 8.2 – Example GenAI application architecture

Let’s go through the steps of this workflow:

1. A **User** connects to a **GenAI application** by sending a prompt.
2. The **GenAI application** sends an **API call** to **Amazon API Gateway**.
3. **Amazon API Gateway** sends a request to a **Lambda** function, which acts as the business logic of the application.
4. **AWS Lambda** sends an API call to AWS Secrets Manager to retrieve the **API keys**.
5. **AWS Lambda** uses the **API keys** to authenticate to **Amazon Bedrock**.

6. **AWS Lambda** connects to **Amazon Bedrock** to select a GenAI model.
7. Amazon Bedrock pulls data from **Amazon DynamoDB**, which acts as a persistent data store that contains the document source used by the **GenAI application**.
8. **Amazon Bedrock** sends an output (the document summary) to the **GenAI application**.
9. The **GenAI application** returns a response to the **User**.

Now that we understand at a high level how GenAI services work, let's move on and review managed GenAI services.

Introduction to managed GenAI services

Each cloud provider implements managed services for running GenAI models, selecting FMs or LLMs, and building GenAI applications.

When you use GenAI services, you need API endpoints that receive prompts (i.e., API calls) from users (or from applications), persistent storage (storing large amounts of data), and FMs or LLMs for generating new content (new images, document summaries, chat bots, etc.).

Considerations for using data to train managed GenAI services

Before you decide to use a managed GenAI service to train GenAI models using data and gain insights, it is important to be able to ask the following questions:

- What data are you planning to use for the training (i.e., is it public data, business-related data, personally identifiable information, healthcare information, financial information, etc.)?
- What is the data classification (i.e., is it public, internal, confidential, highly confidential, etc.)?
- Is the CSP using customers' data to train its model to achieve better results?
- How has the data been protected on the CSP's side (i.e., is the data encrypted, is there a mechanism to control authorization, is an audit trail enabled for the service, etc.)?
- Are there any local laws or regulations that allow or prohibit the use of AI/machine learning services on identifiable data?
- Does the managed GenAI service provide guardrails that will decrease the chance of unwanted output (such as exposing customer data, producing harmful content, or revealing internal business information)?

These questions will allow customers to decide whether they can trust a CSP to train GenAI models (which may contain sensitive business or customer information).

Risks related to the use of GenAI

Before organizations begin using GenAI technology, they need to understand that there are risks involved in this technology.

Among the GenAI risks, there are the following:

- **Misinformation:** Generating false information
- **Bias:** Generating unfair treatment to certain groups in the populations (such as favoring men over women or certain skin colors)
- **Intellectual property:** Generating content based on copyrighted material
- **Privacy:** Generating content based on certain human private information
- **Security vulnerability:** Generating malicious content that will be exploited on someone's computer
- **Regulatory challenges:** Generating content that violates data protection regulations (such as allowing a US citizen to receive information about EU citizens)
- **Misuse:** Generating harmful content (such as hate speech, harassment, etc.)
- **Ethical:** Generative AI can emphasize bias and stereotypes, which may result in unfair or harmful portrayals of race, religion, culture, social class, sex, and gender

High-level best practices for securing GenAI applications

Here are some general best practices for mitigating risks in GenAI applications:

- Implement input validation for all prompts received from all clients to prevent malicious content from being inserted into the GenAI models or into a backend database.
- Implement output validation for all responses returned from the GenAI models to clients to prevent data exfiltration or harmful content.
- Regularly audit and clean training data to remove malicious input.
- Implement data minimization by providing only necessary data to GenAI services.
- Use anonymization and tokenization techniques for sensitive data.
- Implement **role-based access control (RBAC)**, based on the principle of least privilege, when providing access to a GenAI application. Regularly review and update access permissions.
- Log all GenAI interactions for audit purposes and continuously monitor interactions with GenAI services to detect suspicious activity.
- Establish an AI incident response plan with clear steps for identifying, containing, and mitigating incidents.

- Use multiple AI models for layered security, combining generative and discriminative models.
- Regularly audit vendors to ensure compliance with regulations and security standards.
- When embedding open-source binaries or libraries as part of your GenAI application code, perform a regular scan to detect vulnerabilities in open-source code.
- Perform static and dynamic code analysis of your GenAI application code to detect vulnerabilities in your code (such as vulnerabilities to various injection attacks and cross-site scripting). Integrate code scanning as part of the development and deployment processes.
- Implement automated and manual secure code review of all code related to GenAI so you can detect potential vulnerabilities and remediate them before they impact your customers.
- When exposing a GenAI application or GenAI API endpoint to the public internet, protect inbound traffic using DDoS protection and WAF solutions to decrease the risk of network or application layer attacks.
- Continuously monitor inbound traffic to your GenAI application or API endpoints to detect abnormal inbound access requests or spikes.

Here are some additional references:

- OWASP Top 10 for LLM and GenAI (the most critical vulnerabilities related to LLMs):
 - <https://genai.owasp.org/>
 - <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- Guidelines for secure AI system development:
<https://www.ncsc.gov.uk/files/Guidelines-for-secure-AI-system-development.pdf>
- NIST Artificial Intelligence Risk Management Framework:
<https://www.nist.gov/itl/ai-risk-management-framework>

Next, we will review the different offerings of GenAI services from AWS, Azure, and GCP.

Amazon Bedrock

Amazon Bedrock is a fully managed service for building GenAI services based on FMs from leading AI companies (such as Amazon Titan, Anthropic Claude, Cohere, Mistral AI, and more).

Read thorough documentation on security in Amazon Bedrock, which can be found at the following URL:

<https://docs.aws.amazon.com/bedrock/latest/userguide/security.html>

Azure OpenAI

Azure OpenAI is a fully managed service for building GenAI services based on OpenAI models (such as GPT, DALL-E, and more).

Read thorough documentation on security in Azure OpenAI, which can be found at the following links:

- <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/azure-openai-security-baseline>
- <https://learn.microsoft.com/en-us/azure/ai-services/security-features>
- <https://learn.microsoft.com/en-us/azure/ai-services/openai/how-to/use-your-data-securely>

Google Vertex AI

Google Vertex AI is a fully managed service for building GenAI services based on FMs from leading AI companies (such as Google Gemini, Anthropic Claude, Meta Llama, and more).

Read thorough documentation on security in Google Vertex AI, which can be found here:

- <https://cloud.google.com/vertex-ai/generative-ai/docs/learn/responsible-ai>
- <https://cloud.google.com/vertex-ai/generative-ai/docs/access-control>

Next, we will look at how to configure IAM in GenAI services, based on AWS, Azure, and GCP.

Best practices for IAM in GenAI services

Identity and access management is a crucial part when using GenAI services for the following reasons:

- **Data confidentiality:** Many organizations choose GenAI services to train and query models against their internal business or even customers' data, which requires the enforcement of authorization mechanisms.
- **Cost:** The use of GenAI services is not free, and on a production scale, it can be very expensive due to model licensing costs and dedicated hardware, such as GPUs.
- **Abuse and malicious use:** Not all GenAI services are used for legitimate purposes. Unauthorized access to a managed GenAI service can result in data exfiltration or service abuse, which will result in high costs for the organization.

Here are some general best practices for securing IAM in GenAI services:

- Understand which options are available for enforcing authentication and authorization to a GenAI application or managed services
- Enforce authorization for each access request to a GenAI service and follow the principle of least privilege to prevent unauthorized access

Next, we will review how to implement IAM in GenAI services by AWS, Azure, and GCP.

Amazon Bedrock

AWS IAM is a supported service for managing permissions to access Amazon Bedrock.

Read thorough documentation on IAM in Amazon Bedrock, which can be found here:

https://docs.aws.amazon.com/bedrock/latest/userguide/security_iam_service-with-iam.html

Use the following best practices for authentication and authorization in Amazon Bedrock:

- Use identity-based policies to grant the required permissions to Amazon Bedrock and prevent unauthorized access, as explained here:
 - https://docs.aws.amazon.com/bedrock/latest/userguide/security_iam_id-based-policy-examples.html
 - https://docs.aws.amazon.com/bedrock/latest/userguide/security_iam_id-based-policy-examples-agent.html
- Configure an IAM service role for the Amazon Bedrock service to grant Bedrock the required permissions to access AWS resources on behalf of a client or application (such as access to Amazon S3 or access to invoke a Lambda function), as explained here:

<https://docs.aws.amazon.com/bedrock/latest/userguide/security-iam-sr.html>

- Use Amazon API Gateway to authenticate incoming API requests to Amazon Bedrock and protect from unauthorized network access, as explained here:

<https://aws.amazon.com/solutions/guidance/generating-product-descriptions-with-amazon-bedrock/>

Azure OpenAI

Azure controls authorization to Azure OpenAI using Microsoft Entra ID. Permissions are managed using Azure RBAC.

Read thorough documentation on IAM in Azure OpenAI, which can be found here:

<https://learn.microsoft.com/en-us/azure/ai-services/authentication>

Use the following best practices for authentication and authorization in Azure OpenAI:

- Regularly rotate the built-in API keys that come with Azure OpenAI to prevent misuse and unauthorized access, as explained here:

<https://learn.microsoft.com/en-us/azure/ai-services/rotate-keys>

- Use managed identities to grant Azure services (such as Azure VMs and Azure Functions) access to Azure OpenAI, as explained here:

<https://learn.microsoft.com/en-us/azure/ai-services/openai/how-to/managed-identity>

- When granting access to Azure OpenAI, follow the principle of least privilege to prevent unauthorized access, as explained here:

<https://learn.microsoft.com/en-us/azure/ai-services/openai/how-to/role-based-access-control>

- If an application built using Azure OpenAI requires access to other resources using static credentials (such as secrets), use Azure Key Vault for storing, retrieving, and rotating static credentials to prevent unauthorized access, as explained here:

<https://learn.microsoft.com/en-us/azure/ai-services/use-key-vault>

- Use Azure API Management to authenticate incoming API requests to Azure OpenAI and protect from unauthorized network access, as explained here:

- <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/azure-openai-gateway-guide>

- <https://learn.microsoft.com/en-us/azure/api-management/api-management-authenticate-authorize-azure-openai>

Google Vertex AI

Google Cloud IAM is the supported service for managing permissions to Google Vertex AI.

Read thorough documentation on IAM in Google Vertex AI, which can be found here: <https://cloud.google.com/vertex-ai/docs/general/access-control>

Use the following best practices for authentication and authorization in Google Vertex AI:

- Use IAM policies to grant permissions to Vertex AI resources and endpoint, as documented here:
 - <https://cloud.google.com/vertex-ai/docs/general/iam-permissions>
 - <https://cloud.google.com/vertex-ai/docs/general/endpoint-access-control>
- Create a unique identity (a service account) for each newly created GenAI application created using Vertex AI with minimal permissions using the Google Cloud IAM service to prevent unauthorized access by Vertex AI to GCP resources, as explained here:

<https://cloud.google.com/vertex-ai/docs/general/custom-service-account>

- If an application built using Google Vertex AI requires access to other resources using static credentials (such as secrets), use Google Secret Manager for storing, retrieving, and rotating static credentials to prevent unauthorized access, as explained here:

<https://cloud.google.com/vertex-ai/docs/pipelines/secret-manager>

- Use Google Apigee to authenticate incoming API requests to Google Vertex AI and protect from unauthorized network access, as explained in the reference documentation here:

<https://cloud.google.com/apigee/docs/api-platform/integration/gcp-tasks/configure-vertex-ai-predict-task>

In this section, we have learned how to configure IAM in GenAI services, based on AWS, Azure, and GCP infrastructure. Next, we will review data protection in GenAI services.

Best practices for data protection in GenAI services

GenAI is all about data, which is why it is important to protect the data as it traverses from the client, through the GenAI services, to the backend persistent store, and back with a response to the client.

Here are some general best practices for data protection in GenAI services:

- Enforce encryption in traffic and at rest for all data sent to and retrieved from GenAI services
- Keep all traffic to the GenAI services private within the CSP backbone
- Use guardrails mechanisms to filter responses retrieved from GenAI models, to avoid abuse or harmful content
- For highly sensitive environments, deploy GenAI applications on top of virtual machines or container infrastructure that supports confidential computing capabilities to protect data in use

Amazon Bedrock

Because Amazon Bedrock is a managed service, it is located outside the customer's VPC. It is important to protect access to and from Amazon Bedrock.

Use the following best practices for data protection in Amazon Bedrock:

- To secure access from your VPC to Amazon Bedrock, use **AWS PrivateLink**. This approach creates a private communication channel that routes traffic exclusively through the AWS internal network. By utilizing an interface VPC endpoint, you can ensure that data remains within the AWS infrastructure, enhancing security and privacy, as explained here:

<https://docs.aws.amazon.com/bedrock/latest/userguide/usingVPC.html>

- Encrypt all data at rest using AWS KMS (as explained in *Chapter 11, Applying Encryption in Cloud Services*). For sensitive environments containing business or customer data, encrypt data at rest using customer-managed keys, as explained here:

<https://docs.aws.amazon.com/bedrock/latest/userguide/encryption-custom-job.html>

- For highly sensitive environments (containing business or customer data), deploy GenAI applications on instances powered by AWS Nitro System (more on this in *Chapter 11*), as explained here:

<https://aws.amazon.com/ec2/nitro/>

- Use Guardrails for Amazon Bedrock to filter the response and block harmful or abusive content, as explained here:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html>

Azure OpenAI

Because Azure OpenAI is a managed service, it is located outside the customer's VNet. It is important to protect access to and from Azure OpenAI.

Use the following best practices for data protection in Azure OpenAI:

- To secure access from your VNet to the Azure OpenAI, use an Azure private endpoint, which prevents sending network traffic outside your VNet through a secure channel, as explained here:

<https://learn.microsoft.com/en-us/azure/ai-services/cognitive-services-virtual-networks>

- Encrypt all data at rest using Azure Key Vault (as explained in *Chapter 11*). For sensitive environments containing business or customer data, encrypt data at rest using customer-managed keys, as explained here:

<https://learn.microsoft.com/en-us/azure/ai-services/openai/encrypt-data-at-rest>

- For highly sensitive environments (containing business or customer data), deploy GenAI applications on instances that support Azure Confidential Computing, as explained here:

<https://learn.microsoft.com/en-us/azure/confidential-computing/overview>

- Use Azure AI Content Safety to filter the response and block harmful or abusive content, as explained here:

<https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/content-filter>

- To prevent data exfiltration from Azure OpenAI, enable data loss prevention to restrict outbound URLs, as explained here:

<https://learn.microsoft.com/en-us/azure/ai-services/cognitive-services-data-loss-prevention>

Google Vertex AI

Because Google Vertex AI is a managed service, it is located outside the customer's VPC. It is important to protect access to and from Google Vertex AI.

Use the following best practices for data protection in Google Vertex AI:

- To restrict access to Google Vertex AI from trusted sources and reduce the risk of data exfiltration, use VPC Service Controls, as explained here:

<https://cloud.google.com/vertex-ai/docs/general/vpc-service-controls>

- To keep traffic from your VPC to Google Vertex AI API private, use Private Service Connect, as documented here:

<https://cloud.google.com/vertex-ai/docs/general/googleapi-access-methods>

- Encrypt all data at rest using Google Cloud KMS (as explained in *Chapter 11*). For sensitive environments containing business or customer data, encrypt data at rest using customer-managed encryption keys, as explained here:

<https://cloud.google.com/vertex-ai/docs/general/cmek>

- For highly sensitive environments (containing business or customer data), deploy GenAI applications on instances that support Google Confidential Computing, as explained here:

<https://cloud.google.com/confidential-computing/confidential-vm/docs/confidential-vm-overview>

- Use safety filters and attributes to filter the response and block harmful or abusive content, as explained here:

<https://cloud.google.com/vertex-ai/generative-ai/docs/learn/responsible-ai>

In this section, we have learned how to configure data protection in GenAI services – based on AWS, Azure, and GCP infrastructures – including private network access, data encryption, and guardrails, to prevent abuse or harmful content.

Next, we will review the auditing and monitoring of GenAI services.

Best practices for auditing and security monitoring in GenAI services

Auditing is a crucial part of data protection.

There are many topics to review when talking about auditing and monitoring in GenAI services. Here are two of them:

- **User’s activity logs:** To identify abuse attempts against a GenAI service (such as attempts to exfiltrate data from a model)
- **GenAI admin activity:** To detect rogue admin activity (such as disabling guardrails, which may result in customers receiving harmful content as a response from a model)

Follow the best practices here for auditing and monitoring in GenAI services:

- Although admin activity is automatically audited by the CSPs, its log history is limited. As best practice, send all audit and monitoring logs to central storage for further analysis, for incident response, and to comply with various regulations.
- Add threat detection solutions to review the logs and generate prioritized follow-up actions.

Amazon Bedrock

As with any other managed service, AWS allows you to enable logging and auditing using two built-in services:

- **Amazon CloudWatch:** This is a service that allows you to log Amazon Bedrock activity and raise the alarm when predefined activities are detected, such as excessive invocation from specific clients or excessive client-side errors
- **AWS CloudTrail:** This is a service that allows you to monitor API activities (essentially, any action performed on Amazon Bedrock)

Read thorough documentation on auditing and monitoring in Amazon Bedrock, which can be found here:

<https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring.html>

Use the following best practices for auditing and monitoring in Amazon Bedrock:

- Use Amazon CloudWatch alarms to generate notifications about abnormal activities (such as multiple client-side errors or the use of legacy models), as documented here:

<https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-cw.html>

- AWS CloudTrail is enabled by default for Amazon Bedrock. To keep Amazon Bedrock API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here:

<https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html>

- Use Amazon GuardDuty to detect threats in Amazon Bedrock APIs such as abnormal user login hours and the removal of Bedrock guardrails, as explained here: <https://docs.aws.amazon.com/guardduty/latest/ug/ai-protection.html>

Azure OpenAI

Azure allows you to monitor Azure OpenAI using the following services:

- **Azure Monitor:** This service logs access and audit events from Azure OpenAI
- **Threat protection for AI workloads:** This service (part of Microsoft Defender for Cloud), allows you to detect potential threats (such as data leakage and credential theft) in the Azure OpenAI applications

Use the following best practices for auditing and monitoring in Azure OpenAI:

- Enable diagnostic settings for Azure OpenAI and send all logs to a Log Analytics workspace, as explained here:

<https://learn.microsoft.com/en-us/azure/ai-services/openai/how-to/monitoring>

- Enable threat protection for AI workloads in Microsoft Defender for Cloud to identify potential threats in GenAI applications, as explained here:

<https://learn.microsoft.com/en-us/azure/defender-for-cloud/ai-onboarding>

Google Vertex AI

As with any other managed service, Google allows you to enable logging and auditing using the Google Cloud Audit Logs – a service that allows you to audit Vertex AI activities.

Follow these best practices for auditing and monitoring in Google Vertex AI:

- Use Google Cloud Monitoring to monitor the activity of Google Vertex AI, as explained here:

<https://cloud.google.com/vertex-ai/docs/general/monitoring-metrics>

- For sensitive/critical environments, enable data access audit logs (for actions such as listing datasets and getting endpoints), as explained here:

<https://cloud.google.com/vertex-ai/docs/general/audit-logging>

Note

Data access audit logs increase storage costs. Review the business requirements to enable them.

- Use Google Security Command Center to detect and send alerts about AI-related misconfigurations (such as unencrypted datasets and the use of the root user), as mentioned here:

<https://cloud.google.com/security-command-center/docs/security-posture-essentials-secure-ai-template>

In this section, we have learned how to configure auditing and monitoring in GenAI services based on AWS, Azure, and GCP infrastructure, from service monitoring to enabling audit trails for API activities, and finally threat management.

Summary

In this chapter, we have focused on securing GenAI services in AWS, Azure, and GCP.

In each section, we have reviewed how to manage identity management (for authentication and authorization), data protection (such as encryption, network connectivity, and filtering abusing content), and finally, how to configure auditing and logging.

In the next chapter, we will learn how to configure identity and access management in the cloud.

Part 2:

Deep Dive into IAM, Auditing, and Encryption

Securing cloud environments starts with managing identity, monitoring activity, and protecting data. **Identity and Access Management (IAM)** controls who can access resources, auditing helps detect threats, and encryption keeps sensitive information safe. This part explores these key security measures to help organizations strengthen their cloud protection.

This part comprises the following chapters:

- *Chapter 9, Effective Strategies for Implementing IAM Solutions*
- *Chapter 10, Auditing and Threat Management in Cloud Environments*
- *Chapter 11, Applying Encryption in Cloud Services*

Effective Strategies for Implementing IAM Solutions

From *Chapter 2, Securing Compute Services – Virtual Machines* to *Chapter 8, Securing Generative AI Services*, we covered the fundamental building blocks of cloud services (from compute, storage, and networking to generative AI services). However, we cannot talk about cloud services without also discussing identity management.

In this chapter, we will cover various concepts of identity management – from traditional directory services for hybrid environments (based on the Kerberos protocol in the Microsoft environment) to modern cloud-based directory services (based on the SAML or OAuth protocols).

In this chapter, we will cover the following topics:

- Introduction to IAM
- Securing cloud-based IAM services
- Securing directory services

Introduction to IAM

Identity management (IdM) refers to the concept of managing the entire user identity life cycle, including provisioning (that is, account creation), access governance (that is, assigning permissions), authentication and authorization, monitoring and auditing, and, finally, account de-provisioning (that is, when a person leaves the organization, or the account is no longer in use).

Identity and access management (IAM) is made up of the following main concepts:

- **Identity:** This indicates a user, computer, service, or role that wishes to access a system or application and take actions (from accessing shared storage to querying a database and pulling information)

- **Authentication:** This is the process of proving that a specific identity (such as a user) is who they claim to be (for example, providing a username and password that match the user's records on a central user repository)
- **Authorization:** This is the process of granting an authenticated identity (such as a user) permission to take actions on a resource (such as allowing a user to upload a file to an object storage)

Cloud providers take a different approach to IAM, and specifically, to authorization:

- **AWS IAM:** By default, all requests are implicitly denied. This means that if access is not specifically allowed, it will be denied, as explained in detail here:

https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html#policy-eval-denyallow

- **Microsoft Entra ID (formerly Azure AD):** By default, users have a minimal set of permissions to access resources, such as enumerating users in the directory, viewing group attribute properties, and so on, as explained here:

<https://learn.microsoft.com/en-us/entra/fundamentals/users-default-permissions>

- **Google Cloud IAM:** By default, service accounts (a special type of identity for granting services access to GCP resources) have permission to call Google Cloud APIs or the *Editor* role (which allows actions such as creating or deleting resources), as explained here:

<https://cloud.google.com/iam/docs/service-account-types#default>

Before we move on to IdM, let us look at a few examples of failing to manage identities.

Failing to manage identities

Here are a few examples of what can go wrong when we fail to manage identities:

- A new employee has joined an organization. Since the organization does not have a documented and automated IdM workflow, the employee's permissions were copied from a system administrator's profile. The new employee was not aware of the sites they should browse on the internet, and, as a result, they were infected by ransomware. This, in turn, infected the sales database, which was encrypted, and caused downtime for the sales division.
- An employee was able to create an easy-to-guess password because a complex password policy was not enforced at the organizational level. As a result, a hacker was able to guess the employee's password and gain access to a confidential financial report.
- An employee has changed their role in the organization from IT to the development team and kept their privileges to production servers. As a result, the employee now has over-privileges, and they can access and make changes to the production system.

- An employee has left the organization and their account remains enabled. As a result, the employee had access to the corporate CRM system through a VPN, which they used to export the contact details of customers so that they could sell the products of their new employer.

Identities in cloud environments

There are two types of identity when working with cloud environments:

- **Cloud-native identity:** An identity of a persona (such as user, role, application account, etc.) that was created in a cloud-based IAM system and authenticates using modern authentication protocols such as **Security Assertion Markup Language (SAML)**, **OpenID Connect (OIDC)**, or **Open Authorization (OAuth) 2.0**
- **Hybrid identity:** An identity of a persona (such as a user or an application account) that was created on-premises (for example, within Microsoft Active Directory) was synched through federation to a cloud-based IAM system and can authenticate using Kerberos (for legacy systems) or using modern authentication protocols (such as SAML, OIDC, OAuth2, etc.)

Now that we've learned about the types of identity in the cloud, let us talk about the authentication process in the cloud.

Authentication process

During an authentication process, a person proves their identity.

Authentication can happen in various ways:

- **Human authentication** (also known as interactive authentication) using credentials (such as usernames, passwords, and perhaps multi-factor authentication)
- **Non-human authentication** (or non-interactive authentication), such as application-to-application authentication, machine authentication, service account authentication, and so on
- To perform non-interactive authentications, it is possible to use static credentials (such as passwords, secrets, API keys, certificates, etc.)

Authorization process

In the authorization process, we assign a persona with permission to access a resource or a system and take actions. Authorization can be done against a cloud-based service (such as object storage, a database, or an API call) or to an application (such as a SaaS application).

Actions can be anything from an end user who needs view permissions to file metadata, an application service account that needs read and update permissions to a customer database, all the way to an administrator who has full permissions to deploy a new system.

There are two ways to assign permissions to a resource or a system:

- **Role-based access control (RBAC):** A mechanism to control permissions based on a persona's requirements to take action
- **Attribute-based access control (ABAC):** A mechanism to control permissions based on a specific persona's attribute (such as group membership, IP address, whether using MFA or not, etc.)

Next, we will learn how to secure cloud-based IAM services.

Securing cloud-based IAM services

Each cloud provider has its implementation of managed IAM services, in other words, a service for managing authentication and authorization requests.

General best practices for securing IAM services are as follows:

- Enforce authentication for any identity who needs access to resources for authorization and audit purposes.
- When granting permissions to resources, follow the principle of least privilege, (i.e., grant minimal permissions to achieve only the tasks they are required to do).
- Regularly review all identities in your IAM service and disable or delete identities that have not been in use for more than 90 days. This will allow you to revoke accessible or unneeded permissions to your services.
- Prefer to use temporary credentials using roles over static credentials. Temporary credentials allow users and applications a minimal number of permissions to achieve their requirements while decreasing the chance of credentials reuse by unauthorized parties.
- For systems that use static credentials (such as passwords, secrets, access keys, etc.), regularly rotate the static credentials to prevent credential reuse by unauthorized parties.
- For systems that use passwords for human (interactive) authentication, enforce the use of MFA for all users to secure the authentication process and decrease the chance of unauthorized access.
- For scenarios where credentials are based on secrets, access keys, API keys, and so on, avoid storing static credentials as part of code, configuration files, or any other clear-text location, and enforce the use of a secret manager service to decrease the chance of credentials reuse by unauthorized parties (as explained in *Chapter 11, Applying Encryption in Cloud Services*).

Securing AWS IAM

AWS IAM is an Amazon-managed IAM service. It allows you to create users, assign users to groups (for better maintainability of access to resources), assign IAM roles, and configure permissions (using IAM policies).

Here is a list of AWS IAM terminology:

- **Root user:** This is the equivalent of the built-in administrator account with full privileges over the entire AWS account.
- **IAM user:** This is a person or application with permission to access AWS resources. An IAM user usually has credentials (such as a password, access keys, and MFA) unless the user was created using AWS CLI or the AWS API in scenarios where you want to set up the user's identity first and add credentials later based on specific needs.
- **IAM group:** This refers to a group of IAM users to make the permissions management task easier.
- **IAM role:** This indicates an identity that has permission to access resources without any credentials. Usually, you assign an IAM role to an IAM group, IAM user, or a service account that requires temporary permissions.
- **Service account:** This refers to a special type of IAM user, and its purpose is to allow applications to have access to resources, such as IAM roles, which provide temporary security credentials for applications or AWS services to make API calls, or assumed roles, where applications can assume IAM roles to obtain temporary security credentials, similar to how service accounts work in other platforms.
- **IAM policy:** This is a JSON-based definition that sets the permissions for accessing AWS resources. There are two types of IAM policies:
 - **Identity-based policies:** This is a policy attached to a user, group, or role
 - **Resource-based policies:** This is a policy attached to an AWS resource (for example, an Amazon S3 bucket)
- **Identity provider:** This refers to the ability to manage identities outside of AWS while still being able to grant access to AWS resources to the external identities (such as a federation between AWS and Microsoft Entra ID).

AWS IAM evaluates the permissions to a resource, using the following logic:

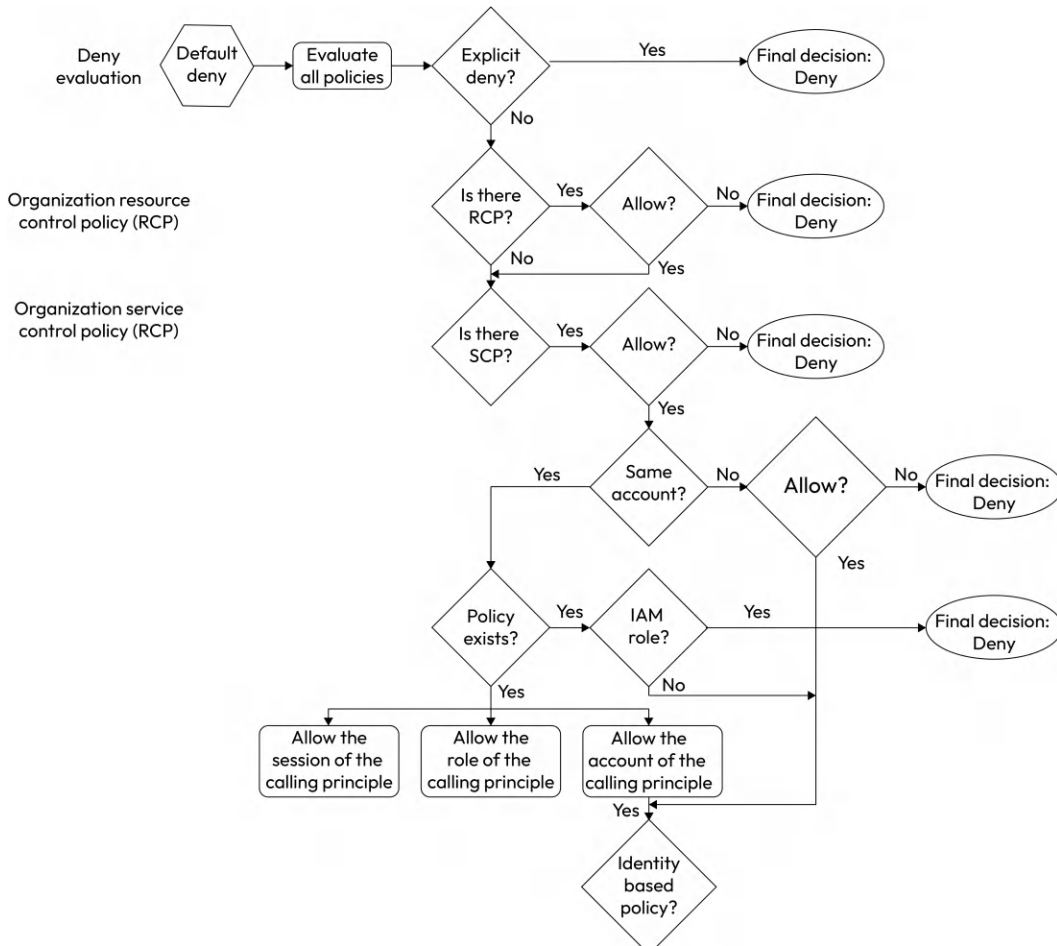


Figure 9.1 – AWS IAM permission evaluation logic – Part 1 (deny policy, RCP, and SCP)

Here is a high-level explanation of the policy evaluation logic:

1. Review the request and make sure there is no implicit denial.
2. Review AWS Organizations **resource control policies (RCPs)** and check whether access to a resource was granted or denied. For more information about AWS RCPs, read *Chapter 17, Security in Large-Scale Environments*.
3. Review AWS Organizations **service control policies (SCPs)** and check whether access for an IAM user or IAM role to a resource was granted or denied. For more information about AWS SCPs, read *Chapter 17*.

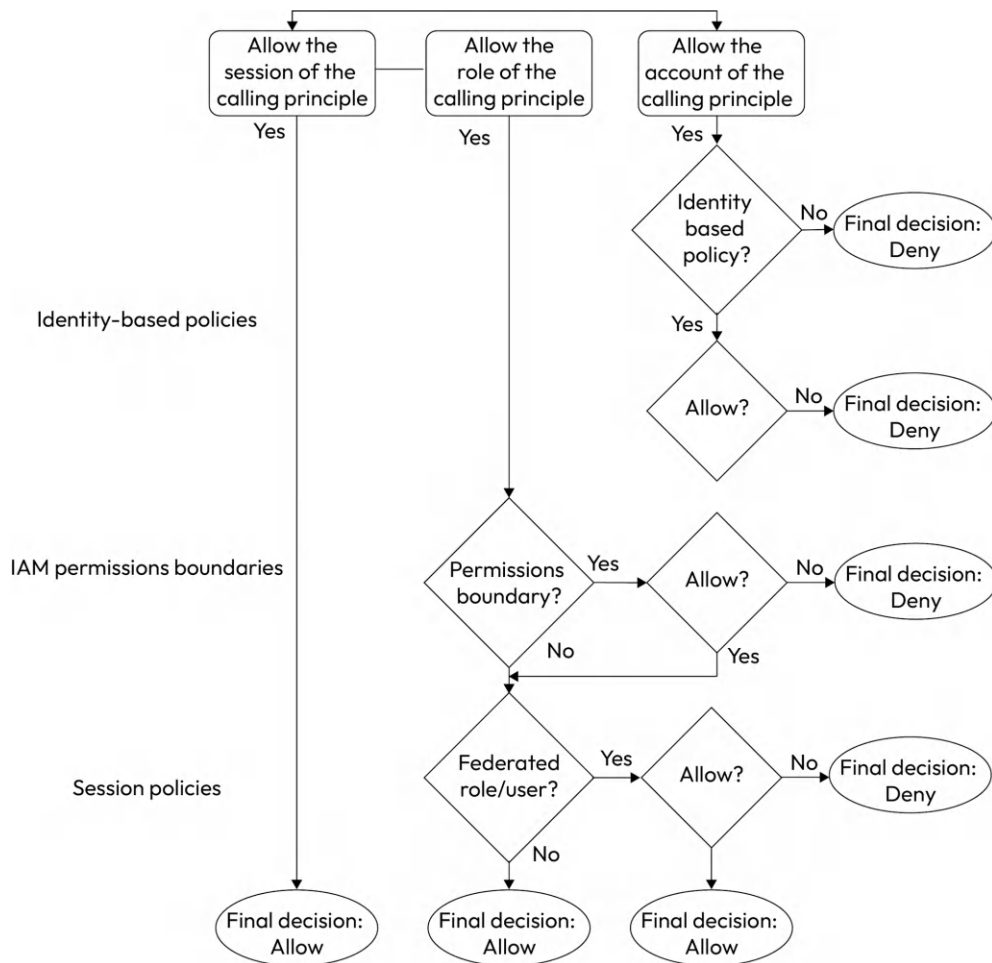


Figure 9.2 – AWS IAM permission evaluation logic – Part 2
(Identity-based policies, IAM permissions, and session policies)

4. Review **Identity-based policies** and evaluate whether the identity (such as a user or a role) has specific allow or deny permission.
5. Review **IAM permissions boundaries** and evaluate the maximum allowed permissions granted to an IAM identity.
6. Review **Session policies** and evaluate whether a federated IAM user/IAM role is restricted or not to access a resource.
7. In the event of no restriction, access to the target resource is granted.

Read thorough documentation on the security of the AWS root user, which can be found here: <https://docs.aws.amazon.com/IAM/latest/UserGuide/root-user-best-practices.html>.

Use the following best practices to secure the AWS root user:

- Avoid using the AWS root user to decrease the chance of a breach in your cloud environment. Instead, create a random password for the AWS root account, and in the rare scenarios where a root account is required, reset the root account password.
- Configure a strong password for the AWS root user to decrease the chance of reuse of the root user's credentials.
- Enable MFA for the AWS root user to decrease the chance of reuse of the root user's credentials.
- Disable and remove all access keys and secret access keys from the AWS root user to decrease the chance of reuse of the root user's credentials.
- Create an IAM user with full privileges and use it instead of the original AWS root user. Protect the IAM *full-admin* user with a strong password and MFA to decrease the chance of a breach in your AWS account.
- Avoid creating access keys and secret access keys for an IAM user with a full IAM admin role.
- Enable auditing on any attempt to use the AWS root user and send alerts to the **Security Operations Center (SOC)** team for both successful or failed attempts for further investigation and analysis of AWS root user access attempts.

Read thorough documentation on security in AWS IAM, which can be found at the following links:

- <https://docs.aws.amazon.com/IAM/latest/UserGuide/security.html>
- <https://docs.aws.amazon.com/IAM/latest/UserGuide/IAMBestPracticesAndUseCases.html>

Use the following best practices to secure AWS IAM:

- Prefer to use **AWS Security Token Service (AWS STS)** to allow short-time access to resources, instead of static credentials, as explained here:

https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp.html

- Use IAM roles to assign permissions to IAM identities, using IAM policies. Follow the principle of least privilege when granting access to resources to avoid unauthorized access, as explained here:

https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies.html

- For AWS-native identities, use identity-based policies to grant permissions to resources, as explained here:

https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_identity-vs-resource.html

- Configure a password policy (which includes the minimum and maximum password age, the minimum password length, and enforces the use of password history and complex passwords) to decrease the chance of unauthorized access to your cloud environment, as explained here:

https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_passwords_account-policy.html

- Enforce MFA for all human (interactive) authentication to decrease the chance of unauthorized access to your cloud environment, as explained here:

https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_mfa.html

- Use IAM roles to allow applications that are deploying on EC2 instances access to AWS resources, as explained here:

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-roles-for-amazon-ec2.html>

- Avoid embedding access keys and secret access keys inside your code. Instead, use secret management solutions such as AWS Secrets Manager to store and retrieve access keys. Rotate access keys periodically to avoid key abuse by malicious internal or external users, as explained here:

https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_access-keys.html

- Use the AWS IAM **AssumeRole** capability to switch between multiple AWS accounts using the same IAM identity, as explained here:

https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html

- Use IAM permissions boundaries to restrict IAM user access to specific AWS resources, as explained here:

https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_boundaries.html

- Use IAM Access Analyzer to regularly scan for external or unused access permissions (including the use of secrets) to an AWS account and generate least-privilege IAM policies, as explained here:

<https://docs.aws.amazon.com/IAM/latest/UserGuide/what-is-access-analyzer.html>

- To secure access from your VPC to AWS STS, use AWS PrivateLink. This keeps traffic internally inside the AWS backbone, through a secure channel, using the interface VPC endpoint, as explained here:

https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_interface_vpc_endpoints.html

- AWS CloudTrail is enabled by default for AWS IAM. To keep IAM API activity for longer than 90 days, for further investigation and analysis, send CloudTrail logs to an Amazon S3 bucket, as explained here:

<https://docs.aws.amazon.com/IAM/latest/UserGuide/cloudtrail-integration.html>

- Use **AWS Security Hub** to manage the security posture of IAM activity in your entire AWS organization (i.e., collecting information from various sources to have an entire security view of your AWS environment), as explained here:

<https://docs.aws.amazon.com/securityhub/latest/userguide/security-iam.html>

Securing Microsoft Entra ID

Microsoft Entra ID (formerly Azure AD) is the Azure-managed directory service. It allows you to create users, assign them to groups (for better maintainability of access to resources), and configure permissions (using an RBAC mechanism).

Entra ID supports the following types of identity models:

- **Cloud-only identity:** The user account is created inside the Entra ID tenant and only exists in the cloud (where access to on-premises is not required)
- **Hybrid identity:** The user account exists in both the Entra ID tenant and on-premises

Here is a list of Entra ID terminology:

- **Microsoft Entra Global Administrator:** This is the built-in administrator account, with full privileges over the entire Azure tenant.
- **User:** This is a person or application with permission to access Azure resources. A user has credentials (such as a password with MFA, or a secret).
- **Group:** This is a group of users to make the permission management task easier.
- **Azure RBAC:** This is an authorization system with built-in permissions to access Azure resources (for example, *Owner*, *Contributor*, *Reader*, and more)
- **Managed Identity:** This refers to a special type of user, and its purpose is to allow applications to have access to resources. There are two types of managed identity:
 - **System-assigned:** This is a service principal attached to a specific Azure resource (such as an Azure VM)
 - **User-assigned:** This is a service principal designed for use with multiple Azure resources (such as Azure App Service)

- **Identity provider:** This refers to the ability to manage identities outside of Microsoft Entra tenant while still being able to grant access to Azure resources to the external identities (such as a federation between AWS and Microsoft Entra ID).
- **Azure Resource Manager:** This is a central management layer for Microsoft Azure cloud services. It provides a unified interface for creating, updating, and deleting resources across Azure's suite of services.

Azure evaluates the permissions to a resource using the following logic:

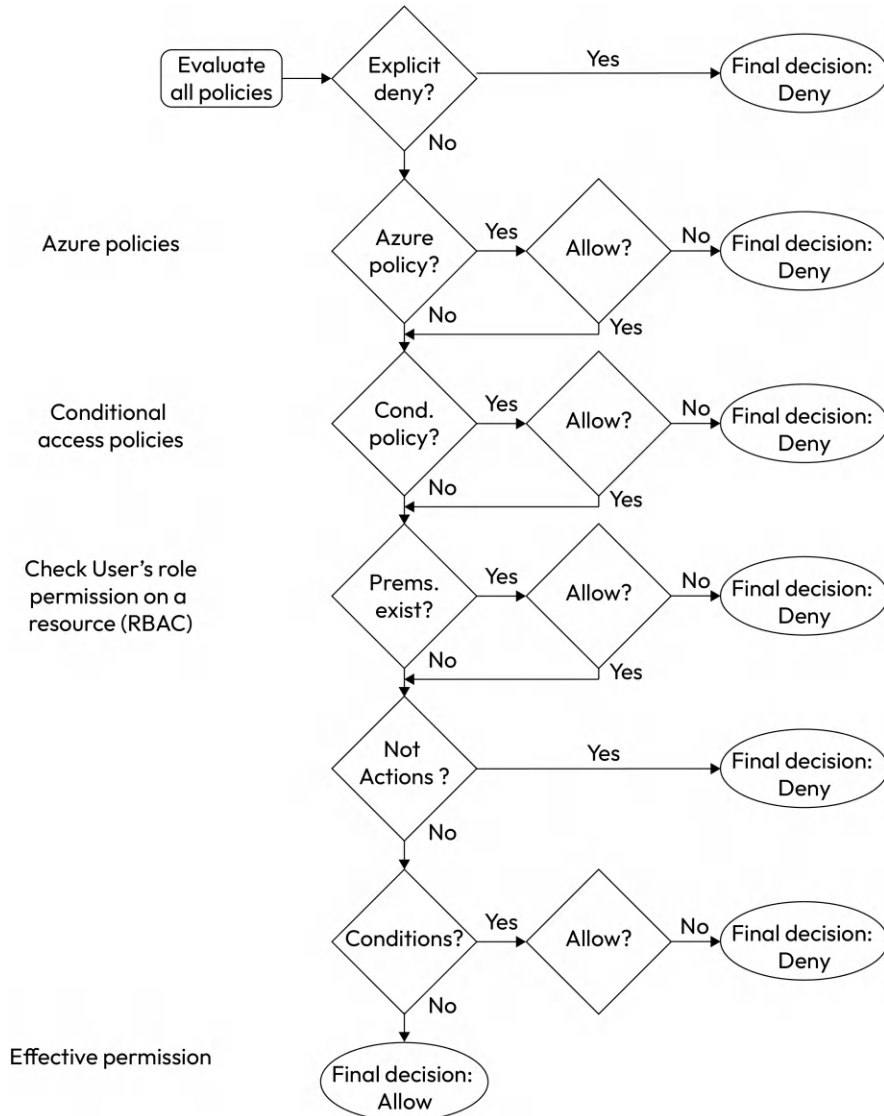


Figure 9.3 – Azure permissions evaluation logic

Here's what the preceding diagram depicts:

- When a user attempts to access a resource, **Azure Resource Manager** first checks for any applicable Azure policies
For more information about Azure policies, read *Chapter 17, Security in Large-Scale Environments*.
- If the request complies with Azure policies, **Microsoft Entra Conditional Access** policies are evaluated
- Conditional Access policies assess various signals such as user/group membership, location, device state, and application being accessed
- If Conditional Access requirements are met, Azure Resource Manager retrieves all relevant role assignments and *deny* assignments for the resource
- If a *deny* assignment applies, access is immediately blocked
- Azure Resource Manager then determines the roles the user has for the resource based on role assignments
- For roles with wildcard actions (*), effective permissions are computed by subtracting `NotActions` from allowed `Actions`
- If the user does not have a role (RBAC) with the required action at the requested scope, access is denied
- If role assignments include conditions, they are evaluated. Access is allowed if conditions are met; otherwise, it is denied

Documentation about the Microsoft Entra Global Administrator elevation process can be found here: <https://learn.microsoft.com/en-us/azure/role-based-access-control/elevate-access-global-admin>.

Use the following best practices for securing the Microsoft Entra Global Administrator:

- Use Microsoft Entra **Privileged Identity Management (PIM)** to configure just-in-time access for the Microsoft Entra Global Administrator
- Limit the Microsoft Entra Global Administrator role to a maximum of five users, as documented here:
<https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/permissions-reference#global-administrator>
- Configure a strong password for all users with the Microsoft Entra Global Administrator role to avoid unauthorized access to your Azure environment

- Enable MFA for all users with the Microsoft Entra Global Administrator role to avoid unauthorized access to your Azure environment
- Enable auditing on any attempt to use the Microsoft Entra Global Administrator role for further investigation and analysis and send alerts to the SOC team for both successful and failed attempts

Next, talking about security in Microsoft Entra ID, thorough documentation can be found in the links here:

- <https://learn.microsoft.com/en-us/entra/architecture/secure-best-practices>
- <https://learn.microsoft.com/en-us/entra/fundamentals/concept-secure-remote-workers>

Use the following best practices for securing Microsoft Entra ID:

- Prefer to use a **Temporary Access Pass (TAP)** to allow short-time access to resources instead of static credentials, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/authentication/howto-authentication-temporary-access-pass>

- Whenever supported, prefer to use passkeys (based on the FIDO2 authentication standard) to perform passwordless login to users instead of using static credentials (such as passwords), as explained here:

<https://learn.microsoft.com/en-us/entra/identity/authentication/concept-authentication-passwordless#passkeys-fido2>

- Use managed identities to allow applications access to Azure resources to allow short-time access to resources instead of using static SAS tokens, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview>

- Use Azure RBAC to assign permissions to Azure identities. Follow the principle of least privilege when granting access to resources to avoid unauthorized access to resources, as explained here:

<https://learn.microsoft.com/en-us/azure/role-based-access-control/best-practices>

- Use **Microsoft Entra Password Protection** to enforce the use of strong passwords on the entire Microsoft Entra tenant and decrease the chance of unauthorized access to your Azure environment, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/authentication/concept-password-ban-bad>

- Enforce MFA for all human (interactive) authentication to decrease the chance of unauthorized access, as explained here:
 - <https://learn.microsoft.com/en-us/entra/identity/authentication/concept-authentication-default-enablement>
 - <https://learn.microsoft.com/en-us/entra/identity/authentication/tutorial-enable-azure-mfa>
- Use Microsoft Entra Conditional Access policies to configure rules for which identities or devices can access Azure resources and in what terms, as explained here:
<https://learn.microsoft.com/en-us/entra/identity/conditional-access/plan-conditional-access>
- Use Microsoft Entra PIM to restrict any privileged access attempt for just-in-time access, as explained here:
<https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/security-planning>
- Create an emergency account (these are special user accounts that should not be used daily, have MFA enabled, and are a member of the global administrator role) to avoid getting locked out of your Microsoft Entra ID, as explained here:
<https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/security-emergency-access>
- Use Microsoft Entra ID Protection to detect any potential vulnerability activities of your Microsoft Entra users and send the audit logs to your SIEM system, as explained here:
<https://learn.microsoft.com/en-us/entra/id-protection/how-to-deploy-identity-protection>
- Use Microsoft Defender for Identity to detect suspicious activities in your on-premises Active Directory and Microsoft Entra ID, as explained here:
<https://learn.microsoft.com/en-us/defender-for-identity/deploy/deploy-defender-identity>
- Enable diagnostic settings for Microsoft Entra ID for further investigation and analysis, and send all logs to a Log Analytics workspace, as explained here:
<https://learn.microsoft.com/en-us/entra/identity/monitoring-health/howto-configure-diagnostic-settings>

- Use Microsoft Entra monitoring and health to review user sign-in logs and detect suspicious sign-in activities, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/monitoring-health/overview-monitoring-health>

- Use Microsoft Sentinel to be able to investigate Entra-related activities, as explained here:

<https://learn.microsoft.com/en-us/azure/sentinel/connect-azure-active-directory>

Securing IAM in GCP

Google Cloud has two services for managing IAM, and they complement each other:

- **Google Cloud IAM:** An authorization system for configuring permissions to GCP resources, based on Google Cloud Identity or Google Workspace. Cloud IAM allows customers to create service accounts to allow applications access to GCP resources, while users (and their passwords) are provisioned on other identity providers such as Google Cloud Identity or Google Workspace.
- **Google Cloud Identity:** An identity provider for managing users (with their passwords) and groups. Google Cloud Identity takes care of the authentication process, and it is integrated with Google Cloud IAM for the authorization process (granting permissions to access GCP resources).

Here is a list of Google Cloud IAM terminology:

- **Member:** This is a Google account (for example, a user), a service account (for applications and virtual machines), or a Cloud Identity with access to GCP resources.
- **User:** This is a person or application (a service account) with permission to access GCP resources. A user has credentials (such as a password and MFA).
- **Group:** This is a group of users to make the permissions management task easier.
- **Role:** This refers to a collection of permissions to access resources.
- **Service account:** This is a special type of IAM user to allow applications access to resources.
- **IAM policy:** This is a JSON-based definition that sets the permissions for accessing GCP resources.
- **Access boundary policy:** Security measures that allow you to downscope the permissions of short-lived credentials. They help prevent unauthorized access, reduce the risk of data exfiltration, and enhance overall security by enforcing the principle of least privilege.

GCP evaluates the permissions to a resource, using the following logic:

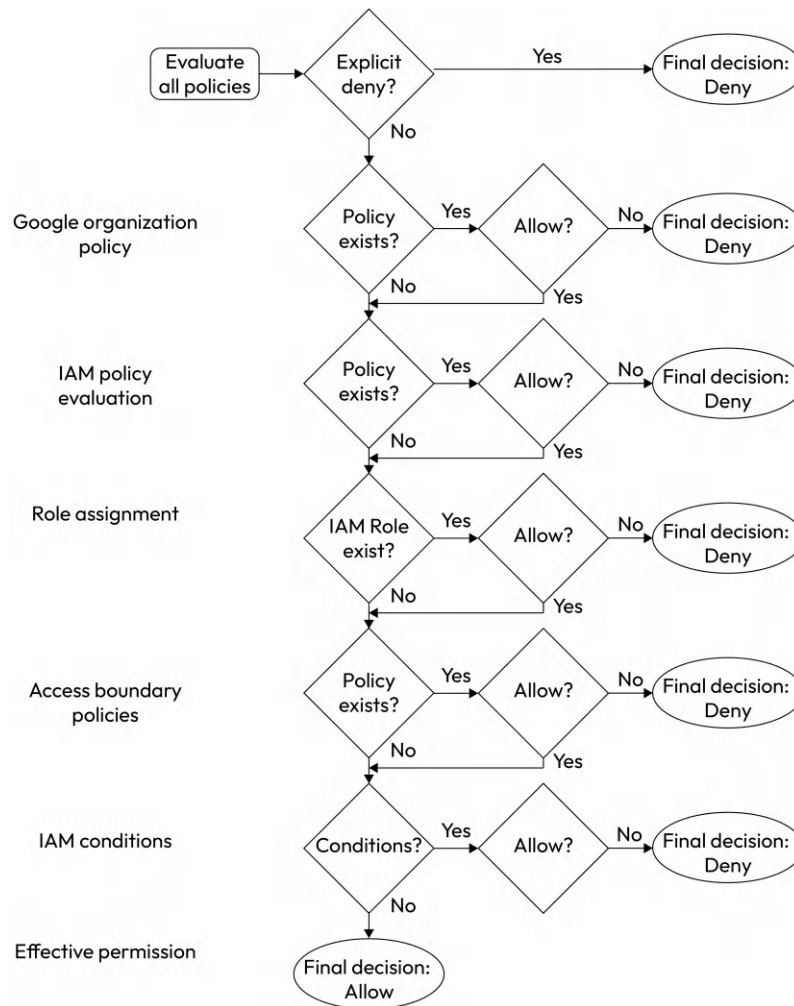


Figure 9.4 – GCP IAM permission evaluation logic

Let's try to understand the logic depicted in the diagram:

- **Organization policy check:** GCP first evaluates any applicable organization policies, which define constraints on resource configurations and usage across the entire organization.

For more information about Google organization policies, read *Chapter 17, Security in Large-Scale Environments*.

- **IAM policy evaluation:** If no *deny* policies apply, GCP evaluates the IAM policies associated with the resource and its parent resources in the hierarchy.

- **Role assignment:** The system determines whether the user has an IAM role that grants the necessary permissions for the requested action.
- **Access boundary policies:** If an applicable role is found, GCP evaluates any access boundary policies in place. These policies define the maximum set of permissions a user can have, even if broader permissions are granted through IAM roles.
- **IAM conditions:** If an *allow* policy exists, GCP evaluates any IAM conditions attached to the role binding (such as resource type, resource name, time of access, or user attributes).
- **Group membership:** If the user belongs to multiple groups with different role assignments, all applicable permissions are aggregated.
- Review access permissions on the target service and check whether there is an allow or deny statement.

Additional references about permission hierarchy can be found at the links here:

- <https://cloud.google.com/resource-manager/docs/organization-policy/understanding-hierarchy>
- <https://cloud.google.com/iam/docs/principal-access-boundary-policies>
- <https://cloud.google.com/iam/docs/resource-hierarchy-access-control>

Thorough documentation on the security of IAM in GCP can be found in the links here:

- <https://cloud.google.com/iam/docs/using-iam-securely>
- <https://cloud.google.com/architecture/identity/best-practices-for-planning>

Use the following best practices for securing IAM in Google Cloud:

- Prefer to use a short-lived credential for a service account instead of static credentials, as explained here:
<https://cloud.google.com/iam/docs/create-short-lived-credentials-direct>
- Use Google Cloud IAM to assign access permissions for GCP resources. Follow the principle of least privilege when granting access to resources to avoid unauthorized access to your GCP resources, as explained here:
<https://cloud.google.com/iam/docs/understanding-roles>

- Use IAM conditions to enforce the location from which a user can access resources in your Google Cloud environment (such as your office network), as explained here:

<https://cloud.google.com/iam/docs/conditions-overview>

- Enforce the use of strong passwords on the entire Google organization to avoid unauthorized access to your GCP environment, as explained here:

<https://support.google.com/cloudidentity/answer/139399?hl=en>

- Enforce MFA for all human (interactive) authentication to avoid unauthorized access to your GCP environment, as explained here:

<https://support.google.com/cloudidentity/answer/175197?hl=en>

- To allow applications access to GCP resources, create a unique service account for each application, and follow the principle of least privilege to avoid unauthorized access to GCP resources, as explained here:

- <https://cloud.google.com/iam/docs/best-practices-service-accounts>

- <https://cloud.google.com/resource-manager/docs/organization-policy/restricting-service-accounts>

- Limit the use of service account keys or avoid them completely to avoid having to expose access to your Google Cloud resources from outside your GCP environment, as explained here:

<https://cloud.google.com/iam/docs/best-practices-for-managing-service-account-keys>

- Use **Privileged Access Manager (PAM)** to restrict any privileged access attempt for just-in-time access, as explained here:

<https://cloud.google.com/iam/docs/pam-create-entitlements>

- Use role recommendations using **IAM recommender** to enforce minimal permissions to Google resources, as explained here:

<https://cloud.google.com/policy-intelligence/docs/role-recommendations-best-practices>

- Use the Policy Analyzer to review IAM policies and understand which identities have access to which GCP resources, as explained here:

<https://cloud.google.com/policy-intelligence/docs/analyze-iam-policies>

- Use the **Policy Simulator** to determine the effect of a policy on your users or service accounts, as explained here:

<https://cloud.google.com/policy-intelligence/docs/iam-simulator-overview>

- To restrict access to Google Cloud IAM from trusted sources and reduce the risk of data exfiltration, use VPC Service Controls, as mentioned here:

<https://cloud.google.com/iam/docs/secure-iam-vpc-sc>

- Enable data access audit logs in Google Cloud Audit Logs (actions such as listing/getting privileged access manager activities, generating access tokens for service accounts, etc.) for further investigation and analysis, as explained here:

<https://cloud.google.com/iam/docs/audit-logging>

Note

Data access audit logs increase storage costs. Review the business requirements to enable them.

- Use Cloud Monitoring to view metrics for service accounts and service account keys and detect suspicious activity of service accounts, as explained here:

<https://cloud.google.com/iam/docs/service-account-monitoring>

- Regularly use **Policy Intelligence** to search for unused service accounts, as explained here:

<https://cloud.google.com/policy-intelligence/docs/service-account-insights>

- Use **Event Threat Detection**, part of Google Security Command Center, to detect and send alerts about suspicious IAM-related activities (such as adding a user to a privileged group, use of IAM service accounts, etc.), as mentioned here:

<https://cloud.google.com/security-command-center/docs/concepts-event-threat-detection-overview>

In this section, we learned how to secure cloud-based IAM services based on the AWS, Azure, and GCP infrastructure. This included account creation, password policy enforcement, authorization mechanisms, and audit IAM-related activities.

Next, we are going to review how to secure directory services.

Securing directory services

The most common directory service deployed on most organizations' on-premises data centers is the Microsoft Active Directory. This service allows you to centrally manage your user and computer identities, join Windows and Linux machines to the Active Directory domain, set password policies (such as the password length, password complexity, and more), and control access to traditional resources (such as Windows file shares, SQL servers, IIS servers, and more).

The most used protocol when working on-premises on traditional Windows applications or services is the Kerberos protocol.

For more information, please refer to the following resources:

- *Active Directory Domain Services Overview:*

<https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/virtual-dc/active-directory-domain-services-overview>

- *Kerberos Authentication Overview:*

<https://learn.microsoft.com/en-us/windows-server/security/kerberos/kerberos-authentication-overview>

Each cloud provider has its own implementation of a managed Active Directory service.

It is important to note that, as a customer, you always have the option to deploy Active Directory domain controllers based on virtual machines and maintain them yourself, as organizations are doing on-premises (this is also known as a self-hosted solution). However, the goal of this book is to show you how things are done using managed services, where, as a customer, you can focus on consuming the IAM service (that is, authenticate and create identities and then grant them permissions) without having to maintain servers (such as availability, patch management, backups, and more).

General best practices for securing managed Active Directory services are as follows:

- Configure a password policy (which includes the minimum and maximum password age, the minimum password length, and enforces the use of password history and complex passwords), to decrease the chance of unauthorized access to your cloud environment
- Configure an account lockout policy (including the number of failed login attempts, the account lockout duration, and whether to allow a reset of the account lockout after a certain amount of time) to decrease the chance of unauthorized access to your cloud environment
- Create Active Directory groups, add users to the groups, and grant permissions over resources (such as access to file servers, login servers, and more) to groups instead of granting permissions to specific users, for easy maintainability of permission management
- Enforce the use of MFA for any human (interactive) logins to cloud resources to decrease the chance of unauthorized access to your cloud environment
- Enable audit for all successful and failed logins and send the audit logs to a central logging service for further investigation and analysis by the SOC team

In the next section, we are going to examine what the best practices are for securing managed directory services from AWS, Azure, and GCP.

Securing AWS Directory Service

AWS Directory Service is the Amazon-managed Active Directory service. Under the hood, AWS is using a cluster of Windows 2019 servers for domain controllers.

AWS offers several deployment options for Active Directory:

- **AWS Directory Service for Microsoft Active Directory:** Suitable for customers who wish to consume a fully managed Active Directory, without having to take care of the maintenance (such as patching and availability)
- **AD Connector:** Suitable for customers who wish to connect AWS services (such as Amazon Workspaces or Amazon EC2) to an on-premises Microsoft Active Directory
- **Simple AD:** Suitable for customers who wish to have a basic Active Directory, based on Samba 4, for user and group creation, joining machines to a central domain, and so on

Thorough documentation on security in AWS Directory Service can be found in the links here:

- <https://docs.aws.amazon.com/directoryservice/latest/admin-guide/security.html>
- https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_security.html

Use the following best practices for securing AWS Directory Service:

- Protect the built-in admin account created during the initial setup of AWS Directory Service, using a long and complex password, to decrease the chance of unauthorized access to your cloud environment. Store the password in a secure vault and regularly rotate the admin password.
- When using AD Connector, regularly rotate the service account used to connect the AD Connector to the on-premises Active Directory to avoid misuse, as explained here:

https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ad_connector_update_creds.html

- Since on-premises Active Directory supports authentication against legacy systems using the LDAP protocol, enforce traffic encryption (i.e., LDAP over TLS) to keep the confidentiality of passwords passing over the network in plain text, as explained here:

https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_ldap.html

- Configure a password policy (which includes the minimum and maximum password age, the minimum password length, and enforces the use of password history and complex passwords), to decrease the chance of unauthorized access to your cloud environment, as explained here:

https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_password_policies.html

- To avoid using a privileged account for a password reset, use the **AWS Delegated Fine-Grained Password Policy Administrators** group to allow your support team permission to reset passwords, as explained here:

<https://docs.aws.amazon.com/directoryservice/latest/admin-guide/delegatepasswordpolicies.html>

- Enforce MFA for all human (interactive) authentication to decrease the chance of unauthorized access to your cloud environment, as explained here:

- https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_mfa.html

- https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ad_connector_mfa.html

- Use VPC security groups to restrict access to the AWS Directory Service domain controllers, from trusted CIDRs (such as private subnets or on-premises) and decrease the chance of direct attack against your managed domain controllers, as mentioned here:

https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_network_security.html

- To secure access from your VPC to AWS Directory Service APIs, use AWS PrivateLink. This keeps traffic internally inside the AWS backbone, through a secure channel, using the interface VPC endpoint, as explained here:

<https://docs.aws.amazon.com/directoryservice/latest/admin-guide/vpc-interface-endpoints.html>

- Enable AWS Directory Service log forwarding and forward the Active Directory logs to Amazon CloudWatch for further Active Directory log analysis, such as tracking failed Active Directory logins, as explained here:

https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_enable_log_forwarding.html

- AWS CloudTrail is enabled by default for AWS Directory Service. To keep the AWS Directory Service API activity for longer than 90 days for further investigation and analysis, send CloudTrail logs to an Amazon S3 bucket, as explained here:

https://docs.aws.amazon.com/directoryservice/latest/devguide/cloudtrail_logging.html

Securing Microsoft Entra Domain Services

Microsoft Entra Domain Services (formerly Azure AD DS) is a managed Active Directory service in Azure. It allows customers the same capabilities as the on-premises Active Directory (such as user creation, group creation, joining machines to an AD domain, etc.) without having to deploy and maintain domain controllers.

Microsoft Entra Domain Services allows legacy infrastructure (such as users created on-premise or VMs joined to an Entra domain) to authenticate using legacy protocols (such as Kerberos) and gain access to Azure native resources (such as Azure Files) using modern protocols (such as OAuth or OpenID connect).

Thorough documentation on security in Microsoft Entra Domain Services can be found here: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/azure-active-directory-domain-services-security-baseline>.

Use the following best practices for securing Microsoft Entra Domain Services:

- Configure a password policy (which includes the minimum and maximum password age, the minimum password length, and enforces the use of password history and complex passwords) to decrease the chance of unauthorized access to your cloud environment, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/domain-services/password-policy>

- Since on-premises Active Directory supports authentication against legacy systems using the LDAP protocol, enforce traffic encryption (i.e., LDAP over TLS) to keep the confidentiality of passwords passing over the network in plain text, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/domain-services/tutorial-configure-ldaps>

- For hybrid environments (on-premises Active Directory and Microsoft Entra ID), sync objects (i.e., users, group membership, and password hashes) using Microsoft Entra Connect to allow on-premises users access to Azure resources, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/domain-services/synchronization>

- To secure access from on-premises to Azure, enforce the use of secure network protocols (such as TLS 1.2 or above), and disable the use of legacy/weak protocols (such as NTLM v1), as explained here:

<https://learn.microsoft.com/en-us/entra/identity/domain-services/secure-your-domain>

- If you have a requirement to allow impersonation on behalf of another user to access resources (for example, an application that needs access to a file server), enable the use of **Kerberos-constrained delegation (KCD)**, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/domain-services/deploy-kcd>

- Use network security groups to restrict access to Microsoft Entra Domain Services domain controllers from trusted CIDRs (such as private subnets or on-premises) to decrease the chance of direct attack against your managed domain controllers, as mentioned here:

<https://learn.microsoft.com/en-us/entra/identity/domain-services/network-considerations>

- If you have a requirement to allow access to legacy applications deployed on-premises from the internet, use Microsoft Entra Application Proxy together with Microsoft Entra Domain Services to allow secured access from the public Internet to on-premises, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/domain-services/deploy-azure-app-proxy>

- Enable diagnostic settings for Microsoft Entra Domain Services for further investigation and analysis and send all logs to a Log Analytics workspace, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/domain-services/security-audit-events>

Securing Google Managed Service for Microsoft Active Directory

Google Managed Service for Microsoft Active Directory (Managed Microsoft AD) is the GCP-managed Active Directory service. It allows customers to connect from Google Cloud to on-premises resources using the same protocols that exist on-premises.

The following is a list of best practices:

- Configure a password policy (which includes the minimum and maximum password age, the minimum password length, and enforces the use of password history and complex passwords) to decrease the chance of unauthorized access to your cloud environment, as explained here:

<https://cloud.google.com/managed-microsoft-ad/docs/how-to-use-fgpp>

- Since on-premises Active Directory supports authentication against legacy systems using the LDAP protocol, enforce traffic encryption (i.e., LDAP over TLS) to keep the confidentiality of passwords passing over the network in plain text, as explained here:

<https://cloud.google.com/managed-microsoft-ad/docs/how-to-use-ldaps>

- To avoid using a privileged account for a password reset, use the delegated administrator account in Google Managed Microsoft AD to allow your support team permission to reset passwords, as explained here:

<https://cloud.google.com/managed-microsoft-ad/docs/how-to-use-delegated-admin>

- Use VPC firewall rules to restrict access to Google Managed Microsoft AD from trusted CIDRs (such as private subnets or on-premises) to decrease the chance of direct attack against your managed domain controllers, as mentioned here:

<https://cloud.google.com/managed-microsoft-ad/docs/firewalls>

- To restrict access to Google Managed Microsoft AD from trusted sources and reduce the risk of data exfiltration, use VPC Service Controls, as mentioned here:

<https://cloud.google.com/managed-microsoft-ad/docs/how-to-use-vpc-service-controls>

- Enable data access audit logs in Google Cloud Audit Logs (actions such as admin activity in Google Managed Microsoft AD) for further investigation and analysis, as explained here:

<https://cloud.google.com/managed-microsoft-ad/docs/audit-logging>

Note

Data access audit logs increase storage costs. Review the business requirements to enable them.

In this section, we learned how to secure Directory Services based on the AWS, Azure, and GCP infrastructure. We examined how to set password policies, how to restrict network access, and, finally, how to audit Active Directory.

Summary

In this chapter, we focused on the various IAM services in AWS, Azure, and GCP.

Identity management services allow us to control access to resources and services in our cloud environment. They provide the least privileged access to resources and monitor what actions were performed using identities in our cloud environments.

We discussed everything from cloud-native identity and access management services to managed Active Directory services for connecting to/from on-premises to cloud environments in hybrid architectures.

In each section, we reviewed best practices for configuration, account management, monitoring, and auditing.

In the next chapter, we will review how to perform auditing and threat detection in the cloud.

Auditing and Threat Management in Cloud Environments

In *Chapters 2, Securing Compute Services – Virtual Machines to 9, Effective Strategies for Implementing IAM Solutions*, we covered the fundamental building blocks of cloud services (from compute, storage, and networking services to identity and access management services).

All services and security controls send their audit logs to a central audit service for alerts and further analysis.

This chapter will cover various concepts of monitoring and auditing from a security perspective – from audit trails to threat detection in cloud environments.

In this chapter, we will cover the following topics:

- Maintaining audit trails
- Controlling access to CSP support engineers
- Conducting threat detection and response
- Managing cloud-native SIEM

Maintaining audit trails

An **audit trail** is a crucial part of security in the cloud. An audit trail in the context of cloud security is about logging activities done in cloud environments, such as user login events (both success and failure), actions taken (who did what and when and what the result was – success or failure). It entails storing events in a central service with restricted user access to logs (according to the principle of *need to know*), raising alerts according to preconfigured rules, and being able to take action.

In cloud environments, resources are based on APIs, including the ability to pull audit logs. We control cloud resources (as we have seen in previous chapters) using security controls (from network access controls through a WAF). All cloud resources can send their audit logs to a central logging service for further analysis (sometimes by default and sometimes by manually enabling audit logging).

An audit trail serves multiple purposes:

- **Accountability:** It allows us to prove who or what (user, application, service account, etc.) performed which action.
- **Incident investigation:** It allows us to review events that happened in a system that led to a security incident, such as privileged escalation.
- **Compliance:** It allows us to meet regulatory requirements for collecting evidence in case of a security incident.
- **Operational transparency:** It provides transparency into system operations by documenting every action taken within the system. This can be useful for internal audits, process improvements, and ensuring that workflows are followed correctly.
- **Forensics analysis:** It allows us to provide a trail of evidence for legal purposes when reviewing a security incident such as a data breach.

General best practices for audit trails in cloud environments

Audit trails for admin activities (such as stopping a VM, deploying a new database, etc.) are enabled by default in most CSP environments, without the customer having the ability to disable this functionality.

Regardless of the cloud environment you are using, you should follow these best practices:

- Categorize logs based on security, application, administrative, data access and modification, transaction history, configuration changes, and so on
- Store all logs in a dedicated account/project, with access permissions to the security team (for incident response and forensics purposes)
- For sensitive/critical environments, enable data access audit logs (actions such as read configuration, create/modify/read user-provided resource data) for further investigation and analysis
- Protect access to audit logs using **role-based access control (RBAC)**, following the principle of least privilege, to avoid unauthorized access
- Encrypt audit logs using encryption in transit and at rest to keep the confidentiality of the logs' content
- Keep audit logs in immutable storage (according to CSP support) to make sure logs will be retained and not accidentally deleted or tampered with

Next, we are going to see what the best practices are for conducting security monitoring and audit trails using AWS, Azure, and GCP built-in managed services.

AWS CloudTrail

AWS CloudTrail is an Amazon-managed audit trail service. It enables you to track all user activity based on an AWS API from both the AWS Management Console and through the API for further analysis and correlation (mostly management-related API events).

Thorough documentation on security in AWS CloudTrail can be found at the following links:

- <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/WhatIsCloudTrail-Security.html>
- <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/best-practices-security.html>

Use the following best practices for securing AWS CloudTrail:

- Use identity-based policies to grant access to objects inside S3 buckets that contain AWS CloudTrail logs and avoid unauthorized access, as explained here:

https://docs.aws.amazon.com/awscloudtrail/latest/userguide/security_iam_id-based-policy-examples.html

- Limit access to CloudTrail logs stored in S3 using an S3 bucket policy, as explained in the following docs:
 - <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/create-s3-bucket-policy-for-cloudtrail.html>
 - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html>
 - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies-vpc-endpoint.html>

- To keep audit trail logs for a long period (according to business or regulation requirements), create a trail by sending all AWS CloudTrail logs from the entire AWS organization to a central S3 bucket, located in a dedicated AWS account, with limited access permissions for the SOC or security teams, as explained here:

<https://docs.aws.amazon.com/awscloudtrail/latest/userguide/creating-an-organizational-trail-in-the-console.html>

- For sensitive/critical environments, enable logging of data events (such as S3 object-level activity, AWS Lambda executions, DynamoDB object-level events, and so on), as explained here:

<https://docs.aws.amazon.com/awscloudtrail/latest/userguide/logging-data-events-with-cloudtrail.html>

Important note

It is recommended to conduct risk analysis on the target service (such as a service that stores or processes credit card information, healthcare data, and more) before enabling data event logging, due to the cost of event storage versus the value that can be achieved from data event logs.

- Enable CloudTrail Insights to detect unusual activity related to *writing* API events, as explained here:
<https://docs.aws.amazon.com/awsccloudtrail/latest/userguide/logging-insights-events-with-cloudtrail.html>
- Send events to CloudWatch logs for further analysis of user activity (such as file deletion from an S3 bucket, changes to a security group, shutting down an EC2 instance, and so on), as explained here:
 - <https://docs.aws.amazon.com/awsccloudtrail/latest/userguide/send-cloudtrail-events-to-cloudwatch-logs.html>
 - <https://docs.aws.amazon.com/awsccloudtrail/latest/userguide/monitor-cloudtrail-log-files-with-cloudwatch-logs.html>
- Enable log file integrity validation for CloudTrail logs to make sure your logs have not been tampered with, as explained here:
<https://docs.aws.amazon.com/awsccloudtrail/latest/userguide/cloudtrail-log-file-validation-enabling.html>
- When storing CloudTrail logs inside an S3 bucket, enable the MFA delete feature to avoid accidental deletion of CloudTrail logs using S3 bucket policies, explained as follows:
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html#example-bucket-policies-MFA>
- In the case that MFA is not required and the business requirement is to keep logs indefinitely while preserving storage cost, configure the S3 bucket that stores CloudTrail logs with object lifecycle policies, which will move old log files to an archive tier, as explained here:
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/how-to-set-lifecycle-configuration-intro.html>
- For data that you need to retain for long periods (due to regulatory requirements), use S3 Object Lock in compliance mode to protect the data from accidental deletion, as explained here:
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>

- Encrypt all CloudTrail logs stored inside a dedicated S3 bucket using a customer-managed key **server-side encryption** with **AWS Key Management Service (SSE-KMS)**, as explained in *Chapter 11, Applying Encryption in Cloud Services*, and as documented here:
 - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingKMSEncryption.html>
 - <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/create-kms-key-policy-for-cloudtrail.html>
- To secure access from your VPC to AWS CloudTrail, use AWS PrivateLink. This keeps traffic internally inside the AWS backbone, through a secure channel, using the interface VPC endpoint, as explained here:
<https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-and-interface-VPC.html>
- Use Amazon GuardDuty to detect anomalous behavior in CloudTrail logs (such as S3 file exfiltration, anomalous credential access to an IAM user, root credential usage, etc.), as documented here:
https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_finding-types-active.html

Note

Amazon GuardDuty can be enabled and used even if CloudTrail logs are not explicitly enabled.

Azure Monitor

Azure Monitor is a fully managed service for collecting, analyzing, and responding to logs from various sources in both Azure ecosystems and on-premises.

Azure Monitor collects logs from infrastructure (such as VMs or databases), platform (such as Azure API Management, Azure AI services, etc.), performance and metrics (such as CPU, memory, and disk), and in the context of this chapter, it collects activity logs (such as Microsoft Entra ID logs, console activity logs, etc.).

Thorough documentation on security in Azure Monitor can be found at the following links:

- <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/azure-monitor-security-baseline>
- <https://learn.microsoft.com/en-us/azure/security/fundamentals/log-audit>

Use the following best practices for securing Azure Monitor:

- Create a central Log Analytics workspace in a dedicated Azure subscription, and forward all activity and audit logs (by configuring diagnostic settings) to the central workspace, as explained here:

- <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/quick-create-workspace>
- <https://learn.microsoft.com/en-us/azure/azure-monitor/essentials/activity-log>
- <https://learn.microsoft.com/en-us/azure/azure-monitor/essentials/diagnostic-settings-policy>

- Make sure all access to the Log Analytics workspace is authenticated using Microsoft Entra ID identities, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/logs/azure-ad-authentication-logs>

- Use RBAC to enforce authorization to Azure Monitor and to the Log Analytics workspace according to business needs, and avoid unauthorized access, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/roles-permissions-security>

- If you are using Azure Monitor Application Insights for live application monitoring, make sure all the Azure services sending telemetry to Application Insights are using managed identities based on Microsoft Entra ID, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/app/azure-ad-authentication>

- For sensitive environments (for example, which contain PII, credit card details, healthcare data, and more), encrypt data at rest using **customer-managed keys** stored inside Azure Key Vault, as explained in *Chapter 11, Applying Encryption in Cloud Services*, and as documented here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/logs/customer-managed-keys>

- Use network security groups (NSGs) and network service tags to restrict access to Azure Monitor from trusted (private subnets), as explained here:

- <https://learn.microsoft.com/en-us/azure/azure-monitor/ip-addresses>
- <https://learn.microsoft.com/en-us/azure/virtual-network/service-tags-overview>

- To secure access from your VNet to Azure Monitor, use an Azure private endpoint, which avoids sending network traffic outside your VNet through a secure channel, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/logs/private-link-security>

- For data that you need to retain for long periods (due to regulatory requirements), use an immutable Blob Storage lock to protect the data from accidental deletion, as explained here:

<https://learn.microsoft.com/en-us/azure/storage/blobs/immutable-storage-overview>

- Send Azure Monitor activity logs (stored in the Log Analytics workspace) to Microsoft Sentinel for further security analysis, as mentioned here:

<https://learn.microsoft.com/en-us/azure/sentinel/design-your-workspace-architecture>

Google Cloud Audit Logs

Cloud Audit Logs allows you to monitor and log user activity logs inside Cloud Logging, a managed log collection and monitoring service.

Cloud Audit Logs collects audit logs from various sources (such as Google App Engine, Google BigQuery, and Google Cloud IAM).

Thorough documentation on security in Cloud Logging can be found at the following link: <https://cloud.google.com/logging/docs/audit/best-practices>

Use the following best practices for securing Cloud Audit Logs/Cloud Logging:

- Use IAM policies to grant permissions to Cloud Audit Logs and avoid unauthorized access, as documented here:

<https://cloud.google.com/logging/docs/access-control>

- To keep audit trail logs for a long period (according to business or regulation requirements), create a central log bucket, located in a dedicated GCP project, with limited access permissions for the SOC or security teams, as explained here:

- <https://cloud.google.com/logging/docs/buckets>

- https://cloud.google.com/logging/docs/export/aggregated_sinks

- <https://cloud.google.com/logging/docs/central-log-storage>

- Limit access to the log bucket using log views, which allows you to configure granular access to the logs, as explained here:

<https://cloud.google.com/logging/docs/logs-views>

- For sensitive/critical environments, explicitly enable **data access audit logs** to log activities performed on Google services (such as read configuration or metadata of resources), as explained here:

<https://cloud.google.com/logging/docs/audit/configure-data-access>

Important note

It is recommended to conduct risk analysis on the target service (such as a service that stores or processes credit card information, healthcare data, and so on) before enabling data event logging due to the cost of event storage versus the value that can be achieved from data event logs.

- Encrypt all **Cloud Logging logs** using customer-managed encryption keys (CMEKs) (see *Chapter 11, Applying Encryption in Cloud Services* for a deeper dive), as explained here:

<https://cloud.google.com/logging/docs/routing/managed-encryption-storage>

- For data that you need to retain for long periods (due to regulatory requirements), use the Bucket Lock feature to protect the data from accidental deletion, as explained here:

<https://cloud.google.com/storage/docs/bucket-lock>

- To restrict access to Cloud Audit Logs to trusted sources and reduce the risk of data exfiltration, use VPC Service Controls, as explained here:

- <https://cloud.google.com/vpc-service-controls/docs/create-service-perimeters>

- <https://cloud.google.com/vpc-service-controls/docs/manage-service-perimeters>

- To keep traffic from your VPC to the Cloud Logging API private, use Private Service Connect, as documented here:

<https://cloud.google.com/vpc/docs/private-service-connect>

Now that we have reviewed the best practices for security monitoring and audit trails in AWS, Azure, and Google Cloud, let us see how we can control access for CSP support engineers.

Controlling access to CSP support engineers

In the previous section, we discussed the audit trail for identities in the cloud eco-system that we, as customers, control (from human identities to applications or service accounts).

When talking about cloud services and the *shared responsibility model*, we understood that as customers of **infrastructure as a service (IaaS)**, we control everything inside a VM and, among others, we are responsible for monitoring activity done above the operating system layer.

With **platform as a service (PaaS)**, we are responsible for monitoring activity done above the platform (such as a managed database) layer, and with **software as a service (SaaS)**, we are responsible for monitoring activity done at the managed service layer (assuming the SaaS vendor supports it).

Let's look at an example use case. Cloud vendor support personnel access customer tenants through controlled procedures when resolving specific service requests, such as restoring deleted mailbox items or files, while adhering to strict data protection and privacy protocols.

Where do we see audit trails of such an event, and how can we approve it in the first place?

If we need to enable access to CSP support engineers and to be able to keep our cloud environment secure, we must follow these best practices:

- **Approval:** All access by external parties (such as a CSP support engineer) must be approved by us (CSP customers). Access must always pass an approval process for each request and must be revoked when no longer required.
- **Authentication and authorization:** All access by external parties must be authenticated and authorized, following the principle of least privilege. We need to be able to identify external access and limit access according to each specific request. Access must be limited in time and must be revoked once it is no longer required.
- **Auditing:** All access must be logged and audited for further review by our SOC teams. Similar to access by internal parties, we must be able to audit which actions were taken by CSP support engineers, from the moment access was granted till the moment access was revoked.

Next, we are going to see what the best practices are for controlling access for external support purposes using AWS, Azure, and GCP built-in managed services.

AWS Support

AWS offers its customers various support options through its web portal. One of the more advanced options is a live troubleshooting session with a support engineer (sometimes referred to in AWS documentation as *AWS Support Recommendations*).

General security best practices for using AWS Support can be found at the following links:

- <https://docs.aws.amazon.com/awssupport/latest/user/security.html>
- <https://docs.aws.amazon.com/awssupport/latest/user/security-for-support-cases.html>

To minimize the risk for your AWS environment, while using a proactive AWS Support session, follow these best practices:

- Before allowing AWS Support Recommendations, temporarily enable access permissions using an IAM policy, as explained here:
<https://docs.aws.amazon.com/awssupport/latest/user/security-support-recommendations.html>
- Use identity-based policies to grant access to AWS Support to be able to access specific resources in your AWS account, following the principle of least privilege, as explained here:
 - https://docs.aws.amazon.com/awssupport/latest/user/security_iam_id-based-policy-examples.html
 - <https://docs.aws.amazon.com/awssupport/latest/user/aws-managed-policies-aws-support.html>
- AWS Support API calls are automatically logged in AWS CloudTrail. To keep audit trail logs for a long period (according to business or regulation requirements), create a trail by sending all AWS CloudTrail logs from the entire AWS organization to a central S3 bucket, located in a dedicated AWS account, with limited access permissions for the SOC or security teams, as mentioned here:
 - <https://docs.aws.amazon.com/awssupport/latest/user/cloudtrail-logging-support-recommendations.html>
 - <https://docs.aws.amazon.com/awssupport/latest/user/logging-using-cloudtrail.html>
 - <https://docs.aws.amazon.com/awssupport/latest/user/logging-using-cloudtrail-support-app.html>

Customer Lockbox (in Azure)

Most support scenarios in Azure do not require support engineer access to customer tenants.

In the case of a support ticket that requires engineer access to a customer tenant (such as a scenario where a customer opens a support ticket and requests to restore an object from a mailbox or SharePoint/OneDrive), the request is sent to an Azure regional manager for approval. If the request is approved, a support engineer receives just enough access to perform the actions required to fulfill the support ticket, and the entire approval process is documented.

Microsoft offers customers a way to intervene in the process, by enabling a license for a feature called **Customer Lockbox**:

- Customer Lockbox for Microsoft Azure (for Azure services)
- Microsoft Purview Customer Lockbox (for Exchange Online, SharePoint, OneDrive, Teams, and Windows 365)
- Customer Lockbox for Microsoft Power Platform (for Power Platform and Dynamics 365)

Customer Lockbox works as per the following steps:

1. A customer submits a ticket to Microsoft support and requests assistance that requires access to their tenant.
2. The ticket is assigned to a support engineer, and since they do not have access to the customer's tenant, they escalate the ticket to a support manager (also known as a regional manager).
3. After the support manager approves the request, an email is sent to a designated approver from the customer's organization, who is asked to grant the support engineer access to their tenant.
4. Once the process is approved, the support engineer gets *just enough time* and *just enough access* to accomplish the requested task.
5. The entire process is logged in our tenant logs (for example, in **Exchange Online logs** or **Azure Activity Logs**) and the temporary permissions are removed.

Thorough documentation on security in Customer Lockbox can be found at the following links:

- <https://learn.microsoft.com/en-us/azure/security/fundamentals/customer-lockbox-overview>
- <https://learn.microsoft.com/en-us/purview/customer-lockbox-requests>
- <https://learn.microsoft.com/en-us/azure/security/fundamentals/customer-lockbox-faq>

To minimize the risk for your Azure environment, while using the Customer Lockbox service, follow these best practices:

- Purchase a license and enable Customer LockBox at the Azure tenant level, which enables this capability for the entire organization, as explained here:
 - <https://learn.microsoft.com/en-us/azure/security/fundamentals/customer-lockbox-overview#enable-customer-lockbox-for-microsoft-azure>
 - <https://learn.microsoft.com/en-us/power-platform/admin/about-lockbox#enable-the-lockbox-policy>
- Assign the **Customer LockBox Access Approver** role to designated employees to be able to receive, review, and approve support tickets that require external access to your Azure tenants, as explained here:
 - <https://techcommunity.microsoft.com/t5/security-compliance-and-identity/customer-lockbox-approver-role-now-available/ba-p/223393>
 - <https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/permissions-reference#customer-lockbox-access-approver>
- Use Microsoft Entra PIM to *temporarily* assign the **Customer LockBox Access Approver** role, and enforce the use of MFA for any user who requests access to this role, as explained here:
 - <https://learn.microsoft.com/en-us/entra/id-governance/privileged-identity-management/pim-how-to-activate-role>
 - <https://learn.microsoft.com/en-us/entra/id-governance/privileged-identity-management/pim-how-to-change-default-settings>
- Use Microsoft Defender for Cloud and enable the Microsoft Cloud security benchmark to monitor the use of Customer Lockbox, as mentioned here:
<https://learn.microsoft.com/en-us/security/benchmark/azure/mcsb-privileged-access#pa-8-determine-access-process-for-cloud-provider-support>
- Connect Microsoft Entra ID, Microsoft Purview (for data governance and loss prevention), and Microsoft Power Platform to Microsoft Sentinel to be able to monitor and get alerts about Customer Lockbox events in near-real time, as explained here:
 - <https://learn.microsoft.com/en-us/azure/sentinel/connect-azure-active-directory>

- <https://learn.microsoft.com/en-us/purview/purview>
- <https://learn.microsoft.com/en-us/azure/sentinel/purview-solution>
- <https://learn.microsoft.com/en-us/azure/sentinel/business-applications/deploy-power-platform-solution>
- <https://learn.microsoft.com/en-us/entra/identity/monitoring-health/howto-access-activity-logs#monitor-events-with-microsoft-sentinel>

Google Access Transparency and Access Approval

To allow review and approval of Google support engineers' access to customers GCP projects, Google offers the following services:

- **Google Access Approval** allows you to control any request by Google support engineers to access your GCP project and resources
- **Google Access Transparency** allows you to review access attempts to your GCP project and resources by Google engineers inside Cloud Logging

To minimize the risk for your GCP environment, while allowing Google engineers access to your environments, follow these best practices:

- Enable Access Transparency to begin logging actions done by Google engineers on your tenant into Cloud Audit Logs, as explained here:

<https://cloud.google.com/assured-workloads/access-transparency/docs/enable>

- Use IAM roles to grant permissions to use Access Approval, following the principle of least privilege, as explained here:

<https://cloud.google.com/assured-workloads/access-approval/docs/access-control>

- In the case of a request to allow a Google engineer access to your GCP projects, temporarily enable Access Approval, as explained here:

<https://cloud.google.com/assured-workloads/access-approval/docs/approve-requests>

- Generate a unique custom signing key for each request for access to your GCP projects to assure the integrity of each request for Access Approval, using Google Cloud KMS, as explained here:

<https://cloud.google.com/assured-workloads/access-approval/docs/review-approve-access-requests-custom-keys>

- Use **Google Security Command Center** to review Access Transparency logs and detect access requests by Google engineers, as mentioned here:

<https://cloud.google.com/security-command-center/docs/concepts-vulnerabilities-findings>

In this section, we have learned how to approve and conduct auditing of requests for allowing CSP engineers access to customer's cloud environments. Next, we are going to review how to conduct threat detection in the cloud.

Conducting threat detection and response

In this section, we will review managed services that allow us to review logs to pinpoint and respond to actual threats. **Threat detection and response (TDR)** services are crucial in cloud environments because they provide real-time monitoring, rapid threat identification, and automated response capabilities to protect sensitive data and infrastructure from constantly evolving cyber threats. We will cover some of the most common recommendations for configuring Amazon GuardDuty, Microsoft Defender, and Google Security Command Center. This section does not replace the official documentation for using these services for threat management and response.

Amazon GuardDuty

Amazon GuardDuty is a threat detection and monitoring service that pulls data from various sources such as VPC Flow Logs, AWS CloudTrail, CloudTrail S3 data event logs, and DNS logs. It uses machine learning to review the logs and alerts us only on events that require further investigation and sometimes actions (such as replacing privileged account credentials or configuring MFA if credentials were compromised).

Amazon GuardDuty combines machine learning capabilities with signature-based detection (such as API action blacklists, detecting bot activity in your AWS environment, and so on). Examples of events include privilege escalation events, stolen credentials, outbound traffic from your environment to an external malicious IP address, and so on.

Thorough documentation on security in Amazon GuardDuty can be found at the following link: <https://docs.aws.amazon.com/guardduty/latest/ug/security.html>

Use the following best practices to configure and secure Amazon GuardDuty:

- Use identity-based policies to grant access for users to be able to maintain and use Amazon GuardDuty, following the principle of least privilege, and avoid unauthorized access, as explained here:

https://docs.aws.amazon.com/guardduty/latest/ug/security_iam_id-based-policy-examples.html

- To protect Amazon GuardDuty findings and retain them for a long period, export all findings to a central S3 bucket, located in a dedicated AWS account, with limited access permissions for the SOC or security teams. Encrypt all GuardDuty findings using a customer-managed key (more on this in *Chapter 11, Applying Encryption in Cloud Services*), as documented here:

- https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_settingup.html#setup-export
- https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_exportfindings.html

- To keep Amazon GuardDuty API calls for a long period (according to business or regulation requirements), create a trail by sending all Amazon GuardDuty logs from the entire AWS organization to a central S3 bucket, located in a dedicated AWS account, with limited access permissions for the SOC or security teams, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/logging-using-cloudtrail.html>

- Enable **S3 protection** in AWS at the entire organization level and on individual S3 buckets that contain sensitive information (such as credit card information, healthcare data, and so on) to allow GuardDuty to monitor and search for malicious activity against an S3 bucket, such as a bucket made public, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/s3-protection.html>

- Enable GuardDuty Malware Protection for S3 on an entire AWS organization level to detect potential malware uploaded or stored in S3 buckets, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/configuring-malware-protection-for-s3-guardduty.html>

- Enable GuardDuty Malware Protection for EC2 to detect potential malware uploaded or stored in EBS volumes attached to EC2 instances, as explained here:

- <https://docs.aws.amazon.com/guardduty/latest/ug/gdu-initiated-malware-scan-configuration.html>
- <https://docs.aws.amazon.com/guardduty/latest/ug/malware-protection-getting-started-on-demand-scan.html>

- Enable EKS Audit Log Monitoring to detect suspicious activity inside Amazon EKS cluster logs (user activity, Kubernetes API, and control plane logs), as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/guardduty-eks-audit-log-monitoring.html>

- For sensitive/critical EC2 instances, enable GuardDuty Runtime Monitoring to detect potential threats by deploying GuardDuty security agents, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/managing-gdu-agent-ec2-automated.html>

- For sensitive/critical Amazon EKS clusters, enable GuardDuty Runtime Monitoring to detect potential threats by deploying GuardDuty security agents, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/managing-gdu-agent-eks-automatically.html>

- For sensitive/critical Amazon ECS clusters (AWS Fargate), enable GuardDuty Runtime Monitoring to detect potential threats by deploying GuardDuty security agents, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/managing-gdu-agent-ecs-automated.html>

- Enable GuardDuty RDS Protection to detect suspicious logins to Amazon Aurora and Amazon RDS databases, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/rds-protection.html>

- Enable Lambda Protection to detect potential threats in Lambda functions, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/configuring-lambda-protection.html>

- Use CloudWatch Events to get notifications such as email notifications via SNS about GuardDuty findings that require immediate response, as explained here:

https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_findings_cloudwatch.html

- Enable integration between Amazon GuardDuty and AWS Security Hub to get a central view of all compliance and security incidents inside a central console, as explained here:

<https://docs.aws.amazon.com/guardduty/latest/ug/securityhub-integration.html>

Microsoft Defender for Cloud

Microsoft Defender for Cloud is a collection of cloud-native tools designed to enhance visibility, provide alerts, and offer recommendations regarding the security posture of customers' cloud environments, including Azure, AWS, GCP, and on-premises setups.

As of writing this book, Defender for Cloud contains solutions for the following services:

- **Defender for API** provides end-to-end protection for APIs, enabling organizations to monitor, detect, and respond to potential security threats throughout an API's lifecycle.
- **Defender for Servers** empowers organizations to proactively enhance their machine security by offering comprehensive, actionable insights that help identify, assess, and systematically mitigate potential vulnerabilities across their computing environment.
- **Defender for Containers** is a comprehensive, cloud-based security platform designed to enhance, oversee, and safeguard containerized resources across diverse computing environments.
- **Defender for Databases** offers comprehensive security coverage for your organization's database infrastructure. This solution provides robust attack detection and threat response capabilities, focusing on widely-used database types within the Azure ecosystem.
- **Defender for App Service** offers seamless, built-in security integration that requires no manual configuration or additional deployment steps. The solution provides automatic, transparent protection for Azure App Service resources, ensuring comprehensive security coverage without requiring complex onboarding processes.
- **Defender for Storage** provides comprehensive protection for Azure storage resources by proactively defending against critical security threats.
- **Defender for Key Vault** is a security solution that provides an additional layer of protection for Azure Key Vault resources by detecting unusual and potentially harmful attempts to access or exploit key vault accounts through advanced threat detection and security intelligence.
- **Defender for Resource Manager** automatically monitors and protects resource management operations across your organization, detecting suspicious activities and potential threats to safeguard the Azure Resource Manager layer.
- **Defender for DNS** provides advanced protection for Azure virtual networks by monitoring DNS traffic and detecting suspicious activities such as data exfiltration, malware communication, and potential cyber threats without requiring additional agents.
- **Defender for Cloud DevOps security** is a comprehensive solution that unifies security management across multi-pipeline environments, providing code-to-cloud contextualization and automated workflows to protect applications and resources throughout the development lifecycle, from code creation to cloud deployment.

Thorough documentation on security in Microsoft Defender for Cloud can be found at the following link: <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/microsoft-defender-for-cloud-security-baseline>

Use the following best practices for configuring and securing Microsoft Defender for Cloud:

- Enable Defender for Cloud on all Azure subscriptions in the organization to gain access to security recommendations for all supported Azure services, as explained here:
 - <https://learn.microsoft.com/en-us/azure/defender-for-cloud/connect-azure-subscription>
 - <https://learn.microsoft.com/en-us/azure/defender-for-cloud/cross-tenant-management>
- Use RBAC to enforce authorization on Microsoft Defender for Cloud for human identities from Microsoft Entra ID or for applications/services accounts using managed identities from Microsoft Entra ID, following the principle of least privilege, to avoid unauthorized access, as explained here:
 - <https://learn.microsoft.com/en-us/azure/defender-for-cloud/permissions>
 - <https://learn.microsoft.com/en-us/azure/defender-for-cloud/tenant-wide-permissions-management>
- To manage permissions for identities across the Azure organization and gain insights into which identity has which permissions, enable Permissions Management (CIEM) for Azure integration with Microsoft Entra ID, as explained here:
<https://learn.microsoft.com/en-us/azure/defender-for-cloud/enable-permissions-management>
- Microsoft Defender for Cloud is built on top of the Log Analytics workspace. In the case that data in Azure Monitor/Log Analytics workspace is encrypted using customer-managed keys, grant Defender for Cloud access to the encryption keys, as mentioned here:
<https://learn.microsoft.com/en-us/azure/azure-monitor/logs/customer-managed-keys>
- To protect Azure VMs from unauthorized access, enable Microsoft Defender for Cloud **just-in-time (JIT)** access, as explained here:
<https://learn.microsoft.com/en-us/azure/defender-for-cloud/just-in-time-access-usage>
- Use the **Microsoft Cloud Security Benchmark (MCSB)** standard recommendations to detect security misconfigurations and remediate them, as explained here:
<https://learn.microsoft.com/en-us/azure/defender-for-cloud/manage-mcsb>

- To gain full security visibility of Azure environments and allow the SOC team to detect and investigate security incidents, connect Microsoft Defender for Cloud with Microsoft Sentinel, as explained here:

`https://learn.microsoft.com/en-us/azure/sentinel/connect-defender-for-cloud`

- To detect secrets (static credentials such as passwords) stored inside Azure VM disks and remediate secrets exposure, run the agentless secrets scanning feature of Defender for Cloud, as explained here:
 - `https://learn.microsoft.com/en-us/azure/defender-for-cloud/secrets-scanning-servers`
 - `https://learn.microsoft.com/en-us/azure/defender-for-cloud/remediate-server-secrets`
- To detect secrets (static credentials such as passwords) stored inside Azure Resource Manager or the AWS CloudFormation stack (i.e., infrastructure as code) and remediate secrets exposure, run the agentless secrets scanning for cloud deployments of Defender for Cloud, as explained here:
 - `https://learn.microsoft.com/en-us/azure/defender-for-cloud/secrets-scanning-cloud-deployment`
 - `https://learn.microsoft.com/en-us/azure/defender-for-cloud/remediate-cloud-deployment-secrets`

Google Security Command Center

Google Security Command Center (SCC) is a threat detection and cloud posture management service that allows you to aggregate logs and configuration from multiple sources (such as Google App Engine, BigQuery, Cloud SQL, Cloud Storage, Cloud IAM, **Google Kubernetes Engine (GKE)**, and so on) into a single console, which enables you to prioritize the work on security configuration, compliance, and threats.

Google SCC provides different capabilities in the following tiers:

- **Standard tier:**
 - Security health check (such as MFA not enforced, opened ports, resources with public IP, etc.)
 - Web security scanner for public URLs or IPs (based on the OWASP Top 10)
 - Integration with **Sensitive Data Protection** (to discover and protect sensitive data)

- Integration with **Google Cloud Armor** (for both DDoS protection and WAF capabilities)
- Integration with **Anomaly Detection** (for detecting anomalies in VMs such as crypto mining)
- Integration with **Policy Controller** (for enforcing policies on GKE clusters)
- Posture management for GKE clusters (missing patches, misconfigurations, etc.)
- **Premium tier:**
 - Attack path simulation (for identifying the path for potential attacks on high-volume resources)
 - Vulnerability management (ability to scan for vulnerabilities based on CVE score)
 - Event Threat Detection (detecting threats by analyzing Google Cloud Logging and Google Workspace logs, combined with threat intelligence and machine learning engine)
 - VM Threat Detection (detecting malicious applications in running VMs)
 - Container Threat Detection (for detecting container runtime attacks such as execution of malicious binary, running reverse shell, etc.)
 - Integration with **Policy Intelligence** (running IAM recommender to detect lateral movement and generate fine-grained IAM policies)
 - Compliance monitoring (generates compliance reports against industry standards such as CIS Benchmarks, NIST, OWASP Top 10, etc.)
 - IaC validation (compares IaC code such as Terraform, against Google organization policies, for securing the GCP environment)
- **Enterprise tier:**
 - Toxic combination: The ability to group security issues from different sources, such as opened ports, combined with public IP to identify and prioritize the handling of a security incident
 - Multi-cloud support: The ability to pull information for GCP, AWS, and Azure environments and detect threats, vulnerabilities, and misconfiguration in a single dashboard
 - Integration with **Google Security Operations**, Google's managed SIEM service for log correlation and security incident and response
 - **Security orchestration, automation, and response (SOAR)** for managing cases for security incidents, configuring response workflows, etc.
 - **Cloud Infrastructure Entitlement Management (CIEM)** for detecting IAM misconfiguration and excessive permissions

Thorough documentation on security in Google SCC can be found at the following link:

- <https://cloud.google.com/security-command-center/docs/concepts-security-command-center-overview>
- <https://cloud.google.com/security-command-center/docs/optimize-security-command-center>

Use the following best practices for configuring and securing Google SCC:

- Following activation of the Google SCC service tier, select the relevant features or services to enable (for both cost reduction and the relevancy of services in your GCP environment), as explained here:

- <https://cloud.google.com/security-command-center/docs/how-to-configure-security-command-center>
- <https://cloud.google.com/security-command-center/docs/cryptomining-detection-best-practices>

- Use Google Cloud IAM to configure the roles for using Google SCC, following the principle of least privilege, as explained here:

<https://cloud.google.com/security-command-center/docs/access-control-org>

- For sensitive/critical environments, explicitly enable data access audit logs to log activities performed on Google SCC (actions such as get source, list findings, calculate security settings, etc.) to gain visibility into non-admin actions that happened in Google SCC, as mentioned here:

<https://cloud.google.com/security-command-center/docs/audit-logging>

Important note

It is recommended to conduct risk analysis on the target service (such as a service that stores or processes credit card information, healthcare data, and so on) before enabling data event logging due to the cost of event storage versus the value that can be achieved from data event logs.

- To gain visibility into the security posture of your GCP environment (such as misconfiguration, vulnerabilities, etc.), use the **security posture** service, as explained here:

<https://cloud.google.com/security-command-center/docs/how-to-use-security-posture>

- Define your organization's high-value assets to be able to simulate potential attack paths and understand your cloud environment's critical assets exposure, as explained here:

<https://cloud.google.com/security-command-center/docs/attack-exposure-define-high-value-resource-set>

- To continuously monitor and detect threats against your GCP environment in near-real time, enable Event Threat Detection, as explained here:

<https://cloud.google.com/security-command-center/docs/how-to-use-event-threat-detection>

- Use Security Health Analytics to detect and fix misconfigurations in various GCP services (such as firewall rules, public Cloud Storage buckets, VMs with public IP addresses, etc.), as explained here:

- <https://cloud.google.com/security-command-center/docs/how-to-use-security-health-analytics>

- <https://cloud.google.com/security-command-center/docs/how-to-remediate-security-health-analytics-findings>

- Use **Web Security Scanner** on test environments before moving an application to production (such as Google App Engine, GKE, or Compute Engine applications) to detect application vulnerabilities, as explained here:

<https://cloud.google.com/security-command-center/docs/how-to-use-web-security-scanner>

In this section, we have learned how to configure threat detection and remediation services based on AWS, Azure, and GCP built-in service capabilities, including getting insights from many logs to detect anomaly behavior of users, VMs, networks, and application services, and respond to threats detected in large environments.

Next, we will review cloud-native SIEM solutions, and how to configure them to aid in an incident response process.

Managing cloud-native SIEM

Security information and event management (SIEM) is a solution for collecting logs, analyzing their content to detect potential security incidents, and allowing security teams to investigate and respond to the discovered incidents.

Cloud-native SIEM is a security solution designed specifically for cloud environments, leveraging cloud infrastructure and services to deliver advanced threat detection, investigation, and response capabilities. It offers scalability, flexibility, and rapid deployment, enabling organizations to monitor and analyze security data from diverse sources across hybrid environments in real time. It is not limited

to a single cloud provider but rather has integration with multiple cloud providers and the ability to collect logs and assist security teams in managing security incidents from on-premises environments.

AWS does not currently have its own cloud-native SIEM service, although it offers customers the following services, which can be integrated with other SIEM services:

- **Amazon CloudWatch:** Collects and stores security logs from various AWS resources, enabling comprehensive monitoring
- **AWS CloudTrail:** Monitors and records account activity across the AWS infrastructure, providing a comprehensive audit trail
- **Amazon GuardDuty:** Delivers intelligent threat detection for AWS environments
- **AWS Security Hub:** Centralizes security alerts and automates security checks across AWS accounts
- **Amazon OpenSearch:** Enables real-time search, monitoring, and analysis of large volumes of data

Additional reference documentation on how to use AWS services to build an equivalent of a SIEM service can be found here:

- *How to use AWS Security Hub and Amazon OpenSearch Service for SIEM:*

<https://aws.amazon.com/blogs/security/how-to-use-aws-security-hub-and-amazon-opensearch-service-for-siem/>

Let us review cloud-native SIEM services from Azure and GCP. We will learn some of the most common recommendations for configuring Microsoft Sentinel and Google Security Operations. However, this section does not replace the official documentation for using these services as a managed SIEM solution.

Microsoft Sentinel

Microsoft Sentinel is a cloud-native SIEM service built upon Azure Log Analytics workspaces. It uses **artificial intelligence (AI)** to detect threats in your Azure environment and assist in responding to detected threats.

Microsoft Sentinel is built on top of the following family of services:

- **Microsoft Defender for Cloud:** An application protection platform for multiple Azure services (such as VMs, containers, APIs, databases, etc.)
- **Microsoft Defender XDR (formerly Microsoft 365 Defender):** A collection of tools for detecting and protecting from threats against endpoints, the Microsoft 365 suite, identity management, vulnerability management, and SaaS applications

In the following diagram, we see a common architecture of Microsoft Sentinel:

- Multiple data sources (Microsoft Defender, Microsoft Entra ID, Azure Blob, and Azure SQL) generate diagnostic logs and send them to a central Log Analytics workspace
- Microsoft Sentinel utilizes the Log Analytics workspace to store, analyze, and query log data from various sources
- A SOC analyst uses Microsoft Sentinel to monitor, detect, investigate, and respond to security threats

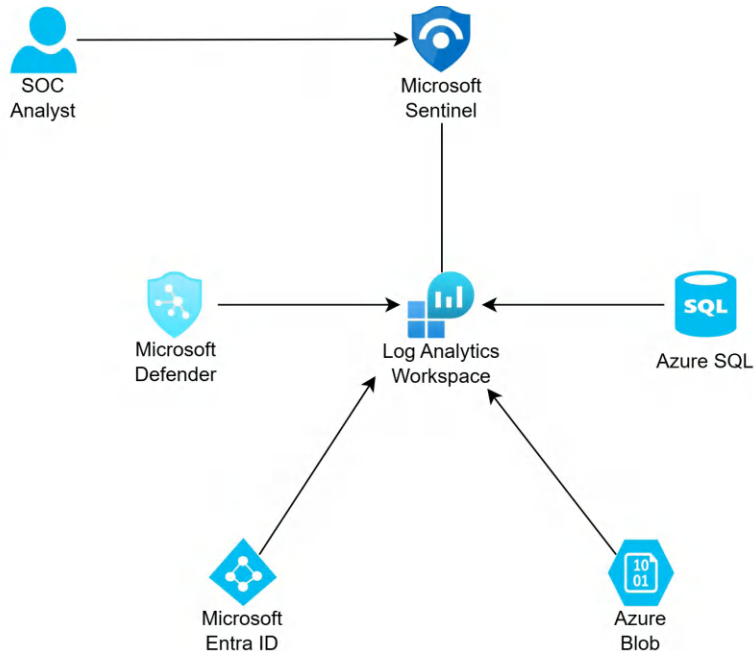


Figure 10.1 – A sample Microsoft Sentinel architecture

Thorough documentation on security in Microsoft Sentinel can be found at the following links:

- <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/microsoft-sentinel-security-baseline>
- <https://learn.microsoft.com/en-us/azure/sentinel/best-practices>

Use the following best practices for configuring and securing Microsoft Sentinel:

- Enable Microsoft Sentinel in Azure at the organizational level to be able to detect and respond to security incidents for all the Azure resources, as explained here:

<https://learn.microsoft.com/en-us/azure/sentinel/quickstart-onboard>

- When creating a Log Analytics workspace for enterprises or large Azure organizations, it is recommended to follow these best practices:
 - Create a central Log Analytics workspace for operational logs from Azure Monitor to be used by operations teams
 - Create a central Log Analytics workspace for Microsoft Sentinel, Microsoft Defender for Cloud, and Microsoft Entra audit logs to be used by security teams
 - For organizations generating more than 1 TB of logs per day, it is recommended to create a dedicated cluster for Azure Monitor logs

For further information, see the following documentation:

<https://learn.microsoft.com/en-us/azure/sentinel/best-practices-workspace-architecture>

- Use Azure RBAC to grant minimal permissions for users to be able to use Microsoft Sentinel and investigate incidents inside the console, following the principle of least privilege, as explained here:

<https://learn.microsoft.com/en-us/azure/sentinel/roles>

- For granting permissions to playbooks (Logic App workflow resources), use managed identities from Microsoft Entra ID, following the principle of least privilege, as explained here:

<https://learn.microsoft.com/en-us/azure/sentinel/automation/authenticate-playbooks-to-sentinel>

- To prioritize risk to sensitive data (such as customers' PII, financial data, healthcare information, etc.), integrate Microsoft Sentinel with **Microsoft Purview**, as explained here:

<https://learn.microsoft.com/en-us/azure/sentinel/purview-solution>

- Microsoft Sentinel is built on top of the Log Analytics workspace. In the case that data in Azure Monitor/Log Analytics workspace is encrypted using customer-managed keys, grant Microsoft Sentinel access to the encryption keys, as mentioned here:

<https://learn.microsoft.com/en-us/azure/sentinel/customer-managed-keys>

- To monitor and detect health issues with Microsoft Sentinel, enable the auditing and monitoring feature, and configure diagnostic settings to store all information inside a Log Analytics workspace, as explained here:

<https://learn.microsoft.com/en-us/azure/sentinel/enable-monitoring>

- Configure Log Analytics data retention to keep accessible data for a short term (up to 2 years), as explained here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/logs/data-retention-configure>

- Configure Auxiliary Logs to keep data for a long period (30 days to 12 years) for sources such as storage access logs, NetFlow logs, proxy logs, firewall logs, and so on, as explained here:

<https://learn.microsoft.com/en-us/azure/azure-monitor/logs/create-custom-table-auxiliary>

- Enable Microsoft Sentinel **User and Entity Behavior Analytics (UEBA)** to detect anomaly behavior of users, hosts, IP addresses, and applications in your Azure environment, as explained here:

<https://learn.microsoft.com/en-us/azure/sentinel/enable-entity-behavior-analytics>

- Integrate Microsoft Sentinel with Microsoft Defender for XDR and Microsoft Defender for Cloud to gain insights about the security posture of your Azure environment, as explained here:

- <https://learn.microsoft.com/en-us/azure/sentinel/microsoft-365-defender-sentinel-integration>

- <https://learn.microsoft.com/en-us/azure/sentinel/ingest-defender-for-cloud-incidents>

- Integrate Microsoft Entra ID with Microsoft Sentinel to gain visibility for authentication and authorization-related events, as mentioned here:

- <https://learn.microsoft.com/en-us/azure/sentinel/data-connectors/microsoft-entra-id>

- <https://learn.microsoft.com/en-us/azure/sentinel/data-connectors/microsoft-entra-id-protection>

- If you are using **Microsoft Power Platform**, integrate it with Microsoft Sentinel to be able to collect logs that will assist you in an incident response process, as explained here:

<https://learn.microsoft.com/en-us/azure/sentinel/business-applications/deploy-power-platform-solution>

- Integrate Microsoft Defender Threat Intelligence with Microsoft Sentinel to be able to correlate events in the Azure environment with **indicators of compromise (IOCs)**, as explained here:

<https://learn.microsoft.com/en-us/azure/sentinel/connect-mdti-data-connector>

- Integrate Microsoft Sentinel with **Microsoft Copilot for Security** to be able to use machine learning capabilities to aid in investigating security incidents, as explained here:

<https://learn.microsoft.com/en-us/azure/sentinel/sentinel-security-copilot>

Google Security Operations

Google Security Operations (SecOps) is a suite of managed services containing the following capabilities:

- **SIEM:** Collects data and ingests it to a central repository for further analysis by SOC teams (the service was formerly named Google Chronicle)
- **SOAR:** For managing cases for security incidents, configuring response workflows, and so on
- **Threat Intelligence:** Correlates events in the GCP environment with IOCs, based on Mandiant threat intelligence.

In the following diagram, we see a common architecture of Google SecOps:

- Multiple sources (Google Cloud IAM, Google Cloud SQL, and Cloud Monitoring) are sending data to a central Cloud Logging bucket
- Google SecOps utilizes a Cloud Logging bucket to store, analyze, and query log data from various sources
- A SOC analyst uses Google SecOps to monitor, detect, investigate, and respond to security threats

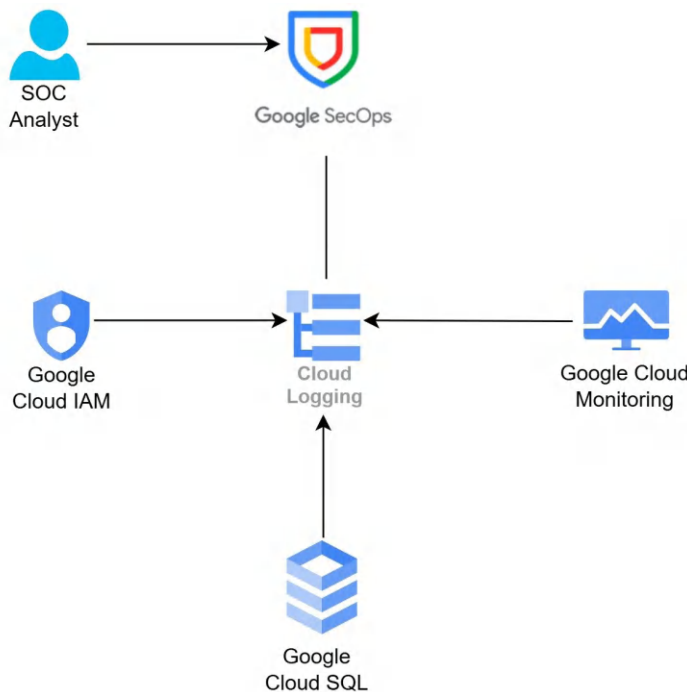


Figure 10.2 – A sample Google SecOps architecture

Use the following best practices for configuring and securing Google SecOps:

- Deploy Google SecOps in a dedicated GCP project to be able to serve your entire GCP organization, as explained here:

<https://cloud.google.com/chronicle/docs/onboard/configure-cloud-project>

- To grant access to identities to be able to connect to Google SecOps, integrate Google SecOps with **Google Cloud Identity**, as explained here:

<https://cloud.google.com/chronicle/docs/onboard/configure-cloud-authentication>

- If you are using a third-party identity provider (such as Microsoft Entra ID) for authentication purposes, integrate Google SecOps with **Google Workforce Identity Federation** to be able to keep a single identity provider, while still being able to grant access to your identities in a federation mode, as explained here:

<https://cloud.google.com/chronicle/docs/onboard/configure-authentication>

- Use IAM permissions to grant permissions to use Google SecOps and avoid unauthorized access, as documented here:

- <https://cloud.google.com/chronicle/docs/onboard/configure-feature-access>

- <https://cloud.google.com/chronicle/docs/reference/feature-rbac-permissions-roles>

- <https://cloud.google.com/chronicle/docs/soar/admin-tasks/permissions/working-with-permission-groups>

- For sensitive/critical environments, explicitly enable data access audit logs to log activities performed on Google SecOps (actions such as admin metadata, read configuration, etc.) to gain visibility into non-admin actions that happened in Google SecOps, as mentioned here:

<https://cloud.google.com/chronicle/docs/administration/audit-logging>

Important note

It is recommended to conduct risk analysis on the target service (such as a service that stores or processes credit card information, healthcare data, and so on) before enabling data event logging due to the cost of event storage versus the value that can be achieved from data event logs.

- In the case of a request to allow a Google engineer access to your GCP SecOps environment, *temporarily* enable access permission, as explained here:

<https://cloud.google.com/chronicle/docs/soar/admin-tasks/permissions/allow-google-support-access>

Note

It is crucial to disable the temporary access when it is no longer required to avoid unauthorized access.

- To assist in an incident investigation process and be able to identify internal resources in your GCP environment, assign labels to internal subnets, as explained here:

<https://cloud.google.com/chronicle/docs/soar/admin-tasks/configuration/manage-networks>

- If you do not need to keep historical data for a long period, configure a data retention policy, as explained here:

<https://cloud.google.com/chronicle/docs/soar/admin-tasks/advanced/define-system-data-retention>

In this section, we have learned how to configure managed SIEM services based on Azure and GCP services, including deployment in a cloud organization, and managing authentication, authorization, and data retention in large environments.

Summary

In this chapter, we have focused on the various monitoring/auditing, threat management, and incident response services in AWS, Azure, and GCP.

We learned how to enable auditing in our cloud environments. We also reviewed the built-in services that enable us to detect and respond to threats in large-scale environments.

We reviewed how to control and audit access to CSP support engineers, and we learned some of the best practices for configuring managed SIEM services in the cloud.

In the next chapter, we will review cryptography in various aspects (encryption fundamentals, differences between symmetric and asymmetric algorithms, key management services, secrets management, and encryption at transit and at rest).

Applying Encryption in Cloud Services

In previous chapters, we covered the fundamental building blocks of cloud services (from compute, storage, and networking services to IAM services, auditing, and threat management).

This chapter will cover various concepts regarding encryption – including the differences between **symmetric encryption** and **asymmetric encryption**, **key management services (KMSs)**, secrets management services, and using encryption in transit, at rest, and in use in cloud environments.

Since encryption is a common security best practice that is used to allow data confidentiality, and since many cloud services already have built-in support for encryption (unlike on-premises environments, which require a lot of effort to maintain encryption keys), it is crucial to understand how encryption works and how it is implemented in the various cloud services.

Failing to encrypt sensitive data (such as credit card information, healthcare data, **personally identifiable information (PII)**, intellectual property, business data, and more) puts us at risk of data exposure, as well as potential lawsuits from customers and regulators.

Encryption in the cloud allows us an additional layer of security in multi-tenancy environments (encryption at rest) and public APIs (encryption in transit).

In this chapter, we will cover the following topics:

- Introduction to encryption
- Best practices for using encryption in transit
- Best practices for using encryption at rest
- Best practices for deploying key management services
- Best practices for deploying secrets management services
- Best practices for using encryption in use

Introduction to encryption

Encryption is the process of converting plaintext into ciphertext and vice versa. The easiest way to explain why we need encryption is to imagine a scenario where we wish to transfer a file containing sensitive information (such as patient medical records) between two computers over an untrusted network such as the public internet, without being revealed by an untrusted third party.

Another example is a retail website, which processes the credit card information of its customers when they purchase products from the website. To follow the **Payment Card Industry Data Security Standard (PCI DSS)**, a standard for storing and processing credit card information, the retail company must encrypt all credit card information in transit and at rest.

Let us use a common three-tier architecture as an example – with front web servers (behind the load balancer for high availability), an application server (for processing the business logic), and a backend database (for storing purchase information).

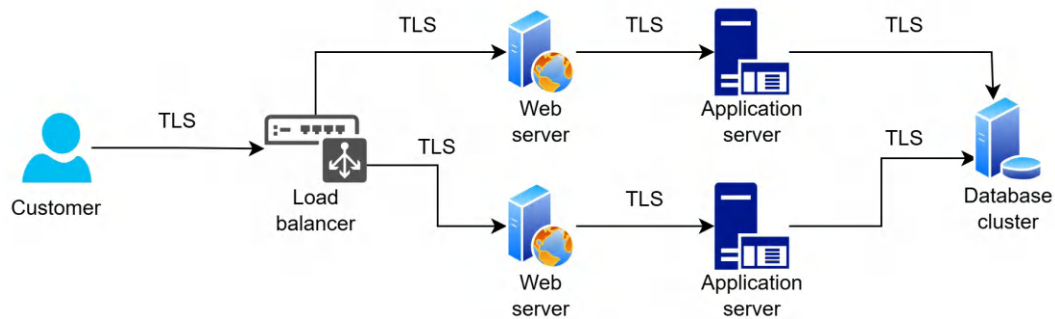


Figure 11.1 – Three-tier architecture

To protect credit card information, we need to do the following:

- While the data is in transit, it should be encrypted between all nodes (web server, load balancers, etc.) in order to guarantee confidentiality and integrity
- If the application server stores credit card information in its local hard drive (even for a short period), we must configure full disk encryption (for encryption at rest)
- We should configure full database encryption on the backend database (for encryption at rest)
- To make sure all the encryption keys are stored securely, we need to store them in a secure location (such as a KMS, as explained later in this chapter)

The following terminology relates to encryption, which we need to understand:

- **Plaintext:** This is the original human-readable form of information.
- **Ciphertext:** This is the result of encryption being done on plaintext – a non-human readable form of information.

- **Encryption:** This is the process of converting plaintext into ciphertext.
- **Decryption:** This is the process of converting ciphertext back into its original form of plaintext.
- **Encryption key:** This is a string that, together with an encryption algorithm, can encode or decode ciphertext.
- **Encryption algorithm:** This is a mathematical algorithm that is used in cryptography that, together with an encryption key, allows us to encrypt or decrypt a message.
- **Key encryption key (KEK):** This is used for generating other encryption keys. The KEK is stored inside the KMS and never leaves it since it wraps (encrypts) other keys in the hierarchy below it.
- **Data encryption key (DEK):** This is used for encrypting the data and is stored near the data itself. The DEK is encrypted using the KEK before it is stored next to the data. KMSs keep a history of DEKs and keep this information in metadata near the data itself, which allows the encrypted service to know which DEK version to use.
- **Master encryption key (MEK):** This is used for encrypting and decrypting the KEK inside the KMS.
- **Key generation:** This is the idea of regenerating new encryption keys to avoid the potential of the key being revealed by an external third party due to a static encryption key (a key that has not been replaced in a very long time) being used for a long time.

For more information, please refer to the following resources:

- *Encryption:* <https://csrc.nist.gov/glossary/term/encryption>
- *Cryptographic algorithm:* https://csrc.nist.gov/glossary/term/cryptographic_algorithm
- *Cryptography:* <https://csrc.nist.gov/glossary/term/cryptography>

Symmetric encryption

Symmetric encryption is a way to transfer data between two parties while using the same encryption key for both the encryption and decryption processes.

Symmetric encryption is commonly used for the following purposes:

- Encrypting data at rest (such as PII, credit card information, healthcare information, and so on)
- Validating that the sender of a message is who they claim to be
- Generating random numbers (as part of the cryptographic process)

The following diagram shows the process of converting plaintext into ciphertext, and vice versa, using symmetric encryption:

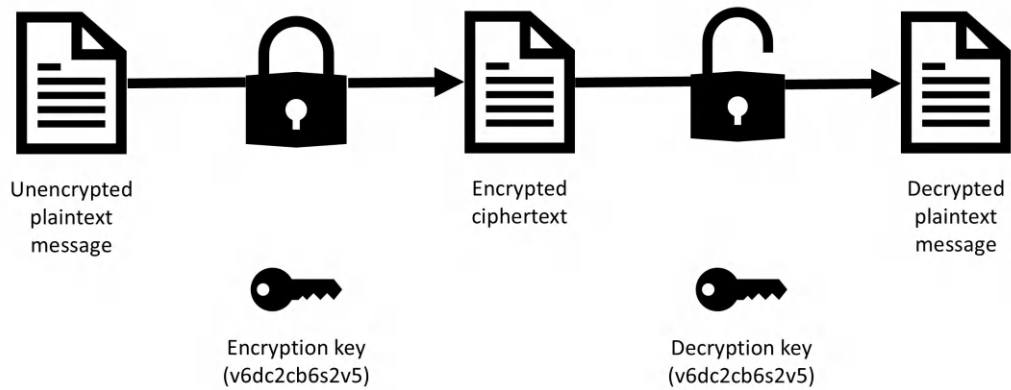


Figure 11.2 – Symmetric encryption

The preceding diagram shows how to encrypt and decrypt data using the same cryptographic key.

The following are the pros of symmetric encryption:

- Small ciphertext size
- Suitable for transferring large amounts of data
- Low CPU resources
- Small encryption key size
- Fast encryption time (compared to asymmetric encryption)

The following are the cons of symmetric encryption:

- Considered less secure than asymmetric encryption since both parties are using the same encryption key
- Considered less confidential than asymmetric encryption since both parties are using the same encryption key

Advanced Encryption Standard (AES)

AES is considered the de facto standard that's used today both on-premises and in the public cloud for encryption at rest. It was developed by NIST in 2001 and comes in variable key sizes – the most used key size is 256-bit.

For more information, please refer to the following resources:

- *Symmetric cryptography*: https://csrc.nist.gov/glossary/term/symmetric_cryptography
- *Advanced Encryption Standard*: https://csrc.nist.gov/glossary/term/advanced_encryption_standard

Asymmetric encryption

Asymmetric encryption is a way to transfer data between two parties. However, unlike symmetric encryption, in asymmetric encryption (also known as **public-key cryptography** or **public-key encryption**), we use two different keys (private and public) that are linked together mathematically.

In asymmetric encryption, the public key can be shared with anyone and is used to encrypt a message. The private key must be safely guarded and is used for two purposes – decrypting the message and signing the message to ensure message integrity (the message was not tampered with in the process).

Asymmetric encryption is commonly used for the following purposes:

- Creating digital signatures
- Distributing session keys for use cases such as the TLS protocol

The following diagram shows the process of converting plaintext into ciphertext, and vice versa, using asymmetric encryption:

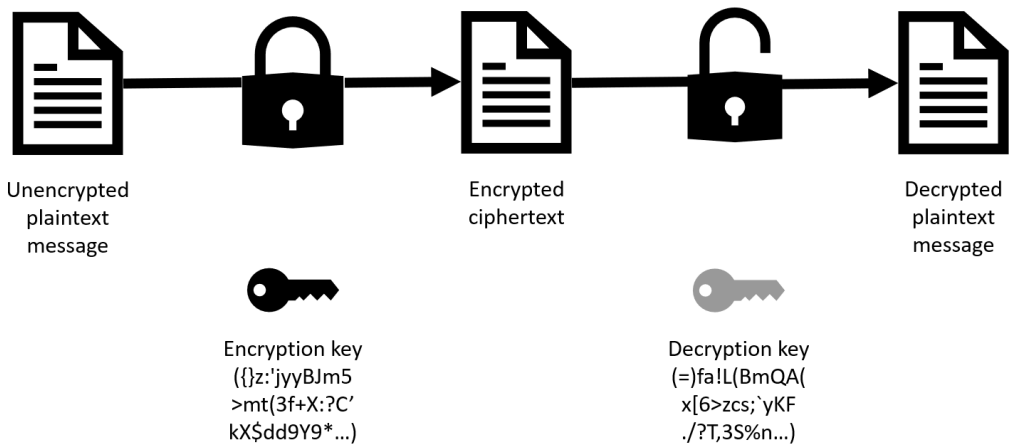


Figure 11.3 – Asymmetric encryption (public-key encryption)

The preceding diagram shows how to encrypt and decrypt data using different cryptographic keys (one for each action).

The following are the pros of asymmetric encryption:

- Large ciphertext size compared to symmetric encryption
- Large encryption key size, for example, RSA 2,048-bit or higher
- Considered much safer than symmetric encryption since there are two keys involved in the process and you cannot reveal the private key from the public key
- Considered more confidential than symmetric encryption since there are two keys and the private key (as implied must be kept private) is required in order to decrypt the message

The following are the cons of symmetric encryption:

- Suitable for transferring small amounts of data
- High CPU resources
- Slow encryption time (compared to symmetric encryption)

Rivest-Shamir-Adleman (RSA)

RSA is one of the most used protocols for encryption at transit, key establishment, and digital signatures. RSA security relies on factoring two large prime numbers (also known as the **factoring problem**). The recommended key size begins at 2,048-bit or higher.

Elliptical curve cryptography (ECC)

ECC is mostly used for key agreements, digital signatures, and pseudo-random generators.

It has a small key size (compared to the RSA algorithm) and is based on the algebraic structure of elliptic curves over finite fields.

The recommended key sizes for ECC are 256-bit (for NSA secret message classifications) and 384-bit (for NSA top-secret message classifications).

For more information, please refer to the following resources:

- *Public-key cryptography*: https://csrc.nist.gov/glossary/term/public_key_cryptography
- *RSA*: https://csrc.nist.gov/glossary/term/rivest_shamir_adleman

In this section, we reviewed the fundamental concepts regarding encryption and the differences between symmetric and asymmetric encryption (including common encryption algorithms). Now, let's dive into the best practices for using encryption.

Best practices for using encryption in transit

The idea behind *encryption in transit* is to allow two parties to share messages over a publicly exposed network, in a secure way, while retaining message confidentiality and integrity.

Internet Protocol security (IPSec)

IPSec is the most used protocol for encryption at transit, mainly for site-to-site VPN and VPN tunnels. IPSec resides on layer 3 of the OSI model.

The following are some best practices regarding IPSec:

- Use the IKEv2 protocol for **security association (SA)**
- Use AES-GCM for encryption
- Use HMAC-SHA256 (or higher) for integrity
- When supported by both the client and the server, use certificate-based authentication instead of a pre-shared key
- Use an up-to-date VPN client (to avoid known vulnerabilities)

For more information, please refer to the following resources:

- *IPSec*: <https://csrc.nist.gov/glossary/term/ipsec>
- *Guide to IPSec VPNs*: <https://csrc.nist.gov/publications/detail/sp/800-77/rev-1/final>

Transport Layer Security (TLS)

TLS is one of the most used protocols for encryption at transit.

It replaced the old **Secure Sockets Layer (SSL)** protocol (which had security flaws) and is now considered the de facto standard for encryption over the public internet.

TLS leverages asymmetric encryption to establish a secure initial connection and negotiate a shared secret key between a client and server. Once the session is established, it switches to symmetric encryption for efficient and secure data transfer.

Even though TLS relies on TCP (the transport layer of the OSI model), it is considered application layer encryption (the HTTPS of the HTTP protocol) or layer 7 of the OSI model.

Common use cases include web browsers (client) and web servers (server) such as e-commerce sites. In the context of cloud services, almost all cloud service providers' APIs use TLS for encryption in transit.

The following are some best practices regarding TLS:

- Websites, APIs, or applications that reside in a private subnet or serve internal organization employees use certificates signed by a trusted internal CA.

Note

As a best practice, avoid using self-signed certificates since they cannot be verified or trusted.

- For any publicly facing websites, APIs, or applications, use a certificate signed by a well-known public CA.
- Make sure that the certificate is valid (the domain name matches the DNS name) and that the certificate has not expired or been revoked.
- Avoid using wildcard certificates – always make sure that the certificate name matches the server's FQDN.
- Avoid using certificates with short validity periods (such as **LetsEncrypt** certificates).
- When using Kubernetes, use mTLS to encrypt traffic between Pods and authenticate both the server and client with each other.
- The minimum recommended TLS protocol version is 1.2, and when you have hardware/software to support it, it is recommended to migrate to TLS 1.3 and drop the support for previous versions of TLS and all versions of SSL protocols.
- Disable the use of weak protocols (any version of SSL and TLS that is lower than 1.2).
- Disable the use of weak cipher suites (such as 3DES/DES/RC2/IDEA).
- Use an RSA private key with a minimum size of 2,048 bits.
- Use the signature algorithm of SHA256.
- Enable the use of **HTTP Strict Transport Security (HSTS)**.
- Enable the use of **perfect forward secrecy (PFS)**.
- Use `ssllabs.com` to test the score of your TLS protocol.
- If you wish to inspect encrypted traffic in cloud services, use SSL termination (for services that support this capability). SSL termination allows you to open encrypted traffic (for example, on your load balancer or frontend web server) by temporarily decrypting the traffic, inspecting it using devices or software that support traffic inspection (such as IPS, WAF, data leak prevention, and more), and finally, encrypt the traffic again before it reaches the next hop (for example, between the load balancer and the web servers).

For more information, please refer to the following resources:

- *TLS*: <https://csrc.nist.gov/glossary/term/tls>
- *SSL and TLS Deployment Best Practices*: <https://github.com/ssllabs/research/wiki/SSL-and-TLS-Deployment-Best-Practices>
- *SSL Server Rating Guide*: <https://github.com/ssllabs/research/wiki/SSL-Server-Rating-Guide>
- *SSL Server Test*: <https://www.ssllabs.com/ssltest>

By now, we've learned about the best practices for encrypting data in transit with the most common protocols – IPSec (for VPN tunnels) and TLS (for applications such as web servers and APIs) – for algorithm selection, as well as to disable the use of weak and vulnerable ciphers. Next, let's look at encryption at rest.

Encryption at rest

The idea behind *encryption at rest* is to protect data once it has been saved into storage or a database or has been accessed by an untrusted third party. Encryption at rest has layers. For example, you may encrypt the database files at rest to protect them from being copied out or accessed from the storage directly. However, there are encryption methods also for authenticated users. For example, passwords are not saved on a database in a plaintext format. Even if the database is encrypted, the passwords are kept encrypted.

Encryption/decryption process

The following workflow explains the process of extracting an encrypted object from object storage:

- The DEK is stored near the object itself and the object metadata specifies which encryption key version to use.
- The entire DEK is wrapped with a KEK, as explained later in this chapter.
- The KEK is stored inside a key-managed service.
- When a request to access an object is made, the authorized service (or application) sends a request to the key-managed service to locate the KEK and decrypt the specific DEK.
- The decrypted DEK is sent via a TLS/SSL channel from the KMS to the object storage. The object itself is decrypted using the decrypted DEK.

The following diagram shows the preceding workflow of extracting an encrypted object from object storage:

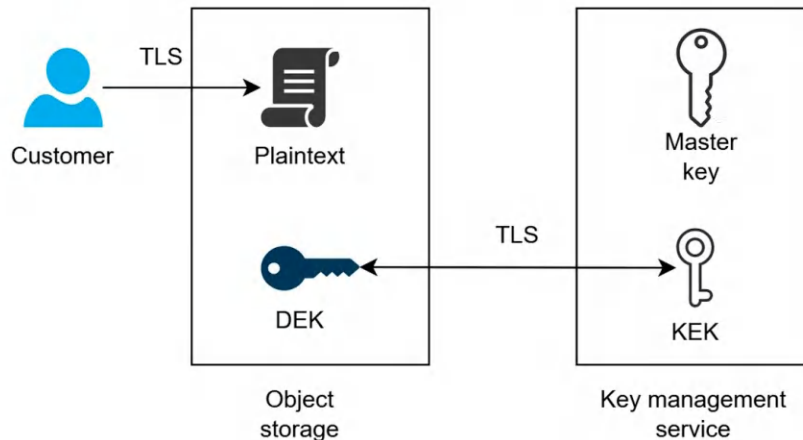


Figure 11.4 – Object storage encryption

Encryption at rest for other persistent storage (such as block storage, file storage, or database storage), works similarly – data is encrypted using a DEK, which is wrapped with a KEK (stored in a KMS, and never leaves the KMS).

Next, we are going to review best practices for deploying KMSs from AWS, Azure, and GCP.

Best practices for deploying KMSs

All major public cloud providers have implementations of a managed KMS – a secured location (or a vault) for generating, storing, and retrieving encryption keys.

General best practices for securing KMSs

At a high level, all KMSs offer the same capability – generating encryption keys inside a secured vault, allowing clients (such as managed services) to pull encryption keys, and the ability to enforce key rotation.

Regardless of the cloud environment you are using, you should follow the following best practices:

- Use **role-based access control (RBAC)** to grant minimal access to generate, request, rotate, or delete encryption keys
- For sensitive/critical environments, use customer-managed keys instead of CSP-owned encryption keys for high assurance of who has access to and controls encrypted data, and the key generation process

- Enforce key rotation to lower the risk of key compromise due to long usage without replacement
- To avoid accidental key deletion (by an authorized or unauthorized party), enable a protection mechanism to keep encryption keys from automatic deletion, by enforcing a minimum number of days before the key is completely deleted from the system
- Enable auditing for all actions related to the KMS, for both accountability (who did what) and for future security incident investigation

Next, we are going to see what the best practices are for conducting security of KMSs using AWS, Azure, and GCP built-in managed services.

AWS KMS

AWS KMS is Amazon's key management service. It allows you to generate and store cryptographic keys in a FIPS 140-2 standard.

AWS KMS generates AES256 symmetric encryption keys for encryption at rest, as well as RSA 2,048-bit (up to 4,096-bit) and ECC 256-bit (up to 512-bit) for encryption, digital signing, and signature verification.

AWS KMS supports the following types of encryption keys:

- **AWS-owned keys:** Keys generated and used by AWS for its internal encryption operations. Data is encrypted, but customers do not have any visibility to those keys.
- **AWS-managed keys:** Keys generated on behalf of customers and stored inside AWS KMS. Customers can use those keys for data encryption, but they do not control aspects of the key creation or rotation process.
- **Customer-managed keys (CMKs):** Keys generated by customers and stored inside AWS KMS. Customers control key creation and rotation processes. The key material for CMKs can be either provided by the customer or by AWS (default).

Thorough documentation on security in AWS KMS can be found at the following links:

- <https://docs.aws.amazon.com/kms/latest/developerguide/kms-security.html>
- <https://docs.aws.amazon.com/kms/latest/developerguide/best-practices.html>
- <https://docs.aws.amazon.com/kms/latest/cryptographic-details/intro.html>

Use the following best practices for securing AWS KMS:

- For sensitive environments (for example, containing PII, credit card details, healthcare data, and more), encrypt data at rest using CMKs, as explained here:

<https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html#customer-cmk>

- When using CMKs, enforce key rotation, to avoid key misuse (when using the same key for a long period), as mentioned here:

<https://docs.aws.amazon.com/kms/latest/developerguide/rotate-keys.html>

- When using CMKs, create a key policy to limit access to encryption keys (per Region), as explained here:

- <https://docs.aws.amazon.com/kms/latest/developerguide/key-policies.html>

- <https://docs.aws.amazon.com/kms/latest/developerguide/key-policy-overview.html>

- <https://docs.aws.amazon.com/kms/latest/developerguide/key-policy-services.html>

- Use IAM policies to grant access to a specific encryption key, following the principle of least privilege, as mentioned here:

- <https://docs.aws.amazon.com/kms/latest/developerguide/iam-policies-overview.html>

- <https://docs.aws.amazon.com/kms/latest/developerguide/iam-policies-best-practices.html>

- <https://docs.aws.amazon.com/kms/latest/developerguide/customer-managed-policies.html>

- For a better understanding of how key policies and IAM policies work when accessing encryption keys, and for troubleshooting purposes, read the following documentation:

<https://docs.aws.amazon.com/kms/latest/developerguide/policy-evaluation.html>

- When designing multi-region architectures (i.e., workload with resources deployed across more than one AWS region), use multi-region keys to be able to encrypt and decrypt data across regions, as explained here:

<https://docs.aws.amazon.com/kms/latest/developerguide/multi-region-keys-overview.html>

- Before permanently closing an AWS account that contains sensitive data, delete the CMKs to avoid revealing your data to an unwanted third party, as explained here:

<https://docs.aws.amazon.com/kms/latest/developerguide/deleting-keys.html>

- If you are using AWS CMKs and you have encryption keys that you are not using, disable them through either the AWS KMS console or through an API call, as explained here:

<https://docs.aws.amazon.com/kms/latest/developerguide/enabling-keys.html>

- Use tagging to have a better understanding of which KMS keys belong to which AWS resource, as explained here:

<https://docs.aws.amazon.com/kms/latest/developerguide/tagging-keys.html>

- To keep the traffic between **Amazon Virtual Private Cloud (Amazon VPC)** and AWS KMS private (i.e., within the AWS backbone), create an interface VPC endpoint, and avoid unauthorized network access to KMS, as explained here:

<https://docs.aws.amazon.com/kms/latest/developerguide/kms-vpc-endpoint.html>

- AWS CloudTrail is enabled by default for AWS KMS. To keep KMS API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, explained as follows:

<https://docs.aws.amazon.com/kms/latest/developerguide/logging-using-cloudtrail.html>

- Use Amazon CloudWatch Logs to generate alerts about key-related activities (such as KMS key pending deletion, key rotation, etc.), as mentioned here:

- <https://docs.aws.amazon.com/kms/latest/developerguide/monitoring-cloudwatch.html>

- <https://docs.aws.amazon.com/kms/latest/developerguide/deleting-keys-creating-cloudwatch-alarm.html>

- For highly sensitive environments, encrypt log data inside Amazon CloudWatch using an AWS KMS CMK and grant the CloudWatch service access to the encryption key, as explained here:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/encrypt-log-data-kms.html>

- Use Amazon S3 bucket keys to create unique data keys for objects inside an S3 bucket, which lowers the number of requests between S3 and AWS KMS (offers better performance for encrypting/decrypting objects, caching encryption closer to data, and lowering the cost per total number of requests to AWS KMS), as explained here:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-key.html>

- Use the AWS IAM Access Analyzer API to detect external entities with access to AWS KMS keys, as explained here:

<https://aws.amazon.com/blogs/security/how-to-use-aws-iam-access-analyzer-api-to-automate-detection-of-public-access-to-aws-kms-keys/>

- Use AWS Config to monitor for AWS KMS configuration changes (such as disabled keys that were enabled), as explained here:

<https://aws.amazon.com/blogs/security/how-to-use-aws-config-to-determine-compliance-of-aws-kms-key-policies-to-your-specifications/>

Azure Key Vault (as a key management service)

Azure Key Vault is the Azure key, secrets, and certificate management service. It allows you to generate and store cryptographic keys in a FIPS 140-2 Level 2 standard.

Azure Key Vault generates RSA 2,048-bit (up to 4,096-bit) and ECC 256-bit (up to 512-bit), for encryption, digital signing, and signature verification.

Azure Key Vault supports the following types of encryption keys:

- **Microsoft-managed keys** (also known as **platform-managed keys**): Keys generated and used by Microsoft for its internal encryption operations. Data is encrypted, but customers do not have any visibility to those keys.
- **CMKs**: Keys generated on behalf of customers and stored inside Azure Key Vault. Customers control key creation and rotation processes.
- **Customer-provided keys**: Keys generated by customers and stored inside the customer's key store. Customers control key creation and rotation processes.

Thorough documentation on security in Azure Key Vault can be found at the following links:

- <https://learn.microsoft.com/en-us/security/benchmark/azure/baselines/key-vault-security-baseline>
- <https://learn.microsoft.com/en-us/azure/key-vault/general/best-practices>

Use the following best practices for securing Azure Key Vault as a KMS:

- Create a separate key vault for each environment (Dev, Test, and Prod), and for sensitive/critical applications, create a separate key vault to decrease the blast radius in case of application/infrastructure compromise.
- For sensitive environments (for example, containing PII, credit card details, healthcare data, and more), encrypt data at rest using CMKs. For more information, see the following link:

<https://learn.microsoft.com/en-us/azure/security/fundamentals/encryption-models>

- Enforce authentication using identities from Microsoft Entra ID for any access to keys inside Azure Key Vault. For non-interactive access (such as applications or services trying to access Azure resources), use managed identities from Microsoft Entra ID, as explained here:

<https://learn.microsoft.com/en-us/azure/key-vault/general/authentication>

- Use Azure RBAC to grant access to keys inside Azure Key Vault, following the principle of least privilege, as explained here:

<https://learn.microsoft.com/en-us/azure/key-vault/general/rbac-guide>

- Use **Microsoft Entra Conditional Access policies** to restrict who can access Azure Key Vault (for example, a VM inside a private subnet), as explained here:

<https://learn.microsoft.com/en-us/azure/key-vault/general/security-features#conditional-access>

- Use Microsoft Entra **Privileged Identity Management (PIM)** to control access to Azure Key Vault. For the highest level of access control to Azure Key Vault, create a different Entra ID group for each key vault (for example, split them between Prod and Dev) and use Microsoft Entra PIM to control **just-in-time (JIT)** access to each key vault by different members of your organization, according to their job requirements (*Owner* for creating new keys, *Reader* for accessing keys, and so on). For more information, see the following link:

<https://learn.microsoft.com/en-us/entra/id-governance/privileged-identity-management/pim-configure>

- As a best practice, rotate all the encryption keys every 365 days (a balance between security and administrative overhead), as explained here:

<https://learn.microsoft.com/en-us/azure/key-vault/keys/how-to-configure-key-rotation>

- Enable soft delete and purge protection to avoid accidental key deletion, as explained here:
<https://learn.microsoft.com/en-us/azure/key-vault/general/soft-delete-overview>
- Use tagging to have a better understanding of which Key Vault key belongs to which Azure resource, as mentioned here:
<https://learn.microsoft.com/en-us/azure/key-vault/keys/about-keys-details>
- Configure the Key Vault firewall to restrict access to the Key Vault service from trusted services or internal IP addresses, as explained here:
 - <https://learn.microsoft.com/en-us/azure/key-vault/general/network-security>
 - <https://learn.microsoft.com/en-us/azure/key-vault/general/access-behind-firewall>
- To secure access from your VNet to Azure Key Vault, use an Azure private endpoint, which avoids sending network traffic outside your VNet through a secure channel, as explained here:
<https://learn.microsoft.com/en-us/azure/key-vault/general/private-link-service>
- Enable diagnostic settings for Azure Key Vault and send all logs to a Log Analytics workspace, as explained here:
<https://learn.microsoft.com/en-us/azure/azure-monitor/essentials/create-diagnostic-settings>
- Use Key Vault insights (within **Azure Monitor**) to gather information about performance, failure, and latency issues, as explained here:
<https://learn.microsoft.com/en-us/azure/key-vault/key-vault-insights-overview>
- Enable **Microsoft Defender for Key Vault** (part of **Microsoft Defender for Cloud**) to be able to detect threats against keys, certificates, connection strings, and so on, as explained here:
<https://learn.microsoft.com/en-us/azure/defender-for-cloud/tutorial-enable-key-vault-plan>

In this section, we have learned how to configure and secure Azure Key Vault for the entire lifecycle of encryption keys.

Google Cloud KMS

Google Cloud KMS is GCP's key management service. It allows you to generate and store cryptographic keys in FIPS 140-2 Level 1 validated as a software-based solution.

FIPS is a US government standard for cryptographic modules (both hardware and software).

Google Cloud KMS generates AES256 symmetric encryption keys for encryption at rest as well as RSA 2,048-bit (up to 4,096-bit) and ECC 256-bit (up to 384-bit) for asymmetric encryption, digital signing, and signature verification.

Google Cloud KMS supports the following types of encryption keys:

- **Default encryption keys (also known as Google-managed encryption keys):** Keys generated and used by Google for its internal encryption operations. Data is encrypted but customers do not have any visibility to those keys.
- **Customer-managed encryption keys (CMEKs):** Keys generated by customers and stored inside Cloud KMS. Customers control key creation and rotation processes.
- **Customer-supplied encryption keys (CSEKs):** Keys generated by customers and provided by the customers to Google Cloud Storage or Google Compute Engine when required for the encryption/decryption process. Customers control key creation and rotation processes.

Thorough documentation on security in Google Cloud KMS can be found at the following link: <https://cloud.google.com/docs/security/key-management-deep-dive>

- Use Google Cloud IAM to configure the roles for using Google Cloud KMS, following the principle of least privilege, as explained here:
 - <https://cloud.google.com/kms/docs/iam>
 - <https://cloud.google.com/kms/docs/reference/permissions-and-roles>

Use the following best practices for securing Google Cloud KMS:

- For maximum security and to enforce the principle of separation of duties, create Cloud KMS in its own GCP project, remove the *Owner* role at the GCP project level, and grant *Organization Admin* at the GCP organization level, to restrict the ability to manage or use encryption keys, as explained here:

<https://cloud.google.com/kms/docs/separation-of-duties>

- For sensitive environments (for example, containing PII, credit card details, healthcare data, and more), encrypt data at rest using CMEKs, as mentioned here:
 - <https://cloud.google.com/kms/docs/cmek>
 - <https://cloud.google.com/kms/docs/use-keys-google-cloud>
- As a best practice, rotate all the encryption keys every 365 days (a balance between security and administrative overhead), as explained here:
<https://cloud.google.com/kms/docs/rotate-key>
- Use labeling to have a better understanding of which Cloud KMS keys belong to which GCP resource, as explained here:
<https://cloud.google.com/kms/docs/creating-managing-labels>
- Explicitly enable **data access audit** logs to log activities that are performed on Google Cloud KMS (such as `List Key`, `Get Key Version`, etc.), as explained here:
<https://cloud.google.com/kms/docs/audit-logging>

Important note

It is recommended to conduct risk analysis on the target service (such as a service that stores or processes credit card information, healthcare data, and so on) before enabling data event logging, due to the cost of event storage versus the value that can be achieved from data event logs.

- If you are not using previous versions of your encryption keys, manually disable them, as explained here:
<https://cloud.google.com/kms/docs/enable-disable>
- To restrict access to Google Cloud KMS from trusted sources and reduce the risk of data exfiltration, use VPC Service Controls, as explained here:
 - <https://cloud.google.com/vpc-service-controls/docs/create-service-perimeters>
 - <https://cloud.google.com/vpc-service-controls/docs/manage-service-perimeters>
- Use the **Google Cloud KMS inventory service** to have an inventory of all keys generated by Cloud KMS, as documented here:
<https://cloud.google.com/kms/docs/audit-logging-kms-inventory>

- Use **Google Security Command Center** to find vulnerabilities in Cloud KMS-related activities (such as KMS key not rotated, publicly accessible KMS key, etc.), as mentioned here:

<https://cloud.google.com/security-command-center/docs/concepts-vulnerabilities-findings>

In this section, we learned how to secure KMS based on AWS, Azure, and GCP infrastructure – from authorization and network access control to auditing and monitoring. Next, we are going to review best practices for deploying secrets management services.

Best practices for deploying secrets management services

All major cloud providers have a secrets management service.

The term **secrets** refers to static credentials such as database credentials, API keys, tokens, usernames/passwords, and more. The fundamental concept behind secrets management services is to allow you to store such sensitive information in a secure location, with authorization and audit mechanisms, while allowing users and service accounts to securely retrieve secrets, without having to store the information in cleartext in configuration files.

Since the cloud environment is highly dynamic, it is recommended to have a central and secure repository for storing, generating, and retrieving secrets.

DevOps teams can benefit from secrets management services by redirecting their code to the secrets management services, instead of embedding sensitive information (secrets, credentials, access keys, and more) inside cleartext code.

Regardless of the cloud environment you are using, you should follow these best practices:

- Avoid storing static credentials in configuration files or code. Always use a secrets manager to generate, retrieve, rotate, and revoke secrets.
- Avoid storing sensitive information (such as secrets) inside your log files.
- Use RBAC to grant minimal access to generate, request, rotate, or delete secrets.
- Enforce secrets rotation to lower the risk of secret compromise due to long usage without replacement.
- Enable auditing for all actions related to the secrets manager service, for both accountability (who did what) and for future security incident investigation.

AWS Secrets Manager

AWS Secrets Manager is Amazon's managed secrets service. AWS Secrets Manager allows you to create, store, rotate, label, and manage the versioning of your secrets. Secrets managed by AWS Secrets Manager are encrypted using AWS KMS.

Thorough documentation on security in AWS Secrets Manager can be found at the following link: <https://docs.aws.amazon.com/secretsmanager/latest/userguide/security.html>

Use the following best practices for securing AWS Secrets Manager:

- Use IAM policies to grant access to a specific secret, following the principle of least privilege, as mentioned here:

https://docs.aws.amazon.com/secretsmanager/latest/userguide/auth-and-access_iam-policies.html

- For sensitive environments (for example, containing PII, credit card details, healthcare data, and more), encrypt secrets using CMKs, as mentioned here:

https://docs.aws.amazon.com/secretsmanager/latest/userguide/manage_update-encryption-key.html

- To centrally store secrets from multiple AWS Regions, enable Replica Secret and update a Replica Secret encryption key, as explained here:

<https://docs.aws.amazon.com/secretsmanager/latest/userguide/replicate-secrets.html>

- Use tagging to have a better understanding of which secret belongs to which AWS resource, as explained here:

https://docs.aws.amazon.com/secretsmanager/latest/userguide/managing-secrets_tagging.html

- To avoid secret compromise due to excessive usage without rotation, enable secret rotation at least every 60 days, as explained here:

https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotate-secrets_managed.html

- AWS CloudTrail is enabled by default for AWS Secrets Manager. To keep Secrets Manager API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here:

<https://docs.aws.amazon.com/secretsmanager/latest/userguide/monitoring-cloudtrail.html>

- To keep the traffic between Amazon VPC and AWS Secrets Manager private (i.e., within the AWS backbone), create an interface VPC endpoint and avoid unauthorized network access to Secrets Manager, as explained here:

<https://docs.aws.amazon.com/secretsmanager/latest/userguide/vpc-endpoint-overview.html>

- Use Amazon CloudWatch Logs to monitor for secrets pending deletion or secret rotation, as explained here:

https://docs.aws.amazon.com/secretsmanager/latest/userguide/monitoring_cloudwatch_deleted-secrets.html

- Use Amazon GuardDuty to detect anomalous behavior related to secrets (such as batch requests for secret retrieval), as mentioned here:

- https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_finding-types-iam.html
- <https://docs.aws.amazon.com/secretsmanager/latest/userguide/monitoring-guardduty.html>

- Use AWS Config to monitor for compliance in AWS Secrets Manager (such as the use of CMKs, keys without rotation configured, etc.), as mentioned here:

<https://docs.aws.amazon.com/secretsmanager/latest/userguide/configuring-awsconfig-rules.html>

Azure Key Vault (as a secrets management service)

As previously mentioned, Azure Key Vault is not only a KMS but also a secrets management and certificate management service.

Use the following best practices for securing Azure Key Vault as a secrets management service:

- Follow all the previously mentioned best practices for securing Azure Key Vault (including authentication, RBAC, network access, auditing, monitoring, and threat management) that are mentioned under *Azure Key Vault (as a key management service)*.

- Best practices for using Azure Key Vault for secrets management can be found at the following link:

<https://learn.microsoft.com/en-us/azure/key-vault/secrets/secrets-best-practices>

- To avoid secret compromise due to excessive usage without rotation, enable secret rotation at least every 60 days, as explained here:

<https://learn.microsoft.com/en-us/azure/key-vault/secrets/tutorial-rotation>

- Use Azure Key Vault to store x509 certificates, such as your organizational PKI CA private key, as documented here:

<https://learn.microsoft.com/en-us/azure/key-vault/certificates/about-certificates>

- To avoid certificate compromise due to excessive usage without rotation, configure automatic certificate rotation, as explained here:

<https://learn.microsoft.com/en-us/azure/key-vault/certificates/tutorial-rotate-certificates>

Google Secret Manager

Google Secret Manager is the GCP-managed secrets service for managing API keys, passwords, certificates, and more. It allows you to create, store, rotate, label, and manage the versioning of your secrets. Secrets managed by Google Secret Manager are encrypted using Google Cloud KMS.

Use the following best practices for securing Google Secret Manager:

- Best practices for using Google Secret Manager can be found at the following link:

<https://cloud.google.com/secret-manager/docs/best-practices>

- Use Google Cloud IAM to configure the roles for using Google Secret Manager, following the principle of least privilege, as explained here:

<https://cloud.google.com/secret-manager/docs/access-control>

- For sensitive environments (for example, containing PII, credit card details, healthcare data, and more), encrypt secrets using CMEKs, as mentioned here:

<https://cloud.google.com/secret-manager/docs/cmek>

- To centrally store secrets from multiple Google regions, create a replication policy, as mentioned here:

<https://cloud.google.com/secret-manager/docs/choosing-replication>

- To avoid secret compromise due to excessive usage without rotation, enable secret rotation at least every 60 days, as explained here:

<https://cloud.google.com/secret-manager/docs/secret-rotation>

- If you have any secrets and you are not using them anymore, disable old secret versions before they are deleted to avoid the possibility that you might need an old version if it is destroyed and the data becomes inaccessible, as explained here:

<https://cloud.google.com/secret-manager/docs/enable-secret-version>

- Enable delay destruction of secret versions to avoid accidental secret deletion, as explained here:
<https://cloud.google.com/secret-manager/docs/delay-destruction-of-secret-versions>
- Use tagging to have a better understanding of which secret belongs to which GCP resource, as explained here:
<https://cloud.google.com/secret-manager/docs/create-and-manage-tags>
- Explicitly enable data access audit logs to log activities that are performed on Google Secret Manager (such as `get secret version`, `get secret location`, etc.), as explained here:
<https://cloud.google.com/secret-manager/docs/audit-logging>

Important note

It is recommended to conduct risk analysis on the target service (such as a service that stores or processes credit card information, healthcare data, and so on) before enabling data event logging, due to the cost of event storage versus the value that can be achieved from data event logs.

- Use the Cloud Asset Inventory service to have an inventory of all secrets generated by Google Secret Manager, as documented here:
<https://cloud.google.com/secret-manager/docs/analyze-resources>
- Use Google Security Command Center to find vulnerabilities in Google Secret Manager activities (such as locating secrets in environment variables), as mentioned here:
<https://cloud.google.com/security-command-center/docs/concepts-vulnerabilities-findings>
- To restrict access to Google Secret Manager from trusted sources and reduce the risk of data exfiltration, use VPC Service Controls, as explained here:
 - <https://cloud.google.com/vpc-service-controls/docs/create-service-perimeters>
 - <https://cloud.google.com/vpc-service-controls/docs/manage-service-perimeters>

In this section, we learned how to manage secrets using managed services in AWS, Azure, and GCP, from authorization to key rotation, encryption, auditing, and threat management. Next, we are going to review encryption in use.

Encryption in use

At this point, we understand the concept of protecting data using encryption in transit and encryption at rest. There is still one place we need to protect data: while the data is being used in the server's memory – that is, *encryption in use* when an application requires access to the decryption key.

This is where confidential computing comes into the picture.

Confidential computing safeguards active data by executing computations within a verified, hardware-based **trusted execution environment (TEE)**. These TEEs create secure, isolated spaces that block unauthorized access or alteration of applications and data during processing.

Different cloud providers implement the concept of confidential computing slightly differently, but at the end of the day, the concept is the same – attestation of access requests to encryption keys, at the hardware level (i.e., CPU), for protecting sensitive data (such as PII, financial data, and more).

In this section, we will review how each cloud provider implements confidential computing.

AWS Nitro Enclaves

Since 2018, all new EC2 instances have been built on top of a new architecture called **AWS Nitro System**, where the main component (also known as the *parent EC2 instance*) is divided into the following components:

- **Nitro cards:** Dedicated cards that control access to the physical host:
 - **Network card:** Provides access to the Amazon VPC
 - **Storage card:** Provides access to EBS storage
 - **Local storage card:** Provides access to local instance storage
- **Nitro hypervisor:** Minimized hypervisor that provides resource isolation between EC2 instances
- **Nitro security chip:** Provides secure boot capabilities, hardware root of trust, and protection from firmware moderation

AWS Nitro system has been supported by all EC2 instances since the beginning of 2018, including RDS, Aurora, ECS, EKS, and Lambda.

In 2020, AWS released **AWS Nitro Enclaves**, which adds another layer of isolation, for memory and CPU, for all applications and users running on the same EC2 instance, and protects sensitive information residing in memory. Once Nitro Enclaves are enabled, an attestation is done using a KMS policy to verify the expected OS image, application, IAM role (for identity access), and instance ID.

In the following diagram, a user is logging in to an EC2 instance and running an application.

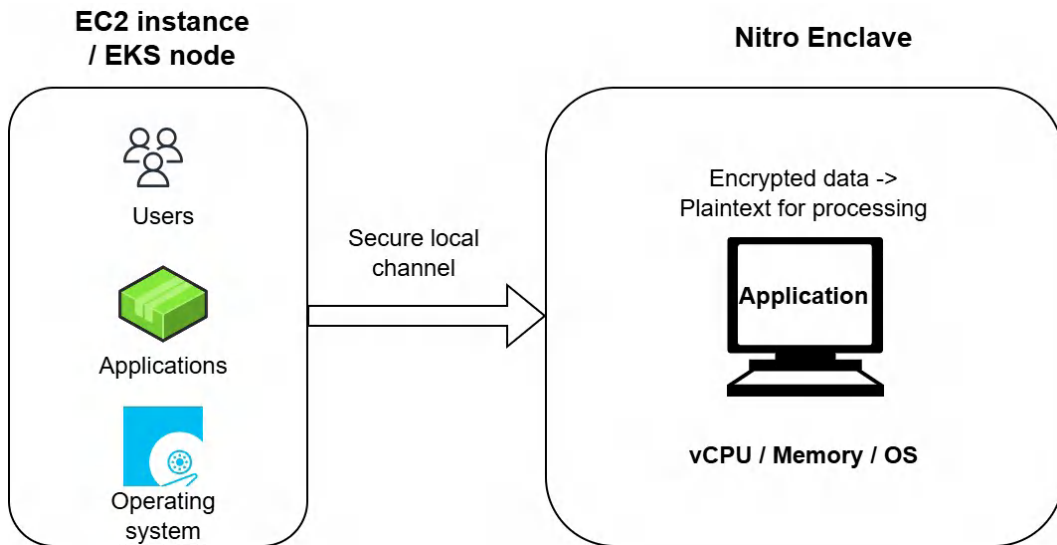


Figure 11.5 – AWS Nitro Enclaves

Once the user wants to gain access to sensitive information, the request is passed to Nitro Enclaves through a secure local channel, which is forwarded to an isolated version of the application in memory that contains the sensitive information.

AWS Nitro System, combined with Nitro Enclaves, provides the following security benefits:

- Secure boot, based on a **trusted platform module (TPM)**
- Cryptographically-signed software and firmware binaries
- Authenticated and encrypted channel to system management APIs
- Traffic encryption between EC2 instances
- Volume encryption at rest, based on AWS KMS
- Isolation from AWS operators
- Protection against EC2 side-channel attacks

Thorough documentation on security in AWS Nitro System and Nitro Enclaves can be found at the following links:

- <https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/security-design-of-aws-nitro-system.html>
- <https://docs.aws.amazon.com/enclaves/latest/user/nitro-enclave.html>

Use the following best practices related to AWS Nitro System:

- Use the latest generation of EC2 instances to gain the full benefit of AWS Nitro System, as documented here:

<https://docs.aws.amazon.com/ec2/latest/instancetypes/ec2-nitro-instances.html>

- For sensitive environments (for example, containing PII, credit card details, healthcare data, and more), enable Nitro Enclaves, as explained here:

<https://docs.aws.amazon.com/enclaves/latest/user/flow.html>

Azure confidential computing

Azure confidential computing uses hardware to isolate data. Data can be encrypted in use by running it in a TEE.

Azure supports multiple types of confidential computing technologies:

- **AMD Secure Encrypted Virtualization-Secure Nested Paging (SEV-SNP)**: Useful for lift-and-shift migration from on-premises to Azure, to build VM-level confidentiality, based on AMD processors
- **Intel Trust Domain Extensions (Intel TDX)**: Useful for lift-and-shift migration from on-premises to Azure, to build VM-level confidentiality, based on Intel processors
- **Intel Software Guard Extensions (Intel SGX)**: Useful for creating VMs and application enclaves, based on Intel processors

Azure confidential computing is not limited to VMs; it can also be used for running Azure AKS worker nodes, and for creating app-enclave aware containers.

In the following diagram, we can see how an Azure AKS confidential compute node isolates two different containers, including their code and data, from each other:

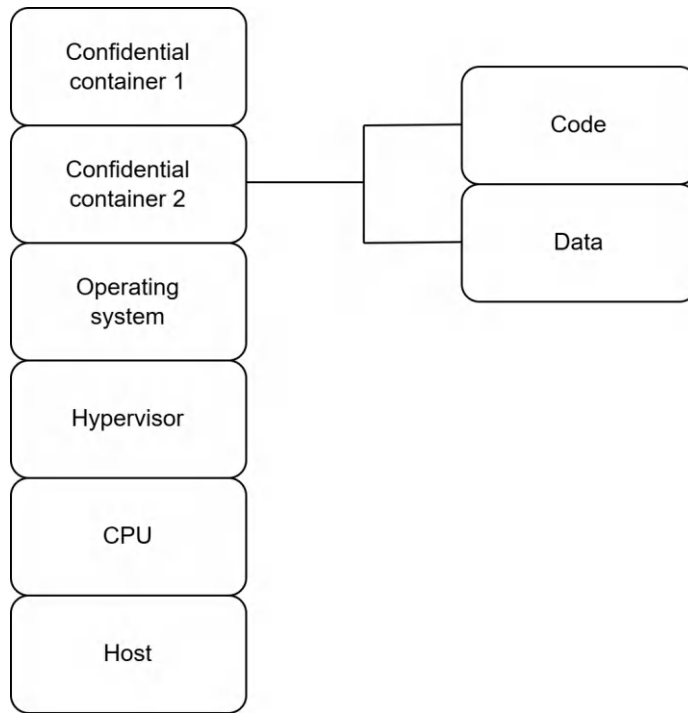


Figure 11.6 – AKS confidential compute node – container isolation

Azure confidential computing provides the following security benefits:

- **Prevents unauthorized access:** All access requests to data must be authorized and attested at the CPU level
- **Regulatory compliance:** It protects customers' information (such as PII, credit card numbers, etc.)
- **Enables secure collaboration:** It allows the secure sharing of data between different parties (such as organizations) over an untrusted network
- **Isolates processing:** It allows the ability to create isolated applications (from VMs to containers) while keeping code and data secured

Thorough documentation on security in Azure confidential computing can be found at the following link: <https://learn.microsoft.com/en-us/azure/confidential-computing/>

Use the following best practices related to Azure confidential computing:

- For sensitive environments (for example, containing PII, credit card details, healthcare data, and more) deployed on top of VMs, use one of the supported Azure confidential VMs, as explained here:

`https://learn.microsoft.com/en-us/azure/confidential-computing/virtual-machine-options`

- For sensitive environments deployed on top of AKS, use confidential VM worker nodes, as explained here:

`https://learn.microsoft.com/en-us/azure/confidential-computing/choose-confidential-containers-offerings`

- For sensitive environments using Azure SQL as a backend database, configure Always Encrypted with secure enclaves, as explained here:

`https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/configure-always-encrypted-enclaves`

- For scenarios that require sharing sensitive data between different organizations, use data cleanrooms, as explained here:

`https://learn.microsoft.com/en-us/azure/confidential-computing/multi-party-data`

Google Confidential Computing

Google Confidential Computing uses hardware to isolate data. Data can be encrypted in use by running it in a TEE.

Google supports multiple types of confidential computing technologies:

- **AMD Secure Encrypted Virtualization (SEV):** Useful for lift-and-shift migration from on-premises to GCP, for high-performance computational tasks, to build VM-level confidentiality, based on AMD processors
- **SEV-SNP:** Useful for lift-and-shift migration from on-premises to Google, to build VM-level confidentiality, based on AMD processors
- **Intel TDX:** Useful for lift-and-shift migration from on-premises to Google, to build VM-level confidentiality, based on Intel processors

Google Confidential Computing is not limited to VMs; it can also be used for running Google GKE worker nodes and Google Dataproc (managed Apache Hadoop clusters).

In the following diagram, we can see how Google Confidential Computing isolates two different applications, located on the same physical host, from each other:

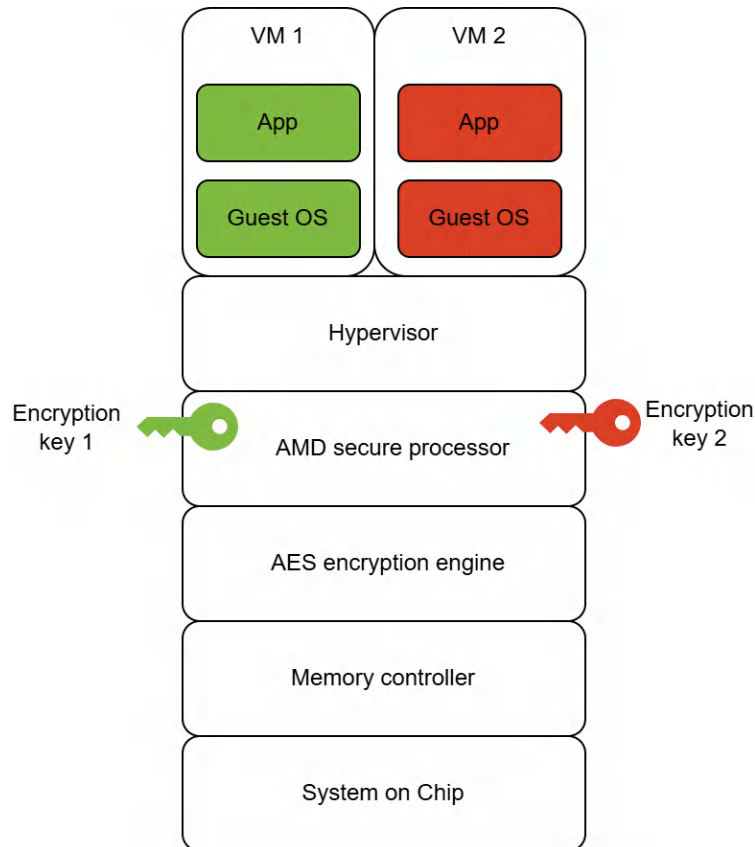


Figure 11.7 – Google Confidential Computing application isolation

Google Confidential Computing provides the following security benefits:

- Isolation between compute nodes (including VMs and containers)
- Attestation for making sure a VM has not been compromised
- Ability to share resources between different parties (organizations) over an untrusted network

Use the following best practices related to Google Confidential Computing:

- For sensitive environments (for example, containing PII, credit card details, healthcare data, and more) deployed on top of VMs, use one of the supported Google Confidential VMs, as explained here:

<https://cloud.google.com/confidential-computing/confidential-vm/docs/create-a-confidential-vm-instance>

- For sensitive environments deployed on top of GKE, use Confidential VM worker nodes, as explained here:

<https://cloud.google.com/kubernetes-engine/docs/how-to/confidential-gke-nodes>

- For sensitive environments based on Apache Hadoop, use Dataproc Confidential Compute, as explained here:

<https://cloud.google.com/dataproc/docs/concepts/configuring-clusters/confidential-compute>

- For scenarios that require sharing sensitive data between different organizations, use Confidential Space, as explained here:

- <https://cloud.google.com/docs/security/confidential-space>
- <https://cloud.google.com/confidential-computing/confidential-space/docs/confidential-space-overview>

In this section, we learned about the various alternatives that AWS, Azure, and GCP offer customers for protecting data in use (from AWS Nitro Enclaves to Azure and Google Confidential Computing).

Summary

In this chapter, we focused on the various encryption alternatives based on AWS, Azure, and GCP.

We began by introducing the concepts of encryption (symmetric and asymmetric algorithms). We continued by introducing the best practices for using KMSs (access control, auditing, and monitoring). Then, we started talking about secrets management services (access control, auditing, and monitoring). Finally, we introduced confidential computing in the cloud.

Throughout this chapter, we discussed encryption at different stages of the data lifecycle, and we concluded with a short section about encryption in use. Following the shared responsibility model, customers can use their encryption keys, which increases their ability to control the data that is stored in the cloud.

Data needs to be protected at all stages:

- **Encryption in transit:** Allows us to protect data as it passes a network
- **Encryption at rest:** Allows us to protect data as it is persistently stored (on disk, object storage, database, etc.), and also to decide who controls the encryption keys (the CSP or us as customers with CMKs)
- **Encryption in use/confidential computing:** Allows us to protect access to data stored in the memory of servers
- **Secrets managers:** Allows us to manage the entire lifecycle of static credentials (such as passwords, API keys, etc.)

Knowing about the available options for encryption will allow you to choose the most suitable solution for each service you are using in the cloud.

In the next chapter, we will review common security threats to cloud computing (data breaches, misconfiguration, insufficient IAM, account hijacking, insider threats, insecure APIs, and the abuse of cloud services).

Part 3:

Threat and Vendor Management

Understanding security threats and working with cloud providers are key to protecting cloud services. Cyber risks such as data breaches, misconfigurations, and insider threats can expose sensitive information. Strong vendor management ensures that cloud providers follow security best practices and meet compliance requirements. This part explores common threats and how organizations can effectively engage with vendors to build a secure cloud environment.

This part comprises the following chapters:

- *Chapter 12, Understanding Common Security Threats to Cloud Services*
- *Chapter 13, Engaging with Cloud Providers*

Understanding Common Security Threats to Cloud Services

In previous chapters, we covered the fundamental building blocks of securing cloud services (including services for compute, storage, networking, **identity and access management (IAM)**, auditing, and threat management), as well as looking at encryption for cloud services.

This chapter will cover the other side of the equation: common security threats to cloud services. We will also consider how to manage these threats by monitoring and responding to them.

Knowing the threats your organization may face when using cloud services will give you an understanding of what to look for and how to better protect your cloud environments in advance. Getting breached is more a question of *when* rather than *if*, so the knowledge presented in this chapter should help you to be prepared for such an eventuality.

In this chapter, we will learn how to detect and mitigate the following:

- Data breaches in cloud services
- Misconfigurations in cloud services
- Insufficient IAM, secrets, and key management in cloud services
- Account hijacking in cloud services
- Insider threats in cloud services
- Insecure **application programming interfaces (APIs)** in cloud services
- The abuse of cloud services
- The MITRE ATT&CK framework

Data breaches in cloud services

A **data breach** is the unauthorized access or disclosure of an organization's data (such as business internal information, customer data, third-party data, and so on). This can result in the exposure of customers' or employees' personal data and lead to financial and reputational damage for an organization. Does this mean that storing our data in the cloud makes it more prone to data breaches? It depends on the cloud service model and the maturity of the **cloud service provider (CSP)**. According to the *shared responsibility model*, when using an **infrastructure-as-a-service (IaaS)** solution, we (customers) oversee the implementation of most of the security controls over the OS.

In a **platform-as-a-service (PaaS)** context, we rely on the CSP in terms of OS hardening, patch management, and backups, but it is very common in a PaaS that we can review audit logs and set proper permissions for who has application-layer access to the service.

In a **software-as-a-service (SaaS)** context, things get tricky. There is no standard for SaaS, meaning anyone who deploys an application on a **virtual machine (VM)** or resides on a public IaaS can declare themselves a SaaS provider. Depending on the maturity of the SaaS provider, we may have the option to add security controls (such as data encryption at rest), configure strong authentication (such as **Security Assertion Markup Language (SAML)** with **multi-factor authentication (MFA)**), connect to audit logs via REST APIs, and others.

Before talking about data breaches, let us try to focus on what data we store in the cloud. The following are a few examples of data we might store in the cloud:

- **Public data:** Data that we can safely store on public websites. If such data is exposed by anonymous users, it will not hurt our organization (this could be news sites, currency rates, the organization's physical address from a *Contact Us* web page, and so on). Even when talking about public data, we need to recall that the data itself may have business value (such as stock or currency values), and in this case, it is more important to keep the data integrity and trustworthiness than the data confidentiality.
- **Intellectual property (IP)/trade secrets:** This is data that we must keep safe, as exposing it will hurt our business and impact our ability to achieve revenue (examples of IP could be data from a pharmaceutical company developing a cure against COVID-19, data from a start-up developing new technology for allowing secure authentication without using passwords, and so on).
- **Personal data:** This includes any identifiable information about people (for example, contact details, credit card information, healthcare data, and so on).

Common consequences of data breaches

As a result of a data breach, several things might happen to our data:

- **Breach of confidentiality:** This could include customer, financial, or healthcare data being exposed, which could then be used by hackers and result in violation of privacy laws.
- **Data integrity:** This could include someone accessing a bank's core systems and manipulating customer financial information; as a result, a customer might notice a change in their account balance, which could result in a breach of customers' trust.
- **Availability:** One of the consequences of a data breach can be the use of ransomware that encrypts the data of the target, making it unavailable, which could result in lawsuits against the organization due to customers being unable to access their data.
- **Financial losses:** Costs related to breach detection, containment, remediation, legal fees, regulatory fines, and loss of business revenue.
- **Reputational damage:** Loss of customer trust, negative media coverage, and long-term damage to the brand's image, which can lead to customer churn and difficulty attracting new customers.
- **Operational disruption:** Downtime caused by the need to investigate and contain the breach, which can halt business operations and reduce productivity.
- **Legal penalties:** Fines and lawsuits due to non-compliance with data protection regulations (for example, the **General Data Protection Regulation (GDPR)** or the **California Consumer Privacy Act (CCPA)**), as well as potential legal actions from affected individuals.
- **Customer churn:** Affected customers may stop doing business with the organization due to loss of trust, leading to reduced sales and market share.
- **Loss of employees:** Key employees may leave due to stress or reputational damage, and it may become harder to attract new talent.
- **Increased insurance premiums:** Cyber insurance premiums may rise following a breach, adding long-term financial strain.

Example of a real event – Drizly

Drizly, an on-demand alcohol delivery service, was breached in 2020. The attack happened through the credentials for a GitHub account that were granted during a hackathon in 2018. An attacker gained access to the GitHub credentials and was able to clone one of Drizly's GitHub repositories, which contained **Amazon Web Services (AWS)** credentials to Drizly's AWS infrastructure. By manipulating a security group, the attacker was able to gain access to a **Relational Database Service (RDS)** database that contained 2.5 million customer records.

Here's an additional reference about the attack:

<https://www.breaches.cloud/incidents/drizly/>

Let us now look at some best practices for detecting and mitigating data breaches such as the one experienced by Drizly.

Best practices for detecting and mitigating data breaches in cloud environments

Before we cover controls for detecting and mitigating data breaches, it is always best practice to adhere to security frameworks (such as **International Organization for Standardization (ISO)** frameworks, the **National Institute of Standards and Technology (NIST)**, and so on) as they cover industry best practices for securing cloud environments.

Here are some controls that allow us to mitigate data breaches:

- **IAM:** IAM allows us to control who has access to our systems (both infrastructure and application-wise). Using IAM, we can enforce **role-based access control (RBAC)** mechanisms to limit permissions to resources, not only by the number of permissions but also by the amount of time to access resources (that is, temporary access permissions).
- **Networking layer:** Configure access controls (**network access control lists (NACLs)**, security groups, and more) to control who can access our resources.
- **Encryption:** Implement encryption in transit and at rest. This allows us to always keep our data confidential.
- **Auditing:** This allows us to keep track of who accessed (or tried to access) our resources and what actions have been carried out (for example, through APIs) related to resources.
- **Threat management:** This allows us to review logs and detect potential threats to our services.
- **Recovery strategy:** Prepare a **disaster recovery plan (DRP)**; for example, technical backups and snapshots of volumes, databases, and processes. This should also include procedures for the recovery of data (for example, which systems to recover first, how to automate recovery using **infrastructure as code (IaC)**, and more).

Here are some best practices to follow:

- **Use central identity management (IdM):** Enforce authentication for any access attempt (human or application/service accounts), which decreases the chance of unauthorized access.
- **Use access controls:** Enforce the maximum allowed permissions to access resources.
- **Use temporary credentials:** Prefer to use roles (such as AWS IAM roles) or temporary credentials instead of long-term credentials to lower the chance of credentials being misused by unauthorized parties.

- **Use key management:** Enforce the use of key management services for controlling the entire life cycle of encryption keys (generate, store, retrieve, rotate, revoke) to mitigate against key theft and misuse by unauthorized parties.
- **Use secrets management:** Enforce the use of secrets management services for controlling the entire life cycle of secrets (generate, store, retrieve, rotate, revoke) to mitigate against secrets theft and misuse by unauthorized parties.
- **Enforce credential rotation:** For any credentials in use (passwords, secrets, API keys, certificates, and so on), enforce automated credential rotation to prevent credentials from being misused by unauthorized parties.
- **Enforce the use of MFA:** For any interactive login (human access), enforce the use of MFA as a strong authentication protection mechanism against authentication misuse, which may allow unauthorized access to resources and applications.

Authentication should be based on at least two out of the following three factors:

- What you know (such as username and password).
- What you have (such as a physical token, smart card, and so on).
- What you are (biometrics such as fingerprint or face recognition).

The use of MFA allows one to strengthen static credentials by using dynamically changed **one-time passwords (OTPs)**, using alternatives such as Google Authenticator or Microsoft Authenticator.

- **Use a secure development life cycle:** Embed this as part of your development process – this includes controls such as the following:
 - **Input validation:** Check and validate what strings and/or data can be inserted into your application and backend database, which could result in data leakage, data manipulation, and more.
 - **Check the integrity of your data:** Make sure stored data has not been changed by unauthorized parties.
 - **Code review:** Use **static application security testing (SAST)**, **dynamic application security testing (DAST)**, or **interactive application security testing (IAST)** to detect and be able to fix vulnerabilities in your code.
 - **Software upgrades:** Scan your infrastructure and application components (such as third-party libraries/binaries) to regularly detect vulnerable components.
 - **Error handling:** Handle all errors and exceptions securely, avoiding information disclosure.
 - **Protection against application-layer attacks:** Detect and mitigate attacks such as code injection attacks, **cross-site scripting (XSS)**, and others, by implementing a **web application firewall (WAF)** in front of any publicly exposed web or API service.
 - **Training:** Train developers on secure coding practices and keep them updated on security trends.

- **Use access control lists (ACLs):** Set network/firewall rules to configure who can access our resources over the network (for both inbound and outbound network traffic).
- **Use audit trails:** These allow you to monitor the access of resources and actions carried out, which could aid in further investigation and IR.
- **Use security information and event management (SIEM):** Send all logs from all services (both infrastructure and application logs) to a central SIEM system to be able to correlate events from different sources and detect and investigate security incidents.
- **Monitor network traffic:** This allows you to record network traffic (source, destination, port, action, and so on) for forensics purposes in case of a security incident investigation.
- **Use threat management services:** These allow you an automated way, using **machine learning (ML)** capabilities, to review logs from various sources, compare the information with **indicators of compromise (IoCs)** from public repositories (information about malicious IP addresses, URLs, and so on), and provide actionable recommendations (what to do in case of a security incident, and where to begin with).
- **Use change management:** Check for deviations from preconfigured settings (such as services that might have public access servers that might need hardening and more).
- **Use data classification:** Data classification allows you to configure security controls according to the data classification (such as public, internal, confidential, and so on). Combine this with data leakage services to assist in the detection of data exfiltration.
- **Configure organization-wide policies:** For example, use inherited policies that enforce who can access which resources and what actions can be taken on resources, enforce auditing, enforce encryption, and so on.
- **Use backups:** This allows you to recover your data and applications after a data breach.

Now that we've discussed data breaches and the main recommendations for identifying and mitigating them, let's talk about misconfigurations.

Misconfigurations in cloud services

Misconfigurations are a common threat when using cloud services. Under the *shared responsibility model*, some common reasons for misconfigurations in cloud services that fall under the customer's responsibility are as follows:

- Lack of knowledge in operating cloud services
- Human error
- Default settings being left in an unsecured state (such as publicly exposing services from the internet)

- Large and complex environments being deployed in a very short time
- Fast and unmanaged changes to cloud environments

Here are some common examples of misconfigurations in cloud services:

- Having overly broad IAM policies (or RBAC policies) – for example, default permissions that allow users to conduct actions on sensitive resources or having more permissions than needed to accomplish their daily tasks
- Object storage is publicly accessible to anyone on the internet
- Snapshots and VM images are publicly accessible to anyone on the internet
- Databases are publicly accessible to anyone on the internet
- Virtual servers are publicly accessible to anyone on the internet using **Secure Shell (SSH)** or **Remote Desktop Protocol (RDP)** ports

Common consequences of misconfiguration

As a result of misconfiguration in cloud environments, several things might happen to our data:

- **Data breach:** An object storage left publicly accessible could lead to data leakage (such as customer or business data) by unauthorized parties
- **Data exfiltration:** Data that was exposed to unauthorized parties
- **Compliance violation:** Regulations such as GDPR, the **Health Insurance Portability and Accountability Act (HIPAA)**, or CCPA look into the causes of data breaches and violations of compliance requirements (for data protection), which may result in financial consequences for the organization
- **A breach in customers' trust:** Organizations suffering from data breaches (due to misconfiguration, among others) may find that their customers stop using their services, defect to competitors, or sue the organization in court

Example of a real event – CommuteAir

CommuteAir, a regional airline from the US, had a security incident in 2023. An attacker scanning the internet for exposed Jenkins servers was able to gain access to the CommuteAir Jenkins server, which was publicly exposed, due to misconfigured security groups.

After the attacker accessed the Jenkins server, they were able to gain access to one of CommuteAir's projects, containing AWS access keys stored hardcoded, which granted them access to AWS resources (including **Simple Storage Service (S3)** buckets, DynamoDB tables, and more).

While reviewing the data, the attacker found production data inside a repository of test data, information about CommuteAir employees, and the US government's *No Fly List* with 1.5 million records of customers' names.

Here's an additional reference about the attack:

<https://www.breaches.cloud/incidents/commuteair/>

Let us now look at some best practices for detecting and mitigating misconfigurations, such as the ones seen in the case of CommuteAir.

Best practices for detecting and mitigating misconfigurations in cloud environments

Here are some controls that allow us to mitigate misconfigurations:

- **IAM:** Similarly to the case of data breach, using IAM we can enforce RBAC mechanisms to limit permissions to resources, making sure we decrease the number of identities (human or application/service accounts) with permissions to make configuration changes (such as who can expose resources to the public internet).
- **Configuration management:** Allows us to set an organization configuration standard (such as object storage configured private, databases encrypted at rest, and so on). Using configuration management tools, we can compare the current configuration with the approved configuration, identify configuration drifts (deviation from the pre-set standard), and fix them (ideally, in an automated way).
- **Change management:** This allows us to make sure that any change is tested, approved, and documented.

Here are some best practices to follow:

- Use ACLs to restrict access to resources (that is, which employee can make which change).
- Enforce the use of MFA for any privileged activity in the cloud environment (whether the access is done from the console or using a **software development kit (SDK)**).
- Implement privileged IdM mechanisms to ensure that any privileged activities require a workflow approval and documentation process.
- Enforce encryption mechanisms to protect data so that even if an object storage or a database becomes publicly accessible (due to a misconfiguration), the data will remain inaccessible to unauthorized parties.
- Use organizational configuration policies to enforce the desired configurations, such as encrypted storage, blocking public access to resources, and more (as explained in *Chapter 17, Security in Large-Scale Environments*).

- Use IaC for automation and standard configurations (as explained in *Chapter 17, Security in Large-Scale Environments*).
- Use **cloud security posture management (CSPM)** to detect misconfigurations (as explained in *Chapter 15, Managing Multi-Cloud Environments*).
- Scan for misconfigurations (such as the use of vulnerable components) as part of the build process.
- Use automated tools to review API calls stored in audit logs that document configuration changes (using ML capabilities) and send alerts to the **security operations center (SOC)** team about changes that require immediate attention from the infrastructure teams.
- Send all logs to a central SIEM system to be able to correlate between security incidents (such as opened object storage that allows data exfiltration) and allow investigation of security incidents.
- Use change management to check for deviations from preconfigured settings (such as services that might have public access or servers that might need hardening and more).
- Limit access to production environments to the minimum required (both by using minimal permissions and using network access controls) – do not allow developers or end users access to production environments.
- Conduct employee training on the proper use of the various cloud services.
- Regularly review which identities exist in your organization that require access to cloud resources and manage these as required.
- Segment the network according to roles (for example, configure network segmentation between front web servers and backend database) to limit the blast radius in case a misconfiguration exposes one segment of the cloud environment.
- Regularly review NACLs (both inbound and outbound) to detect unauthorized access and limit the risk of data exfiltration.
- Regularly review logging configuration to detect services and applications that have had their logging capabilities disabled, which will not allow documentation of changes.
- Regularly conduct external vulnerability assessments to detect misconfigurations that expose your cloud environments to unauthorized access.
- Conduct penetration tests (by external **service providers (SPs)**) at least once a year or when there is a major configuration change (such as replacement of authentication/authorization mechanism, replacement of technology such as switching from VMs to containers or **functions as a service (FaaS)**) to identify potential misconfigurations.

We now know what misconfigurations are and how we can identify and mitigate them. Next, we'll look at insufficient IAM.

Insufficient IAM, secrets, and key management

Insufficient IAM can happen in a scenario where we have many user identities (such as in an enterprise organization) but we fail to properly manage the identities. Or, we might use cryptography to protect sensitive data but fail to follow secrets or key rotation best practices and, as a result, increase the chance of data exposure by unauthorized parties.

Common consequences of insufficient IAM and key management

Here are some common consequences of insufficient IAM and key management:

- Failing to follow the **principle of least privilege (PoLP)** leads to excessive permissions being granted.
- Failing to configure access controls – for example, allowing unauthorized access to sensitive data (such as **personally identifiable information (PII)**, credit card data, healthcare data, and so on), which leads to exposed credentials.
- Failing to enforce password policy (for example, allowing short passwords, not enforcing password rotation, allowing password reuse, and so on) leads to unauthorized access to company resources.
- Encrypting data but keeping the same cryptographic key without key rotation can potentially lead to breaches in data confidentiality (for example, using the same certificate to authenticate a service account to an application, and once the certificate is stolen, it will allow access on behalf of the service account).
- Not enforcing the use of MFA can lead to unauthorized access to resources and data.
- Embedding credentials or cryptographic keys inside code and scripts can lead to the unintentional exposure of credentials to unauthorized parties.
- Not configuring audit trails leads to a lack of visibility of which identities have access to which resources. Without audit trails, it's hard to accept/deny that someone has accessed a resource.

Example of a real event – Football Australia

In 2022, Football Australia, the national governing authority for the sport, left AWS access keys on its public website. The access keys granted access to 126 S3 buckets that contained personal information, contracts, and documents of football players.

The incident was published by *Cybernews*, an independent cybersecurity research publication, and in 2024, Football Australia officially disclosed the incident.

Here's an additional reference about the attack:

<https://www.breaches.cloud/incidents/footballaustralia/>

Let us now look at some best practices for detecting and mitigating insufficient IAM access management.

Best practices for detecting and mitigating insufficient IAM, secrets, and key management in cloud environments

Here are some controls that allow us to mitigate insufficient IAM, secrets, and key management:

- **IAM:** Similarly to previous risks, using IAM we can enforce RBAC mechanisms to limit permissions to resources, making sure we decrease the number of identities (human or application/service accounts) with permissions to make configuration changes (such as who can deploy new resources, who can access customer data stored in a backend database, and so on)
- **Privileged IdM:** This allows us to enforce a workflow for requesting, approving, and documenting privilege requests
- **Key management:** Allows us to manage the entire life cycle of encryption keys (generate, store, retrieve, rotate, and revoke) in a central and secure location
- **Secrets management:** Allows us to manage the entire life cycle of secrets, passwords, API keys, and so on (generate, store, retrieve, rotate, and revoke) in a central and secure location
- **Auditing trail:** This allows us to document all actions done on a system (from the infrastructure to the application layer), which will assist in incident investigation

Here are some best practices to follow:

- Manage the entire identity life cycle for your employees and any sub-contractors who have access to your systems and data – this should include recruitment, account provisioning, access management to resources, and, finally, account de-provisioning when an employee or sub-contractor has left the organization or no longer needs access to systems and data.
- Use a central repository for identities, where you provision and de-provision accounts and set password policies.
- Use federation to allow access from on-premises environments to cloud environments and resources and between external partners' or customers' IAM systems and your cloud environments.
- Block the use of weak and legacy authentication protocols (such as **New Technology LAN Manager (NTLM)** and clear text **Lightweight Directory Access Protocol (LDAP)**).
- Enforce the use of a password policy (for example, by stipulating a minimum password length, minimum and maximum password age, password history, complex passwords, account lockout and release settings, and more).
- Enforce RBAC, following PoLP when granting permissions to resources.
- Use **attribute-based access control (ABAC)** to limit an identity's permission not just to a specific role but to identities that meet certain conditions (such as group membership, users who authenticated using MFA, certain time of the week, and so on).
- Use privileged IdM tools to enforce proper permission request, review, and approval workflow.

- Use conditional access services (such as Microsoft Entra Conditional Access) to make sure that access to resources (such as admins trying to gain access to manage resources) will have to meet certain conditions (such as using MFA from a well-known workstation, containing an organization certificate, at certain hours, and so on).
- Enforce a mechanism (such as Azure Customer Lockbox) that requires approval for CSP engineer access requests to the customer's cloud environment.
- Follow the concept of **segregation of duties (SoD)** to prevent single users from having privileges to conduct sensitive actions (such as a user with the permissions both to generate encryption keys and use encryption keys).
- Enforce the use of MFA for any interactive access to cloud environments.
- Prefer to use temporary credentials (such as AWS **Security Token Service (STS)**) instead of long-term credentials to limit the amount of time an identity has permissions to take actions over resources (in case the credentials get stolen).
- Audit users' actions and correlate their activities using an SIEM system that will allow you to detect anomalous behavior (such as user logins after working hours, users trying to access resources for the first time, and so on).
- Correlate audit logs with a threat management service that uses an ML engine to review logs and identify anomalies (such as the use of a root/admin account, privileged escalation attempts, credentials that have not been replaced, and so on).
- Review user login activities. If a user hasn't used their account for several weeks, perhaps it is time to lock or disable the account due to inactivity, which is also a part of the *least privilege* principle.
- Implement identity governance – do the right users have the right roles at the right time?
- Review user access privileges, locate accounts with default or increased permissions that are not needed, and fine-tune the accounts' privileges.
- Use a secure and central repository (that is, secrets manager) for generating, storing, retrieving, and rotating encryption keys.
- Use a secrets management service to generate, store, retrieve, and rotate secrets, passwords, API keys, and access keys inside code to access resources, while keeping secrets secured and encrypted.
- Encrypt all sensitive information (such as PII, credit card numbers, healthcare data, and so on).
- Use a secure and central repository for generating, storing, retrieving, and rotating secrets (passwords, credentials, access keys, and others).
- Scan your code and configuration files to detect hardcoded credentials (passwords, access keys, API keys, secrets, or certificates), and once detected, revoke all static credentials and either replace them with temporary credentials or enforce the use of key or secrets management services.

Next, let's see how we can mitigate account hijacking.

Account hijacking in cloud services

Account hijacking happens when an account (either belonging to a human or a system/application/service account) is compromised and an unauthorized identity gains access to use resources and data on behalf of the (usually high-privileged) compromised account.

Some common methods of account hijacking are as follows:

- **Phishing attacks** against a system administrator's account, allowing an attacker to gain access to databases with customer data
- **Credential stuffing**, where a privileged account fails to replace its credentials, the credentials are leaked by unauthorized parties, and they are used to breach the organization's systems
- **Brute-force attack**, which is an attack that uses an automated script to guess admin passwords until a weak password (such as short, not complex, or even a commonly used password) is found and can be used by an attacker
- **Social engineering**, where an attacker manipulates an admin account to reveal their password (such as entering a password on a website controlled by the attacker), which can later be used by the attacker to gain access to the organization's systems
- **Malware and key logger**: A scenario where an attacker manipulates an admin to execute malicious code while running in a privileged session, allowing the attacker to record and steal the admin credentials

Common consequences of account hijacking

Here are some common consequences of account hijacking:

- Unauthorized access to resources
- Data exposure and leakage
- Data deletion
- System compromise
- Identity theft
- Ransomware or malicious code infection
- Account lockout
- **Denial of service (DoS)**
- Denial of wallet (there can be a huge cloud spend due to resource misuses such as crypto mining)
- Website defacement

Example of a real event – Retool MFA

In 2023, Retool, a software development platform, fell victim to a spear phishing attack. An attacker was able to gain access to a Retool engineer's Google account, which provided the attacker access to the engineer's software-based MFA tokens.

Once the attacker had access to the MFA tokens, they were able to connect to Retool's VPN and use the engineer's privileges (which happened to be internal admin privileges) to conduct an account takeover attack and, as a result, were able to access some of Retool's customers' data.

One of Retool's customers was Fortress Trust, a financial infrastructure for digital assets. As a result of the attack on Retool, the attacker was able to steal 15 million dollars in cryptocurrency.

Here's an additional reference about the attack:

<https://www.breaches.cloud/incidents/retool-mfa/>

Let us now look at some best practices for detecting and mitigating account hijacking.

Best practices for detecting and mitigating account hijacking in cloud environments

Here are some controls that allow us to mitigate account-hijacking attacks:

- **Hardware-based MFA tokens:** These are considered more secure than software-based **temporary OTPs (TOTPs)**, which are often synced to a cloud service (such as Google Workspace).
- **Passwordless login:** Passkeys (based on the **Fast IDentity Online 2 (FIDO2)** standard) are not vulnerable to interception (compared to passwords or SMS OTP). They are cryptographically encrypted, and a common way to unlock them is by using biometrics (such as Face ID or fingerprint).
- **Auditing and anomalous behavior:** One of the ways to detect account hijacking is by enabling auditing on all users' activity and using ML engines to detect anomalous behavior (such as a user trying to log in over the weekend, a user trying to access a resource they never used in the past, and so on) and alert the SOC team.
- **Strong password policy:** Enforcing long and complex passwords for all users and requiring password rotation (for example, every 90 days) will decrease the chance of account hijacking. (*Note:* passwords are not a foolproof protection mechanism.)
- **Break-glass mechanisms:** Review and approval workflows for granting privileges to environments (such as production environments) will decrease the chance of an unauthorized identity gaining high-privilege permissions.

- **Access controls:** Using access controls, organizations can limit the permissions granted to identities (both human and application/service accounts), and by doing so, decrease the chance of account hijacking.
- **Education and awareness:** By investing in employee awareness, employees (from all types of privileges) will be aware of potential risks (such as the potential that their credentials will get stolen) and safe practices such as avoiding opening emails from unknown sources, avoiding clicking on attachments, or installing software from untrusted parties.

Here are some best practices to follow:

- Enforce the use of strong passwords (for example, by stipulating a minimum password length, minimum and maximum password age, password history, complex passwords, account lockout and release settings, and more)
- Use services such as Microsoft Entra Password Protection to be able to detect easily guessed passwords or passwords that were stolen and can be found on the dark web
- Enforce the use of hardware-based MFA token devices, which are less prone against compromise and token attacks (compared to software-based tokens)
- Prefer passwordless login (that is, passkeys) for any system that supports the FIDO2 authentication standard
- Avoid using SMS-based OTPs, which are prone to cyberattacks from SIM swapping to **Signaling System No. 7 (SS7)** protocol attacks, as recommended by the NIST guidelines:

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-63b.pdf>

- Prefer short-lived tokens over static/long-lived credentials, which are prone to abuse since they are not been rotated for a very long time
- If you need to use secrets to gain access to resources, enforce the use of a secrets management service to manage the entire secrets life cycle (generate, store, retrieve, rotate, and revoke)
- When granting permissions to employees, use RBAC, and follow PoLP to keep permissions to the bare minimum an employee requires to perform their tasks
- Follow the concept of SoD to prevent single users from having the privilege to conduct sensitive actions (such as a user with the permissions both to generate encryption keys and use encryption keys)
- Use services such as Microsoft Entra Conditional Access to enforce different levels of access and different authentication methods when users are connecting from the corporate network versus an unknown IP address
- Use break-glass mechanisms to make sure that any request for a privileged account will be reviewed, approved, and audited to decrease the chance of abusing permissions, which may lead to account hijacking

- Enforce session management on any access to resources, which sets the maximum amount of time an identity can remain logged in to a system
- Audit user actions and correlate their activities using a SIEM system that will allow you to detect anomalous behavior (such as user logins after working hours, users trying to access resources for the first time, and so on)
- Use automated tools to review login attempts and send alerts to the SOC team in case of multiple failed logins from the same source (which may indicate unauthorized access and require immediate investigation and response)
- Review the account recovery process to make sure the process is not prone to attacks, such as the use of easily guessed answers to questions that will allow account recovery by unauthorized parties (which may be publicly available)
- Regularly review permissions granted to employees, specifically highly privileged accounts (which include permissions to make configuration changes, access to customers' data, and so on), and revoke unneeded permissions
- Invest in employee awareness to allow all your employees to detect phishing attempts, avoid opening emails from unknown sources, avoid running unknown executables, and avoid plugging in removable media from unknown sources
- Invest in **business continuity (BC)** planning to prepare for recovery after a system is compromised (for example, rebuilding systems, recovering data from backups, changing credentials, and so on)

While account hijacking is an external threat, cloud services can also be vulnerable to insider threats, on both the customer side and the CSP side. Let's learn about these next.

Insider threats in cloud services

An **insider threat** is a concept where an authorized employee (that is, an *insider*) acts (either maliciously or accidentally) in a way that they are not supposed to.

Some common examples of insider threats are as follows:

- An administrator clicks on a phishing email from an unknown source, and as a result, a file server gets infected by ransomware, and all the files are encrypted
- An employee with the privilege to access an accounting system leaves their laptop unattended, and an unauthorized person takes over their laptop and steals customer data
- A sub-contractor with access to databases with customer email addresses exports customer data and sells it on the dark web
- An administrator with access to backup files decides to delete all backup files out of spite before leaving the organization

Common consequences of insider threats

Some common consequences of insider threats are as follows:

- Loss of data
- Data leakage
- System downtime
- Loss of company reputation
- Monetary loss due to lawsuits

Example of a real event – First Republic Bank

In 2020, a terminated employee from First Republic Bank used a laptop they received from the bank as part of their work, after getting fired, to log in to the bank's VPN.

Once the ex-employee was inside the bank's network, they logged in to a jump-box (a machine used to allow access to internal resources), impersonated other employees using a Linux admin account, terminated running **Elastic Compute Cloud (EC2)** instances using an automated script, deleted code repositories, and locked internal users from their ability to use an Amazon Elastic MapReduce (EMR) service, used by the bank employees.

Here's an additional reference about the attack:

<https://www.breaches.cloud/incidents/first-republic/>

Let us now look at some best practices for detecting and mitigating insider threats such as the one experienced by First Republic Bank.

Best practices for detecting and mitigating insider attacks in cloud environments

Here are some controls that allow us to mitigate insider attacks:

- **IAM:** Allows you to control the entire identity life cycle (provision, enable, disable, de-provision)
- **RBAC:** Allows you to restrict access for all identities according to their business roles, following PoLP
- **Network segmentation:** Allows you to define which areas of the network (for example, specific subnets) can be accessed by which identities (such as allowing a user access to resources that are labeled with a specific tag), and, in the case of an insider threat, the user will be limited to certain areas of the network and not be able to access all of the organization's resources

- **Regularly review access permissions** for all identities and automatically revoke permissions when not required (such as in a scenario where an employee was fired or switched departments and does not require permissions to resources)
- **Enable auditing:** Allows recording of all actions done in the system (from the infrastructure to the application layer), which allows further analysis as part of an IR process
- **Anomalous behavior:** An automated process to review identity behavior (from time of the day/week/month an action was taken, amount of access to resources, and so on) to be able to identify and automatically alert the SOC team if further actions are required (such as disable accounts, force a password reset, revoke permissions, and so on)
- **Session management:** Allows you to enforce the maximum amount of time that an identity can access a system and automatically log off sessions after they have expired
- **Data loss prevention (DLP):** Allows you to detect when internal data (from customer data to business-related data) was exfiltrated outside the organization's boundaries (or an employee tried to exfiltrate data and failed)
- **IR plan (IRP):** A documented process (which requires regular testing to keep it effective) to allow the organization to respond to a security breach (such as an insider threat), respond to the event, and be able to overcome and return systems to their expected behavior (from system availability, data confidentiality, and so on)
- **Security awareness training:** Allows the organization to keep employees alerted on unexpected behavior (such as being aware that an employee who left the firm is trying to access the organization's systems – physically or via the organization's computers)

Here are some best practices to follow:

- Invest in background screening before hiring employees or sub-contractors.
- Use IAM to manage the entire life cycle of all identities with access to the organization's resources.
- Regularly review all identities to make sure they are still required. Disable identities not required and revoke all their permissions before deleting them.
- Enforce the use of strong passwords (for example, by stipulating a minimum password length, minimum and maximum password age, password history, complex passwords, account lockout and release settings, and more).
- Audit user actions and correlate their activities using a SIEM system that will allow you to detect anomalous behavior (such as user logins after working hours, users trying to access resources for the first time, and so on).
- Use ML capabilities to automate the process of audit log review, identify anomalous behavior (such as an employee trying to access a resource at an unusual date of the week, or a resource they never tried to access before), and automatically alert the SOC team to investigate and act (such as disable an account, revoke access permissions, and so on).

-
- Enforce the use of MFA to minimize the risk of attackers taking control of an internal employee's account without the employee's knowledge to access internal systems and data.
 - Use RBAC to grant permissions to identities according to their business roles and follow PoLP when granting permissions to resources.
 - Regularly review all permissions and create a process for removal of unnecessary permissions to resources to prevent unauthorized access.
 - Follow the concept of SoD to prevent single users from having the privilege to conduct sensitive actions (such as a user with the permissions both to generate encryption keys and use encryption keys).
 - Use services such as Microsoft Entra Conditional Access to enforce different levels of access and different authentication methods when users are connecting from the corporate network versus an unknown IP address.
 - Use break-glass mechanisms to make sure that any request for a privileged account will be reviewed, approved, and audited to decrease the chance of abusing permissions, which may lead to account hijacking.
 - Use a DLP solution to review all access attempts to sensitive data (such as customer data or business-related data), alert the SOC team about potential data exfiltration, and block it from unauthorized parties before data leaves your organization.
 - Regularly conduct risk assessments to evaluate your entire cloud environment (who has access to resources, what type of access, what is the potential damage if an identity takes over resources, and what are the methods to mitigate the risk of unauthorized access).
 - Invest in employee awareness to allow all your employees to detect phishing attempts, avoid opening emails from unknown sources, avoid running unknown executables, and avoid plugging in removable media from unknown sources.
 - Invest in BC planning to prepare for recovery after a system is compromised (for example, rebuilding systems, recovering data from backups, changing credentials, and so on).
 - Encrypt sensitive customer data and make sure access to encryption keys is limited to authorized personnel only, with audits for the entire key encryption/decryption process. Store the encryption keys in a key management service to manage the entire life cycle of the keys (generate, store, retrieve, rotate, and revoke).
 - For application/service accounts that require access to static credentials (secrets, passwords, API keys, and so on), store all static credentials in a secrets management service and enforce strict authorization mechanisms for any identity who requires access to the secrets management, for the entire life cycle of the secrets (generate, store, retrieve, rotate, and revoke).
 - Segment your network to subnets, to limit potential access an identity has over your entire cloud environment.

- Store backups off-site and audit all access to backups, to prevent unauthorized access to backups, which may lead to data compromise or inaccessibility.

Next, let's learn how to deal with insecure APIs in cloud services.

Insecure APIs in cloud services

In today's world, modern developments are based on APIs to communicate between system components. The fact that APIs are publicly exposed makes them an easy target for attackers trying to access a system and cause damage.

Some common examples of attacks exploiting insecure APIs are as follows:

- Due to a lack of input validation, an attacker can misuse an exposed API and inject malicious code through the API into a backend database
- Due to a lack of input validation, an attacker can perform an SQL injection through an exposed API and exfiltrate customer data from a retail site
- Due to a lack of application access control mechanisms, an attacker can use an API to penetrate a cloud service by using a low-privilege account
- An attacker located an API key stored in an open source code repository and was able to run remote commands against an internal system, using the permissions that the API key had

Common consequences of insecure APIs

Some common consequences of insecure APIs are as follows:

- Data breach
- Data leakage
- Data exfiltration
- Damage to data integrity
- DoS

Example of a real event – Trello

In 2024, a hacker was able to gain access to Trello, a project management tool developed by Atlassian. The hacker accessed Trello through an insecure API endpoint, which allowed them unauthorized access.

Through the API endpoint, the hacker was able to link email addresses to Trello's customers, which resulted in data exposure of 15 million Trello customers (containing their full names and email addresses).

Here's an additional reference about the attack:

<https://hackread.com/trello-data-breach-hacker-dumps-users-personal-info/>

Let us now look at some best practices for detecting and mitigating insecure APIs seen in the Trello case.

Best practices for detecting and mitigating insecure APIs in cloud environments

Here are some controls that allow us to mitigate insecure APIs:

- **Authentication and authorization:** Use protocols such as **Open Authorization 2.0 (OAuth 2.0)** or **JSON Web Token (JWT)** to enforce access permissions for identities trying to access any API.
- **RBAC:** Use RBAC mechanisms to limit access to APIs and which actions can be taken by identities against APIs.
- **Encryption:** Protect data confidentiality by enforcing encryption in transit for all APIs.
- **Input validation:** Sanitize all input coming to API endpoints (such as character types, minimum and maximum number of characters, and so on) to mitigate the risk of injection attacks trying to manipulate data from internal systems, causing data leakage or tampering with data integrity.
- **Rate limiting:** Allows you to configure the maximum number of requests a client can make at each time (can mitigate attacks such as brute force or massive data leakage).
- **Auditing:** Document all attempts to access an API endpoint (and from the API endpoint to backend systems) and which actions were taken (get data, update data, delete data, and so on). This can aid in both troubleshooting and as part of an IR process.
- **Error handling:** Avoid exposing detailed errors to customers through failed API requests to limit potential attackers' ability to learn about the system and potential vulnerabilities within the system.
- **Software development life cycle (SDLC):** Implement a secure design life cycle that contains threat modeling at the design phase and code review at the build and pre-release phases. Embed tools such as SAST or DAST to detect vulnerabilities in code (such as injection attacks, insecure authentication, and so on).
- **Session management:** Enforce session expiration and invalidation to make sure a client does not stay logged in to an API indefinitely.
- **API gateway:** Make sure all APIs are exposed behind an API gateway, which allows you to add authentication and authorization layers for client requests and rate limiting for API calls from clients.

- **WAF:** Allows you protection from the most used application-layer attacks (such as injection attacks).
- **Distributed DoS (DDoS) protection:** Allows you protection from network-based attacks (such as volumetric attacks trying to consume the entire API endpoint network bandwidth).

Here are some best practices to follow:

- **Use a secure development life cycle:** Embed this as part of your development process – this includes controls such as the following:
 - **Authentication and authorization:** Enforce who can access the API.
 - **Input validation:** Check and validate what strings and/or data can be inserted into the API to mitigate the risk of injection attacks.
 - Conduct code reviews for all your APIs to mitigate the risk of code vulnerabilities.
 - Perform schema validation on the server side to make sure only well-known field sizes, characters, or regular expressions can pass through the API.
 - Embed SAST and DAST as part of the SDLC process as an automated solution for detecting code vulnerabilities.
 - Embed **software composition analysis (SCA)** tools as part of the SDLC process to detect third-party code, libraries, and binaries containing known vulnerabilities.
 - Conduct internal and external penetration testing against the API endpoint to detect vulnerabilities and be able to mitigate them before they can be exploited.
- Authenticate all client requests to the API endpoint using protocols such as OAuth 2.0 or JWT.
- Use RBAC to grant permissions for sending API requests to an API endpoint, following PoLP.
- When granting access to APIs using API keys, manage the entire life cycle of the API key inside a secrets management solution (generate, store, retrieve, revoke, delete) and configure the API key rotation process to avoid using the same API key over a long time (which makes it prone to misuse or unauthorized access).
- Prefer to use short-lived API keys (which require constant API key rotation) to mitigate against API key misuse by unauthorized parties.
- Regularly review who has access to the API keys, rotate keys that are in use, and revoke keys that are not in use or not needed.
- For human (interactive) authentication to an API endpoint, enforce the use of MFA for sending API requests to the API endpoint (to limit the potential of unauthorized access).
- Enforce session expiration and invalidation to limit the maximum amount of time a client can consume data from an API endpoint to mitigate against mass data leakage attacks.

- Encrypt data in transit by default for all APIs to retain data confidentiality. Use the latest encryption algorithms and cipher suites. For more information, see <https://www.ssl.com/guide/tls-standards-compliance/>.
- Sign each message through the API using a cryptographic key to avoid data tampering or changes to data integrity.
- Use an API gateway in front of any exposed APIs to add a layer of authentication, authorization, and throttling to protect from scenarios where a specific source (such as a client IP) is trying to extract data from the API or cause DoS to the API endpoint.
- Use a WAF to protect against well-known application-layer attacks. Examples of well-known attacks against API endpoints can be found at <https://owasp.org/www-project-api-security/>.
- Use DDoS protection services to protect the API service from DoS attacks.
- Enforce a rate limit on the API to decrease the chance of automated attacks (such as brute force, mass data exfiltration, and so on).
- Limit the type of HTTP methods to the minimum required (for example, GET without POST or DELETE) according to business needs (read, update, delete, and so on), which limits attackers' ability to cause damage through API calls.
- Send generic error messages in case a client is having issues connecting to an API endpoint to limit potential attackers' ability to gain information about the API and backend systems, which will allow the attacker a way to exploit the API endpoint (such as knowing which endpoint version an organization is using and what known vulnerabilities exist for that version).
- Audit the use of APIs and backend systems to detect anomalous behavior (such as brute-force attacks or data exfiltration). Use ML capabilities to review audit logs and send alerts on anomalies to the SOC team for further investigation and response (such as blocking source IP, rotating an API key, revoking an API key, and so on).
- Prepare an IRP that will instruct the organization how to handle attacks against an API (from blocking source IPs, revoking access permissions, redirecting traffic to another version of the API endpoint, and so on).

Finally, let's try to understand security concerns related to the abuse of cloud services.

The abuse of cloud services

The abuse of cloud services is about using the scale of the cloud provider's resources and the multi-tenancy architecture to conduct malicious activities.

Some common examples of the abuse of cloud services are as follows:

- Using the cloud to deploy multiple servers and conducting DDoS attacks
- Using the cloud to deploy multiple expensive servers for crypto mining
- Using the cloud to spread email spam and phishing attacks
- Using the cloud for brute-force attacks on passwords

Common consequences of abuse of cloud services

Some common consequences of the abuse of cloud services are as follows:

- Misconfiguration (such as leaving resources publicly exposed on the internet), which could lead to massive data breaches and exfiltration of sensitive data (from customer- to business-related data)
- DDoS attacks use the mass resources that exist in the cloud, which could lead to loss of service availability to other customers' services exposed from the internet
- Monetary loss due to the use of cloud resources being exploited for crypto mining without the customer's awareness
- The use of cloud resources to host malicious sites for credential harvesting
- The use of managed cloud email services to spread bulk spam or phishing emails to customers around the world
- The use of stolen API keys (found on public code repositories) to delete or manipulate data in cloud applications

Example of a real event – UNC2903

In 2021, Mandiant (part of Google Cloud) discovered a new threat actor called UNC2903, which attempts to steal credentials using Amazon's **Instance Metadata Service (IMDS)**, a component of EC2 instances used to access instance metadata (such as internal IP address, key pair name, IAM roles, and so on).

The threat actor began by scanning vulnerabilities in a database management tool written in PHP. Once the exploit of the publicly facing web servers succeeded, the attacker was able to gain access to AWS API credentials, which granted the attacker access to customers' AWS accounts.

Here's an additional reference about the attack:

<https://www.breaches.cloud/incidents/unc2903/>

Let us now look at some best practices for detecting and mitigating the abuse of cloud resources seen in the UNC2903 case.

Best practices for detecting and mitigating abuse of cloud services

Here are some controls that allow us to mitigate the abuse of cloud services:

- **IAM:** Enforces authentication and authorization for any access to cloud resources
- **RBAC:** Allows configuring privileges to cloud resources based on business needs, following PoLP
- **Audit:** Allows documenting all access requests to cloud resources for accountability (who did what) and assists in an IR process
- **Encryption:** Allows making sure data remains confidential and accessible only for identities with access to the encryption keys
- **Anomaly detection:** Automated way to scan cloud environment and their logs to detect potential abuse of cloud resources
- **Awareness:** Educate employees on the proper use of cloud services and the consequences of not following security best practices (from unauthorized access to data to abuse of cloud resources, which costs the organization a lot of money)

Here are some best practices to follow:

- Authenticate all access to cloud resources to validate that all requests are originating from known sources (human or application/service accounts).
- Enforce strong password policies (such as password length, password complexity, and password rotation).
- Enforce the use of MFA (preferred FIDO2 authentication standard) for any interactive login to cloud resources.
- Prefer to use short-lived credentials (originating from a secrets management service) over static credentials to prevent credentials from being exposed or leaked and used by an unauthorized party.
- Regularly review all identities in the system, disable unneeded identities, and revoke all permissions that are not needed anymore.
- Enforce RBAC on any access to cloud resources and limit permissions according to business requirements, following PoLP, to avoid unauthorized access.
- Follow the concept of SoD to prevent single users from having the privileges to conduct damaging actions (such as a user with the permissions both to deploy an expensive VM and to log in to the machine for ongoing maintenance).
- Use ACLs and set network/firewall rules to configure who can access your resources (for both inbound and outbound network traffic).

- Encrypt all data at transit and rest to retain data confidentiality. Manage the entire encryption key life cycle (generate, store, retrieve, rotate, revoke, and delete) inside a key management service.
- Manage all static credentials (such as secrets, passwords, API keys, and so on) inside a secrets management service for the entire secret life cycle (generate, store, retrieve, rotate, revoke, and delete).
- When using Amazon EC2 instances, make sure all instances are using IMDSv2 to access instance metadata, which enforces session-based authentication to the instance metadata.
- Use audit trails to allow you to monitor the access of resources and actions carried out.
- Enable review, approval, and audit of all access attempts by CSP engineers to gain access to your cloud environments. Send all logs to a central SIEM system for further review and as part of IR.
- Use **threat intelligence (TI)** services to review audit logs, detect anomalies (such as excessive use of email services, crypto mining activity, brute-force attempts, and so on), and send alerts to the SOC team for further investigation and immediate response (such as disabling account, blocking firewall rules, rotating credentials, and so on).
- Regularly scan for vulnerabilities (such as exposed resources, missing security patches, and so on) and fix discovered vulnerabilities.
- Regularly deploy security patches on both the infrastructure (such as VMs and containers) and the application layer (such as versions of Java, Python, PHP, and so on).
- Regularly conduct internal and external penetration testing against all your cloud environments to identify potential threats and remediate the findings.
- Use DDoS protection services to protect your cloud environment from DoS attacks.
- Use WAF services to detect and protect against application-layer attacks.
- Configure billing alerts to get notified in advance about any increase in resource consumption.
- Invest in employee awareness regarding the cloud provider's acceptable terms of use.

In this section, we discussed various types of threats to cloud services and how we can detect and mitigate most of them. Next, we will review the MITRE ATT&CK framework that maps adversarial **tactics, techniques, and procedures (TTPs)** specific to cloud environments, enabling organizations to better understand and mitigate security risks across various cloud services.

The MITRE ATT&CK framework

The **MITRE ATT&CK** framework is a knowledge base of hacking techniques. The cloud matrix of MITRE ATT&CK contains the following attack techniques:

- Initial access
- Execution
- Persistence
- Privilege escalation
- Defense evasion
- Credential access
- Discovery
- Lateral movement
- Collection
- Exfiltration
- Impact

Understanding attack techniques allows us to understand the results of attacks. For example, using the *abuse of credentials* allows an attacker to gain persistent access to our system.

Another attack example is *cloud object storage discovery*, which may be used by an attacker to gain access to all objects inside a cloud storage instance.

Reviewing all attack techniques will allow us to understand which security controls we can implement to mitigate potential attacks in our cloud environment.

For more information, refer to the following resources:

- *MITRE ATT&CK, Cloud Matrix*: <https://attack.mitre.org/matrices/enterprise/cloud/>
- *MITRE ATT&CK, IaaS Matrix*: <https://attack.mitre.org/matrices/enterprise/cloud/iaas/>
- *MITRE ATT&CK, SaaS Matrix*: <https://attack.mitre.org/matrices/enterprise/cloud/saas/>

Summary

In this chapter, we have focused on common security threats to cloud services, including data breaches, misconfiguration, insufficient IAM, account hijacking, insider threats, insecure APIs, and the abuse of cloud services. For each of the identified threats, we reviewed the potential consequences of the threat, a common example of the threat, and best practices to detect and mitigate the threat.

Knowing the most common threats you face when using cloud services and the various built-in cloud service capabilities will allow you to better protect your cloud environment.

In the next chapter, we will learn how to engage with public cloud providers and what important questions to ask a cloud provider before the engagement phase, important topics to embed inside a contractual agreement with the cloud provider, and penetration testing in cloud environments.

Engaging with Cloud Providers

In previous chapters, we have covered cloud infrastructure fundamentals and common threats in cloud environments. This chapter will cover fundamental steps before working with cloud services, such as engaging with cloud providers.

In the traditional data center, we used to control everything – from physical to logical security controls; however, with **cloud service providers (CSPs)**, to get assurance, there are several options, such as the following:

- Conduct a risk assessment before engaging with a cloud provider – one good option is to review **System and Organization Controls (SOC) 2 Type 2** reports (what controls the cloud provider has set and how effective they are). SOC reports will be reviewed later in this chapter.
- Have a good contract that sets the obligations of the cloud provider (such as a **service-level agreement (SLA)** for handling security incidents and an SLA to notify us as customers).
- Conduct an audit on the CSP's security controls and their effectiveness (according to the CSP's supported capabilities to provide such information).
- Conduct a penetration test at least once every 12 months on the system that stores or processes our data.

Next is a common cloud engagement workflow, showing the different stages of engagements from the initial CSP selections to the ongoing assessment of the existing CSP:

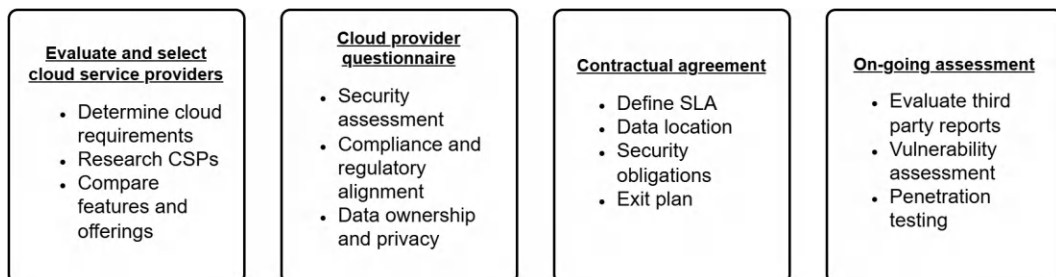


Figure 13.1 – A CSP engagement workflow

In this chapter, we will delve into the topics mentioned in the CSP engagement workflow, covering the following:

- Choosing a cloud provider
- What is a cloud provider questionnaire?
- What are SOC reports?
- Tips for contracts with cloud providers
- Conducting penetration testing in cloud environments

Choosing a cloud provider

Before choosing a cloud provider, we need to ask ourselves the following question: *What are we trying to achieve by migrating to the cloud or from using the cloud?*

There are a few steps that we need to take before we begin working with a cloud provider:

- Make a choice based on the business goals set forth by the business.
- Get familiar with a potential cloud provider and try to understand its maturity level.
- Sign a contract with a cloud provider that will protect us as customers. Although in the case of **infrastructure-as-a-service (IaaS)**/**platform-as-a-service (PaaS)** providers, beginning to work with a CSP requires payment methods (unless your organization is able to sign an enterprise agreement and make changes to the formal contract), it is crucial to sign a contract with **software-as-a-service (SaaS)** providers and make sure the contract contains sections about data protection and the obligations of the CSP.
- Conduct an assessment (such as an audit, vulnerability scanning, or penetration testing) of the cloud environment to make sure security controls are effective.

What is the most suitable cloud service model for our needs?

This is the first question we need to ask ourselves. If we are still taking the first steps in the cloud, have legacy systems that we wish to migrate to the cloud, and don't have dependencies in specific vendors' hardware (such as IBM Mainframe, Oracle SPARC, and so on), then IaaS is the most suitable model.

If our on-premises servers are based on x86 architecture and can be virtualized, we should be able to migrate or rebuild our systems based on any of the major IaaS providers, in a migration approach called *lift and shift* (that is, deploying similar architecture of **virtual machines (VMs)** to what we had in the local data center).

One of the topics we should consider is our ability to connect to a cloud provider in a hybrid model and to extend our on-premises data center – what are the potential cloud providers' hybrid cloud capabilities?

Perhaps we have already chosen a cloud provider, but now we have a business or technical requirement to work with a specific service (such as the **Internet of Things (IoT)**, **machine learning (ML)**, data analytics, and so on) that is considered superior, and it exists on another cloud provider. In this case, we need to consider multi-cloud architecture and review both cloud providers' multi-cloud capabilities.

We must remember that according to the shared responsibility model, we are responsible for the entire operating system maintenance, such as patch management, software upgrades, backup, hardening, and availability.

If we are planning to build new applications, based on **function-as-a-service (FaaS)** architecture, or if we prefer to use managed services (such as database services, machine learning, and data analytics), then PaaS should be our preferred service model (combined with IaaS for VMs).

If we choose the PaaS model, we need to consider how are we planning to connect our development (or **Continuous Integration/Continuous Delivery (CI/CD)**) infrastructure to the cloud environment and how are we planning to connect our applications (whether they are located on-premises or in our IaaS environment) to the PaaS services.

Perhaps the easiest service model for organizations that wish to consume cloud services or replace a legacy on-premises system, such as **customer relationship management (CRM)**, **enterprise resource planning (ERP)**, and messaging, with a fully managed service is to choose the SaaS model.

When choosing a mature SaaS provider (as explained later in this chapter, based on the answers we received from the **service provider** to the questionnaire mentioned in the *What is a cloud provider questionnaire?* section), we have the benefits of consuming a service without the burden of ongoing maintenance, adding security controls, taking care of availability, and so on.

If we do decide to use a SaaS model, we need to understand that not all SaaS providers offer us the same level of security. Not all SaaS providers follow security best practices such as complete data isolation in a multi-tenancy environment (complete separation between customers of the service), protection against infrastructure-based or application-based attacks, audit reports, and security certifications.

Note

Even if we choose the SaaS model, we as customers are still the owners of the data, and we are responsible for deciding which data to store in the cloud and who has access to the data (and what type of permissions).

At a high level, when comparing between different offerings of different cloud providers and cloud service models, we should consider the following:

- **Comparison of cloud service models:** A direct comparison of IaaS, PaaS, and SaaS in terms of management responsibilities, flexibility, and use cases would be helpful
- **Cost considerations:** While the text mentions the benefits of each model, it doesn't discuss the cost implications of choosing one model over another
- **Scalability:** The scalability aspects of each model could be elaborated upon
- **Security considerations:** Although security is mentioned for SaaS, a more comprehensive discussion of security implications for each model would be beneficial
- **Vendor lock-in:** The risks of vendor lock-in, especially for PaaS and SaaS models, could be addressed
- **Compliance and data governance:** Information on how each model affects compliance requirements and data governance strategies would be valuable
- **Performance considerations:** A discussion on how each model might affect application performance could be added
- **Skills and expertise required:** Information on the level of in-house expertise required for managing each model would be useful
- **Exit strategy:** Guidance on planning for potential future transitions between cloud providers or back to on-premises infrastructure could be included

Data privacy and data sovereignty

Before choosing a public cloud provider (from any of the service models), we need to consult with our **data protection officer (DPO)**, our legal department, or an expert on data privacy to check what laws and regulations our organization is required to adhere to regarding our industry, our country, and the data we are planning to store or process in the cloud.

Since data stored in the cloud can (potentially) be replicated around the world, we need to understand whether there are laws that require us to keep our data locally in a specific country. Whether we serve customers in a specific country or multiple locations (or perhaps globally), we need to know whether a cloud provider has a region in specific countries. Data residency and various privacy laws require keeping data in a specific country (that is, in a specific cloud region).

There are scenarios where we have regulation requirements (such as the **Payment Card Industry (PCI)** and the **Federal Risk and Authorization Management Program (FedRAMP)**), and we need to build an entire cloud environment to be compliant with such regulations – in this case, we need to check whether the cloud provider has environments compliant with those regulations in specific regions. One good example is AWS GovCloud, Azure for the US government, or Google for Government, which are dedicated cloud environments just for government agencies. Another example

is AWS China and Microsoft Azure in China, which are separate cloud environments for storing and processing data in China.

For the IaaS and PaaS models, it should not be an issue since most IaaS or even PaaS services are regional – once we choose a target region, our data will not leave the region (unless we choose a global service or we decide to replicate data between regions to get closer to our customers).

One alternative for making sure our data will not be accessible outside a specific region is to encrypt our data and store the encryption keys in a specific region, making sure that our data will not be accessible or be available outside a specific region.

For SaaS services, there is no one answer to whether we can keep data sovereignty. Some SaaS providers replicate our data across regions to allow better service availability, or they might store our backups in a different region to better protect our data. The best solution for IaaS and PaaS services is to read the vendor's documentation.

For SaaS services, other than reading the vendor's documentation, I strongly recommend consulting with the SaaS vendor and asking specific questions regarding data privacy or data sovereignty before signing a contract with SaaS vendors.

Auditing and monitoring

As we previously learned in *Chapter 10, Auditing and Threat Management in Cloud Environments*, auditing is crucial to protect cloud environments, to gain insights about what is going on in your cloud environment, and as part of **incident response (IR)** and forensics if the cloud environment has been breached.

Mature cloud providers will offer you documented APIs to collect *activity audit logs* to your **security information and event management (SIEM)** systems for further analysis of who logged in to the service and what actions were taken (both successes and failures).

Another type of log is a *service log* (such as object storage, load balancers, and managed databases). Most mature cloud providers offer us documented APIs to pull those logs.

Another type of audit log is a *host log* (such as an operating system, security, and application log). Look for a cloud provider that offers you a way to collect host logs and store them in a centralized logging service that exposes an API. This will allow you to connect to the API and pull the logs to your SIEM systems for further analysis.

From an infrastructure point of view, look for cloud providers who offer you documented APIs to collect performance data to a central observability service and allow you to anticipate performance issues before you get complaints from your customers. A spike in performance might indicate abnormal behavior such as crypto-mining activity.

From an application point of view, look for cloud providers who offer you documented APIs to collect application insights, error logs, and statistics about the number of failed actions inside the system that enable you to ingest data into an **application performance management (APM)** tool.

From a financial point of view, look for cloud providers who offer you documented APIs to collect billing information that enable you to ingest data into cost management tools, generate billing alerts, and so on.

An important thing to consider regarding logs is cost. Some of the services offer us a free tier (such as lasting for 30 days), and if we wish to store logs for a long time (for example, due to a regulation requirement), we need to consider the cost.

Another important thing to consider is log retention. Cloud providers (such as SaaS providers) might keep logs for 60-90 days. If we are required to keep logs for a longer period, we need a way to pull those logs and copy them to an external log repository.

Migration capabilities

When evaluating a cloud provider, it is important to review what your migration capabilities are in an on-premises environment or even from hosting on another cloud provider.

Consider topics such as the following:

- **Secure data transfer:** A mechanism for transferring files into cloud storage and retaining data confidentiality in transit and at rest
- **Database migration:** Managed services to allow the transfer of an entire database (including database schema) into a managed service in the cloud, or migration between one database engine to another
- **VM migration:** The ability to transfer (in either real time or offline) an entire server stack from on-premises to the cloud can be useful for both **disaster recovery (DR)** and migration scenarios

Understanding the available migration alternatives will allow us an easy migration and adoption of the public cloud.

Additional cloud migration strategies can be found at the following links:

- AWS migration strategies
<https://docs.aws.amazon.com/prescriptive-guidance/latest/large-migration-guide/migration-strategies.html>
- Azure migration overview
<https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/migrate/>
- *Migrate to Google Cloud: Best practices for validating a migration plan*
<https://cloud.google.com/architecture/migration-to-google-cloud-best-practices>

Authentication

Before using a cloud service, we must consider how we are going to authenticate our internal users or our external customers. We discussed authentication in *Chapter 9, Effective Strategies for Implementing IAM Solutions*.

When evaluating a cloud service, we need to check what authentication mechanisms are available. Some cloud providers offer us internal authentication and authorization capabilities that come built in as part of the cloud solution.

As best practice, we should avoid using cloud services that do not support standard authentication mechanisms, such as Microsoft Entra ID or any of the known **Security Assertion Markup Language (SAML)**-based or **Open Authorization 2.0 (OAuth 2.0)** protocols, which allows us to federate our existing directory service into the cloud service without having to provision new accounts for our end users.

For external customers, we should choose solutions that support the **OpenID Connect (OIDC)** protocol, which allows customers to authenticate using their personal Facebook, Apple, LinkedIn, Twitter, and other **identity providers (IdPs)**.

In this section, we discussed methods and best practices to consider before choosing a suitable cloud provider. Let's take this a step further and discuss questions we can ask cloud providers directly.

What is a cloud provider questionnaire?

To evaluate a cloud provider, before engagement, it is very common to send the cloud provider a questionnaire with all the topics your team believes need to be asked to get familiar with the cloud provider.

The **Cloud Security Alliance (CSA)** offers organizations and CSPs the following tools:

- **Cloud Controls Matrix (CCM)**: A detailed framework of security controls aimed at establishing essential security principles across 17 key domains, such as data protection, **identity management (IdM)**, and risk management, tailored specifically for cloud computing environments.
- **Consensus Assessments Initiative Questionnaire (CAIQ)**: A self-evaluation tool derived from the CCM. It includes a series of yes/no questions aligned with the security domains outlined in the CCM, enabling CSPs to assess and showcase their security practices.

For more information, see the following link:

<https://cloudsecurityalliance.org/artifacts/cloud-controls-matrix-v4>

Mature cloud providers have, most likely, shared information about their compliance and security controls. A good public repository of CAIQ questionnaires can be found at the following link:

<https://cloudsecurityalliance.org/star/registry>

When reviewing a potential cloud provider, consider the *shared responsibility model*. The answers of the cloud provider will allow us a clear understanding of what the cloud provider's responsibilities are (for protecting our data at the infrastructure level) versus our responsibilities (and capabilities) as customers for protecting our data (within our cloud environments).

Questions are a part of a risk assessment process, and they can come from various domains. In this section, we will review common questions that we should ask most cloud providers:

- **Cloud infrastructure-related:**

- What is the cloud service model (IaaS, PaaS, and SaaS)? This allows us to understand the *shared responsibility model*.
- What is the cloud infrastructure provider's name? This is relevant for SaaS providers, deployed on top of IaaS/PaaS providers.
- Where are the cloud provider's data center/region locations? This information allows us to understand our compliance with data residency or data sovereignty laws and regulations.

- **Architecture-related:** Next are some sample architecture-related questions. Organizations should adjust the questions as per their requirements:

- Does the cloud provider offer full multi-tenant separation between customers (compute, storage, database, and networking), or do all the CSP customers share the same infrastructure (such as storage, database, and so on)? The answer to this question allows us to understand the risk we are taking when storing our data in the public cloud.
- Does the cloud provider offer separation between production and development or test environments (compute, storage, database, and networking)? This is mostly relevant for the SaaS model since in IaaS/PaaS, we choose to deploy separate environments.

- **Network-related:**

- Does the cloud provider offer site-to-site (or point-to-site) VPN connectivity from on-premises (or from anywhere on the internet) to the cloud environment? This is useful for designing hybrid architecture from on-prem to the cloud.
- Does the cloud provider offer a dedicated network connection (such as AWS Direct Connect, Azure ExpressRoute, and Google Cloud Interconnect) from on-premises to the cloud environment? This is useful for understanding our ability to securely connect to the cloud environment using a private connection with low network latency.

-
- **Data-related:** Next are some sample data-related questions the author has collected and used in his cloud provider questionnaire over the years:
 - Does the cloud provider have an official privacy policy?
 - Does the cloud provider have a full-time DPO?
 - Is the cloud infrastructure service provider compliant with data privacy laws and regulations (such as the **General Data Protection Regulation (GDPR)** in Europe and the **California Privacy Rights Act (CPRA)** in California)? Through this, we can understand how can we meet data protection regulations, at least on the cloud infrastructure level).
 - Does the cloud provider offer a built-in mechanism for data classification (such as **personally identifiable information (PII)**, credit card information, and healthcare)?
 - Does the cloud provider offer documentation on how to deploy and protect PCI environments (such as containing, storing, or processing credit card information)? (This allows us to understand how can we meet the **PCI Data Security Standard (PCI DSS)**, at least on the cloud infrastructure level.)
 - Does the cloud provider offer documentation on how to deploy and protect environments containing personal information (such as European citizens' information as defined by the GDPR)?
 - Does the cloud provider offer documentation on how to deploy and protect environments containing healthcare information (such as US healthcare information as defined by the **Health Insurance Portability and Accountability Act (HIPAA)**)?
 - **API-related:**
 - Does the cloud offer document APIs for streaming audit logs to a SIEM system? This is useful for collecting all security-related logs in a central SIEM.
 - Does the cloud offer documented APIs for collecting performance data? Although this is not security-related, it is important to know that we will have the ability to collect and review performance data from our applications for troubleshooting purposes.
 - Does the cloud offer document APIs for collecting application insights? Again, this is not security-related, but the ability to collect and review near-real-time data from our applications is important for troubleshooting.
 - Does the cloud offer document APIs for deploying new environments in an automated way (that is, IaaS)? This is useful for deploying multiple environments or versions of our applications on an automated and large scale.

- **Authentication- and authorization-related:**

- Does the cloud provider support standard authentication and authorization protocols for connecting from on-premises or any type of external environment to the cloud (protocols such as SAML and OAuth 2.0)? This allows us to maintain IdM using a single IdP.
- Does the cloud provider support authentication protocols (such as OIDC) for authenticating external identities (such as Facebook, Apple, LinkedIn, and Google)? This allows us a standard mechanism to authenticate our customers and third-party providers by centrally granting them access permissions, without having to manage external identities.
- Does the cloud provider support **role-based access control (RBAC)**? This is critical for assigning permissions according to business needs and for following the **principle of least privilege (PoLP)**.
- Does the cloud provider allow managing permissions to cloud resources based on IdP groups (such as Microsoft Entra ID groups) and match groups to roles? This is useful for creating federated **identity and access management (IAM)**.
- Does the cloud provider support **multi-factor authentication (MFA)** for interactive login? It allows us to protect the login process from unauthorized access.
- Does the cloud contain default passwords? If so, can we disable or replace all default passwords? This practice allows us to protect the login process from unauthorized access.
- Does the cloud contain default accounts (such as administrator and root)? If so, can we disable or remove those accounts? Doing so allows us to decrease the risk of unauthorized access.

- **Session management:**

- Does the cloud provider support automatic session disconnection after a pre-defined period of inactivity?
- Does the cloud provider generate random and time-restricted session IDs for each identity authenticated to the service?
- Does the cloud provider enforce session disconnection at the end of each session?
- Does the cloud provider enforce a single session for each authenticated identity?

- **Encryption at transit:**

- Does the cloud provider enforce encryption in transit using secured protocols (such as **Transport Layer Security (TLS)** 1.2 or above) for each of the supported services?
- If the cloud provider is exposing TLS/HTTPS services to the internet, do all services (or URLs) achieve a score of A+ in an SSL Labs test (<https://www.ssllabs.com/ssltest>)?

- **Encryption at rest:**

- Does the cloud provider offer encryption at rest for databases? (This allows us to retain data confidentiality for data stored in databases.)
- Does the cloud provider offer encryption at rest for object storage? (This allows us to retain data confidentiality for data stored in object storage services.)
- Does the cloud provider offer encryption at rest for file-sharing protocols (such as **Network File System (NFS)** or **Common Internet File System (CIFS)/Server Message Block (SMB)**)? (This allows us to retain data confidentiality for data stored in file storage services.)
- For each of the supported mechanisms for encryption at rest, does the cloud provider offer a key rotation mechanism? (This allows us to avoid key reuse and unauthorized access to encrypted data.)
- For each of the supported mechanisms for encryption at rest, does the cloud provider support customer-managed keys? (This allows us to control the key generation and revocation process for sensitive data such as customers' PII, financial records, and so on.)

- **IR:**

- Does the cloud provider have built-in mechanisms to detect security incidents on its infrastructure or the infrastructure serving its customers?
- Does the cloud provider have a documented IR process?
- Does the cloud provider have a dedicated **IR team (IRT)**?
- What is the cloud provider's SLA for handling security incidents?
- What is the cloud provider's SLA for handling critical vulnerabilities?

- **Penetration testing and security assessment:**

- Has the cloud provider conducted a security assessment by an external auditor in the past 12 months?
- If the cloud provider conducted security assessments in the past, were all vulnerabilities fixed?
- Has the cloud provider conducted penetration testing by an external auditor in the past 12 months?
- If the cloud provider conducted penetration testing in the past, were all vulnerabilities fixed?
- Will the cloud provider allow its customers to schedule penetration testing?

- **Secure development life cycle:**

- Does the cloud provider conduct ongoing training for its employees (and its third-party suppliers) regarding secure development?
- Does the cloud provider follow known secure development life cycle methodology (such as Microsoft **Security Development Lifecycle (SDL)**, the **National Institute of Standards and Technology Secure Software Development Framework (NIST SSDF)**, or some other methodology)?
- Does the cloud provider implement **static application security testing (SAST)** as part of its secure development life cycle?
- Does the cloud provider implement **dynamic application security testing (DAST)** as part of its secure development life cycle?
- Does the cloud provider implement **interactive application security testing (IAST)** as part of its secure development life cycle?
- Does the cloud provider implement **software composition analysis (SCA)** as part of its secure development life cycle?
- How does the cloud provider store secrets (such as API keys, privileged credentials, and **Secure Shell (SSH)** keys) as part of its infrastructure and code?
- What security controls have been implemented by the cloud provider for handling file uploads?

- **Audit and monitoring:**

- Does the cloud provider keep an audit trail of every login attempt to its services (both successes and failures)?
- Does the cloud provider keep an audit trail of actions done on its services (both successes and failures)?
- What records are saved by the cloud provider as part of its logging process?
- Does the cloud provider keep a record of actions done by its system administrators (including third-party suppliers), with access to customers' infrastructure and data?

- **Business continuity (BC):**

- Does the cloud provider share information about service outages and historical information about past outages?
- Does the cloud provider have documented BC and DR processes?
- What is the cloud provider's **Recovery Time Objective (RTO)**?

- What is the cloud provider's **Recovery Point Objective (RPO)**?
- Does the cloud provider have documented change management processes?
- Does the cloud provider have documented backup processes?
- **Compliance-related:**
 - What cloud and cloud security-related **International Organization for Standardization (ISO)** standard does the cloud provider have from the past 12 months?
 - Has the cloud provider published a CSA **Security, Trust, Assurance, and Risk (STAR)** report?
 - Has the cloud provider published a SOC 2 report in the past 12 months?
 - Does the cloud provider have a documented information security policy?
 - Does the cloud provider have a dedicated **chief information security officer (CISO)/chief strategy officer (CSO)**?
 - What logical controls are implemented by the cloud provider to protect its infrastructure?
 - What logical controls are offered by the cloud provider to protect customers' data?
 - What logical controls have been enforced on cloud provider employees with access to customers' infrastructure and data (such as anti-malware, device control, full disk encryption, MFA, and a VPN client)?
 - What physical controls are implemented by the cloud provider to protect its data centers?
 - Does the cloud provider perform an ongoing vulnerability assessment?
 - Does the cloud provider perform ongoing patch management?
 - Does the cloud provider's infrastructure (compute, storage, and network) comply with known hardening standards (such as **Center for Internet Security (CIS)** benchmarks, NIST, and more)?
 - Does the cloud provider conduct ongoing training for its employees (and its third-party suppliers) regarding information security and data privacy?

In this section, we have covered many questions to assist organizations in the reviewing process of cloud providers. Most of the topics can be answered by reviewing mature cloud providers' SOC 2 Type 2 reports or the CSA CCM. I strongly recommend conducting a cloud provider assessment – a questionnaire is one of the simplest ways to understand the maturity level and transparency of cloud providers.

What are SOC reports?

SOC is an auditing process that verifies whether your service providers implement proper security measures to safeguard your organization's data, ensuring both the protection of your business interests and the privacy of its clients. SOC has three levels of reports:

- **SOC 1** focuses on the controls that are critical to a service organization's financial reporting. A SOC 1 Type 1 report evaluates the design and implementation of a service organization's internal controls over financial reporting at a specific point in time, while a SOC 1 Type 2 report assesses both the design and operating effectiveness of those controls over a defined period, typically 6 to 12 months.

The primary audience for SOC 1 reports includes the following:

- Management of the service organization
 - External auditors of the service organization
 - Management of the service organization's customers
 - External auditors of the service organization's customers
- **SOC 2** evaluates controls related to the security, availability, processing integrity, confidentiality, and privacy of data. SOC 2 Type 1 evaluates controls at a specific point in time. SOC 2 Type 2 evaluates controls not just at a specific point in time but also the effectiveness of controls.

Note

Due to the sensitive content of SOC 2 Type 2 reports, CSPs tend to share those reports under a signed **non-disclosure agreement (NDA)**.

The primary audience for SOC 2 reports includes the following:

- Management of the service organization's customers
 - External auditors of the service organization's customers
 - Business partners
 - Prospective customers
 - Compliance regulators
- **SOC 3** is a high-level summary of an organization's controls without disclosing sensitive or detailed information.

The audience for SOC 3 reports is broad and includes the following:

- General public
- Potential customers
- Business partners
- Shareholders
- Marketing audiences

For more information, see the following link:

<https://www.aicpa-cima.com/resources/landing/system-and-organization-controls-soc-suite-of-services>

Tips for contracts with cloud providers

A good contract protects your organization when engaging with cloud providers since you do not have control over your data, located in a remote data center. You cannot always influence contracts with cloud providers.

To consume IaaS services, all you need is a credit card number. As a small organization, you will get the same contract as all other customers receive, and once you agree with the contractual terms, you cannot make any changes.

If you can commit to a large consumption of cloud services, you may contact your IaaS provider and sign an agreement, such as the AWS **Enterprise Discount Program (EDP)** or **Azure Enterprise Agreement (Azure EA)**, to be able to negotiate pricing terms.

To use SaaS services, you sign a contract with your chosen SaaS provider, where you can influence the contract terms (not always, but it is very common) – much more than just pricing topics.

In this section, we will review common topics that should be negotiated and embedded in contracts with cloud providers:

- **SLA:**
 - What is the guaranteed uptime percentage for your cloud services?
 - What security measures and standards are included in the SLA?
 - What is the process for reporting and escalating SLA violations?

- **Pricing and payment terms:**
 - What is the pricing structure, including any additional costs for data transfer or storage?
- **Exit terms:** Consider the following questions to understand the conditions under which you can terminate your contract with the CSP and what consequences it may have:
 - What are the customers' options for terminating a contract?
 - Are there any penalties from the customers' side for terminating a contract?
- **Data location:**
 - Does the contract specify what the exact data location is (that is, the country)?
 - Does the contract contain a **data processing agreement** for setting where customers' data is located and whether data can be processed outside a specific country?
 - Does the contract specify how the cloud provider will protect customers' data?
- **Data deletion:**
 - Is the cloud provider obligated to erase customers' data when terminating a contract?
 - What data erase standards does the cloud provider use when terminating a contract with its customers?
 - How long will the cloud provider keep customers' data after terminating a contract?
- **Data export:**
 - Is there an obligation from the cloud provider's side to return customers' data after contract termination?
 - What options does the cloud provider offer its customers to export a backup of their data after terminating a contract?
- **DR and BC:**
 - What are your RTOs and RPOs for different types of disasters?
 - What is your process for failover and failback in the event of a disaster?
 - Do you provide automated testing capabilities for DR scenarios? How frequently can we conduct these tests?
 - What monitoring and alerting systems do you have in place to detect potential issues before they escalate into disasters?

- **Supply chain security:**
 - How do you assess and manage risks associated with your own suppliers and third-party vendors?
 - How do you handle access management for third-party vendors who may need to interact with our data or systems?
 - What is your process for conducting regular security assessments of your supply chain partners?
- **IR:**
 - Does the contract specify what the cloud provider's obligations are regarding handling security incidents for the systems that store or process customers' data?
 - What is the cloud provider's SLA for fixing security vulnerabilities on the systems that store or process customers' data?
 - What is the cloud provider's SLA for notifying its customers about security incidents that affect the systems that store or process customers' data?
 - Does the contract specify what the cloud provider's third-party suppliers' obligations are regarding systems that store or process customers' data?
 - Does the contract contain information about security awareness, privacy, and secure development for the cloud provider's employees and third-party providers?

For more information, please refer to the following resources:

- *Custom terms and pricing in AWS Marketplace:*
<https://aws.amazon.com/marketplace/features/custom-terms-pricing/>
- *Azure EA agreements and amendments:*
<https://learn.microsoft.com/en-us/azure/cost-management-billing/manage/ea-portal-agreements>

We have reviewed important topics that should appear in a contract with cloud providers to protect customers' data while being stored or processed by them and to define each party's obligations as part of the contract and the shared responsibility model. Next, we dive into how we can use penetration testing to measure the effectiveness of a CSP's security provisions.

Conducting penetration testing in cloud environments

One of the ways to raise our assurance with a cloud provider is to conduct a penetration test to measure the effectiveness of their security controls. Penetration testing is a proactive security measure that involves simulating real-world cyberattacks to identify and address vulnerabilities throughout the development process. Penetration testing should be conducted on a regular basis, following the limitations that each cloud service model has, as explained next.

Let us review how to handle penetration testing in the various cloud service models and following the shared responsibility model:

- **IaaS:** We oversee the OS layer and the network environment around the VMs or containers. A penetration test allows us to measure the effectiveness of the security controls we have implemented.
- **PaaS:** In PaaS environments, specifically in FaaS, we are not in charge of the lower layer of the infrastructure; however, since we import our code, we are in charge of making sure we follow a secure development life cycle (conducting code reviews, making sure we are not using vulnerable third-party packages/binaries/libraries, and so on).
- **SaaS:** We are only in charge of inserting data and controlling access to the service. A penetration test allows us to measure how the SaaS provider protects our data.

If we expose a service to the internet or use a SaaS service, we need to evaluate topics such as the following:

- Is the service allowing only the minimum number of ports/protocols to/from the public internet? This can be achieved using vulnerability scanners or using **cloud security posture management (CSPM)** solutions.
- Is the service enforcing proper authentication controls to make sure only authenticated identities can access it? This will allow us to avoid unauthenticated or weak access attempts.
- Is the service enforcing proper authorization controls to make sure authenticated identities have the minimum amount of access required to achieve their tasks? This will allow us to avoid unauthorized access, such as privileged escalation attempts.
- Is the service performing proper input validation to make sure all data inserted into the system is sanitized and the system is protected from injection attacks, **cross-site scripting (XSS)**, and any other attacks originating from improper input validation? This will allow us to ensure our application and its code are protected from various injection attacks that may lead to data exfiltration or data tampering attacks.
- Is the service performing data sanitization before inserting data into a backend database – data that potentially can return to the end customer and impact them? This will allow us to ensure that data cannot be executed as scripts on the customer's side, an attack that may lead to data exfiltration such as credentials on the customer's side.
- Is the service keeping an audit log of any access attempt and request performed by its end customer? This will assist us as part of troubleshooting and as part of an IR process.

A penetration test, together with a vulnerability assessment, allows us to measure the service security controls; however, we must always remember that cloud environments are different than on-premises. By design, cloud environments are public and shared with many customers, which makes them prone to exposure by external attackers.

We must follow the cloud provider's terms of use. Unless we are using a SaaS service from one of the hyper-scale cloud providers, the chances are that the SaaS service is deployed above one of the hyper-scale IaaS providers, meaning we must follow the entire supply chain – both lower-layer IaaS and upper-layer SaaS providers' terms of use.

Mature cloud providers build their services in a fully multi-tenant model, meaning that each customer will have its dedicated infrastructure (such as servers, databases, storage, and network separation).

It is not that rare to come across a SaaS provider that offers us its services while sharing resources with its customers (such as sharing virtual servers, storage, and databases).

In scenarios where the SaaS provider hasn't embraced modern cloud architecture (that is, full data isolation in a multitenancy environment), one customer being targeted by an attacker will increase the chances of our organization being impacted while working with the same SaaS provider.

Most IaaS providers will allow you to run security tools against virtual servers in your IaaS environment, port scanning, or application-layer tests (such as injection attacks) against virtual servers in your IaaS environment. This will allow us adequate assurance about the controls we have implemented in our IaaS environments.

SaaS providers may allow you to conduct security tests against your environment, assuming the SaaS was originally built in a fully-isolated multi-tenant environment. This will allow us assurance about the controls implemented by the SaaS provider.

Likely, any cloud provider will not allow you to run **denial-of-service (DoS)**-type attacks against the cloud infrastructure or any type of attempt to breach or impact other customers' environments or resources.

For more information, please refer to the following resources:

- *AWS – Penetration Testing:*
<https://aws.amazon.com/security/penetration-testing/>
- *Azure – Penetration Testing Rules of Engagement:*
<https://www.microsoft.com/en-us/msrc/pentest-rules-of-engagement>
- *Google – Cloud Security FAQ:*
<https://support.google.com/cloud/answer/6262505>
- *Cloud Penetration Testing Playbook:*
<https://cloudsecurityalliance.org/artifacts/cloud-penetration-testing-playbook>

In this section, we have reviewed the concept of penetration testing, which allows you to test the effectiveness of cloud service security controls. The most important thing to remember is to always follow the cloud provider's terms of use.

Summary

In this chapter, we focused on engagement with cloud providers. We reviewed topics that will assist an organization in choosing a cloud provider (always focusing on business goals).

We then reviewed the cloud provider questionnaire, which allows customers to get familiar with and select mature cloud providers once they have decided to sign up with one.

We reviewed important topics that should appear in contracts with cloud providers, ensuring that both the legal and purchase departments have a solid understanding of what to look out for in the contractual phase.

Finally, we reviewed our ability as customers to measure the effectiveness of cloud environments (whether they are IaaS environments that we deploy and control or SaaS services that we use) by conducting penetration testing on cloud environments and the importance of following the cloud vendor's terms of use.

Understanding the topics discussed in this chapter will give customers confidence in using cloud services that store or process data outside their data centers.

In the next chapter, we will review hybrid clouds (such as hybrid cloud strategy, IdM, network connectivity, storage and compute-related services in hybrid cloud environments, and monitoring and security-related topics).

Part 4:

Advanced Use of Cloud Services

As more businesses use the cloud, keeping everything secure becomes even more important. Hybrid and multi-cloud setups mean companies work across different platforms, creating new security challenges. DevSecOps helps teams build security into their development process right from the start. For large-scale cloud environments, automation and strong security measures are needed to keep data safe. This part covers the best ways to handle security in advanced cloud systems.

This part comprises the following chapters:

- *Chapter 14, Managing Hybrid Clouds*
- *Chapter 15, Managing Multi-Cloud Environments*
- *Chapter 16, Implementing DevSecOps*
- *Chapter 17, Security in Large-Scale Environments*

Managing Hybrid Clouds

In the previous chapters, we covered cloud infrastructure fundamentals, common threats in cloud environments, and how to engage with cloud providers. In this chapter, we will discuss **hybrid clouds**.

A hybrid cloud is a combination of an on-premises data center or private cloud and a public cloud environment. It is considered an extension of our local data center and we must keep this in mind as we take the necessary steps to manage, maintain, and secure our infrastructure and services across the entire hybrid solution.

We are not only extending our local data center infrastructure to the cloud but also extending our security boundaries to the cloud, so we wish to have a central way to control security operations in a hybrid architecture.

In the following diagram, we can see an example of a hybrid architecture, based on Azure:

- A **Customer** is connecting from the public internet and reaches **Azure Front Door**, as both a global DNS service and a global load-balancer
- **Azure Front Door** routes the traffic to either the **On-Premises Data Center** or an Azure region (according to customer proximity)
- Both locations are connected via a site-to-site **VPN**, for sync purposes
- In both locations, there are load-balancers, and a farm of application servers, with a backend SQL database
- The databases sync their information to keep data up to date
- Authentication on-premises is done via Microsoft **Active Directory**, which is synced to **Microsoft Entra Domain Services**, to keep identities, passwords, and group membership (for access permissions to both application and infrastructure layers)

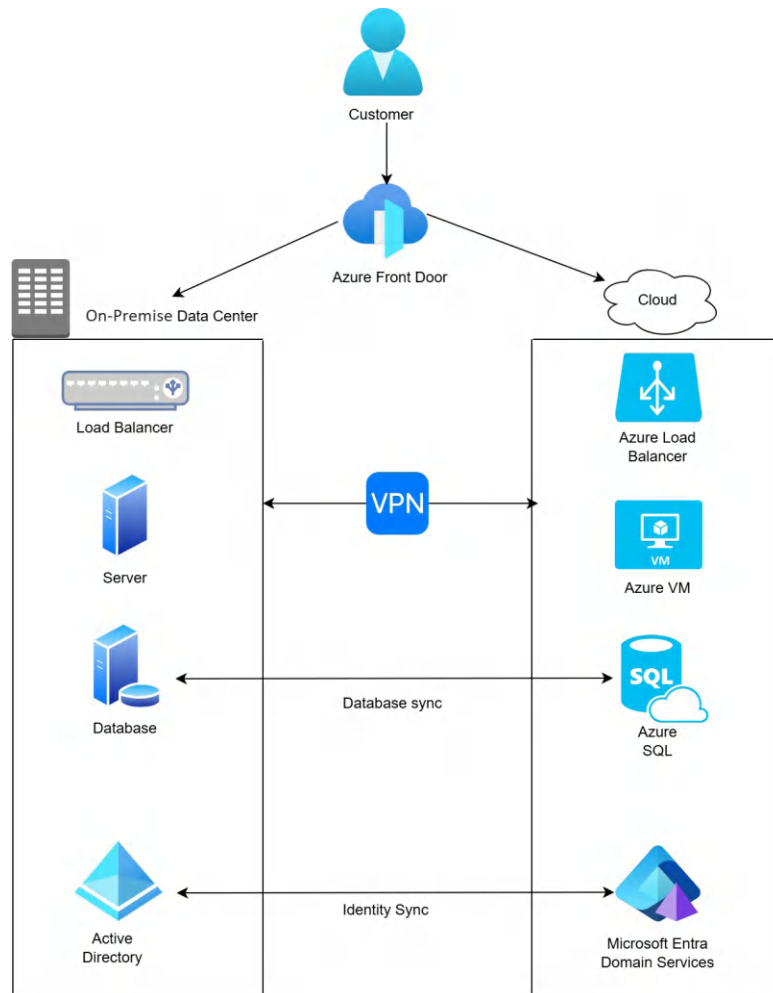


Figure 14.1 – Sample hybrid architecture between on-premises and Azure

In this chapter, we will cover the following topics:

- Hybrid cloud strategy
- Identity management over hybrid cloud environments
- Network architecture for hybrid cloud environments
- Storage services for hybrid cloud environments
- Computing services for hybrid cloud environments
- Securing hybrid cloud environments

Hybrid cloud strategy

Before we start using a hybrid cloud setup, we should think about what we want to achieve with it. We also need to figure out which resources can be moved to the cloud and which should stay on our local servers.

Cloud-native solutions for hybrid environments have the following benefits:

- Integration of both on-premises resources with cloud resources
- Built-in integration with cloud services
- Virtually unlimited capacity (compute and storage) for storing logs and running correlations between events
- Virtually unlimited storage capacity for storing data (for backups, regulation compliance, disaster recovery, and more)
- Support for federated identity management (allows a single identity to access resources in hybrid environments)

Let's look at some of the most common use cases for choosing hybrid cloud solutions.

Cloud bursting

The idea behind **cloud bursting** is to allow applications that run on-premises to burst into the cloud when there is a need for extra resource capacity – both planned and unplanned.

Some examples of cloud bursting are as follows:

- A large demand for cloud resources for a short period during the Black Friday weekend until Cyber Monday
- A campaign where a phone company ships a new product to the market
- Large batch processing, such as genetic research

Backup and disaster recovery

The cloud is an excellent solution for backup and/or disaster recovery for your on-premises environments. It has (almost) no limitations for storing long-term backups, and it is highly suitable for building entire environments (when using Infrastructure as Code, as explained in *Chapter 17, Security in Large-Scale Environments*) within minutes to serve as disaster recovery.

Archive and data retention

Regulation might require us to archive logs and retain them for a long period (years).

The cloud offers us a cheap solution for archiving storage tiers, combined with strong protection (encryption at rest using customer-managed keys, RBAC authorization, and an audit trail).

Distributed data processing

If there is a need for processing large amounts of data (such as *Apache Hadoop*), high-performance computing (such as weather forecasting), or machine learning (such as GenAI using large language models), the cloud can be a perfect solution.

Whenever we do not have the necessary resources in our local data center, we can deploy clusters in the cloud, perform our data analysis process, and, by the end of the process, erase all the unneeded resources to save money.

Application modernization

To avoid on-premises and legacy infrastructure constraints, the cloud allows us to re-architect our applications and switch to modern architectures such as microservices or even **Function as a Service (FaaS)**.

Some workloads can be migrated to the cloud (such as modern developments), while other workloads must remain on-premises (due to regulations that force us to keep data on-premises) – such a mixed environment creates a hybrid architecture.

Creating a cloud strategy helps organizations decide what to do with their existing applications and systems:

- **Retire:** Decide which applications are no longer needed
- **Retain:** Decide which applications are needed but the migration to the cloud will be too complex and should remain on-premises
- **Replace:** Decide which applications can be replaced with managed services or SaaS solutions
- **Rehost:** Decide which applications can be *lifted and shifted* with minimal effort to the cloud
- **Re-platform:** Decide which applications need minimal changes to migrate to the cloud (for example, an OS upgrade)
- **Refactor:** Decide which application requires significant modifications to migrate to the cloud (for example, switching from a monolith to a microservice architecture)
- **Reimagine:** Decide which applications need to be rebuilt from scratch to benefit from cloud advantages (such as elasticity, multi-region, and more)

For more information, please refer to the following resources:

- *Building Your Hybrid Cloud Strategy with AWS*: <https://aws.amazon.com/blogs/publicsector/building-your-hybrid-cloud-strategy-with-aws/>
- *Azure hybrid options*: <https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/hybrid-considerations>
- *Plan a hybrid and multi-cloud strategy*: <https://cloud.google.com/architecture/hybrid-multicloud-patterns/strategy>

A hybrid cloud strategy helps organizations decide what to do with existing applications and set security policies for storing data in the cloud. This lets organizations leverage cloud advantages (such as scale and short purchase processes) while still keeping legacy applications on-premises.

Identity management over hybrid cloud environments

One of the first things to decide on, before using the hybrid cloud, is identity management. Organizations would like to keep their existing identity provider, have a single identity for each of their end users (while preserving existing credentials), and still be able to access resources in the cloud.

Identity management in hybrid cloud environments can be split into the following areas:

- **Directory replication**: Extending the on-premises directory into the cloud with either one-way replication or synchronization between the two
- **Federated authentication**: This on-premises component brokers the user authentication to the cloud using **Security Assertion Markup Language (SAML)**, **OpenID Connect (OIDC)**, or some other protocol

Some of the benefits of using centralized identity management are as follows:

- A single place to provision or de-provision identities
- Reusing strong credentials and authentication capabilities
- Centralization of access audits
- Avoid supporting every cloud identity mechanism

Best practices for identity in a hybrid environment

Before we look at the different components that each **cloud service provider (CSP)** implements for managing hybrid identities, let us review some of the best practices:

- **Centralized identity management:** Select a central identity provider, to manage identities over the hybrid environment. For most organizations managing legacy applications on-premises, the most common choice is Microsoft Active Directory, but it can also be Okta, Ping, OneLogin, and so on.
- **Authorization:** When granting permissions for identities to take actions over the entire hybrid environment, always follow the principle of least privilege to minimize the risk of unauthorized access to company resources or data.
- **Permission review process:** It is crucial to regularly review all permissions granted to identities (for both users and high privileges for admins), to make sure all permissions have business justifications. When permissions are no longer needed, they must be revoked.
- **Just-in-time (JIT) access:** When granting high privileges (such as admin access to manage production servers), it is crucial to implement a break glass mechanism, where privileges need to follow a request, review, approval, and documentation process, and access is granted for a short amount of time, for specific resources.
- **Single-Sign-on (SSO):** By implementing an SSO solution, we can minimize the number of credentials users need to manage.
- **Multi-Factor Authentication (MFA):** It is crucial to enforce the use of MFA for all interactive login, to mitigate against credential theft and account hijack, which may lead to unauthorized access to company resources or customer data.
- **Encryption in transit:** Always pass authentication information through secured protocols – from TLS for SAML-based authentication, through Kerberos inside site-to-site VPN tunnels (over IPsec), to LDAP over SSL (LDAPS).
- **Auditing:** It is crucial to enable auditing for all access requests (both successful and failed ones) on the entire hybrid environment (i.e., both on-premises and in all cloud environments). Audit logs should be sent to a central **security information and event management (SIEM)** system, to allow further investigation as part of incident response.

Managing identity over hybrid AWS environments

On-premises environments can have various types of identity providers – from Active Directory (for Windows-based applications) to LDAP-based identity directories (mostly for Linux-based applications). An organization may also consider **Security Assertion Markup Language (SAML)**-based authentication for cloud-native services.

Amazon supports the following identity management solutions for hybrid environments:

- **AWS IAM with Active Directory Federation Services (ADFS):** Gives you the ability to use AWS IAM SAML-based directory service to manage your on-premises identities (from the on-premises Active Directory). Use AWS IAM with ADFS if you have a complex on-premises AD setup and need granular control over AWS permissions based on AD groups.
- **AWS Managed Microsoft AD:** A fully managed Active Directory allows you to manage user identities from on-premises AD or a local LDAP provider (usually for Linux-based applications). Use AWS Managed Microsoft AD if you need a full-fledged AD environment in AWS for applications or EC2 instances that require AD authentication.
- **AD Connector:** A proxy service that allows you to manage AWS services such as Amazon WorkSpaces, Amazon EC2, and more using the on-premises Active Directory. Use AD Connector if you want to use your on-premises AD for AWS authentication without storing directory data in the cloud.
- **AWS IAM Identity Center:** This allows you to centrally create identities across the AWS organization or bring identities from Microsoft Active Directory or other identity providers (such as OKTA, OneLogin, and others). Use AWS IAM Identity Center for managing access across multiple AWS accounts and applications, especially if you are using AWS Organizations.

Here's a common authentication flow in a hybrid environment:

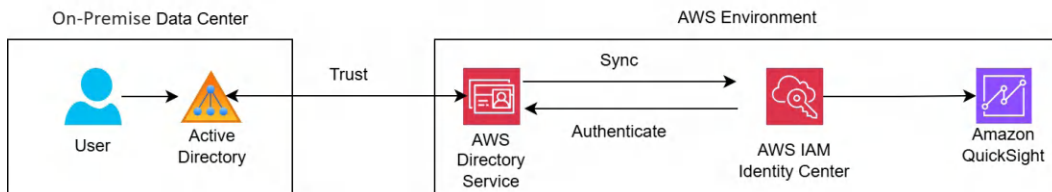


Figure 14.2 – Sample hybrid authentication in AWS

Note

The authentication flow concept is the same for all cloud providers. However, as each cloud provider goes through different service names, we have described the flow for all mentioned cloud providers for clarity.

Let's understand what's happening here:

- A **User** from on-premises authenticates to **Active Directory** using Kerberos authentication
- The **Active Directory** is synced to the **AWS Directory Service** in a trust relationship
- The **AWS Directory Service** is synced to **AWS IAM Identity Center**

- **AWS IAM Identity Center** converts a Kerberos authentication token to a SAML token
- Using a SAML token, the user can authenticate to **Amazon QuickSight** (which supports SAML authentication), to be able to consume information for business intelligence (BI) purposes

Although most of the security best practices for AWS Directory Service were already mentioned in *Chapter 9, Effective Strategies for Implementing IAM Solutions*, let us recap the main best practices, as they apply to hybrid environments:

- Protect the built-in admin account created during the initial setup of AWS Directory Service, using a long and complex password, to decrease the chance of unauthorized access to your cloud environment. Store the password in a secured vault, and regularly rotate the admin password.
- When using AD Connector, regularly rotate the service account used to connect AD Connector to the on-premise Active Directory to avoid misuse, as explained here: https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ad_connector_update_creds.html.
- Configure a password policy (which includes the minimum and maximum password age, the minimum password length, and enforces the use of password history and complex passwords), to decrease the chance of unauthorized access to your cloud environment, as explained here: https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_password_policies.html.
- To avoid using a privileged account for a password reset, use the **AWS Delegated Fine-Grained Password Policy Administrators** group to allow your support team permission to reset passwords, as explained here: <https://docs.aws.amazon.com/directoryservice/latest/admin-guide/delegatepasswordpolicies.html>.
- Enforce MFA for all human (interactive) authentication to decrease the chance of unauthorized access to your cloud environment, as explained here:
 - https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_mfa.html
 - https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ad_connector_mfa.html
- Enable AWS Directory Service log forwarding and forward the Active Directory logs to Amazon CloudWatch for further Active Directory log analysis, such as tracking failed Active Directory logins, as explained here: https://docs.aws.amazon.com/directoryservice/latest/admin-guide/ms_ad_enable_log_forwarding.html.

Thorough documentation on security in AWS IAM Identity Center can be found at the following link: <https://docs.aws.amazon.com/singlesignon/latest/userguide/security.html>

Use the following best practices for securing AWS IAM Identity Center:

- Use identity-based policies to grant minimal access to manage AWS IAM Identity Center, as explained here: <https://docs.aws.amazon.com/singlesignon/latest/userguide/iam-auth-access-using-id-policies.html>
- Enforce the use of MFA for all interactive accounts to mitigate against unauthorized access to your AWS environment, as explained here: <https://docs.aws.amazon.com/singlesignon/latest/userguide/mfa-getting-started.html>
- Create permissions to manage AWS IAM Identity Center, following the principle of least privilege, to avoid unauthorized access, as explained here: <https://docs.aws.amazon.com/singlesignon/latest/userguide/get-started-create-a-permission-set.html>
- Configure session duration for accounts with access to AWS accounts, to automatically disconnect sessions, after a pre-configured amount of time (i.e., for the duration needed by users to perform their work on AWS accounts), as explained here: <https://docs.aws.amazon.com/singlesignon/latest/userguide/howtosessionduration.html>
- AWS CloudTrail is enabled by default for AWS IAM Identity Center APIs. CloudTrail logs provide a detailed record of API calls made by users and services in your AWS account. To keep IAM Identity Center API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here: <https://docs.aws.amazon.com/singlesignon/latest/userguide/sso-info-in-cloudtrail.html>.
- Enable AD sync and configurable AD sync error logs, to aid in an incident response process, as explained here: <https://docs.aws.amazon.com/singlesignon/latest/userguide/logging-ad-sync-errors.html>

Managing identity over hybrid Azure environments

Azure supports the following identity management solutions for hybrid environments:

- **Microsoft Entra ID:** This offers SAML-based authentication. When synced with on-premises Active Directory (using **Microsoft Entra Connect**), you can have a single identity with access (according to business needs) to both on-premises resources and cloud-based resources (such as Azure resources and any SAML-based cloud service). Use this service when you need identity management for cloud-native applications and services.
- **Microsoft Entra Domain Services:** This is a managed domain service that allows legacy protocols such as Kerberos (for Windows-based applications) or LDAP (for Linux-based applications) to be integrated. Use this service when you need to support legacy applications that rely on Active Directory features.

Here's a common authentication flow in a hybrid environment:

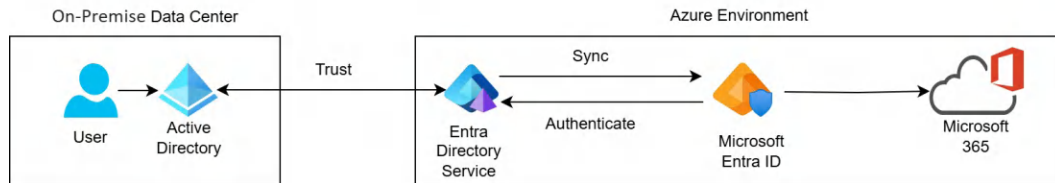


Figure 14.3 – Sample hybrid authentication in Azure

Let's understand what's happening in the flow:

- A **User** from on-premises authenticates to **Active Directory** using Kerberos authentication
- The **Active Directory** is synced to **Microsoft Entra Directory Service** in a **Trust** relationship
- **Microsoft Entra Directory Service** is synced to **Microsoft Entra ID** and converts a Kerberos authentication token to a SAML token
- Using a SAML token, the user can authenticate to **Microsoft 365** (which supports SAML authentication), to be able to consume Microsoft Teams services

Although most of the security best practices for Microsoft Entra Domain Service were already mentioned in *Chapter 9, Effective Strategies for Implementing IAM Solutions*, let us recap the main best practices, as they apply to hybrid environments:

- For hybrid environments (on-premises Active Directory and Microsoft Entra ID), sync objects (i.e., users, group membership, and password hashes) using Microsoft Entra Connect, to allow on-premises users access to Azure resources, as explained here: <https://learn.microsoft.com/en-us/entra/identity/domain-services/synchronization>
- To secure access from on-premises to Azure, enforce the use of secure network protocols (such as TLS 1.2, or above), and disable the use of legacy/weak protocols (such as NTLM v1), as explained here: <https://learn.microsoft.com/en-us/entra/identity/domain-services/secure-your-domain>
- Use network security groups to restrict access to Microsoft Entra Domain Services domain controllers, from trusted CIDRs (such as private subnets or on-premises) to decrease the chance of direct attack against your managed domain controllers, as mentioned here: <https://learn.microsoft.com/en-us/entra/identity/domain-services/network-considerations>
- Enable diagnostic settings for Microsoft Entra Domain Services for further investigation and analysis, and send all logs to a Log Analytics workspace, as explained here: <https://learn.microsoft.com/en-us/entra/identity/domain-services/security-audit-events>

Managing identity over GCP hybrid environments

Google supports the following identity management solutions for hybrid environments:

- **Google Cloud Identity:** This is a service for managing identities from both GCP and external identity providers, with support for SSO (for cloud services), and MFA. Use Cloud Identity if you want a comprehensive identity management system that integrates well with Google Cloud and other SaaS applications.
- **Workforce Identity Federation:** This is a service that allows you to use an external identity provider for authenticating to Google Cloud without synchronizing user accounts. Use Workforce Identity Federation if you want to enable access to Google Cloud using your existing identity provider without extensive synchronization.
- **Google Cloud Directory Sync (GCDS):** This is a tool that synchronizes users, groups, and other data from the on-premises Active Directory or LDAP server to Google Cloud. Use GCDS if you need to maintain your on-premises directory as the source of truth and require detailed control over synchronization.
- **Google Managed Microsoft Active Directory:** This is a fully managed Active Directory that allows you to manage user identities from on-premises Active Directory. Use Managed Microsoft AD when you want the benefits of Active Directory in Google Cloud without the complexities of managing AD infrastructure.

Here's a common authentication flow in a hybrid environment:

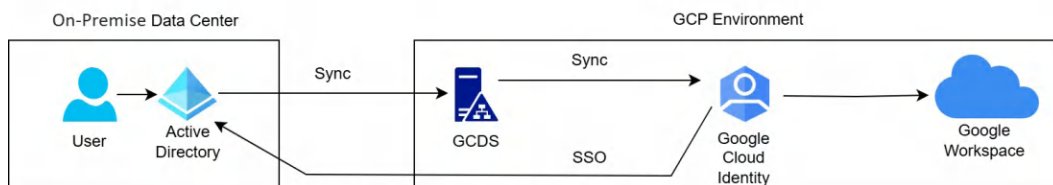


Figure 14.4 – Sample hybrid authentication in GCP

Let's understand what's happening here:

- A **User** from on-premises authenticates to **Active Directory** using Kerberos authentication
- The **Active Directory** is synced to Google Cloud using **Google Cloud Directory Sync (GCDS)**
- GCDS is synced to **Google Cloud Identity**, which converts a Kerberos authentication token to a SAML token
- Using a SAML token, the user can authenticate to **Google Workspace** (which supports SAML authentication) to be able to consume messaging services

Although most of the security best practices for Google Managed Microsoft AD were already mentioned in *Chapter 9, Effective Strategies for Implementing IAM Solutions*, let us recap the main best practices, as they apply to hybrid environments:

- Since on-premise Active Directory supports authentication against legacy systems using LDAP protocol, enforce traffic encryption (i.e., LDAP over TLS), to keep the confidentiality of passwords passing over the network in plain text, as explained here: <https://cloud.google.com/managed-microsoft-ad/docs/how-to-use-ldaps>
- To avoid using a privileged account for a password reset, use the delegated administrator account in Google Managed Microsoft AD, to allow your support team permission to reset passwords, as explained here: <https://cloud.google.com/managed-microsoft-ad/docs/how-to-use-delegated-admin>
- Use VPC firewall rules to restrict access to Google Managed Microsoft AD, from trusted CIDRs (such as private subnets or on-premises) to decrease the chance of direct attack against your managed domain controllers, as mentioned here: <https://cloud.google.com/managed-microsoft-ad/docs/firewalls>
- To restrict access to Google Managed Microsoft AD from trusted sources and reduce the risk of data exfiltration, use VPC Service Controls, as mentioned here: <https://cloud.google.com/managed-microsoft-ad/docs/how-to-use-vpc-service-controls>

Use the following best practices for securing Google Cloud Directory Sync:

- Create a dedicated account for GCDS, following the principle of least privilege, to avoid unauthorized access, as explained here: <https://support.google.com/cloudidentity/answer/6126573>
- Use Configuration Manager to set up GCDS, as documented here: <https://support.google.com/cloudidentity/answer/6162412>
- Enforce traffic encryption (i.e., LDAP over TLS), to keep the confidentiality of passwords passing over the network in plain text, as mentioned here: <https://support.google.com/cloudidentity/answer/6123896>

Use the following best practices for securing Google Cloud Identity:

- Protect admin accounts using 2-Step Verification, and regularly rotate all admin account passwords, to avoid unauthorized access, as explained here: <https://support.google.com/cloudidentity/answer/9011373>
- Enforce **multi-factor authentication (MFA)** for all human (interactive) authentication to avoid unauthorized access to your GCP environment, as explained here:
 - <https://support.google.com/cloudidentity/answer/175197?hl=en>
 - <https://support.google.com/cloudidentity/answer/9176657>

- Forward all logs to Google Cloud Logging, and from Cloud Logging to a central SIEM system, to aid in an incident response process, as mentioned here: <https://support.google.com/a/answer/4579579>

In this section, we reviewed the different alternatives AWS, Azure, and GCP offer customers to manage identities in a hybrid environment. Selecting authentication and authorization methods allows organizations to reuse their existing identity providers and keep a single identity for each of their end users in a central repository.

The second important thing to consider when building a hybrid architecture is how to connect from the on-premises environment to the cloud. Let's dive into that now.

Network architecture for hybrid cloud environments

The recommended way to connect to cloud environments, considering the cloud as an extension of the local data center, is to use a secure and permanent network connection – either a site-to-site VPN or a dedicated connection. A secured and permanent connection will allow you to set access control (layer 4 firewall rules) between on-premises segments and cloud segments and retain access to resources in the cloud (or allow access from cloud resources to the on-premises environment) in terms of business needs.

Following are some of the solutions that you can choose depending on the specific situation you are dealing with:

- You should choose a VPN solution in the following situations:
 - You need fast deployment
 - You are OK passing an IPSec tunnel over the internet
 - You do not have bandwidth requirements
 - You are looking for a low-cost solution
- You should choose a dedicated private connection in the following situations:
 - You have bandwidth requirements (you need a solution with a fixed bandwidth)
 - You would like to have SLAs on the network connectivity
 - You would like to have private connectivity between your on-premises environment and the cloud environment (i.e., not passing through the public internet)

The following are some best practices to keep in mind:

- Always think about **Classless Inter-Domain Routing (CIDR)** before configuring network segments in the cloud to avoid IP conflicts between the on-premises environment and the cloud. CIDR helps you efficiently allocate IP addresses and avoid overlap, ensuring that all devices can communicate smoothly without running into issues caused by duplicate IP addresses.
- Use **network ACLs** and **route tables** to configure network access rules between the on-premises environment and the resources inside your VPC/VNet.
- For redundancy, plan failover connectivity – from multiple VPN connections or redundant interconnects using different network providers or a combination of dedicated private connectivity and VPN connectivity.

Connecting the on-premises environment to AWS

Amazon offers the following services to allow hybrid connectivity:

- **AWS Site-to-Site VPN:** This is a fully managed VPN service that provides connectivity from the customer gateway (customer side of the VPN) over an IPsec tunnel. Use AWS Site-to-Site VPN for simple, secure connections with moderate bandwidth needs.
- **AWS VPN CloudHub:** This is a hub and spoke model that uses AWS-managed VPNs to connect Amazon VPC to multiple customer data centers, each with its own IPsec tunnels. Use AWS VPN CloudHub if you have multiple branch offices needing to connect to AWS and each other.
- **AWS Transit Gateway:** This is a central hub to route traffic between VPCs, VPNs, and Direct Connect connections. Use AWS Transit Gateway for complex network architectures with multiple VPCs and on-premises locations.
- **AWS Direct Connect:** This is a dedicated connection from the on-premises environment to one or more VPCs in the same region in a pre-defined network bandwidth. Use AWS Direct Connect if you require high bandwidth, low latency, and consistent network performance.

The following are some best practices to keep in mind:

- Use a dual site-to-site VPN connection, along with **AWS Transit Gateway**, for connection redundancy between your on-premises environment and the cloud
- Use multiple **Direct Connect (DX)** connections with the DX gateway so that you have connection redundancy between your on-premises environment and the cloud, as explained here: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/disaster-recovery-resiliency.html>
- Configure routing and security groups to restrict access to AWS resources, as explained here: <https://repost.aws/knowledge-center/routing-dx-private-virtual-interface>

In the following diagram, we can see how AWS Transit Gateway can be used as a central connectivity hub from various network connections (from site-to-site VPN, AWS Direct Connect, and various AWS VPCs):

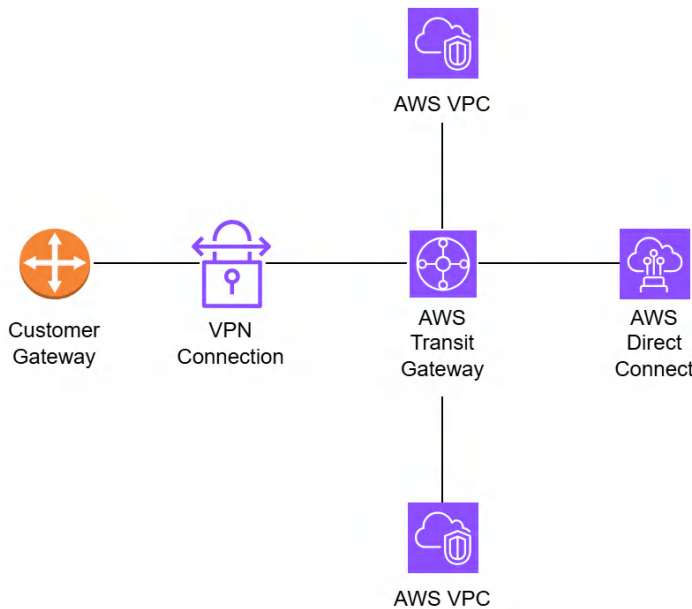


Figure 14.5 – Sample AWS Transit Gateway architecture

Although we covered most of the security best practices for securing AWS Site-to-Site VPN in *Chapter 6, Securing Networking Services – Part 1*, let us recap the main best practices, as they apply to hybrid environments:

- Encrypt traffic over the VPN tunnel using the AES256-GCM-16 algorithm and SHA2-256 (at a minimum) as a hashing algorithm, as documented here: <https://docs.aws.amazon.com/vpn/latest/s2svpn/your-cgw.html>
- For non-sensitive environments, use pre-shared keys to authenticate to the site-to-site VPN tunnel, as explained here: <https://docs.aws.amazon.com/vpn/latest/s2svpn/vpn-tunnel-authentication-options.html#pre-shared-keys>
- For highly sensitive environments, use a private certificate from the **AWS Certificate Manager (ACM) Private Certificate Authority (CA)** service, as explained here: <https://docs.aws.amazon.com/vpn/latest/s2svpn/vpn-tunnel-authentication-options.html#certificate>
- Restrict access to AWS resources inside your AWS environment using Amazon VPC security groups, as explained here: <https://docs.aws.amazon.com/vpn/latest/s2svpn/SetUpVPNConnections.html#vpn-configure-security-groups>

Here are some of the best practices for configuring AWS Transit Gateway connections:

- Use security groups to restrict access to AWS resources inside VPCs, as explained here: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-sg-updates-update.html>
- A complex environment with multiple connections through AWS Transit Gateway may contain multiple route tables. As best practice, it is recommended to regularly export the route tables to an S3 bucket, as part of disaster recovery processes, as explained here: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-export-route-tables.html>

Here are some of the best practices for configuring AWS Direct Connect:

- To keep data confidentiality, encrypt data in transit for AWS Direct Connect using an AWS Site-to-Site VPN, as mentioned here: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/encryption-in-transit.html>
- Use AWS IAM identity-based policies to control who can create or modify AWS Direct Connect settings, as explained here: https://docs.aws.amazon.com/directconnect/latest/UserGuide/security_iam_id-based-policy-examples.html
- Use Amazon CloudWatch alarms to generate alerts via Amazon SNS, when certain metrics pass a pre-defined value (for example, when the state of a Direct Connect connection is down for five consecutive one-minute periods), as explained here: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/creating-alarms.html>
- AWS CloudTrail is enabled by default for AWS Direct Connect. To keep AWS Direct Connect API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as mentioned here: https://docs.aws.amazon.com/directconnect/latest/UserGuide/logging_dc_api_calls.html

Connecting the on-premises environment to Azure

Azure offers the following services to allow hybrid connectivity:

- **Azure VPN Gateway:** This is a fully managed VPN service that provides connectivity from the on-premises environment to the cloud over IPsec tunnels. Use this alternative for smaller-scale deployments, or when lower bandwidth requirements (up to 10 Gbps) are sufficient.
- **Azure ExpressRoute:** This offers a dedicated connection from the on-premises environment to one or more Azure VNets in a pre-defined network bandwidth. Use this alternative for large enterprises with high bandwidth needs or when predictable performance and lower latency are critical.
- **Azure Virtual WAN:** This is a comprehensive networking service that brings together many networking, security, and routing functionalities. Use this alternative for large, distributed organizations with multiple sites and complex networking needs or when you need a fully meshed and transit-oriented network architecture.

The following are some best practices to consider:

- Use a dual site-to-site VPN connection, along with **Azure Virtual WAN**, for connection redundancy between your on-premises environment and the cloud. This way, if one connection goes down, the other can still keep your data flowing smoothly between your local environment and the cloud, maintaining reliable and continuous connectivity.
- Configure multiple connections from your on-premises environment (through your service provider's network) to the Azure ExpressRoute circuit for connection redundancy between your on-premises environment and the cloud.

In the following diagram, we can see how Azure Virtual WAN can be used as a central connectivity hub from various network connections (from site-to-site VPN, Azure Express Route, and various Azure VNets):

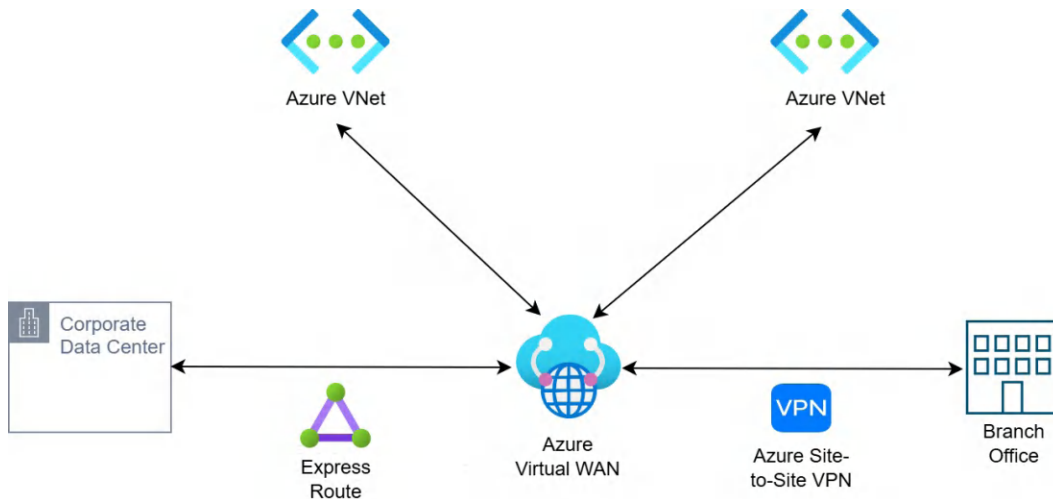


Figure 14.6 – Sample Azure Virtual WAN architecture

Although most of the security best practices for securing Azure Site-to-Site VPN were already mentioned in *Chapter 6*, let us recap the main best practices, as they apply to hybrid environments:

- Encrypt traffic over the VPN tunnel using the GCM/AES256 algorithm and SHA256 as a hashing algorithm, as documented here: <https://learn.microsoft.com/en-us/azure/vpn-gateway/vpn-gateway-about-vpn-devices#ipsec>
- Enable diagnostic settings for Azure VPN Gateway, and send all logs to a Log Analytics workspace, as explained here: <https://learn.microsoft.com/en-us/azure/azure-monitor/essentials/diagnostic-settings>

- Use Azure Monitor to monitor the VPN tunnels (for example, it could send alerts when the amount of traffic in bytes is above a pre-defined threshold), as mentioned here: <https://learn.microsoft.com/en-us/azure/vpn-gateway/vpn-gateway-howto-setup-alerts-virtual-network-gateway-metric>

Here are some of the best practices for configuring Azure Virtual WAN:

- When using Azure Virtual WAN with Azure ExpressRoute, encrypt data in transit using IPsec to keep data confidentiality, as explained here: <https://learn.microsoft.com/en-us/azure/virtual-wan/vpn-over-expressroute>
- Limit routes from Azure to on-premises to your private CIDRs, to avoid unauthorized traffic to your resources, as explained here: <https://learn.microsoft.com/en-us/security/zero-trust/azure-virtual-wan>
- Limit traffic between Azure VNets according to business needs, to avoid unauthorized traffic to your resources, as explained here: <https://learn.microsoft.com/en-us/azure/virtual-wan/scenario-isolate-vnets>

Here are some of the best practices for configuring Azure ExpressRoute:

- Encrypt data in transit using a site-to-site VPN combined with Azure ExpressRoute, to keep data confidentiality, as explained here: <https://learn.microsoft.com/en-us/azure/vpn-gateway/site-to-site-vpn-private-peering>
- For high resiliency, configure both Azure ExpressRoute and site-to-site VPN connections in different **availability zones (AZs)**, to avoid a scenario where an AZ fails and all connections from the on-premises to Azure go down, as explained here: <https://learn.microsoft.com/en-us/azure/vpn-gateway/create-zone-redundant-vnet-gateway>

Connecting the on-premises environment to GCP

GCP offers the following services to allow hybrid connectivity:

- **Google Cloud VPN:** This is a fully managed VPN service that provides connectivity from the on-premises environment to the cloud over an IPsec tunnel. Use Cloud VPN for cost-effective, secure connections with lower bandwidth needs.
- **Google Cloud Interconnect:** This offers a dedicated connection from the on-premises environment to one or more VPCs in a pre-defined network bandwidth. Use Cloud Interconnect when requiring higher performance, security, and bandwidth for mission-critical and highly sensitive workloads.

The following are some best practices to consider:

- For redundancy, plan for failover connectivity – either configure multiple VPN connections or multiple Google Cloud Interconnect using different network providers or a combination of Google Cloud Interconnect and a VPN connection
- Use a **high availability (HA)** VPN for connection redundancy between your on-premises environment and the cloud
- Use **Dedicated Interconnect** for connection redundancy between your on-premises environment and the cloud

Although most of the security best practices for securing Google Cloud VPN were already mentioned in *Chapter 6*, let us recap the main best practices, as they apply to hybrid environments:

- Restrict access to GCP resources inside your GCP environment using VPC firewall rules, as explained here: <https://cloud.google.com/network-connectivity/docs/vpn/how-to/configuring-firewall-rules>
- Restrict the IP addresses of the peer VPN gateway to prevent unauthorized VPN tunnels, as explained here: <https://cloud.google.com/network-connectivity/docs/vpn/how-to/restrict-peer-ip-addresses>
- Encrypt traffic over the VPN tunnel using the AES-GCM-16-256 algorithm and SHA2-256 (at a minimum) as a hashing algorithm, as documented here: <https://cloud.google.com/network-connectivity/docs/vpn/concepts/supported-ike-ciphers>

Here are some of the best practices for configuring Google Cloud Interconnect:

- Restrict which Google VPCs can use Cloud Interconnect, to avoid unauthorized network access, as explained here: <https://cloud.google.com/network-connectivity/docs/interconnect/how-to/restricting-usage>
- To keep data confidentiality, encrypt data in transit using HA VPN over Cloud Interconnect, as explained here: <https://cloud.google.com/network-connectivity/docs/interconnect/how-to/ha-vpn-interconnect-deploy-process>

Now that we have decided on identity management and network topology, the next thing we need to consider is how to use hybrid clouds for data transfer and storage.

Storage services for hybrid cloud environments

In this section, we will review the various options for data transfer.

When considering hybrid storage connectivity, consider the following:

- Bandwidth/latency/time to transfer
- Use of public versus private connectivity
- Moving files versus file synchronization
- Encryption requirements (in transit and at rest)
- Access control for a hybrid solution
- Supported protocols (NFS, CIFS, HTTP(S), and others)

The following are some best practices to keep in mind:

- Always choose secured protocols (such as TLS, SFTP, or FTPS) when transferring files to the cloud, to keep data confidentiality while traversing the network
- Encrypt data at rest when it is stored in the cloud, to keep data confidentiality
- Always configure permissions on cloud resources, according to your business needs, and follow the principle of least privilege, to avoid unauthorized access
- Enable auditing on any access to cloud resources, and send all logs to a central SIEM system, to aid in an incident response process

Connecting to storage services over AWS hybrid environments

Amazon offers the following services to transfer data to the cloud in a hybrid architecture:

- **AWS Storage Gateway:** This is a virtual appliance for installing on-premises environments that allows access from the on-premises environments to an object storage service (Amazon S3/Glacier), file storage (Amazon FSx for Windows), block storage (Amazon EBS), and a backup service (AWS Backup). Use AWS Storage Gateway when you need to extend on-premises storage to the cloud and require low-latency access to cloud data for on-premises applications.
- **AWS DataSync:** This is a data transfer service between the on-premises environment and object storage (Amazon S3), NFS file storage (Amazon EFS), and SMB file storage (Amazon FSx for Windows). Use AWS DataSync when you need to perform large-scale data transfers to or from AWS.
- **AWS Transfer Family:** This offers a file transfer service between the on-premises environment and object storage (Amazon S3) or NFS file storage (Amazon EFS), above the SFTP, FTPS, and FTP protocols. Use AWS Transfer Family when you need to support standard file transfer protocols (SFTP, FTPS, FTP), and when you require a managed service for secure file transfers to and from S3 or EFS.

In the following diagram, we can see an NFS file server uploading files to an AWS Storage Gateway virtual application, which transfers the files and uploads them into an S3 bucket:



Figure 14.7 – AWS Storage Gateway architecture

Here are some of the best practices for securing AWS Storage Gateway:

- Encrypt data in transit and at rest, to keep data confidentiality, as explained here: <https://docs.aws.amazon.com/storagegateway/latest/vgw/encryption.html>
- Use identity-based policies to grant minimal access to manage the AWS Storage Gateway, as explained here: https://docs.aws.amazon.com/storagegateway/latest/vgw/security_iam_id-based-policy-examples.html
- AWS CloudTrail is enabled by default for AWS Storage Gateway APIs. CloudTrail logs provide a detailed record of API calls made by users and services in your AWS account (such as view, search, download, etc.). To keep Storage Gateway API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here: <https://docs.aws.amazon.com/storagegateway/latest/vgw/logging-using-cloudtrail.html>

Here are some of the best practices for securing AWS DataSync:

- Encrypt data in transit (according to the target protocol – NFS, SMB, or HTTP), to keep data confidentiality, as explained here: <https://docs.aws.amazon.com/datasync/latest/userguide/encryption-in-transit.html>
- Encrypt data at rest, to keep data confidentiality while stored in one of AWS's storage services, as explained here: <https://docs.aws.amazon.com/datasync/latest/userguide/encrypting-data.html>
- Use identity-based policies to grant minimal access to manage AWS DataSync, as explained here: <https://docs.aws.amazon.com/datasync/latest/userguide/managing-access-overview.html>

Here are some of the best practices for securing AWS Transfer Family:

- Use the latest cryptographic algorithms to protect data transfer to/from target servers from weak/vulnerable algorithms, as explained here: <https://docs.aws.amazon.com/transfer/latest/userguide/security-policies.html>
- Use the latest cryptographic algorithms to protect data transfer using SFTP from weak/vulnerable algorithms, as explained here: <https://docs.aws.amazon.com/transfer/latest/userguide/security-policies-connectors.html>
- Encrypt data inside target S3 buckets using AWS KMS managed keys (SSE-KMS), as explained here: <https://docs.aws.amazon.com/transfer/latest/userguide/encryption-at-rest.html>
- Regularly rotate the SSH keys to avoid key reuse, as explained here: <https://docs.aws.amazon.com/transfer/latest/userguide/keyrotation.html>
- Use identity-based policies to grant minimal access to manage the AWS Transfer Family resources, as explained here: https://docs.aws.amazon.com/transfer/latest/userguide/security_iam_id-based-policy-examples.html

Connecting to storage services over Azure hybrid environments

Azure offers the following services to transfer data to the cloud in a hybrid architecture:

- **Azure Data Box Gateway:** This is a service for installing on-premises environments that allows you to copy data from the on-premises environment to Azure Blob Storage using the NFS or SMB protocol. Use Data Box Gateway when you need to continuously transfer large amounts of data to Azure Blob Storage.
- **Azure File Sync:** This is a data copy service that is installed as an agent on Windows machines and allows you to copy files over the SMB or NFS protocol to Azure Files. Use Azure File Sync when you want to centralize your file shares while maintaining local access performance, and you need to synchronize file data across multiple sites.

Here are some of the best practices for securing Azure Data Box Gateway:

- Regularly rotate the Data Box Gateway device password to avoid credential leakage and reuse by unauthorized parties, as explained here: <https://learn.microsoft.com/en-us/azure/databox-gateway/data-box-gateway-manage-access-power-connectivity-mode#manage-device-access>
- Regularly rotate the passwords for the users on Data Box Gateway, to avoid credential leakage and reuse by unauthorized parties, as explained here: <https://learn.microsoft.com/en-us/azure/databox-gateway/data-box-gateway-manage-users>
- Generate the Data Box Gateway device activation key using Azure Key Vault, to keep it secure and unique

- Connect Data Box Gateway to a dedicated Azure storage account, where you can configure encryption at rest and control RBAC with identities from Microsoft Entra ID
- Use Azure Key Vault to manage the storage account access keys, as explained here: <https://learn.microsoft.com/en-us/azure/storage/common/storage-account-keys-manage>
- When creating an NFS share, limit the CIDR of source IP addresses who can access the NFS share from your private subnets, to avoid authorized network access
- When using an SMB share, use SMB 3.0 with encryption, to protect data in transit
- Regularly update Data Box Gateway to the latest version, as explained here: <https://learn.microsoft.com/en-us/azure/databox-gateway/data-box-gateway-apply-updates>

Here are some of the best practices for securing Azure File Sync:

- To keep network access to Azure File Sync private, configure private network endpoints, as explained here: <https://learn.microsoft.com/en-us/azure/storage/file-sync/file-sync-networking-endpoints>
- Enforce the use of secure transfer, to protect data confidentiality, as explained here: <https://learn.microsoft.com/en-us/azure/storage/common/storage-require-secure-transfer>
- Enforce encryption in transit using TLS 1.2, as explained here: <https://learn.microsoft.com/en-us/azure/storage/common/transport-layer-security-configure-minimum-version>
- Encrypt data at rest using Microsoft-managed keys (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here: <https://learn.microsoft.com/en-us/azure/storage/common/storage-service-encryption>
- For sensitive environments (for example, that contain PII, credit card details, healthcare data, and more), encrypt data at rest using **customer-managed keys (CMKs)** stored inside Azure Key Vault (as explained in *Chapter 11*), as documented here: <https://learn.microsoft.com/en-us/azure/storage/common/customer-managed-keys-overview>
- To avoid accidental deletion of files from Azure Files (SMB only), enable soft delete for Azure file shares, as explained here: <https://learn.microsoft.com/en-us/azure/storage/files/storage-files-enable-soft-delete>
- Use Microsoft Defender for Storage to detect potential threats on storage accounts (such as malicious files or data exfiltration), as explained here: <https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-storage-introduction>

Connecting to storage services over GCP hybrid environments

GCP offers the **Storage Transfer Service** to transfer data to the cloud in a hybrid architecture. It allows you to transfer large amounts of data from an on-premises environment to Google Cloud Storage.

Here are some of the best practices for securing Storage Transfer Service:

- Use **Cloud IAM** roles to restrict access to Storage Transfer Service resources, as documented here: <https://cloud.google.com/storage-transfer/docs/access-control>
- When using transfer agents, restrict the agent directory access according to business needs, as explained here: <https://cloud.google.com/storage-transfer/docs/on-prem-agent-details>
- For sensitive environments (for example, that contain PII, credit card information, healthcare data, and more), using Google Secret Manager to pass Amazon S3 or Azure credentials, encrypt credentials at rest using a CMEK in Google Cloud KMS (as explained in *Chapter 11, Applying Encryption in Cloud Services*), as documented here: <https://cloud.google.com/storage-transfer/docs/cmek>
- For sensitive/critical environments, enable data access audit logs (actions such as Get Operations, or Read Operations), as explained here: <https://cloud.google.com/storage-transfer/docs/audit-logging>

Note

Data access audit logs increase storage costs. Review the business requirements to enable them.

- For sensitive/critical environments, enable Cloud Logging for Storage Transfer Service (actions such as find, copy, or delete), as explained here: <https://cloud.google.com/storage-transfer/docs/transfer-logs>

Note

Cloud Logging for Storage Transfer Service increases storage costs. Review the business requirements to enable it (usually for troubleshooting purposes).

- To restrict access to Google Cloud Storage Transfer Service to trusted sources and reduce the risk of data exfiltration, use VPC Service Controls, as explained here: <https://cloud.google.com/storage-transfer/docs/on-prem-vpc-sc>

In this section, we reviewed the various options that AWS, Azure, and GCP offer for transferring data from on-premises environments to the cloud in a hybrid cloud scenario.

Computing services for hybrid cloud environments

The hybrid cloud architecture is not just limited to connecting on-premises environments to the cloud for resource consumption. It can also be used in scenarios where an organization would like to keep its data locally (due to regulatory restrictions or network latency) while still benefitting from cloud capabilities and, perhaps sometime in the future, being able to migrate data and resources to the cloud.

Using computing services over AWS hybrid environments

Amazon offers the following services in a local deployment topology:

- **AWS Outposts:** This is a fully managed service that contains the same type of compute, storage, database, and networking capabilities that are deployed on-premises in the shape of a physical rack. Use for applications requiring low latency, local data processing, or data residency compliance.
- **Amazon ECS Anywhere:** ECS Anywhere extends the capabilities of Amazon ECS to on-premises environments, allowing users to manage containers on any infrastructure while using the same control plane as in AWS. It's ideal for organizations that prefer a straightforward container management solution without the overhead of managing Kubernetes.
- **Amazon EKS Anywhere:** EKS Anywhere enables users to create and manage Kubernetes clusters on their on-premises infrastructure, providing a consistent experience with Amazon EKS in the cloud. It's suitable for teams with existing knowledge of Kubernetes who can navigate its complexities effectively.

Here are some of the best practices for securing AWS Outposts:

- Connect AWS Outposts to an AWS region close to your physical location
- To keep the connectivity between AWS Outposts and AWS secured, use service link private connectivity, as explained here: <https://docs.aws.amazon.com/outposts/latest/userguide/region-connectivity.html>
- Use a **VPC**, **security groups**, and a **local gateway (LGW)** to restrict access between your on-premises data center resources and the resources that have been deployed inside AWS Outposts, as explained here: <https://docs.aws.amazon.com/outposts/latest/userguide/routing.html>
- Use **encrypted protocols** (such as TLS) between your local data center and AWS Outposts
- Encrypt **data at rest** for resources that are deployed inside AWS Outposts (the same way you would do in the cloud)
- Use identity-based policies to grant minimal access to manage AWS Outposts, as explained here: https://docs.aws.amazon.com/outposts/latest/userguide/security_iam_id-based-policy-examples.html

- AWS CloudTrail is enabled by default for AWS Outposts APIs. CloudTrail logs provide a detailed record of API calls made by users and services in your AWS account. To keep AWS Outposts API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here: <https://docs.aws.amazon.com/outposts/latest/userguide/logging-using-cloudtrail.html>

Although most of the security best practices for securing Amazon ECS were already mentioned in *Chapter 3, Securing Compute Services – Containers and Kubernetes*, most recommendations for securing Amazon ECS Anywhere are the same. Let us recap the main best practices, as they apply to hybrid environments:

- Grant minimal IAM permissions for managing Amazon ECS Anywhere, to avoid unauthorized access to the environment accessible by the ECS Anywhere, as explained here: <https://docs.aws.amazon.com/AmazonECS/latest/bestpracticesguide/security-iam.html>
- Grant minimal IAM permissions for the Amazon ECS Anywhere service (for running tasks, accessing S3 buckets, monitoring using CloudWatch Events, and so on), to avoid unauthorized access to AWS resources, as follows: <https://docs.aws.amazon.com/AmazonECS/latest/bestpracticesguide/security-iam-roles.html>
- Secrets (such as API keys or database credentials) are a common way to authenticate ECS tasks against other resources or applications. Avoid storing secrets as part of code or configuration files. Instead use services such as AWS Secrets Manager or AWS Systems Manager Parameter Store, for storing, retrieving, and rotating secrets, to avoid unauthorized access to static credentials, as explained here: <https://docs.aws.amazon.com/AmazonECS/latest/bestpracticesguide/security-secrets-management.html>
- AWS CloudTrail is enabled by default for Amazon ECS. To keep ECS API activity for longer than 90 days, for further investigation and analysis, send CloudTrail logs to an Amazon S3 bucket. For more information about ECS and CloudTrail logs, see the following: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/logging-using-cloudtrail.html>.

Here are some of the best practices for securing Amazon EKS Anywhere:

- Use the **Cilium CNI** to restrict access between Pods by setting network policies (at layers 3 and 4 of the OSI model), and avoid unauthorized network access to your Pods: <https://anywhere.eks.amazonaws.com/docs/clustermgmt/networking/networking-and-security/>
- Encrypt the etcd at rest, to protect the configuration of the Kubernetes cluster, as explained here: <https://anywhere.eks.amazonaws.com/docs/getting-started/optional/etcdencryption/>

- Run the **kube-bench** tool regularly to check for compliance with CIS Benchmarks for Kubernetes, as follows: <https://anywhere.eks.amazonaws.com/docs/clustermgmt/security/kube-bench/>
- Use AWS IAM Authenticator, to authenticate identities from your AWS environment to the EKS Anywhere cluster, as explained here: <https://anywhere.eks.amazonaws.com/docs/clustermgmt/security/cluster-iam-auth/>

Using computing services over Azure hybrid environments

Azure offers the following services in a local deployment topology:

- **Azure Stack Hub:** This is a fully managed service containing the same type of compute, storage, database, and networking capabilities that are deployed on-premises in the shape of a physical rack. It provides a consistent cloud experience, enabling users to deploy applications and services using the same tools and APIs as in Azure.
- **Azure Arc:** This is a service that allows unified management of your resources – both on Azure and on-premises (configuring Azure policy, managing VMs, Kubernetes clusters, and databases from a single console). Use Azure Arc when you have existing infrastructure that you want to manage alongside your Azure resources.

Here are some of the best practices for securing Azure Stack Hub:

- Use Azure Stack Hub's built-in network ACLs to restrict access between your on-premises data center resources and resources that are deployed inside Azure Stack Hub.
- Encrypt all data in transit using the TLS protocol when you're passing traffic between Azure Stack Hub and your local data center, as explained here: <https://learn.microsoft.com/en-us/azure-stack/operator/azure-stack-security-configuration>
- Connect Azure Stack Hub to Microsoft Entra ID, to manage the identities who have access to maintain Azure Stack Hub, as explained here: <https://learn.microsoft.com/en-us/azure-stack/operator/azure-stack-identity-architecture>
- Azure Stack Hub uses secrets to communicate with external services in the Azure ecosystem (such as App Service, Event Hub, MySQL, SQL server, etc.). To avoid secret compromise and reuse by unauthorized parties, regularly rotate secrets, as explained here: <https://learn.microsoft.com/en-us/azure-stack/operator/azure-stack-rotate-secrets>.
- Use Microsoft Defender for Endpoint to protect VMs that are deployed inside Azure Stack Hub and make sure that you keep the antivirus up to date, as explained here: <https://learn.microsoft.com/en-us/azure-stack/operator/azure-stack-security-av>

Here are some of the best practices for securing Azure Arc:

- To keep the traffic between Azure and the on-premises servers private (i.e., not passing over the public internet), configure a private endpoint, as explained here: <https://learn.microsoft.com/en-us/azure/azure-arc/servers/private-link-security>
- Configure the Azure Arc agents to authenticate to Azure using Microsoft Entra ID managed identities, and use RBAC to configure least privileged roles, as explained here:
 - <https://learn.microsoft.com/en-us/azure/azure-arc/servers/security-identity-authorization>
 - <https://learn.microsoft.com/en-us/azure/azure-arc/servers/managed-identity-authentication>
- To allow secure remote control to Azure Arc-enabled servers, configure remote access using SSH protocol, as explained here: <https://learn.microsoft.com/en-us/azure/azure-arc/servers/ssh-arc-overview>
- Centrally configure policies using Azure Policy, both on VMs and Kubernetes clusters, to configure settings for Azure Arc-enabled resources, as explained here: <https://learn.microsoft.com/en-us/azure/azure-arc/servers/policy-reference>
- Protect your servers and containers using Microsoft Defender for Cloud, both on Azure and on-premises, as explained here: <https://learn.microsoft.com/en-us/azure/defender-for-cloud/quickstart-onboard-machines>

Using computing services over GCP hybrid environments

GCP offers **Google Distributed Cloud (GDC)** in a local deployment topology. It is a platform that provides fully managed software and hardware solutions, allowing organizations to focus on application development without the overhead of infrastructure management. GDC is based on GKE, with the ability to deploy on-premises.

Here are some of the best practices for securing GDC (GKE on-premises):

- To keep the traffic between GKE on-premises and Google Cloud private, configure Private Google Access, and pass the traffic inside Google Cloud Interconnect or Google Cloud VPN, as explained here: <https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/concepts/connect-on-prem-gcp>
- Connect the GKE on-premises cluster to an external identity provider supporting OIDC protocol, for central identity and access management: <https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/installing/identity-manage>

- Use **Workload Identity Federation** for GKE on-premises, and grant minimal permissions to allow GKE Pods access to GCP resources, to avoid unauthorized access to GCP resources, as explained here: https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/installing/keyless#workload_identity
- In case you need to allow traffic outside the GKE on-premises cluster (such as traffic to Google Cloud or on-premises resources), configure an egress NAT gateway, as explained here: <https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/how-to/egress-nat>
- Store container images for applications to be deployed on top of GKE on-premises in a private container registry (with minimal permissions to push new images or pull images from the registry). To configure authentication between the GKE on-premises and a private container registry, follow the instructions here: <https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/how-to/configure-node-private-reg>.
- Use **Binary Authorization for Google Distributed Cloud** to deploy container images from a signed and trusted source, as explained here: <https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/how-to/binary-authorization-policy>
- Regularly rotate the GKE on-premises cluster secrets, to avoid credential reuse and access by unauthorized parties, as explained here: <https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/how-to/update-secrets>
- Enable logging for GKE, for further investigation and analysis of Kubernetes cluster activity, as explained here: <https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/how-to/audit-logging>
- For sensitive/critical environments, enable data access audit logs (actions such as read cluster configuration), for further investigation and analysis, as explained here: <https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/how-to/audit-logging-api>

In this section, we reviewed some of the alternatives to using cloud resources locally based on AWS, Azure, and GCP services. Each cloud provider takes a different approach to hybrid cloud. AWS and Azure offer a physical rack to be deployed on-premises and can run VMs, storage, and databases. Both AWS and GCP offer solutions to allow you to deploy Kubernetes clusters on-premises with the same APIs and capabilities as the cloud version.

Securing hybrid cloud environments

When it comes to talking about securing hybrid cloud environments, we are looking for solutions that can manage your entire environment (both on-premises and the cloud) in a centralized way.

Securing AWS hybrid environments

AWS offers the following services for managing security in hybrid environments:

- **AWS Systems Manager:** This allows you to manage VMs from compliance, patch management, and hardening perspectives (a central location for running scripts over hybrid environments)
- **AWS Secrets Manager:** This offers a central and secured location for managing secrets (credentials, API keys, and more) over hybrid environments
- **AWS Elastic Disaster Recovery:** This provides secure data replication for disaster recovery between on-premises environments and the cloud
- **Amazon CloudWatch Agent:** This allows you to collect operating system logs from VMs in hybrid environments

The following are some best practices to consider:

- Deploy AWS Systems Manager Agent on both EC2 instances in the cloud and on your on-premises servers (Windows or Linux)
- Use AWS Systems Manager to set a patch baseline and get notified about VMs' missing security patches (afterward, you can push security patches using Systems Manager)
- Use AWS IAM to grant minimal permissions to allow your local VMs to communicate with AWS services
- Encrypt all traffic between your local Systems Manager agents and the AWS Systems Manager service using TLS
- Use AWS Systems Manager Change Manager to check for configuration changes on all your VMs (both in the cloud and on-premises)
- Use AWS Secrets Manager to generate, revoke, store, and retrieve sensitive information (such as database credentials, API keys, and more) instead of storing secrets hardcoded on your VMs
- Use AWS CloudTrail to audit API activities of AWS Secrets Manager (such as who generates secrets, who accesses secrets, and more)
- Use AWS Elastic Disaster Recovery to build a DR solution for your on-premises VMs by replicating entire VMs to the cloud

- Deploy Amazon CloudWatch Agent on both EC2 instances in the cloud and on your on-premises servers (Windows or Linux)
- Use Amazon CloudWatch Logs as a central log repository where you can create alerts on activities such as failed logins, security-related events from the OS security event logs, and more.

For more information, please refer to the following resources:

- *Setting up AWS Systems Manager for hybrid environments*: <https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-managedinstances.html>
- *AWS Secrets Manager Features*: <https://aws.amazon.com/secrets-manager/features/>
- *AWS Elastic Disaster Recovery*: <https://aws.amazon.com/disaster-recovery/>
- *Installing the CloudWatch agent on on-premises servers*: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/install-CloudWatch-Agent-on-premise.html>
- *What are Amazon CloudWatch Logs?*: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/WhatIsCloudWatchLogs.html>

Securing Azure hybrid environments

Azure offers the following services for managing security in hybrid environments:

- **Microsoft Defender for Cloud**: A comprehensive security solution designed to protect cloud workloads across various environments, including multi-cloud and hybrid setups.
- **Microsoft Sentinel**: A cloud-native **security information and event management (SIEM)** solution that provides advanced security analytics and threat detection capabilities.

The following are some best practices to consider:

- Deploy **Azure Arc** and enable **Microsoft Defender** on all your virtual machines and containers that are on-premises and in the cloud
- Deploy an **Azure Monitor agent** on your on-premises servers to allow security information to be received inside the Microsoft Defender for Cloud console
- Use the built-in vulnerability assessment that comes with Microsoft Defender for Cloud to detect vulnerabilities in your on-premises environment and the cloud
- Use Microsoft Defender for SQL to scan for vulnerabilities on all your SQL databases – both on-premises and in the cloud

- Use Microsoft Defender for Containers to defend all your Kubernetes clusters – both on-premises and in the cloud
- Connect all Microsoft Defender for Cloud services to Microsoft Sentinel to be able to correlate between security incidents and response to security alerts – both on-premises and in the cloud

For more information, please refer to the following resources:

- *What is Microsoft Defender for Cloud?*: <https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-cloud-introduction>
- *What is Microsoft Sentinel?*: <https://learn.microsoft.com/en-us/azure/sentinel/overview>
- Azure Monitor Agent overview: <https://learn.microsoft.com/en-us/azure/azure-monitor/agents/azure-monitor-agent-overview>

Securing GCP hybrid environments

GCP offers the following services for managing security in hybrid environments:

- **Google Cloud Endpoints**: A service for securing, monitoring, analyzing, and setting quotas for APIs – both in the cloud and on-premises.
- **Private Google Access**: Allows private access from on-premises environments to Google APIs and services.

The following are some best practices to consider:

- When you're using Google Cloud Endpoints, remember the following:
 - Encrypt traffic using SSL with **Extensible Service Proxy (ESP)** to keep the confidentiality of sensitive information (such as credentials, PII, and more)
 - Use ESP to validate authentication requests to the API backend (such as Firebase authentication, Auth0, Google ID token authentication, and more)
 - If you're using API keys, restrict access to those API keys using the Service Control API
 - Use Google Cloud IAM to restrict access to APIs
- When you need to access Google APIs or services from the on-premises environments, remember the following:
 - Connect to your VPC using Cloud VPN or Cloud Interconnect
 - Redirect traffic through `private.googleapis.com` for APIs that do not support VPC service controls

- Redirect traffic through `restricted.googleapis.com` for APIs that support **VPC service controls**
- Use firewall rules to allow access from the on-premises environment to your VPC

For more information, please refer to the following resources:

- *Cloud Endpoints documentation*: <https://cloud.google.com/endpoints/docs>
- *Configuring Private Google Access for on-premises hosts*: <https://cloud.google.com/vpc/docs/configure-private-google-access-hybrid>

In this section, we reviewed, at a high level, several security-related services from AWS, Azure, and GCP that enable you to have central management over your hybrid environments – both on-premises and cloud resources.

Summary

In this chapter, we focused on hybrid clouds. We reviewed the importance of having a hybrid cloud strategy to allow organizations to adopt cloud services and hybrid solutions. We also discussed the various identity and access management solutions from AWS, Azure, and GCP, which allow organizations to have a central directory service so that they can keep a single identity for each end user.

We looked at the various methods that AWS, Azure, and GCP use to help organizations connect their on-premises environment to the cloud in a hybrid architecture. We also discussed the various storage services that allow organizations to transfer data to the cloud. Then, we dived into various computing services that allow organizations to control all their compute needs using the same technology and capabilities – both on-premises and in the cloud.

Finally, we reviewed several services from AWS, Azure, and GCP that allow organizations to achieve a single pane of glass for managing security both on-premises and in the cloud.

Understanding the topics mentioned in this chapter will provide organizations with the necessary tools for when they build a hybrid cloud architecture.

In the next chapter, we will review multi-clouds (including identity management, vulnerability and patch management, configuration management, monitoring, and network-security-related topics).

Managing Multi-Cloud Environments

In the previous chapter, we covered how to connect your on-premises data centers to AWS, Azure, and GCP, and how to create a hybrid cloud architecture. In this chapter, we will cover one of the most complex scenarios in the cloud – managing **multi-cloud environments**. Multi-cloud is about using more than one cloud solution or cloud provider to set a common environment, infrastructure, or even an application layer.

Multi-cloud provides organizations with benefits such as the ability to select cheaper service alternatives (different **cloud service providers (CSPs)** offer different service costs), or unique capabilities (such as the ability to select a **generative artificial intelligence (GenAI) large language model (LLM)** that exists on a specific CSP), but it also has challenges (such as knowledge skill gaps, egress data cost, and so on) that organizations need to be aware of. We will delve deeper into these benefits and challenges. In this chapter, we will cover the following topics:

- Multi-cloud strategy
- **Identity and access management (IAM)** over multi-cloud environments
- Network architecture for multi-cloud environments
- Data security in multi-cloud environments
- Cost management in multi-cloud environments
- **Cloud security posture management (CSPM)**
- **Cloud infrastructure entitlement management (CIEM)**
- Patch and configuration management in multi-cloud environments
- The monitoring and auditing of multi-cloud environments

In the diagram shown next, we can see an example of a multi-cloud architecture, with a public application deployed into both AWS and Azure:

- A customer is connecting from the public internet and reaches a DNS service, resolving the application name.
- The DNS service routes the traffic to either AWS or Azure in a round-robin fashion (that is, half of the traffic is routed to AWS and half is routed to Azure).
- The application is deployed on top of a Kubernetes cluster (in AWS, it uses **Elastic Kubernetes Service (EKS)**, and in Azure, it uses **Azure Kubernetes Service (AKS)**).
- The application stores its persistent data in a backend MySQL database (in AWS, it uses **Amazon Relational Database Service (RDS)**, and in Azure, it uses Azure Database for MySQL).

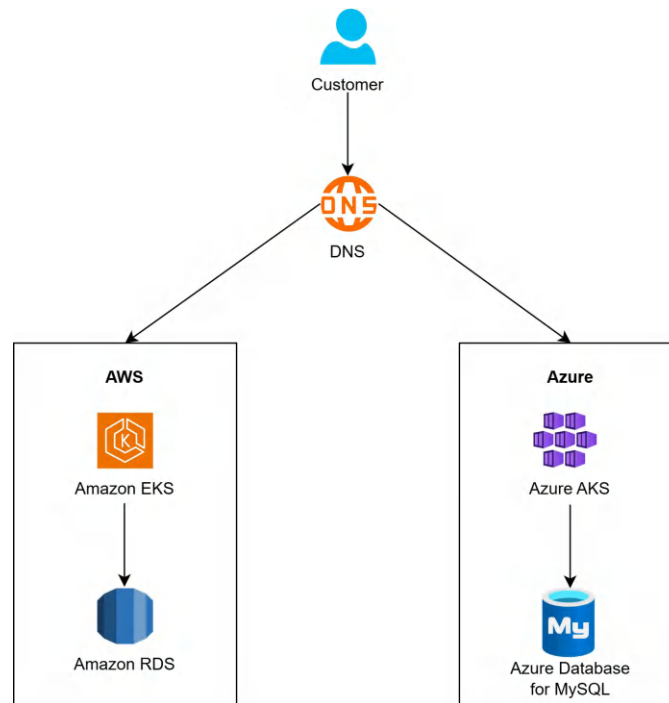


Figure 15.1 – Sample multi-cloud architecture application

Multi-cloud strategy

Organizations taking their first steps using cloud services consider a multi-cloud approach as a solution for protecting themselves against a vendor lock-in scenario – a scenario where an entire infrastructure or applications rely on a single cloud provider's ecosystem. One of the questions organizations ask themselves is this: What do we do if the cloud provider goes out of business or makes changes to its

service price, meaning that it is no longer beneficial for us to continue consuming services from this CSP? How do we migrate our assets to another cloud provider? In theory, a multi-cloud approach might resolve vendor lock-in risk; however, organizations need to be aware of the complexities that come with multi-cloud environments.

Business continuity (BC) is a significant concern for organizations. A multi-cloud architecture enables the deployment of workloads across multiple cloud providers, which helps mitigate potential disruptions. This approach reduces the impact of issues such as outages, ensuring that operations remain uninterrupted even if one provider experiences a problem, such as an authentication service failure. A multi-cloud architecture allows us to create **disaster recovery (DR)** solutions, whereby one cloud provider is configured as a failover for another cloud provider. These architectures, mostly based on containers, allow us portability between cloud providers while decreasing the dependencies on a specific cloud ecosystem (from **virtual machines (VMs)**, storage services, managed databases, and so on). Multi-cloud allows us to have the benefits of each cloud provider's ecosystem, including the latest versions of Kubernetes, the most scalable NoSQL database, and the latest GPU processors.

Before using a multi-cloud architecture, we need to ask ourselves: what are we trying to achieve through a multi-cloud strategy? Some of the most common use cases for choosing a multi-cloud strategy are discussed in the following sections.

Freedom to select a cloud provider

Most cloud providers offer us the same fundamental services (such as compute, storage, and database). Having the freedom to select a cloud provider allows us to decide for each workload where we wish to deploy our resources (VMs, containers, database, and so on), in case one of the cloud providers changes its **service-level agreement (SLA)** or pricing model.

Freedom to select your services

In case one of the cloud providers offers a certain service that is not available on other cloud providers, such as data analytics for large datasets, specific LLMs, or a **function as a service (FaaS)** that supports certain development languages not supported by other cloud providers, we can opt in for a multi-cloud strategy.

Data sovereignty

Even though most major cloud providers have data centers in central locations in the US and Europe, there are cases where an organization is required by local laws or regulations to keep customer data locally in a specific country. There may be scenarios where a specific cloud provider has no local presence in one of your customer's regions, making a multi-cloud approach a suitable solution. In such cases, workloads can be deployed on another cloud provider within the same region as the customer's data to meet regulatory requirements.

BC and DR

Storing data in multiple locations, with multiple cloud providers, will decrease the risk of an organization losing data or overcoming cloud provider service availability in a certain region or even global service availability issues.

Next are two common alternatives for achieving BC through multi-cloud:

- **Active-active architecture:** In this architecture, the application and its underlying infrastructure are deployed on top of multiple CSPs. Traffic is been routed through a global DNS service to each of the CSPs. The architecture at each CSP has similar components (such as VMs, containers, community database engines, and so on.). The data is synchronized between CSPs on a regular basis.
- **Active-passive architecture:** In this architecture, one of the CSPs is configured as the primary one to receive traffic from customers, while the other CSP is considered for DR purposes.

Improving reliability

Even though most major cloud providers offer **distributed denial-of-service (DDoS)** protection services, CSPs might suffer from service outages, internet providers may encounter latency, or workloads may fail for various reasons.

To improve reliability, consider topics such as the following alternatives using multi-cloud architectures:

- **Data replication:** How do you replicate data between cloud providers, and how does it impact the cost of implementing such a solution (redundant storage, data transfer cost, and so on)?
- **Failover scenarios:** What DNS strategy should be implemented to ensure effective failover in a multi-cloud environment, considering factors such as latency, traffic routing, and service continuity?
- **Automation:** What tools or services will you use for failover between providers?
- **Knowledge:** Does your IT or development staff have enough knowledge to deploy and maintain infrastructure on top of multiple CSPs?

Having the ability to switch workloads to another cloud provider (assuming you have already built an entire production environment on another cloud provider and synced data) will allow you to mitigate the risk of having your services unavailable (though it increases the cost and complexity of the overall solution).

Identity management

A multi-cloud strategy must address **identity management (IdM)**. Each cloud provider has its own IAM solution, which causes challenges and opportunities for a centralized identity life cycle (onboarding, access management, and offboarding) and credential management. These challenges increase the motivation to centralize a single IAM solution that must be integrated with each cloud provider.

When designing a multi-cloud architecture, consider the following:

- **Central identity provider (IdP):** Select a single IdP (such as Microsoft Entra ID, which is commonly used by many organizations), and manage all identities in this IdP.
- **Identity federation:** Create identity federation, where the central IdP is trusted by all other CSPs used by the organization. Identity federation allows you **single sign-on (SSO)** against the central IdP, and once a federation is established, you can grant permissions to resources across all CSPs using the same identity.

Data security

Data security in a multi-cloud strategy involves implementing comprehensive measures to protect sensitive information across multiple cloud environments. This approach requires organizations to establish consistent security policies, employ strong encryption techniques, and utilize centralized management tools to ensure uniform protection across diverse cloud platforms. Common examples of data security in a multi-cloud environment are as follows:

- **Data residency:** How to keep data residency while working with multiple cloud providers (for example, keep EU citizens' data in an EU data center)?
- **Data replication:** How to keep data secured (encrypted and with proper authorizations) when data has been replicated between cloud providers and within a single cloud provider over multiple regions?
- **Authorization:** How to manage access control of data over multiple cloud providers?
- **Data protection:** How to enforce encryption, key management, and secrets management over multiple cloud providers?
- **Auditing:** How to keep audit access to data over decentralized audit solutions?
- **Misconfiguration:** How to handle misconfiguration when working with multiple cloud providers, each with its services and capabilities?

Asset management

A multi-cloud strategy makes asset management a challenge for any organization. An organization must have visibility or a real-time, centralized inventory of its entire cloud environment – what assets it has, where its assets are deployed (which cloud provider and which account), who owns its assets, and who has access to its assets. When an organization decides to work with multiple cloud providers, it is recommended to set standards for resource tagging (such as Dev/Test/Prod, **business unit (BU)** name, application name, and so on) across all cloud providers to be able to easily filter resources/assets.

Skills gap

A multi-cloud strategy must address the fact that working with multiple cloud providers requires hands-on experience in architecture, development, and operations, working with multiple cloud solutions. An organization must invest in employee training as an ongoing process to allow employees the necessary knowledge to maintain multiple cloud platforms. An important topic to invest in is automation, to scale the impact of employees and to reduce the operational burden.

For more information, refer to the following resources:

- *Proven Practices for Developing a Multicloud Strategy*
<https://aws.amazon.com/blogs/enterprise-strategy/proven-practices-for-developing-a-multicloud-strategy/>
- *Prepare your environment for a hybrid and multicloud scenario*
<https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/scenarios/hybrid/ready>
- *Plan a hybrid and multicloud strategy*
<https://cloud.google.com/architecture/hybrid-multicloud-patterns/strategy>

A multi-cloud strategy helps organizations decide in which scenario to select a different cloud provider or a different cloud service. Such a strategy must address data security topics (including data residency, availability, and protection mechanisms). Another important topic that organizations should consider is the fact that a multi-cloud approach demands highly experienced and trained employees, and as such, organizations must invest in employee training.

Next, we will review how each of the CSPs allows customers to manage IAM in a multi-cloud environment.

IAM over multi-cloud environments

One of the first things to decide on before using a multi-cloud architecture is IAM. Organizations would like to keep their existing IdP, have a single identity for each of their end users (while preserving existing credentials), and still be able to access resources in the cloud. If an organization is already using Microsoft 365 for managing mailboxes and collaboration, consider using Microsoft Entra ID and its central IdM service.

Microsoft Entra ID (formerly Azure AD) is the most used IdP. It supports identity federation to most major cloud providers and can integrate with most **software-as-a-service (SaaS)** solutions. Other popular IdM providers that are outside the scope of this book are Okta, Ping Identity, and OneLogin, which allow you a universal directory service for managing your users, groups, and devices, enforcement of **multi-factor authentication (MFA)**, and SSO for multiple cloud providers and SaaS applications.

IAM in a multi-cloud environment involves two phases:

- **Central IdP:** The action of identifying all personas (both human and service/application accounts) against a central IdP, which enforces credential management (such as password policy, MFA, checking if accounts are enabled/disabled, and auditing of login requests).
- **Authorization:** The action of validating each persona's permissions to access resources. In a multi-cloud environment, authorization is enforced on the cloud provider, which manages permissions for its resources. For example: A user performs login against Microsoft Entra ID, and permissions on AWS resources are controlled using AWS IAM identity-based policies.

In the following diagram, we can see a common IAM workflow for accessing data inside an Amazon S3 bucket:

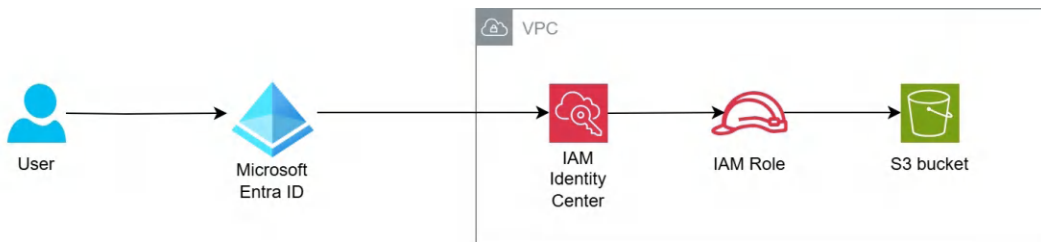


Figure 15.2 – An IAM workflow for reading objects from S3

Let's try to understand the workflow:

- A user in an Azure environment is authenticating against Microsoft Entra ID
- Microsoft Entra ID is synced with AWS IAM Identity Center
- The user's request is sent from the Azure environment to AWS and is being evaluated against an AWS IAM role
- The request has been evaluated on the AWS side, and assuming the user has an IAM role to read the content of objects inside the S3 bucket, the request succeeds and the user can read data from S3

Use the following best practices for securing IAM in a multi-cloud environment:

- Select a central IdP to manage the entire life cycle of identities in the organization
- On the central IdP, enforce storage authentication measures (such as complex passwords, password rotation, and MFA)
- When granting permissions to resources, follow the **principle of least privilege (PoLP)** to avoid unauthorized access or excessive permissions to a resource

- Regularly review all identities, and disable/delete unneeded identities to avoid unauthorized access
- Regularly review permissions to resources in the entire multi-cloud environment and remove unneeded permissions to avoid unauthorized access
- Enable audit trail on both authentication and authorization access to resources and send all audit logs from the entire multi-cloud environment to a central **security information and event management (SIEM)** system to aid in an **incident response (IR)** process

Next, let's see how we can manage identity, specifically in AWS, Azure, and GCP.

Managing identity in AWS over multi-cloud environments

AWS provides the following services to manage identity over multi-cloud environments:

- **AWS IAM Identity Center:** A service that allows SSO across AWS organizations or connection to identities in external cloud identity services based on the **Security Assertion Markup Language (SAML)** protocol, such as Microsoft Entra ID or Google Workspace. Use AWS IAM Identity Center when you need to manage workforce access to AWS resources and business applications centrally.
- **Amazon Cognito:** A service that allows SSO for web or mobile applications while allowing customers to connect using external IdPs, such as Apple, Facebook, or Google, or SAML-based authentication, such as Microsoft Entra ID. Use Amazon Cognito when you are building customer-facing applications that require user authentication, profile management, and social media integration.

Although most of the security best practices for AWS IAM Identity Center were already mentioned in *Chapter 14, Managing Hybrid Clouds*, let us recap the main best practices, as they also apply to multi-cloud environments:

- Use identity-based policies to grant minimal access to manage AWS IAM Identity Center, as explained here:

```
https://docs.aws.amazon.com/singlesignon/latest/userguide/iam-auth-access-using-id-policies.html
```

- Enforce the use of MFA for all interactive accounts to mitigate against unauthorized access to your AWS environment, as explained here:

```
https://docs.aws.amazon.com/singlesignon/latest/userguide/mfa-getting-started.html
```

- Create permissions to manage AWS IAM Identity Center, following PoLP, to avoid unauthorized access, as explained here:

```
https://docs.aws.amazon.com/singlesignon/latest/userguide/get-started-create-a-permission-set.html
```

- Configure session duration for accounts with access to AWS accounts, to automatically disconnect sessions after a preconfigured amount of time (that is, for the duration needed by users to perform their work on AWS accounts), as explained here:

<https://docs.aws.amazon.com/singlesignon/latest/userguide/howtosessionduration.html>

Next are some of the most common best practices for securing Amazon Cognito:

- Use identity-based policies to grant minimal access to Amazon Cognito resources and avoid unauthorized access, as explained here:

https://docs.aws.amazon.com/cognito/latest/developerguide/security_iam_id-based-policy-examples.html

- When working with Amazon Cognito, if you need to configure fine-grained access to resources based on external identity attributes (such as a tag that correlates between a user and a free or premium tier, for accessing services), use **attribute-based access control (ABAC)** in conjunction with the IAM policy (such as conditions in the IAM policy), as explained here:

<https://docs.aws.amazon.com/cognito/latest/developerguide/attributes-for-access-control.html>

- When working with Amazon Cognito to allow access to web or mobile applications, enforce the use of MFA to the user pool to allow strong authentication, as explained here:

<https://docs.aws.amazon.com/cognito/latest/developerguide/user-pool-settings-mfa.html>

- Enable password reuse prevention to mitigate against a scenario where a user is reusing historical passwords when required to replace their password, which weakens the authentication process and may allow unauthorized access, as explained here:

<https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pool-settings-advanced-security-password-reuse.html>

- Use Amazon Cognito compromised credentials protection features to detect whether a user's credentials were compromised and block access attempts of that user, as explained here:

<https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pool-settings-compromised-credentials.html>

- AWS CloudTrail is enabled by default for Amazon Cognito APIs. CloudTrail logs provide a detailed record of API calls made by users in your AWS account (such as login attempts, reset passwords, and so on). To keep Amazon Cognito API activity for longer than 90 days, send CloudTrail logs to an Amazon S3 bucket, as explained here:

<https://docs.aws.amazon.com/cognito/latest/developerguide/logging-using-cloudtrail.html>

- The Amazon Cognito API is directly exposed to the public internet. To protect the endpoint API from unauthorized access, use an AWS **Web Application Firewall (WAF)** web **access control list (ACL)** to protect from unwanted requests, as explained here:

`https://docs.aws.amazon.com/cognito/latest/developerguide/user-pool-waf.html`

Managing identity in Azure over multi-cloud environments

Azure provides the following services to manage identity over multi-cloud environments:

- **Microsoft Entra ID (formerly Azure AD):** An Azure IdP that allows access to Azure resources or SaaS applications using external SAML-based IdPs such as AWS IAM Identity Center or Google Identity. Use Microsoft Entra ID when you need to manage identities for employees, partners, or other organizational members.
- **Microsoft Entra External ID:** Microsoft Entra External ID integrates robust solutions for engaging with individuals outside your organization. These identities may include accounts issued by corporations or governments, as well as social IdPs such as Google or Facebook. Use External ID to implement authentication and **customer IAM (CIAM)** into your application.

Although most of the security best practices for Microsoft Entra ID were already mentioned in *Chapter 9, Effective Strategies for Implementing IAM Solutions*, let us recap the main best practices as they also apply to multi-cloud environments:

- Use Azure **role-based access control (RBAC)** to assign permissions to Microsoft Entra identities. Follow PoLP when granting access to resources to avoid unauthorized access to resources, as explained here:

`https://learn.microsoft.com/en-us/azure/role-based-access-control/best-practices`

- Use Microsoft Entra Password Protection to enforce the use of strong passwords on the entire Microsoft Entra tenant and decrease the chance of unauthorized access to your Azure environment, as explained here:

`https://learn.microsoft.com/en-us/entra/identity/authentication/concept-password-ban-bad`

- Enforce MFA for all human (interactive) authentication to decrease the chance of unauthorized access, as explained here:

- `https://learn.microsoft.com/en-us/entra/identity/authentication/concept-authentication-default-enablement`

- `https://learn.microsoft.com/en-us/entra/identity/authentication/tutorial-enable-azure-mfa`

- Use Microsoft Entra Conditional Access policies to configure rules for which identities or devices can access Azure resources, and in what terms, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/conditional-access/plan-conditional-access>

- Use Microsoft Entra **Privileged Identity Management (PIM)** to restrict any privileged access attempt for **just-in-time (JIT)** access, as explained here

<https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/security-planning>

- Use Microsoft Entra ID Protection to detect any potential vulnerability activities of your Microsoft Entra users and send the audit logs to your SIEM system, as explained here:

<https://learn.microsoft.com/en-us/entra/id-protection/how-to-deploy-identity-protection>

- Use Microsoft Defender for Identity to detect suspicious activities in your on-premises Active Directory and Microsoft Entra ID, as explained here:

<https://learn.microsoft.com/en-us/defender-for-identity/deploy/deploy-defender-identity>

- Enable diagnostic settings for Microsoft Entra ID for further investigation and analysis, and send all logs to a Log Analytics workspace, as explained here:

<https://learn.microsoft.com/en-us/entra/identity/monitoring-health/howto-configure-diagnostic-settings>

The following are some of the most common best practices for securing Microsoft Entra External ID:

- Create an emergency account (these are special user accounts that should not be used daily, have MFA enabled, and are a member of the administrator role) to avoid getting locked out of your Entra External ID service, as explained here:

<https://learn.microsoft.com/en-us/entra/external-id/customers/how-to-manage-admin-accounts>

- Use Azure RBAC to assign permissions to Microsoft Entra External ID identities. Follow PoLP when granting access to resources to avoid unauthorized access to resources, as explained here:

<https://learn.microsoft.com/en-us/entra/external-id/customers/how-to-use-app-roles-customers>

- Enforce MFA for all human (interactive) authentication to decrease the chance of unauthorized access, using conditional access policies, as explained here:

<https://learn.microsoft.com/en-us/entra/external-id/customers/how-to-multifactor-authentication-customers>

- Enforce MFA for any human (interactive) authentication originating from business partners (external B2B guest users) to decrease the chance of unauthorized access, as explained here:

<https://learn.microsoft.com/en-us/entra/external-id/b2b-tutorial-require-mfa>

- When designing web or mobile applications, consider using browser-delegated authentication to have reach authentication and security capabilities (such as **one-time password (OTP)**, social IdP sign-in, SSO, and more), as explained here:

<https://learn.microsoft.com/en-us/entra/external-id/customers/concept-native-authentication>

- Use Microsoft Entra ID Protection to detect any potential vulnerability activities of your Microsoft Entra External ID users and send the audit logs to your SIEM system, as explained here:

<https://learn.microsoft.com/en-us/entra/external-id/customers/how-to-identity-protection-customers>

- To allow organizational users access to external applications on partner networks using Entra External ID, use tenant restrictions v2 to limit the access according to the business use case, as explained here:

<https://learn.microsoft.com/en-us/entra/external-id/tenant-restrictions-v2>

- Implement a WAF solution to protect the Entra External ID publicly exposed endpoint from a web application-related attack, as explained here:

<https://techcommunity.microsoft.com/blog/identity/built-in-security-controls-for-external-facing-apps/4175879>

- Enable diagnostic settings for Entra External ID for further investigation and analysis, and send all logs to a Log Analytics workspace, as explained here:

<https://learn.microsoft.com/en-us/entra/external-id/customers/how-to-azure-monitor>

- When using Entra External ID to authenticate web applications, create separate environments – split between development, testing, and production. This will decrease the blast radius in unauthorized access to any of the environments.

Managing identity in GCP over multi-cloud environments

Google provides the following services to manage identity over multi-cloud environments:

- **Google Cloud Identity:** A platform that provides comprehensive IAM capabilities for organizations, supporting user management, SSO, MFA, and integration with external IdPs such as Microsoft Entra ID. Use Cloud Identity when you need a unified identity solution for both GCP services and non-Google Workspace users.

- **Google Identity Platform:** A CIAM solution aimed at developers who want to implement authentication features in their applications. It provides user-friendly authentication services that can be easily integrated into web and mobile apps. Use Identity Platform for consumer-facing applications needing secure sign-in.

Although most of the security best practices for Google Cloud Identity were already mentioned in *Chapter 14, Managing Hybrid Clouds*, let us recap the main best practices as they also apply to multi-cloud environments:

- Protect admin accounts using 2-step verification, and regularly rotate all admin account passwords to avoid unauthorized access, as explained here:

<https://support.google.com/cloudidentity/answer/9011373>

- Enforce MFA for all human (interactive) authentication to avoid unauthorized access to your GCP environment, as explained here:

- <https://support.google.com/cloudidentity/answer/175197?hl=en>

- <https://support.google.com/cloudidentity/answer/9176657>

- Forward all logs to Google Cloud Logging, and from Cloud Logging to a central SIEM system to aid in an IR process, as mentioned next:

<https://support.google.com/a/answer/4579579>

Next are some of the most common best practices for securing Google Identity Platform:

- Enforce the use of MFA for all external identities to decrease the chance of unauthorized access, as explained here:

<https://cloud.google.com/identity-platform/docs/admin/enabling-totp-mfa>

- Enable email enumeration protection to decrease the chance of credentials theft using a brute-force attack (that is, an attacker trying to guess which users exist, and once confirmed, will try to guess their passwords and gain unauthorized access), as explained here:

<https://cloud.google.com/identity-platform/docs/admin/email-enumeration-protection>

- To protect web-based applications from automated attacks (such as brute-force attacks), implement reCAPTCHA, as explained here:

<https://cloud.google.com/identity-platform/docs/recaptcha-enterprise>

Selecting an authentication and authorization method allows an organization to reuse its existing IdPs and to keep a single identity for each of its end users in a central repository. When possible, it is recommended to manage user identities using a central IdM system for provisioning and de-provisioning on your chosen cloud IdP.

The second important thing to consider when building a multi-cloud architecture is how to set up network connectivity between the various cloud providers and between on-premises infrastructure and cloud providers. Let's dive into that next.

Network architecture for multi-cloud environments

The recommended way to connect to cloud environments is a secure and permanent network connection using a dedicated interconnect (such as AWS **Direct Connect**, Azure ExpressRoute, or Google Interconnect) for connecting from the on-premises to the cloud or using a site-to-site VPN (either from on-premises to the cloud or between cloud providers).

A secured and permanent connection will allow access control (layer 4 firewall rules) to be set between cloud providers and cloud segments and keep access to resources in the cloud (or allow access to cloud resources) for business needs.

When considering network architecture, you need to think about the following:

- Which resources should be kept private (such as data transfer and backend service-to-service communication)?
- Which resources can be public (such as public API calls, frontend services, and VPN endpoints)?
- How will the network architecture adapt to changes (such as adding additional VPCs/VNets)?
- How do we achieve resiliency in multi-cloud network architecture?
- How do we achieve visibility and monitoring capabilities in a multi-cloud network architecture?

Next are some best practices for configuring multi-cloud network architecture:

- Always think about **Classless Inter-Domain Routing (CIDR)** before configuring network segments in the cloud to avoid IP conflict between the cloud providers.
- Use **network ACLs (NACLs)** and route tables to configure network access rules between the CSPs and the resources inside your VPC/VNet.
- Design resilient architecture to avoid outages or availability problems on the connection between the cloud providers. One alternative can be to use two dedicated private connections, and another alternative can be to create a primary connection based on a dedicated private connection, with a site-to-site VPN tunnel as a backup connection.
- Monitor the connection between the CSPs to be able to detect connection failures that may impact customers (usually internal customers).

Although most concepts related to network connectivity were explained in *Chapter 14, Managing Hybrid Clouds*, let us recap the various alternatives:

- **AWS Site-to-Site VPN:** A fully managed VPN service that provides connectivity over an IPsec tunnel. Use AWS Site-to-Site VPN for simple, secure connections with moderate bandwidth needs.
- **AWS Direct Connect:** A dedicated connection with a pre-defined network bandwidth. Use AWS Direct Connect if you require high bandwidth, low latency, and consistent network performance.
- **Azure VPN Gateway:** A fully managed VPN service that provides connectivity from the on-premises environment to the cloud over IPsec tunnels. Use this alternative for smaller-scale deployments or when lower bandwidth requirements (up to 10 Gbps) are sufficient.
- **Azure ExpressRoute:** A dedicated connection from the on-premises environment to one or more Azure VNets in a pre-defined network bandwidth. Use this alternative for large enterprises with high bandwidth needs or when predictable performance and lower latency are critical.
- **Google Cloud VPN:** A fully managed VPN service that provides connectivity from the on-premises environment to the cloud over an IPsec tunnel. Use Cloud VPN for cost-effective, secure connections with lower bandwidth needs.
- **Google Cloud Interconnect:** A dedicated connection from the on-premises environment to one or more VPCs in a pre-defined network bandwidth. Use Cloud Interconnect when requiring higher performance, security, and bandwidth for mission-critical and highly sensitive workloads.

Creating network connectivity between AWS and GCP

To create an IPsec VPN tunnel between AWS and GCP, follow these high-level guidelines:

- Create a dedicated AWS VPC with a new subnet for the VPN tunnel
- Create a dedicated GCP VPC with a new subnet for the VPN tunnel
- Restrict access to AWS resources inside your AWS environment using Amazon VPC security groups, as explained here:

<https://docs.aws.amazon.com/vpn/latest/s2svpn/SetUpVPNConnections.html#vpn-configure-security-groups>

- Restrict access to GCP resources inside your GCP environment using VPC firewall rules, as explained here:

<https://cloud.google.com/network-connectivity/docs/vpn/how-to/configuring-firewall-rules>

- If network redundancy is not required, create a connection between an AWS virtual private gateway (using an AWS-managed VPN) and a Google Cloud VPN

- If network redundancy is required, create a connection between an AWS transit gateway and a Google **high availability (HA)** VPN

In the following diagram, we can see a site-to-site connection between GCP and AWS:

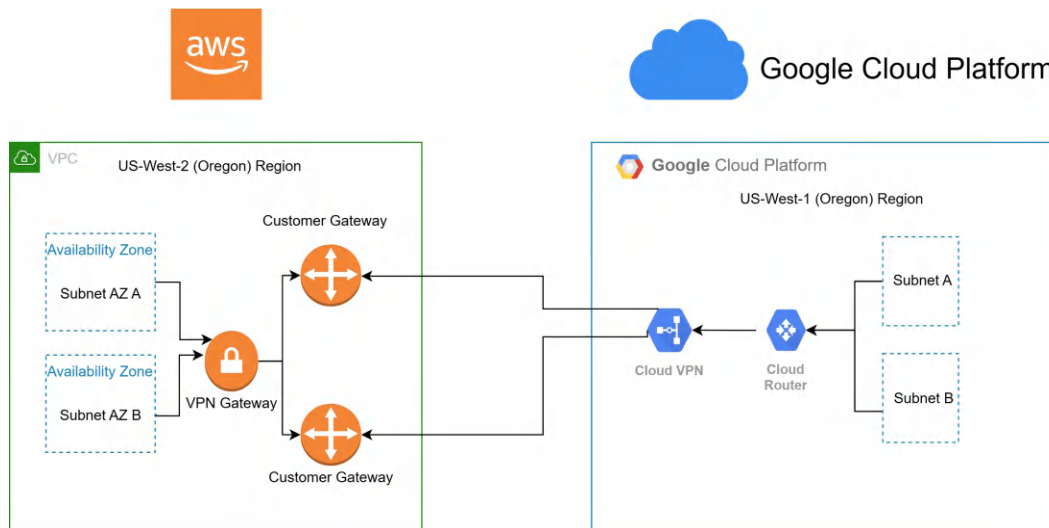


Figure 15.3 – Network connectivity from GCP to AWS

To create a secure connection between GCP and AWS using an HA VPN, you will need the following components:

- **HA VPN gateway:** Set up in GCP to manage the VPN connections
- **Cloud router:** Used for dynamic routing in GCP
- **Virtual private gateway:** Set up in AWS as the endpoint for the VPN connection

Follow the next steps to establish a connection:

1. **Create a VPC network in GCP:** This is where your resources will be hosted.
2. **Set up an HA VPN gateway:** Deploy the HA VPN gateway and configure the cloud router in GCP.
3. **Create a customer gateway in AWS:** This represents the GCP side of the VPN connection.
4. **Create a virtual private gateway in AWS:** This acts as the entry point for the VPN connection.
5. **Attach the virtual private gateway to your VPC:** Connect it to your existing VPC in AWS.
6. **Create VPN connections:** Establish multiple tunnels for redundancy, ensuring that if one tunnel fails, traffic can still flow through another.

For more information, refer to the following resource:

Create HA VPN connections between Google Cloud and AWS

<https://cloud.google.com/network-connectivity/docs/vpn/tutorials/create-ha-vpn-connections-google-cloud-aws>

To create a dedicated private connection between AWS and GCP, follow these high-level guidelines:

- Use multiple Direct Connect connections with the Direct Connect gateway so that you have connection redundancy between AWS and GCP, as explained here:

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/disaster-recovery-resiliency.html>

- Restrict access to AWS resources inside your AWS environment using Amazon VPC security groups, as explained here:

<https://repost.aws/knowledge-center/routing-dx-private-virtual-interface>

- Use multiple Google Interconnect connections to have connection redundancy between AWS and GCP, as explained here:

<https://cloud.google.com/network-connectivity/docs/interconnect/how-to/dedicated/creating-redundant-interconnect>

- Restrict access to GCP resources inside your GCP environment using VPC firewall rules, as explained here:

<https://cloud.google.com/network-connectivity/docs/interconnect/concepts/overview>

For more information, refer to the following resource:

- *Connect to Amazon Web Services:* <https://cloud.google.com/network-connectivity/docs/interconnect/how-to/cci/aws/connectivity-overview>

Creating network connectivity between AWS and Azure

To create an IPsec VPN tunnel between AWS and Azure, follow these high-level guidelines:

- Create a dedicated AWS VPC with a new subnet for the VPN tunnel
- Create a dedicated Azure VNet with a new subnet for the VPN tunnel
- Create a connection between an AWS virtual private gateway (using an AWS-managed VPN) and Azure VPN Gateway

Important note

For network redundancy, create two VPN tunnels.

- Restrict access to AWS resources using Amazon VPC security groups, as explained here:
<https://docs.aws.amazon.com/vpn/latest/s2svpn/SetUpVPNConnections.html#vpn-configure-security-groups>
- Restrict access to Azure resources using a **network security group (NSG)**, as documented here:
<https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview>

For more information, refer to the following resources:

- *How to connect AWS and Azure using a BGP-enabled VPN gateway*
<https://learn.microsoft.com/en-us/azure/vpn-gateway/vpn-gateway-howto-aws-bgp>
- *How to create a VPN between Azure and AWS using only managed solutions:*
<https://techcommunity.microsoft.com/t5/fasttrack-for-azure/how-to-create-a-vpn-between-azure-and-aws-using-only-managed/ba-p/2281900>

To create a dedicated private connection between AWS and Azure, follow these high-level guidelines:

- Use multiple Direct Connect connections with the Direct Connect gateway so that you have connection redundancy between AWS and Azure, as explained here:
<https://docs.aws.amazon.com/directconnect/latest/UserGuide/disaster-recovery-resiliency.html>
- Restrict access to AWS resources inside your AWS environment using Amazon VPC security groups, as explained here:
<https://repost.aws/knowledge-center/routing-dx-private-virtual-interface>
- Use multiple Azure ExpressRoute connections to have connection redundancy between AWS and Azure, as explained here:
<https://learn.microsoft.com/en-us/azure/expressroute/designing-for-high-availability-with-expressroute>

For more information, refer to the following resource:

- *Designing private network connectivity between AWS and Microsoft Azure:*

<https://aws.amazon.com/blogs/modernizing-with-aws/designing-private-network-connectivity-aws-azure/>

How to create network connectivity between Azure and GCP

To create an IPsec VPN tunnel between Azure and GCP, follow these high-level guidelines:

- Create a dedicated Azure VNet with a new subnet for the VPN tunnel
- Create a dedicated GCP VPC with a new subnet for the VPN tunnel
- Restrict access to GCP resources inside your GCP environment using VPC firewall rules, as explained here:

<https://cloud.google.com/network-connectivity/docs/vpn/how-to/configuring-firewall-rules>

- Restrict access to Azure resources using an NSG, as documented here:
<https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview>
- If network redundancy is not required, create a connection between Azure VPN Gateway and Google Cloud VPN
- If network redundancy is required, create a connection between Azure VPN Gateway and Google HA VPN

Important note

For network redundancy, create two VPN tunnels.

For more information, refer to the following resource:

- *Create HA VPN connections between Google Cloud and Azure:*

<https://cloud.google.com/network-connectivity/docs/vpn/tutorials/create-ha-vpn-connections-google-cloud-azure>

To create a dedicated private connection between Azure and GCP, follow these high-level guidelines:

- Use multiple Azure ExpressRoute connections to have connection redundancy between Azure and GCP, as explained here:

<https://learn.microsoft.com/en-us/azure/expressroute/designing-for-high-availability-with-expressroute>

- Use multiple Google Interconnect connections to have connection redundancy between Azure and GCP, as explained here:

<https://cloud.google.com/network-connectivity/docs/interconnect/how-to/dedicated/creating-redundant-interconnect>

- Restrict access to GCP resources inside your GCP environment using VPC firewall rules, as explained here:

<https://cloud.google.com/network-connectivity/docs/interconnect/concepts/overview>

For more information, refer to the following resource:

- *Connect to Microsoft Azure:*

<https://cloud.google.com/network-connectivity/docs/interconnect/how-to/cci/azure/connectivity-overview>

According to your business requirements, create a secure and redundant connection between cloud providers using a site-to-site VPN connection (which passes over the public internet) or a dedicated private connection (when low and promised latency is required).

Next, we will review how to secure data when working in a multi-cloud environment.

Data security in multi-cloud environments

Data security in the cloud relates to protecting data as it transfers over the network, is at rest, and is in use.

Data classification and government

Data classification in a multi-cloud environment is crucial for organizing and protecting sensitive information across diverse cloud platforms, but it presents unique challenges due to the constantly changing nature of cloud infrastructures and the potential for data sprawl. Next are some recommendations to address this challenge:

- **Establish data classification levels:** Data classification levels typically include categories such as *Public*, *Internal*, *Confidential*, and *Restricted*, each representing increasing levels of sensitivity and requiring progressively stringent security measures to protect the information from unauthorized access or disclosure
- **Implement data classification policies:** Data classification policies establish guidelines for categorizing an organization's information based on sensitivity levels, enabling proper handling, storage, and protection of data while ensuring compliance with regulatory requirements and enhancing overall data security

- **Deploy data discovery tools:** Data discovery tools in a multi-cloud environment enable organizations to locate, catalog, and analyze data across diverse cloud platforms, facilitating efficient data management, governance, and insights generation while addressing challenges such as data sprawl and inconsistent security policies

Encryption in transit

To secure your cloud environment, make sure that all data traverses through secured protocols. Some alternatives to secured protocols are as follows:

- Enforce the use of **Transport Layer Security (TLS)** 1.2 (or newer versions of TLS) for accessing resources over your entire multi-cloud environment
- Configure IPsec VPN tunnels between cloud providers (for connections that pass through the public internet)

Encryption at rest

All major cloud providers have their **key management services (KMSs)** for managing keys, secrets, credentials, and so on. Although the built-in KMS can store, manage, retrieve, and rotate keys and secrets, they are integrated into their own cloud provider's ecosystem. When selecting a solution for handling encryption at rest, look for the following capabilities:

- The ability to encrypt data at rest over multiple cloud providers – each cloud provider has its own KMS. If you need to share encryption keys across multiple cloud providers, you need to deploy a third-party solution to achieve this goal.
- The ability to handle the scale of your cloud environment (generating thousands of keys from multiple cloud providers).

Important note

Consider a **hardware security module (HSM)**-based solution when you need to comply with regulations (such as the **Payment Card Industry Data Security Standard**, or **PCI DSS**) since HSM devices are tamper-resistant and tamper-proof.

- The ability to create, store, retrieve, and rotate both keys and secrets from multiple cloud provider services.
- The ability to configure RBAC – who can generate keys/secrets on which cloud account or cloud provider.
- The ability to generate a full audit log for the KMS solution for further investigation.

Encryption in use

The concept of encryption in use is to be able to protect data while running in memory on a VM or a container. The best-known concept for encryption in use is confidential computing. Common confidential computing options are as follows:

- AWS Nitro Enclaves
- Azure Confidential Computing
- Google Confidential Computing

To protect data in multi-cloud environments, combined with central provisioning solutions, aim to deploy VMs or containers that support one of the aforementioned confidential computing solutions.

In this section, we have reviewed various ways to protect data in cloud environments. Enforcing the use of encryption in all layers (at transit, at rest, and in use) will allow you to protect your cloud environment, even when working in multi-cloud environments. Next, we're going to talk about managing costs in multi-cloud environments.

Cost management in multi-cloud environments

Although **cost management** (also known as **financial management** or **FinOps**) may not seem directly related to security, it has implications for a multi-cloud environment.

Cost management is part of having visibility in your entire multi-cloud environment, from a cost point of view. All major cloud providers have built-in cost management services that allow you the necessary visibility of your cloud accounts, subscriptions, or projects.

When organizations are moving from a single cloud provider (in many cases, with multiple accounts at the same cloud provider, for different environments or business needs) to a multi-cloud one, they begin to realize that they lack the necessary visibility regarding cloud cost.

Some of the concerns organizations are having are as follows:

- Data transfer cost (also known as egress data)
- Redundant resources being spent (such as multiple log systems, backup services, and anti-malware solutions)

The possible risks of missing cost visibility are as follows:

- High payments to multiple cloud providers, without knowing what resources are been consumed and whether we need those resources.
- Anomalous spending, such as charges in unexpected regions, unexpected services, or anomalous use of disk volumes, which may be **indicators of compromise (IoCs)**, specifically when working in a multi-cloud environment.

- A potential denial-of-wallet scenario, where a hacker gains access to one of the cloud accounts and consumes expensive compute resources for crypto mining. Without proper visibility or an alerting mechanism, by the time we become aware that someone is using our resources, we will have to pay thousands of dollars to the cloud providers.

Here are some best practices to follow:

- Deploy a cost management solution that supports multiple cloud providers and offers a central solution to configure budgets, cost alerts, and visibility (both dashboards and reports), to be able to manage an entire cloud infrastructure while using multiple cloud providers.

For more information about cost management solutions for multi-cloud environments, see the following link:

<https://www.newhorizons.com/resources/blog/multi-cloud-cost-optimization>

- Tag all resources in all your cloud accounts, and all your cloud providers, at the creation/provisioning stage.
- Configure a budget for each of your cloud accounts. Once the budget has been configured, generate an automatic budget alert once a certain threshold of the budget (such as 75%) has passed.
- Deploy automation capabilities to remove unused resources (such as VMs, containers, and managed databases). Automation will allow you to deploy development or testing environments when resources are needed.
- Review cost recommendation reports and follow the recommendations, such as the following:
 - Rightsizing a VM to a smaller VM type (with less CPU or memory).
 - Shut down underutilized VMs.
 - Change the purchase options of VMs from on-demand (or pay as you go) to a reserved instance (a commit for 1 or 3 years in advance) or even spot, according to the expected VM runtime.
 - Switch to cheaper block storage (disk volumes) according to the actual usage (disk utilization).
 - Migrate objects on object storage services to a cheaper tier (from hot or real-time storage to cool or near-real-time, or even to cold or archive storage).
 - Delete unneeded backups or snapshots.

It is important to have visibility of your cloud costs over multi-cloud environments, as part of multi-cloud governance. It will allow you to mitigate the risk of high payments or potential hidden crypto mining in one of your cloud accounts.

Next, we will understand how to gain central visibility into the security posture of cloud environments.

CSPM

As part of having visibility in your multi-cloud environments, over the past couple of years, a new concept has emerged.

There are multiple offers from many third-party CSPM vendors, but the fundamental idea behind all CSPM solutions is to have visibility in misconfigurations – a crucial topic when working in a multi-cloud environment and managing multiple cloud accounts. When selecting a CSPM solution, look for the following capabilities:

- Support for multiple cloud providers and visibility into your entire cloud assets over a multi-cloud environment.
- Visibility into misconfigurations (such as an opened port and publicly accessible object storage).
- Visibility into IAM (such as API keys located in publicly accessible object storage, access keys or passwords that were not rotated in the past 90 days, users' permissions, and accounts missing MFA).
- The ability to assess risks over your entire multi-cloud environment.
- Get insights into mitigation actions: your staff needs to know what actions to take to resolve a misconfiguration or potential risk.
- The ability to prioritize risk mitigations: your staff needs to know what risks need to be handled immediately.
- Contextualized visibility: a publicly accessible web server must be patched first, while servers that reside in a private subnet, protected with strict access network controls, can be patched later.
- The ability to remediate misconfigurations automatically.
- Get insights into attack vectors: understand how a potential attacker can take advantage of misconfiguration to breach your cloud environment.
- Visibility into vulnerable components (such as missing security patches on a VM or an outdated third-party library inside a container).
- Check for compliance in your entire multi-cloud environment against well-known security frameworks (such as the **International Organization for Standardization (ISO) 27001** standard) and regulations (such as the **General Data Protection Regulation (GDPR)** and the **Health Insurance Portability and Accountability Act (HIPAA)**).
- Continuous monitoring of your entire multi-cloud environment: detecting threats and misconfiguration must be an ongoing process.
- Integration with SIEM systems for further investigation.
- Integration with a ticketing system for submitting tasks to allow your staff to handle alerts from the CSPM solution.

For more information, refer to the following resources:

- *Cloud security posture management (CSPM):*
<https://learn.microsoft.com/en-us/azure/defender-for-cloud/concept-cloud-security-posture-management>
- *Security posture overview:*
<https://cloud.google.com/security-command-center/docs/security-posture-overview>

Now that we've learned the concept of CSPM solutions and the CSPM capabilities to watch out for when selecting a CSPM solution, let's dive into CIEM solutions.

CIEM

CIEM solutions allow us to monitor and manage identities (both human and machine) and access privileges in a multi-cloud environment, from a central console, while applying PoLP access to cloud infrastructure.

Common use cases for using a CIEM solution include the following:

- **Managing overprivileged access:** A former employee retains access to sensitive cloud resources after leaving the company. A CIEM solution is able to detect the inactive status of the user and automatically revoke their access rights, mitigating the risk of data breaches or insider threats.
- **Addressing shadow IT:** Employees use unauthorized cloud applications, creating security vulnerabilities. CIEM tools continuously monitor for unauthorized applications and alert security teams to investigate and remediate these instances, enhancing overall cloud security.
- **Temporary elevated access:** A developer needs temporary **Secure Shell (SSH)** access to a production machine for troubleshooting. A user requests temporary access through a CIEM solution, which grants permissions for a limited time and automatically revokes them once the task is completed, ensuring tight control over sensitive resources.
- **Multi-cloud management:** Organizations operate across multiple cloud platforms, complicating entitlement management. CIEM provides centralized visibility and control over entitlements across various cloud environments, simplifying management and enhancing security operations.

When selecting a CIEM solution, look for the following capabilities:

- Support for multiple cloud providers
- An inventory of existing entitlements
- Detecting and remediating IAM misconfigurations
- The ability to control access to resources, services, and administrative accounts

- Identifying risks associated with configuration errors
- Identifying shadow admin accounts in a multi-cloud environment
- Suggestions for policy correction
- Autogeneration of access policies according to actual identity needs
- Detecting excessive permissions to identities
- Detecting external and cross-account access permissions

For more information, refer to the following resources:

- *CIEM on AWS*:
<https://aws.amazon.com/products/security/ciem-on-aws/>
- *What's Microsoft Entra Permissions Management*:
<https://learn.microsoft.com/en-us/entra/permissions-management/overview>
- *Overview of Cloud Infrastructure Entitlement Management*:
<https://cloud.google.com/security-command-center/docs/ciem-overview>

Next, let's learn about patch and configuration management in the context of multi-cloud.

Patch and configuration management in multi-cloud environments

One of the challenges in multi-cloud environments is managing software updates and configuration changes centrally while dealing with diverse operating systems, compute services (VMs, containers, and FaaSs), and varying infrastructure, APIs, and deployment mechanisms across multiple cloud providers.

Next is a list of services that can be used in a multi-cloud environment to deploy security patches and improve security posture and operational efficiency by automating and simplifying the update management process:

- AWS Systems Manager Patch Manager:
<https://aws.amazon.com/blogs/mt/use-aws-systems-manager-for-multi-cloud-operations-management-part-1/>

- Azure Update Manager (combined with Azure Arc):

<https://learn.microsoft.com/en-us/azure/update-manager/workflow-update-manager>

When selecting a vendor-agnostic patch management solution, look for the following capabilities:

- The ability to centrally manage patch deployment from a central place
- Support for deploying security patches on both Windows and Linux platforms
- Support for asset inventories
- Support for patch rollback in case of a problematic security patch
- Support for deploying security patches over secured protocols (such as the TLS tunnel between the central patch server and the remote VM)
- The ability to deploy security patches to a group of servers (such as different environments and a different patch cycle to avoid breaking availability in case of server clustering)
- The ability to deploy third-party security patches (such as patches to self-managed database servers, web servers, and client tools)
- The ability to integrate in a **continuous integration/continuous delivery (CI/CD)** pipeline for deploying patches during image creation of both VMs and containers
- The ability to configure least privilege to access remote VMs over your entire multi-cloud environment for patch deployment
- The ability to configure a maintenance window for patch deployment (to avoid deploying patches during working hours)
- The ability to generate compliance reports (such as what percentage of your servers were already deployed with a patch and which servers are still missing a patch)
- The ability to integrate a patch management solution with a ticketing system to allow your staff to take care of vulnerable VMs

Another topic that needs to be addressed when working with development teams is taking care of third-party libraries, also known as **software composition analysis (SCA)**.

When selecting a vendor-agnostic SCA solution, look for the following capabilities:

- The ability to centrally manage your entire multi-cloud environment from a central location
- The ability to integrate with multiple source code repositories
- The ability to integrate with multiple container registries in multi-cloud environments
- The ability to search for outdated third-party library repositories

- The ability to detect both a vulnerable or outdated third-party library and all its dependencies
- The ability to configure credentials with minimum permissions to access a remote source code or container registries
- The ability to integrate the SCA solution with CI/CD processes (in multi-cloud and multi-account environments, it is common to have multiple CI/CD pipelines)
- The ability to generate reports to allow you to see which vulnerable components exist in your development of production environments
- The ability to break a build process if you are using vulnerable components
- The ability to set the vulnerability severity that is allowed in your environments before breaking a build process (for example, do not allow the completion of a build process if critical vulnerabilities exist in one of the open source libraries or dependencies)
- The ability to integrate an SCA solution with a ticketing system to allow your staff to take care of vulnerable components as part of the build process

Configuration and change management are also crucial topics to take care of in multi-cloud environments. When selecting a vendor-agnostic configuration and change management solutions, look for the following capabilities:

- The ability to manage your entire multi-cloud environment from a central place
- The ability to configure least privilege to access assets in multiple cloud providers and multiple cloud accounts
- The ability to detect compliance against well-known standards, such as a **Center for Internet Security (CIS)** or **National Institute of Standards and Technology (NIST)** benchmark
- The ability to detect a configuration change in a multi-cloud environment
- The ability to automate a configuration change in multiple cloud accounts
- The ability to integrate a configuration or change management solution with a ticketing system to allow your staff to take care of changes
- The ability to integrate with an SIEM system to raise alerts when a configuration change is detected in one of your systems

In this section, we learned about patch management and configuration/change management in a multi-cloud environment. Since multi-cloud environments face many challenges in these areas, we reviewed important capabilities to look out for when searching for a solution to manage security patches and configuration management.

Finally, let's talk about monitoring and auditing in the context of multi-cloud.

The monitoring and auditing of multi-cloud environments

Monitoring is a crucial part of multi-cloud visibility. It can have multiple layers, such as the following:

- Resource utilization (performance logging)
- Monitoring running applications for errors (application logging)
- Security auditing and logging to detect security incidents

When selecting a vendor-agnostic security monitoring solution for multi-cloud environments, look for the following capabilities:

- The ability to connect to multiple cloud providers, using the native cloud provider APIs
- The ability to connect to remote APIs using secured protocols (such as TLS 1.2 or newer versions)
- Built-in connectors for multiple cloud solutions (both common **infrastructure as a service (IaaS)**/**platform as a service (PaaS)** and SaaS)
- The ability to receive feeds from threat detection services such as Amazon GuardDuty, Microsoft Defender for Cloud, and Google Security Command Center
- The ability to receive feeds from third-party vulnerability management solutions
- The ability to receive feeds from configuration management solutions to detect a misconfiguration or deviation from standards
- The ability to detect anomalous user behavior (such as a user trying to access a resource for the first time or outside normal working hours) and raise an alert
- The ability to centrally store all logs encrypted at rest, to avoid unauthorized access to sensitive information (such as customer data)
- The ability to configure RBAC of the monitoring solution to various employees in various roles (such as IT staff, security analysts, and auditors) and different places in your organization (for example, not all teams need access to view information about your entire cloud environment)
- The ability to receive real-time and continuous monitoring from your entire multi-cloud environment
- The ability to create correlation rules between various log sources (such as audit logs, endpoint detection solutions, and network traffic flow logs)
- The ability to run automated remediation actions (such as closing an open port and disabling a compromised account)
- The ability to create alerts based on logs and send them to a central SIEM system
- The ability to collect performance benchmarking from multiple resources across multiple CSPs in a central location
- The ability to integrate with a ticketing system to allow your staff to take care of discovered security incidents

These capabilities are important to look out for when searching for a solution for security monitoring in multi-cloud environments.

Summary

In this chapter, we focused on multi-cloud environments. We reviewed the importance of having a multi-cloud strategy to allow organizations to adopt multi-cloud environments. We also discussed the various IAM solutions from AWS, Azure, and GCP that allow organizations to have a central directory service that can keep a single identity for each end user.

We looked at the various methods from AWS, Azure, and GCP to allow organizations to connect between different cloud environments on different cloud providers using a site-to-site VPN tunnel or a dedicated private connection. We also discussed the various data security mechanisms (encryption at transit, encryption at rest, and confidential computing). Then, we reviewed the importance of cost management in multi-cloud environments. We reviewed the concept of CSPM and CIEM. Finally, we looked at patch management, configuration management, and monitoring in multi-cloud environments.

Understanding the topics mentioned in this chapter will provide organizations with the necessary tools when they build multi-cloud architectures.

In the next chapter of the book, we will review what DevSecOps is and how to implement it when working in cloud environments.

Implementing DevSecOps

In the previous chapters, we covered hybrid cloud and multi-cloud architectures and security aspects. In this chapter, we will explain how to embed security as part of the development process – **DevSecOps**.

DevSecOps is much more than a technology or a product – it is a cultural change (as we will explain in this chapter) that aims to enable development and embed security as part of a development process.

DevSecOps is a software development approach that integrates security practices into the DevOps process. In this chapter, we will review the cultural change that is DevSecOps. We will then have an overview of the implementation of a secure **continuous integration/continuous deployment (or delivery) (CI/CD)** pipeline, and we will review some services from AWS, Azure, and GCP that can assist organizations in implementing DevSecOps.

In this chapter, we will cover the following topics:

- Introduction to DevSecOps
- DevSecOps best practices – people
- DevSecOps best practices – processes
- DevSecOps best practices – technology

Introduction to DevSecOps

Before we deep dive into DevSecOps, let us understand the background of this concept.

Traditionally, development processes were very long. Applications were built from a complex monolith (that is, all components such as frontend, business logic, and database were tightly coupled, which made upgrades/changes a complex task). Due to the long development and test processes, organizations used to release new versions of their applications every couple of months (sometimes several times a year).

New techniques of software development, such as DevSecOps, drive significant business value by enabling faster delivery of secure applications, improving operational efficiency, and fostering innovation, ultimately contributing to increased revenue growth and competitive advantage in the digital marketplace.

In the traditional software development processes, we had teams that were in charge of the development itself and operations teams in charge of provisioning and maintaining the infrastructure. However, this dynamic changed with the emergence of DevOps.

DevOps and CI/CD

DevOps is a cultural change that aims to improve collaboration between Dev and Ops teams to allow frequent application releases.

The goal of the cultural change is to break silos between developers, testers, and operations teams (fewer delays and miscommunications between the teams).

To meet the fast development and deployment phase, modern applications implement the concept of CI/CD:

- **CI:** A process of regularly integrating code changes, from the moment a developer pushes code to a Git repository (most used code repos are based on Git) till the code (with its dependencies) is packaged into an artifact.
- **Continuous delivery (CD):** A process of making sure a product is fully tested (while still allowing rollback to previous versions in case of issues with the new version) before deploying an artifact to a target environment (such as dev, test, or even prod). In many cases, the output (artifact) remains ready to be deployed but not deployed to production.
- **Continuous deployment (CD):** A similar concept to continuous delivery, but the result is deployment to production. Useful for organizations with mature development processes that sometimes deploy new versions of their applications, several times a day.

A CI/CD pipeline is a continuous process, as you can see in the following diagram:

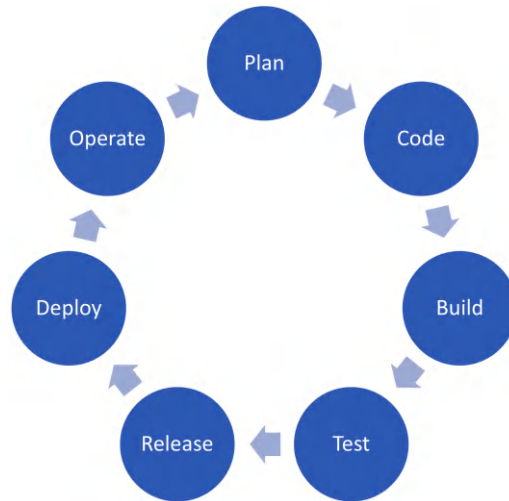


Figure 16.1 – A common CI/CD pipeline

CI includes the *plan*, *code*, *build*, and *test* phases. *Continuous delivery* focuses on the *release* phase, while *continuous deployment* covers the *deploy* phase. The *operate* phase is part of maintenance. Let us take a look at what each phase entails:

Note

The CI/CD process has been briefly outlined here to provide an overview of how the modern development process works. We will explore the CI/CD workflow in detail and analyze its integration with security in the *DevSecOps best practices – processes* section of this chapter.

1. **Plan:** During the planning phase, gathering system requirements is a critical step that involves collaborating with stakeholders to identify, document, and prioritize the functional and non-functional needs of the software system, ensuring that the final product aligns with business objectives and user expectations.
2. **Code:** In the coding phase, developers write and refine code based on the established requirements, leveraging **version control systems (VCSs)** such as Git to manage changes, facilitate collaboration, and maintain a comprehensive history of the code base, which enables efficient tracking and rollback of modifications when necessary.
3. **Build:** The build phase involves compiling the source code, integrating external dependencies, and packaging all components into a deployable artifact, often utilizing automated build tools to ensure consistency and reproducibility across different environments, thereby reducing potential errors and streamlining the development process.

4. **Test:** The testing phase ensures the build meets functional, performance, and security requirements by executing various tests, such as unit, integration, and **user acceptance testing (UAT)**, to identify and resolve defects early, guaranteeing a reliable and high-quality product before deployment.
5. **Release:** Creating an artifact (package) and storing it inside an artifact repository.
6. **Deploy:** Deploying artifact to production.
7. **Operate:** Managing software releases (software upgrades).

Now that we have explained what DevOps is and how a modern development process works (based on CI/CD), let us explain what DevSecOps means.

The goals of DevSecOps

In the same way that DevOps is a cultural change that aims to combine the developers with the operations team, DevSecOps is a practice that aims to embed security into the process.

The primary goal of DevSecOps is to integrate security practices seamlessly into every stage of the **software development life cycle (SDLC)**, from planning to deployment and monitoring.

In the DevSecOps mindset, security teams evolve into integral advisors within the development process, collaborating seamlessly to enhance security practices and facilitate smoother transitions to production rather than acting as barriers to new releases.

Next are some of the most common benefits of DevSecOps culture:

- **Early vulnerability detection:** Ability to detect vulnerabilities earlier in the development process (instead of the final phase before going live in production or, worse, after the system is already in production).
- **Faster remediation:** Ability to remediate vulnerabilities faster; that is, during the development, build, and pre-deployment stage.
- **Better team collaboration:** Instead of having different silos for developers, operational, and security teams, have a close team working on a project together, exchanging ideas and resolving issues before they impact the development process.
- **Security awareness:** Having security experts part of development teams raises awareness of topics such as security best practices and secure coding.
- **Continuous security integration:** DevSecOps emphasizes integrating security practices throughout the entire SDLC, not just at specific points. This continuous approach ensures that security is an ongoing consideration rather than an afterthought.
- **Cost reduction:** By identifying and addressing security issues early in the development process, DevSecOps can significantly reduce costs associated with fixing vulnerabilities later in the production stage or after deployment.

- **Improved compliance and governance:** DevSecOps practices help organizations maintain compliance with regulatory requirements by incorporating automated compliance checks and enforcing security policies throughout the development process.
- **Enhanced agility and speed:** Contrary to the misconception that security slows down development, DevSecOps actually improves the agility and speed of development teams by integrating security seamlessly into existing workflows.
- **Automation of security processes:** DevSecOps promotes the automation of security testing, vulnerability scanning, and compliance.

In the next sections, we will review how DevSecOps impacts people, processes, and technology. Let's start by understanding its impact on people.

DevSecOps best practices – people

Employees are one of the most valuable assets in any organization. A cultural change in the organization, as it relates to DevSecOps, begins by changing the way people look at security and security teams.

Adopting a security-by-design approach

Although you might be thinking that mandatory security training will provide the necessary awareness about security and application security practices, there are better ways to do so.

To adopt a *security-by-design* approach, the following is recommended:

- Security teams should be an integral part of the development process from the design stage and for the entire development until moving to the production and maintenance stages
- Having a security team as part of the development team allows effective communication of security issues (from sharing knowledge about new vulnerabilities published on the internet to explanation of how to implement security controls)

Shared responsibility

In DevSecOps culture, security is shared among all members of the team:

- **Development teams:** From implementing secure coding practices to code review processes
- **Operational teams:** From using secured protocols (such as TLS) to deploy infrastructure to ongoing vulnerability scanning and patch deployment
- **Security team:** From training about security best practices to ongoing monitoring and **incident response (IR)** processes

Cultural shift

To implement the DevSecOps cultural shift, organizations need to change the way they look at security:

- **Embracing security teams:** Shift the perception of security teams from being obstacles to valuable partners, enabling a faster and more secure development process.
- **Embracing a continuous learning mindset:** Evolving technologies and new attack methods require all team members to continue learning how to implement security controls (in the past, organizations had to learn how to deal with SQL injection, and now they need to learn how to deal with prompt injection attacks).
- **Continuous improvement:** Vulnerabilities and attacks are getting more and more sophisticated, environments (from dev to prod) are changing, and drifts from secure or standard organization configuration happen all the time, so it is crucial to invest in automated detection of changes and continuously close security gaps.
- **Promoting a culture of transparency and trust:** Shifting from being suspicious of security teams to welcoming them as trusted team members, while keeping open and transparent discussions.

Empowering development and operations teams

The role of security teams is to support and enable development and operations teams.

Here are a couple of ways to achieve this goal:

- **Setting guardrails and becoming advisors:** Just being a security observer does not empower Dev and Ops teams. Instead, security teams should set guardrails (such as using **policy as code (PaC)**, mentioned in *Chapter 17, Security in Large-Scale Environments*). For example: a developer will be able to deploy a new VM or new application, but without the option to assign a public IP address.
- **Providing training:** Most universities and colleges do not provide enough training about how to write secure coding and how to avoid issues such as buffer overflows, injection attacks, and unauthorized or excessive permissions. Security teams should provide regular training about security best practices, new security controls, lessons learned from recently published breaches, and more.
- **Security champions:** Since no organization has enough security experts to cover all development teams' work, as part of DevSecOps, (ideally) each development team should have at least one person who will become a security champion. This person should come with a development background and a passion for security to be able to speak with developers in their language, explaining security concepts and how to implement them.

In this section, we explained DevSecOps from a people (employee) point of view. Now, let us review DevSecOps from a process point of view.

DevSecOps best practices – processes

We previously mentioned a CI/CD pipeline as a common way to develop and deploy modern applications.

Now, let us deep dive into each step in the CI/CD pipeline and see how we can implement security into the process.

Before we deep dive into DevSecOps and how to embed security controls as part of a CI/CD pipeline, let us review a sample CI/CD workflow:

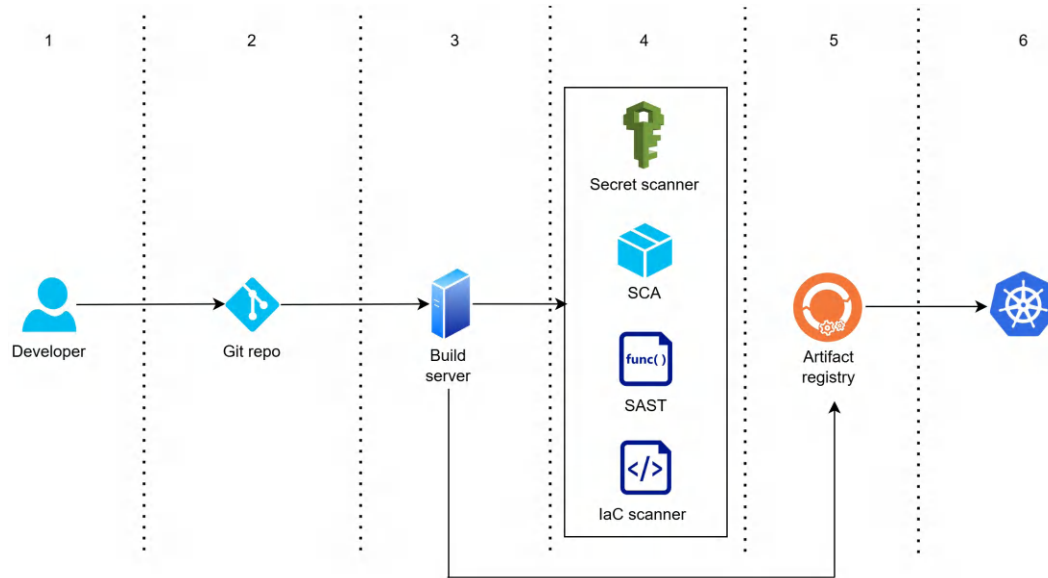


Figure 16.2 – A sample DevSecOps pipeline

Let's understand what's happening in this workflow:

1. A developer writes code in their IDE.
2. The developer commits the code changes to a Git repository.
3. A build server initiates a build process.
4. The build server initiates various security tests (from detecting secrets in configuration, code vulnerabilities, vulnerable packages, and misconfiguration in **infrastructure as code (IaC)**).
5. Once all security tests pass, the build server creates a container image and uploads it to an artifact registry.
6. An artifact is deployed to a Kubernetes cluster as a fully running application.

Next, we will review how to add security controls at each of the CI/CD stages.

Implementing security as part of the planning phase

During the planning phase, we review things such as the following:

- **Application requirements:** From infrastructure (should we use **virtual machines (VMs)**, containers, or a **function as a service (FaaS)**), development language (should we choose Java, Python, Rust, and so on), and application capabilities (how will the customer log in, where do we store persistent data, do we need a caching mechanism, and so on).
- **Testing requirements:** What testing do we need to validate that our application works as expected (from functional tests, UATs, performance tests, and so on), and will we implement the various tests as part of the CI/CD process (which tools at which phase; what do we do about failed tests, and so on)?

From a security point of view, there are things we should review, such as the following:

- **Security requirements:** How to implement authentication and authorization (that is, what **identity provider (IdP)** to use, and what mechanism will enforce **role-based access control (RBAC)**), how to implement data protection (such as encryption in transit, at rest, or even in use in memory), how to implement auditing (and where to send the audit logs), how to implement complementary controls (such as **web application firewall (WAF)**, **distributed denial-of-service (DDoS)** protection, vulnerability scanners, and so on).
- **Secure SDLC (SSDLC):** How to capture code vulnerabilities as early as possible in the development process, how to mitigate them, required security training for development teams, and so on.
- **Threat modeling:** Even before writing code and during the planning phase, it is possible to conduct threat modeling, trying to anticipate how the application will behave and what can go wrong (from privileged escalation to data leakage, and so on).

Implementing security as part of the coding phase

During the coding phase, a developer writes code in their **integrated development environment (IDE)**, such as Visual Studio, IntelliJ IDEA, and so on.

In some scenarios, the developer begins writing code from scratch, but commonly, the developer pulls code from a central code repository, makes changes (such as debugging past discovered issues in code), and pushes it back (commit) to the code repository.

Modern code repositories are based on Git, which allows central code storage, version control, and code sharing among development team members.

Since 2023, is it common to see that organizations began to embed **generative AI (GenAI)**-based tools, integrated into the developer's IDE (such as GitHub Copilot, Amazon Q Developer, and so on), which assist in writing code (by suggesting lines of code based on past organization code or samples from public code repositories).

During the coding phase, it is crucial to implement secure code review as early as possible in the development process by embedding **static application security testing (SAST)** tools inside the developer's IDE. SAST add-ons to IDE assist developers by discovering code vulnerabilities during the code-writing process and suggesting recommendations to fix discovered vulnerabilities, even before committing the code to the code repository.

For further reading about the SAST process, see the following links:

- Amazon CodeWhisperer (now part of Amazon Q Developer) security scans:
<https://docs.aws.amazon.com/codewhisperer/latest/userguide/security-scans.html>
- Code scanning in GitHub Advanced Security for Azure DevOps:
<https://learn.microsoft.com/en-us/azure/devops/repos/security/github-advanced-security-code-scanning>

Implementing security as part of the build phase

During the build phase, we gather code (stored in a code repository) together with third-party libraries and binaries (and all their dependencies) to create an artifact (application package).

Sometimes, an artifact is a compiled binary that can later be deployed on a target machine, and sometimes, an artifact is a container image (containing all the requirements for a running application) to be stored as an artifact registry.

Some main security aspects include the following :

- **Proper permissions:** Authorization to the code repository (source code) and artifact registry (final artifact) is restricted to authorized identities only. In the dev stage, a developer should be able to push code to the code repo; however, in production, it is recommended that only authorized identities (usually a service account running in a non-interactive mode) can pull artifacts and deploy them to production.
- **Change management:** Version control allows us to store different versions of an application, audit who made a change, and be able to revert to a previous version of code/application, in case problems were detected (such as application bugs or security vulnerabilities in code).

- **Secure code review:** The build phase is also a good time to review the code and run SAST tools to detect vulnerabilities in the code before compiling it into artifacts. SAST analyzes source code, bytecode, or binary code without executing the application. The reason for running SAST at the build stage is that it requires access to the source code. SAST can run automatically as part of the CI stage; however, the results of a test require manual review to remove false positives (an automated tool does not always understand the entire application end to end) and to prioritize fixing identified vulnerabilities. Some identified issues can be resolved by the developers themselves, but some are more complex and require the assistance of an application security expert or a DevSecOps expert.
- **Vulnerable package detection:** It is very common for development teams to use third-party libraries, binaries, and their dependencies as part of developed code. Such third-party components may contain vulnerable or outdated packages. To address this issue, it is crucial to embed **software composition analysis (SCA)** tools, which scan all third-party components against known vulnerability databases, alert on vulnerable components, and provide recommendations for newer versions of those components (assuming all components and their dependencies are still being maintained by their owners or by the community).
- **Package inventory:** Another issue with the use of third-party components is software supply chain attacks – using vulnerable third-party components. Before handling vulnerable components, we need to figure out which components we are using. For this, we can implement **software bill of material (SBOM)** tools, which scan our artifacts and generate reports (usually in JSON format) about all detected binaries, their versions, and their license (helping organizations avoid legal consequences such as lawsuits, injunctions, monetary damages, and potential loss of **intellectual property (IP)** rights due to non-compliance with open source licenses).

Implementing security as part of the test phase

The test phase, as its name implies, is the stage where we run various tests, such as the following:

- **Unit tests:** Focus on testing individual components or functions. Help developers catch basic errors early in the development process.
- **Integration tests:** Test the interaction between various modules or services. Ensure that integrated parts of the system function as expected.
- **Functional tests:** Simulate real user scenarios and workflows. Verify that the application meets specified functional requirements.
- **Performance tests:** Assess response times, throughput, and resource usage. Help identify bottlenecks and scalability issues.
- **Acceptance tests:** Ensure the software aligns with user expectations and business goals.

From a security point of view, the test phase is ideal for running the following tests:

- **Dynamic application security testing (DAST):** DAST scans the running application externally, searching for security vulnerabilities and weaknesses. It runs after the application is deployed to a test or staging environment or during the testing/QA phase. It uses a black-box testing approach (that is, it does not require deep insights into the application configuration). Use DAST for identifying runtime and configuration vulnerabilities.
- **Interactive application security testing (IAST):** IAST combines elements of both SAST and DAST, performing testing via instrumentation added to the application's code during runtime. It is performed during runtime and security testing or the testing/QA phase. It requires access to the application's runtime environment (that is, artifact). Use IAST for detailed, context-aware security testing.

Implementing security as part of the release phase

At the release stage, we have an artifact (from a VM image, container image, or application package) in the artifact repository/registry. We use the artifact as the source of deployment of new versions of applications.

Most common modern applications are deployed in an automated way using IaC (as explained in *Chapter 17, Security in Large-Scale Environments*).

From a security point of view, the release phase is ideal for the following:

- Scanning the content of a VM or container images to detect vulnerable components (such as vulnerable third-party components) and fix them (replace them with newer and stable versions) before deploying to production
- Scanning the content of IaC to detect security misconfigurations (such as allowing public object storage buckets, publicly accessible resources, unencrypted data stores, and so on)
- Scanning all code and configuration files to detect static credentials (such as secrets, passwords, API keys, access keys, and so on), replacing them with temporary credentials pulled from a secrets manager service (as explained in *Chapter 11, Applying Encryption in Cloud Services*)
- Penetration testing can be done at the release phase to detect vulnerabilities that have not been detected in previous stages (such as the behavior of an authenticated user in the system and which actions can be carried out with their privileges)

Implementing security as part of the deployment phase

Release (that is, continuous delivery) and deploy (that is, continuous deployment) are close stages with small differences – most organizations will complete the CI/CD phase in delivery, having an artifact that was either waiting to be deployed or deployed to lower environments (dev, test), while mature organizations may end up in the deploy phase, deploying a fully running application to production.

As such, most security tests are identical between release (delivery) and deployment (deployment), as previously explained.

Implementing security as part of the maintenance phase

Maintenance is the phase after an application is deployed to production and is being used by customers.

At this stage, the operations team takes responsibility for the application and its underlying infrastructure, overseeing the following:

- Ongoing software/component updates (such as replacing a package with a newer and more stable component)
- Ongoing monitoring to detect performance bottlenecks, application or infrastructure errors, and so on

From a security point of view, the maintenance phase is ideal for the following:

- **Security monitoring:** This is performed to detect potential security issues (such as unauthorized access, privileged escalation, data leakage, and so on) by collecting audit logs and sending them to a central **security information and event management (SIEM)** solution for further investigation by the **security operations center (SOC)** team.
- **Vulnerability scanning:** This is performed to detect configuration or software vulnerabilities (solutions such as **cloud security posture management (CSPM)** products).
- **Patch management:** Deploying security hotfixes to resolve identified vulnerabilities.
- **Penetration testing:** To ensure optimal security, it is considered best practice to conduct penetration testing before moving an application to production and then continue with regular assessments (typically annually) once the application is live. This proactive approach helps identify and address vulnerabilities throughout the application's life cycle, enhancing the overall security posture.

In this section, we explained DevSecOps from a process point of view (that is, how to embed security as part of a CI/CD pipeline). Now, let us review DevSecOps from a technology point of view.

DevSecOps best practices – technology

In the previous section, we explained how to embed security as part of a CI/CD pipeline.

Now, let us review some of the services offered by AWS, Azure, and GCP to assist in DevSecOps.

Amazon Q Developer

This is a GenAI developer assistant that can be integrated into a developer's IDE and offers code suggestions.

Combined with **Amazon CodeGuru Security**, a developer gets security capabilities such as SAST scanning to detect code vulnerabilities, secrets scanning to detect secrets embedded in code, and IaC scanning to detect misconfiguration and security issues inside IaC.

For more information, see the following links:

- <https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/what-is.html>
- <https://docs.aws.amazon.com/codeguru/latest/security-ug/what-is-codeguru-security.html>

GitHub Copilot

This is a GenAI developer assistant that can be integrated into a developer's IDE and offers code suggestions.

Combined with **GitHub Advanced Security**, a developer gets security capabilities such as SAST scanning to detect code vulnerabilities, secrets scanning to detect secrets embedded in code, and dependency review to detect vulnerable packages.

For more information, see the following links:

- <https://docs.github.com/en/copilot/about-github-copilot/what-is-github-copilot>
- <https://docs.github.com/en/get-started/learning-about-github/about-github-advanced-security>

Amazon Inspector

Amazon Inspector is a vulnerability scanning service, capable of detecting software vulnerabilities in **Elastic Compute Cloud (EC2)** instances, container images in Amazon **Elastic Container Registry (ECR)**, and Lambda functions.

Amazon Inspector can be integrated during the build process and afterward as part of regular scanning of the mentioned services.

Amazon Inspector can also generate regular SBOM reports to collect information about components inside VMs and container images.

For more information, see the following link:

<https://docs.aws.amazon.com/inspector/latest/user/what-is-inspector.html>

Microsoft Defender for Cloud

Defender for Cloud is a suite of services providing security insights into various services in Azure, AWS, and GCP (with a focus on Azure services):

- **Defender for Servers** allows you to detect vulnerabilities and misconfigurations in Windows and Linux servers deployed in Azure, AWS, GCP, and on-premises
- **Defender for Containers** allows you to detect vulnerabilities and misconfigurations in Kubernetes clusters and in the context of CI/CD in container images in Azure, AWS, GCP, and on-premises
- **Defender for Cloud DevOps Security** allows you visibility into the security posture of the DevOps process (such as dependency vulnerability scans and IaC scans to detect misconfigurations and detect secrets inside code)

For more information, see the following link:

<https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-cloud-introduction>

Google Artifact Analysis

Artifact Analysis (part of **Artifact Registry**) allows you to scan the content of container images (and detect vulnerable packages) and container image OS vulnerabilities (for **Google Kubernetes Engine (GKE)**).

Artifact Analysis can also generate regular SBOM reports to collect information about components inside container images.

For more information, see the following link:

<https://cloud.google.com/artifact-analysis/docs/artifact-analysis>

Open source tools

There are many DevSecOps tools available in the market (some are commercial, and some are open source) that we can embed as part of a CI/CD pipeline.

Next are some of the tools available as open source solutions:

- **Bearer:** An open source tool that can serve as a scanner (for detecting code vulnerabilities).

For more information, see the following link:

<https://github.com/bearer/bearer>

In the following example output, you can see how Bearer scans a vulnerable application called juice-shop (<https://github.com/juice-shop/juice-shop>) and is able to detect both code vulnerabilities and secrets embedded in the code:

```
Security Report
=====

Rules:
https://docs.bearer.com/reference/rules [v0.46.5]

Language  Default Rules  Custom Rules  Files
JavaScript 87             0             588
Python     88             0             3

CRITICAL: Usage of hard-coded secret [CWE-798]
https://docs.bearer.com/reference/rules/javascript_express_hardcoded_secret
To ignore this finding, run: bearer ignore add c095a5cc1948bae67c38b5c5d85eb4f4_0

File: juice-shop/lib/insecurity.ts:56

56 export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256' })

CRITICAL: Unsanitized user input in 'eval' type function [CWE-95]
https://docs.bearer.com/reference/rules/javascript_lang_eval_user_input
To ignore this finding, run: bearer ignore add 07e54b106f46e8d2158d7cc39518dab0_0

File: juice-shop/routes/b2bOrder.ts:21

21         vm.createContext(sandbox)
```

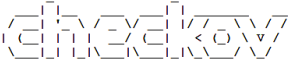
Figure 16.3 – Output of Bearer scanner (SAST and hardcoded secret in code)

- **Checkov:** An open source tool that can serve as a SAST scanner (for detecting code vulnerabilities), SCA scanner (for detecting vulnerabilities in open source packages), and IaC scanner (for detecting misconfigurations in IaC).

For more information, see the following link:

<https://github.com/bridgecrewio/checkov>

In the following example output, you can see how Checkov is scanning a vulnerable IaC application called terragoat (<https://github.com/bridgecrewio/terragoat>) and is expecting an Amazon S3 bucket to be encrypted using a **customer-managed key (CMK)**, and the test failed:



```

By Prisma Cloud | version: 3.2.342

terraform scan results:

Passed checks: 0, Failed checks: 1, Skipped checks: 0

FAILED for resource: aws_s3_bucket_object.data_object
File: /terraform/aws/s3.tf:23-40
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/aws-general-policies/bc-aws-general-106

23 | resource "aws_s3_bucket_object" "data_object" {
24 |   bucket = aws_s3_bucket.data.id
25 |   key    = "customer-master.xlsx"
26 |   source = "resources/customer-master.xlsx"
27 |   tags = merge({
28 |     Name      = "${local.resource_prefix.value}-customer-master"
29 |     Environment = local.resource_prefix.value
30 |   }, {
31 |     git_commit = "d68d2897add9bc2203a5ed0632a5cdd8ff8cefb0"
32 |     git_file   = "terraform/aws/s3.tf"
33 |     git_last_modified_at = "2020-06-16 14:46:24"
34 |     git_last_modified_by = "nimrodkor@gmail.com"
35 |     git_modifiers = "nimrodkor"
36 |     git_org       = "bridgecrewio"
37 |     git_repo      = "terragoat"
38 |     yof_trace     = "a7f01cc7-63c2-41a8-8555-6665e5e39a64"
39 |   })
40 | }
```

Figure 16.4 – Output of Checkov scanner (IaC code misconfiguration)

- **Trivy:** An open source tool that can serve as an SCA scanner (for detecting vulnerabilities in open source packages), IaC scanner (for detecting misconfigurations in IAC), Kubernetes misconfiguration scanner, and SBOM scanner (for collecting information about packages inside containers).

For more information, see the following link:

<https://trivy.dev>

- **Syft:** An open source tool that can serve as an SBOM scanner (for collecting information about packages inside containers).

For more information, see the following link:

<https://github.com/anchore/syft>

- **KICS:** An open source tool that can serve as an IaC scanner (for detecting misconfigurations in IaC).

For more information, see the following link:

<https://checkmarx.com/product/opensource/kics-open-source-infrastructure-as-code-project/>

- **Kubescape:** An open source tool that can serve as a Kubernetes configuration scanner (for detecting misconfiguration and compliance issues in Kubernetes).

For more information, see the following link:

<https://www.armosec.io/kubescape/>

- **Gitleaks:** An open source tool that can serve as a secret scanner (for detecting passwords, API keys, and tokens in Git repositories).

For more information, see the following link:

<https://github.com/gitleaks/gitleaks>

In this section, we reviewed various services from AWS, Azure, and GCP, together with open source tools that can be integrated into a CI/CD pipeline and embed security tests (from code vulnerabilities, vulnerable packages, misconfigurations, and secrets in code/configuration).

Summary

In this chapter, we focused on DevSecOps. We started by talking about DevOps (a cultural change meant to break silos between developers and operations teams) and then explored how DevSecOps comes into the picture.

We explained how DevSecOps is an organization's cultural change and how to make security teams part of development and operational teams. We looked at DevSecOps from the process perspective and how to implement security as part of a CI/CD pipeline for developing modern applications. Finally, we looked at some tools – from AWS, Azure, and GCP and open source tools – that can be integrated as part of the development process.

In the final chapter of the book, we will try to understand how to implement security in large-scale cloud environments.

Security in Large-Scale Environments

In the previous chapters, we covered cloud services, from the basic building blocks (such as compute, network, storage, and security) to hybrid and multi-cloud architectures.

In the final chapter of this book, we will cover security in large-scale environments. Previously, we looked at managing an account based on a single cloud provider while having all the resources located in a single place. In this chapter, we will zoom out of this picture and look at large-scale cloud environments – having multiple accounts, for different departments or different business needs, often in multiple regions around the world.

The following are some of the questions organizations often ask themselves about managing large-scale cloud environments:

- How do I enforce governance over multiple accounts?
- How do I get visibility into my entire cloud environment?
- How do I manage a multi-region environment?
- What automation capabilities do I have?

We will try to address those questions in this chapter.

In this chapter, we will cover the following topics:

- Managing governance and policies at a large scale
- Automation using infrastructure as code and policy as code

Managing governance and policies at a large scale

When organizations begin to adopt cloud services, it is highly recommended to plan and consider cloud governance (*asset deployment, data security, risk management*, and more), even before they find themselves managing multiple cloud environments and multiple accounts.

Failing to plan leads to ad hoc configurations, which create several challenges. These include a lack of uniformity, resulting in heterogeneous assets that require protection; a lack of repeatability; unnecessary redundancy; an expanded attack surface; and technical debt, such as overlapping CIDR blocks, which can be difficult to resolve.

Some of the important terminologies related to governance are as follows:

- **Landing zones:** These are preconfigured environments that are provisioned through code (policies, best practices, guidelines, and centrally managed services), as explained later in this chapter.
- **Guardrails:** This is an automated way to detect or enforce the desired configuration (for example, make sure that a newly deployed VM does not have a public IP, or encrypt a database at rest during creation time, etc.) before changes reach the production environment (allowing employees to do their job with minimum restrictions imposed on them).

Note

We will review guardrails in the section dealing with automation using **infrastructure as code (IaC)** and **policy as code (PaC)**, later in this chapter.

These are some of the benefits of using landing zones:

- They provide a consistent approach to setting up and configuring cloud environments, ensuring all deployments follow best practices and security standards.
- They establish security baselines and policies from the outset, reducing the risk of vulnerabilities and breaches.
- They offer an automated way to structure accounts according to best practices.
- They can be designed to isolate resources between different environments or projects, enhancing security and operational efficiency.
- They provide the following common types of shared or preconfigured account baselines:
 - **Network:** A placeholder for network-related services (such as a VPN gateway)
 - **Identity and access management:** A placeholder for services such as directory or IAM services, password policies, and more
 - **Management and monitoring:** A placeholder for services such as central log management, monitoring, alerting, and more

- **Preconfigured account baselines:** An example is a vending machine with automated integration, along with other shared services (compliance, security controls, networking, monitoring, and more)
- They enable a central billing configuration for configuring budget notifications (such as when the monthly bill exceeds the preconfigured value, which may require a review of used resources). By efficiently allocating resources and implementing standardized tagging, landing zones help organizations track and manage cloud spending more effectively.
- They enable setting limits on resource usage (such as how many VMs can be created by users in a specific account).
- They enforce the use of tagging (prohibit creating untagged resources).

The following diagram shows a sample landing zone based on AWS infrastructure:

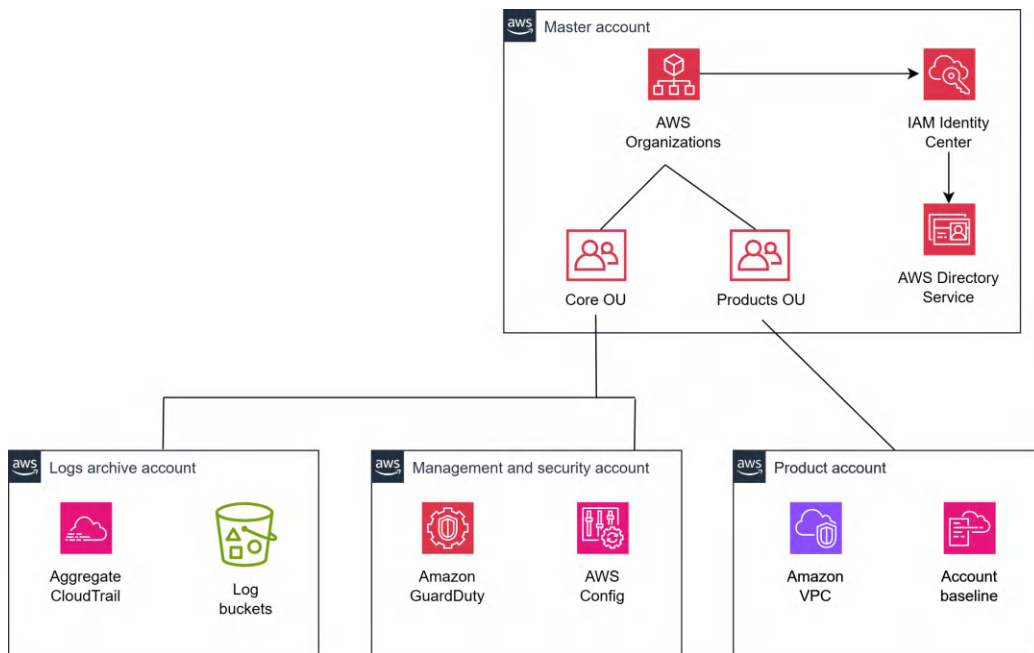


Figure 17.1 – AWS landing zone architecture

The preceding diagram shows the AWS landing zone concept. We will discuss this later in this chapter, under *AWS Landing Zone Accelerator*.

In the preceding diagram, we can see an AWS organization structure with the following:

- **Master account:** Contains cross-organization services such as AWS Organizations (with the underlying organizational units), the main SSO service (IAM Identity Center), connected to AWS Directory Service (for legacy applications or for connecting to an on-premises active directory)

- **Log archive account:** A central account for collecting audit logs (using CloudTrail), stored in an S3 bucket
- **Management and security account:** A central account for security services such as Amazon GuardDuty (a central service for detecting malicious activity from the entire organization), and AWS Config (for centrally detecting configuration drifts)
- **Product account:** An account deployed for a specific application, containing Amazon VPC (for configuring virtual networks such as private and public subnets), and a CloudFormation template for configuring baselines (such as IAM roles, KMS keys for encryption, log retention, etc.)

Next, we will review how AWS allows customers to manage governance.

Governance in AWS

In this section, we will review the AWS services that allow governance at a large scale. The following diagram depicts AWS's multi-account hierarchy:

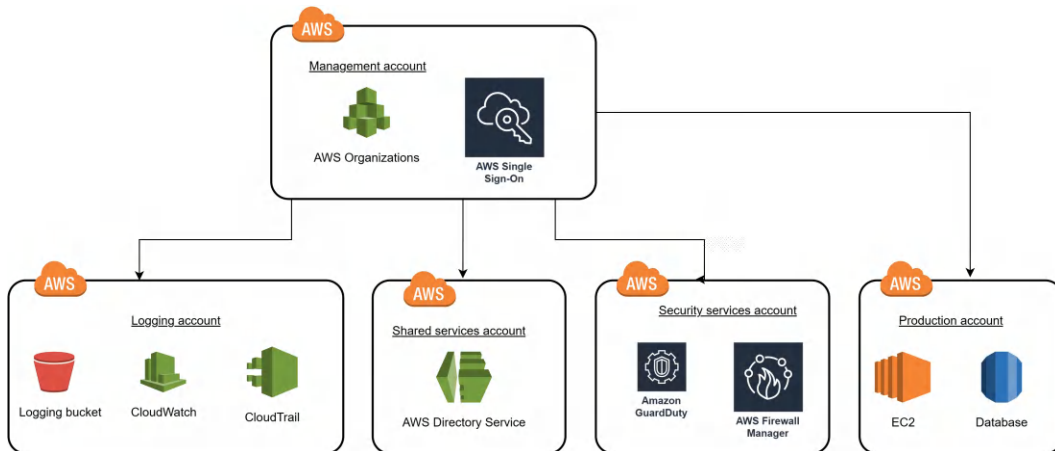


Figure 17.2 – AWS multi-account hierarchy

Before we move on to explaining governance in AWS, let us review the AWS terminology that is part of the AWS Organizations hierarchy (placeholders for AWS accounts and resources within AWS accounts):

- **Organization:** An organization is a collection of AWS accounts that can be managed centrally and organized into a hierarchical, tree-like structure.
- **Root AWS account:** The root AWS account, also known as the management account, is the primary account used to create and manage an AWS organization.

- **Organizational unit (OU):** An OU in AWS Organizations is a container for grouping AWS accounts with similar operational or security requirements. OUs provide a hierarchical structure within an organization, allowing for more efficient management and governance of multiple AWS accounts.

AWS Organizations

AWS Organizations provides customers with a central way to provision AWS accounts according to various business needs (such as IT services, marketing, HR, and more), which suits the ability to scale to multiple accounts.

Once we create a hierarchy of accounts, AWS Organizations allows us a central way to configure the following:

- **IAM:** The ability to centrally manage identities across the AWS organization, using a single identity provider, such as AWS IAM Identity Center as explained in *Chapter 14, Managing Hybrid Clouds*
- **Security and compliance:** The ability to centrally configure guardrails of the maximum allowed permissions to access and use resources across the entire AWS organization, as explained later in this chapter under *Automation using IaC and PaC*
- **Auditing:** The ability to centrally configure audit logs using AWS CloudTrail, as explained in *Chapter 10, Auditing and Threat Management in Cloud Environments*
- **Billing:** The ability to configure a central billing account for managing costs and gain discounts for the entire AWS organization

For more information, please refer to the following resources:

- *AWS Organizations FAQs:*
<https://aws.amazon.com/organizations/faqs/>
- *Building Multi-Region and Multi-Account Tools with AWS Organizations:*
<https://aws.amazon.com/blogs/architecture/field-notes-building-multi-region-and-multi-account-tools-with-aws-organizations/>
- *Security in AWS Organizations:* <https://docs.aws.amazon.com/organizations/latest/userguide/security.html>
- *AWS Organizations – Checklist for 2021:*
<https://www.chrisfarris.com/post/aws-organizations-in-2021/>

AWS Landing Zone Accelerator

AWS Landing Zone Accelerator is a framework that helps organizations set up a secure and well-structured multi-account environment in AWS.

For more information, please refer to the following resource: <https://docs.aws.amazon.com/solutions/latest/landing-zone-accelerator-on-aws/solution-overview.html>.

AWS Control Tower

The most common way to deploy AWS landing zones is by using **AWS Control Tower**.

AWS Control Tower is a service that allows organizations to create multi-accounts by following AWS best practices.

AWS Control Tower has the following capabilities:

- Built on top of AWS Organizations, which allows hierarchical account creation, including the ability to centrally enforce security and compliance best practices
- Use **AWS Account Factory** for provisioning new accounts inside the AWS organization in an automated and standard way
- Use **AWS CloudFormation** for provisioning resources within the AWS organizations using IaC
- Use **AWS Service Catalog** to centrally enforce which services are allowed to be consumed within AWS organizations, such as VM images, databases, allowed software, and so on
- Allows you to centrally configure **Amazon CloudWatch** (for monitoring) and **AWS CloudTrail** (for audit logs) at the AWS Organizations level
- Allows you to configure guardrails (maximum allowed permissions to consume resources) using service control policies (SCP) and resource control policies (RCP), as explained later in this chapter

For more information, please refer to the following resources:

- *AWS Control Tower FAQs*: <https://aws.amazon.com/controltower/faqs/>
- *Best Practices for AWS Control Tower Administrators*: <https://docs.aws.amazon.com/controltower/latest/userguide/best-practices.html>

Best practices for governance in AWS Organizations

In previous sections, we reviewed AWS Organizations, landing zones, and AWS Control Tower. Let us summarize everything by providing best practices for securing large-scale AWS environments.

Use the following best practices for governance and security in AWS environments:

- Carefully design the AWS organization account structure, to avoid having to restructure everything once resources have already been created and consumed. Create the first account in the organization as the root (or top account in the hierarchy), and make it the payer account (i.e., the account that centrally manages the billing for all accounts in the organization).

At the second level in the AWS organization structure, consider creating OUs according to business units or according to the different stages of the SDLC (such as Dev, Test, and Prod), to make it easy to enforce policies at the OU level (such as making Prod more restrictive and Dev/Test less restrictive and allowing experimentation). For more information, see https://docs.aws.amazon.com/organizations/latest/userguide/orgs_tutorials_basic.html.

- Select a Region to be your home region for deploying resources such as landing zones, location of the default VPC, and logging (for Amazon CloudWatch and AWS CloudTrail), as explained here: <https://docs.aws.amazon.com/controltower/latest/userguide/tips-for-admin-setup.html>
- Keep the root AWS account (in the hierarchy) solely for administrative actions that affect the entire AWS organization, and avoid creating resources in the top (root) AWS account, as explained here:
 - https://docs.aws.amazon.com/organizations/latest/userguide/orgs_best-practices_mgmt-acct.html
 - <https://aws.amazon.com/blogs/aws/centrally-managing-root-access-for-customers-using-aws-organizations/>
- Integrate AWS Organizations with AWS IAM Identity Center to centrally manage identities and their permissions to resources on the entire AWS organization, including SSO for simple authentication, and enforcement of MFA for all interactive logins, as explained here: <https://docs.aws.amazon.com/controltower/latest/userguide/sso-guidance.html>
- Set up a password policy (password length, complexity, and lock-out policies) for all the identities in the AWS organization, as explained here: <https://docs.aws.amazon.com/singlesignon/latest/userguide/password-requirements.html>
- If there is a business requirement to share data across regions (for example, cross-region replication of S3 buckets, sync of RDS databases, etc.), and assuming all data at rest is encrypted by default using encryption keys stored inside **AWS KMS**, create multi-region keys, as explained here: <https://docs.aws.amazon.com/kms/latest/developerguide/multi-region-keys-overview.html>

- Plan the CIDR allocation for your entire AWS organization, to avoid IP conflict when configuring VPC peering, as explained here: <https://docs.aws.amazon.com/controltower/latest/userguide/vpc-ct-cidr.html>
- Use a central S3 bucket in a dedicated AWS account, and create a trail to send all AWS CloudTrail logs from the entire AWS organization to the central S3 bucket, to be able to collect audit logs from all accounts in the organization, as explained here: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_cloudtrail-integration.html
- Configure Amazon CloudWatch cross-account observability to gain insights into CloudWatch logs from the entire AWS organizations, as explained here: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-Unified-Cross-Account.html>
- Decide which changes in resources are critical enough to capture at the AWS organizational level (due to the cost of **AWS Config** for a large number of changes), and create an AWS Config rule to send the information to a central location, as explained here: <https://docs.aws.amazon.com/config/latest/developerguide/config-rule-multi-account-deployment.html>
- Integrate **AWS Security Hub** with AWS Organizations to gain visibility into the security posture of your entire organization, as explained here: <https://docs.aws.amazon.com/securityhub/latest/userguide/central-configuration-intro.html>
- Integrate **Amazon GuardDuty** with AWS Organizations to have continuous monitoring of security events (from privileged escalations to Bitcoin mining) at the entire organization, as explained here:
 - https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_organizations.html
 - <https://aws.amazon.com/blogs/mt/automating-amazon-guardduty-deployment-in-aws-control-tower/>
- At the root AWS account in the AWS organization, centrally control credentials for all root users in the entire organization, as explained here: <https://aws.amazon.com/blogs/security/secure-root-user-access-for-member-accounts-in-aws-organizations/>
- Use **AWS Service Catalog** to configure which resources (such as VM images, servers, software, and databases) can be deployed by users in your AWS organization, as explained here: <https://docs.aws.amazon.com/organizations/latest/userguide/services-that-can-integrate-servicecatalog.html>
- Select **service control policies (SCPs)** to enforce policies and guardrails in your AWS organization. SCPs will be reviewed later in this chapter (under the **AWS SCPs** section).

- Configure S3 lifecycle policies to migrate less frequently used object storage data to the archive tier, as mentioned here: <https://docs.aws.amazon.com/controltower/latest/userguide/configure-log-retention.html>
- To simplify the creation and reuse of VPC security groups between different AWS accounts in the AWS organization, use VPC associations and security group sharing, as explained here: <https://aws.amazon.com/blogs/networking-and-content-delivery/simplify-amazon-vpc-security-groups-management-with-vpc-associations-and-security-groups-sharing/>

In this section, we reviewed the various alternatives that AWS offers customers to manage multi-account environments at a large scale.

Governance in Azure

In this section, we will review Azure services that allow governance at a large scale.

The following diagram shows the Azure multiple subscription hierarchy:

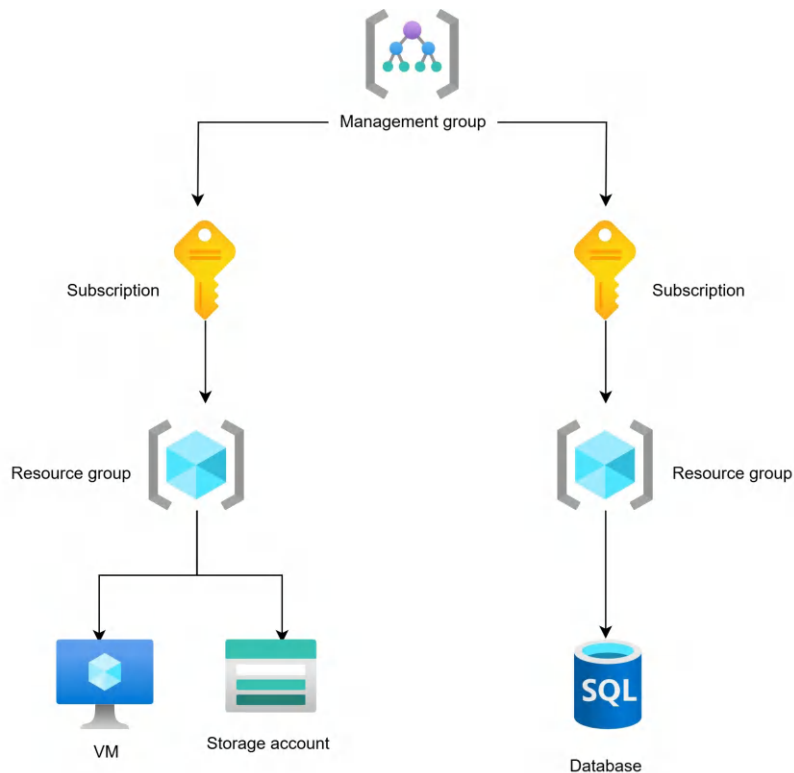


Figure 17.3 – Azure multiple subscription hierarchy

Before we move on to explaining governance in Azure, let us review the Azure terminology that is part of the Azure resource hierarchy (placeholders for Azure subscriptions, and resources within Azure subscriptions):

- **Management groups:** Azure management groups are logical containers that provide a hierarchical structure for organizing and managing Azure subscriptions. They offer a governance scope above the subscription level, allowing for more efficient control of access, policies, and compliance across multiple subscriptions.
- **Subscriptions:** Azure subscriptions are logical containers that group related Azure resources for billing, access management, and organizational purposes.
- **Resource groups:** Azure resource groups are logical containers for organizing and managing related resources with an Azure subscription. They serve as a fundamental organizational unit within the Azure ecosystem.

Azure management groups

Azure management groups are a logical way to group multiple Azure subscriptions that have similar uses (such as all production subscriptions, all development subscriptions, and more). Once these subscriptions have been grouped, you can apply governance conditions to the management groups.

Azure management groups allow us to do the following:

- Achieve a hierarchical structure for organizing subscriptions and resources and applying policies and access controls that automatically inherit child resources and subscriptions.
- Integrate with **Microsoft Entra ID** for user and group management. Assign RBAC to users and groups to manage access across the hierarchy, to control users' access to subscriptions within a management group, according to business requirements and the least privilege principle.
- Integrate with **Azure Policy** to centrally enforce settings over all the subscriptions inside a management group (as explained in the *Azure Policy* section).

For more information, please refer to the following resources:

- *What are Azure management groups?:* <https://learn.microsoft.com/en-us/azure/governance/management-groups/overview>
- *Protect your resource hierarchy:* <https://learn.microsoft.com/en-us/azure/governance/management-groups/how-to/protect-resource-hierarchy>

Azure landing zones

Azure landing zones are a structured approach to designing and implementing a cloud environment in Microsoft Azure.

For more information, please refer to the following resource:

- *What is an Azure landing zone?:* <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/landing-zone/>

Azure Lighthouse

Azure Lighthouse is a service that simplifies the management of multiple Azure environments. It is particularly useful for service providers or companies that need to manage multiple Azure tenants (i.e., multiple and isolated instances of Microsoft Entra ID).

Azure Lighthouse has the following capabilities:

- **Azure delegated resource management:** This allows you to manage customer Azure resources securely from within your tenant, without switching contexts or control planes. You can delegate customer subscriptions and resource groups to specific users and roles in the managing tenant, with the ability to revoke access when needed.
- **Cross-tenant visibility:** This feature allows you to access and manage resources across multiple tenants without switching between tenants or subscriptions, improving efficiency in resource management.
- **Role-based access control (RBAC):** Azure Lighthouse supports RBAC, enabling you to delegate specific management tasks to different users or teams, ensuring that only authorized personnel can access and manage resources.

For more information, please refer to the following resource:

- *What is Azure Lighthouse?:* <https://learn.microsoft.com/en-us/azure/lighthouse/overview>

Best practices for governance in Azure environments

Now that we have explored Azure management groups, landing zones, and Azure Lighthouse, let's wrap up by highlighting best practices for securing large-scale Azure environments.

Use the following best practices for governance and security in Azure environments:

- Keep the hierarchy flat, with no more than three to four levels, to reduce complexity:
 - Configure a default management group for new subscriptions to prevent them from going under the root management group
 - Establish a platform management group under the root of common platform policies and role assignments
 - Use resource tags for horizontal navigation across the hierarchy instead of creating a complex structure

For more information, see <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/landing-zone/design-area/resource-org-management-groups>.

- Integrate Azure organizations with **Microsoft Entra ID** to centrally manage identities and their permissions to resources on the entire Azure environment, including SSO for simple authentication, and enforcement of MFA for all interactive logins, as explained here: <https://learn.microsoft.com/en-us/entra/fundamentals/concept-secure-remote-workers>.
- Enable **Azure RBAC** authorization for your entire Azure environment to refine user privileges, as explained here: <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/landing-zone/design-area/identity-access-landing-zones>.
- Set up password policy (password length, complexity, and lock-out policies) for all the identities in Microsoft Entra ID, as explained here: <https://learn.microsoft.com/en-us/entra/identity/authentication/concept-sspr-policy>.
- If there is a business requirement to share data across regions (for example, cross-region replication of **Azure Blob Storage**, sync of **Azure SQL Database**, etc.), and assuming all data at rest is encrypted by default using encryption keys stored inside **Azure Key Vault**, use Key Vault replication across regions, as explained here: <https://learn.microsoft.com/en-us/azure/key-vault/general/disaster-recovery-guidance>.
- Plan the CIDR allocation for your entire Azure environment, to avoid IP conflict when configuring VNet peering, as explained here: <https://learn.microsoft.com/en-us/azure/virtual-network/concepts-and-best-practices>.
- Use a central **Log Analytics workspace** in a dedicated Azure subscription to be able to collect audit logs from all accounts in the organization, as explained here: <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/workspace-design>.
- Use **Microsoft Entra PIM** as **just-in-time (JIT)** access to Azure resources and as a mechanism for approving least privileged access within Azure, as explained here: <https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/security-planning>.
- Enable **Microsoft Defender for Cloud** for all Azure subscriptions, to allow the security posture of your entire Azure environment, as explained here: <https://learn.microsoft.com/en-us/azure/defender-for-cloud/connect-azure-subscription>.
- Use Azure landing zones to configure Azure Key Vault settings for your entire Azure environment, as explained here: <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/landing-zone/design-area/encryption-and-keys>.

- Centrally configure cost management on your entire Azure environment to gain visibility into resource consumption, as explained here: <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/azure-best-practices/track-costs>.

With that, we reviewed the various alternatives that Azure offers customers to manage multi-subscription environments at a large scale. Let's see what the next section holds.

Governance in Google Cloud

In this section, we will review the GCP services that allow governance at a large scale.

The following diagram shows the Google Cloud resource hierarchy:

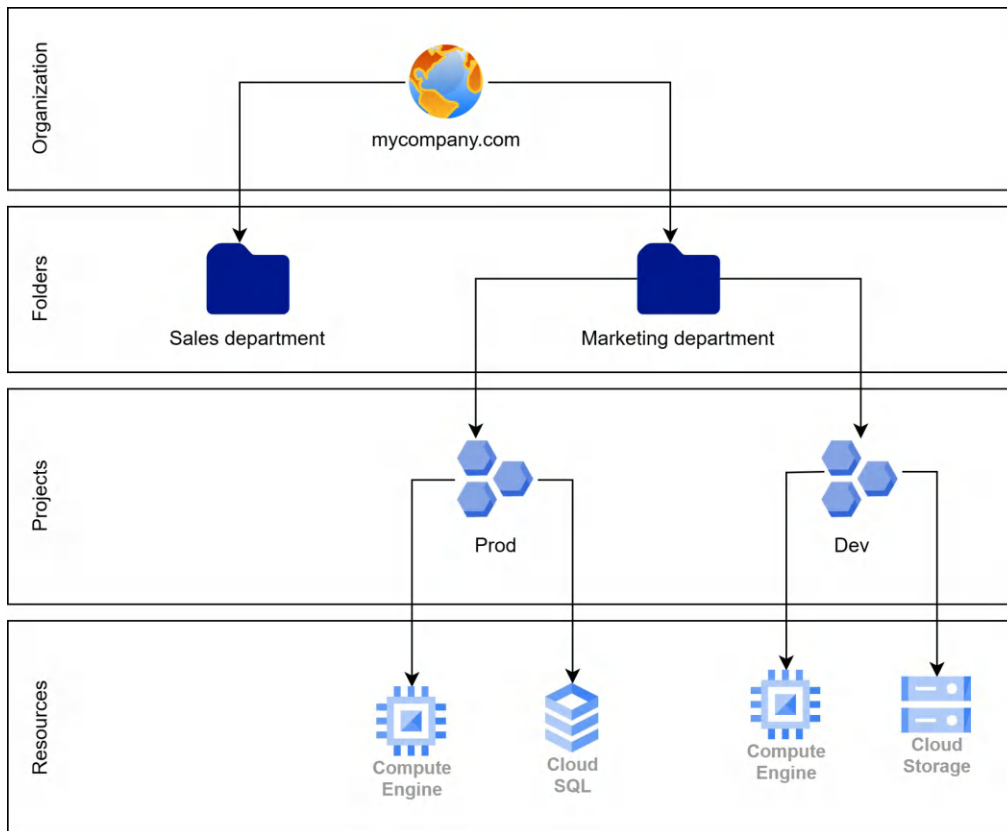


Figure 17.4 – Google Cloud resource hierarchy

Before we move on to exploring governance in GCP, let us review the Google Cloud terminology that is part of the Google Cloud resource hierarchy (placeholders for GCP projects, and resources within GCP projects):

- **Organization:** The root node of the resource hierarchy and represents the top-level container for all GCP resources within a company or entity
- **Folder:** Organizational units that provide an additional layer of hierarchy between the organization and projects
- **Project:** Fundamental organizational units that serve as containers for managing resources, services, and applications
- **Resource:** Any component created, managed, or consumed within Google Cloud, such as Compute Engine instances or Cloud Storage buckets

Google Resource Manager

Managing large-scale environments based on Google Cloud means managing multiple GCP projects in a single Google organization.

Google Resource Manager allows customers to centrally create and manage multiple GCP projects. In some cases, projects are grouped by business needs into a logical grouping called *folders* (a folder for all production GCP projects, a folder for all development GCP projects, and more).

For more information, please refer to the following resource:

- *Resource Manager documentation:* <https://cloud.google.com/resource-manager/docs>

Google landing zones

Google Cloud landing zones are foundational blueprints for cloud adoption on GCP. They provide a structured approach to setting up and configuring cloud environments, ensuring consistency, security, and scalability across an organization's GCP infrastructure.

For more information, please refer to the following resource:

- *Landing zone design in Google Cloud:* <https://cloud.google.com/architecture/landing-zones>

Best practices for governance in Google environments

In previous sections, we reviewed Google Resource Manager and Google landing zones.

Let us summarize everything by providing best practices for securing large-scale Google environments.

Use the following best practices for governance and security in Google environments:

- Implement a well-structured resource hierarchy using the *organization* resource as the root, followed by *folders* and *projects*:
 - Map your enterprise's operational structure to Google Cloud using this hierarchy for better management and control
 - Use folders to group related projects and apply consistent policies across them

For more information, see <https://cloud.google.com/architecture/landing-zones/decide-resource-hierarchy>.

- Integrate your Google organization with **Google Cloud Identity** to centrally manage identities and their permissions to resources on the entire Google environment, including SSO for simple authentication and enforcement of MFA for all interactive logins, as explained here: <https://cloud.google.com/resource-manager/docs/creating-managing-organization>.
- Enforce the use of two-step verification for all interactive logins in the entire Google organization, as explained here: <https://support.google.com/cloudidentity/answer/9176657?hl=en>.
- Use **Google Cloud IAM** to configure permissions for identities over the entire Google environment, following the principle of least privilege, as explained here: <https://cloud.google.com/resource-manager/docs/access-control-org>.
- Use **Google Privileged Access Manager (PAM)** as JIT access to Google resources, and as a mechanism for approving least privileged access within Google organization, as explained here: <https://cloud.google.com/iam/docs/pam-overview>.
- Plan the CIDR allocation for your entire Google environment to avoid IP conflict when configuring VPC peering, as explained here: <https://cloud.google.com/architecture/security-foundations/networking>.
- Create a central log bucket, located in a dedicated GCP project with limited access permissions for the SOC or security teams, as explained here:
 - <https://cloud.google.com/logging/docs/buckets>
 - https://cloud.google.com/logging/docs/export/aggregated_sinks
 - <https://cloud.google.com/logging/docs/central-log-storage>
- Integrate **Google Cloud audit logs** with **Google Security Operations** to assist SOC teams in an incident response process for the entire Google organization, as explained here: <https://cloud.google.com/chronicle/docs/ingestion/cloud/ingest-gcp-logs>.

- Activate **Google Security Command Center** on the entire Google organization to gain visibility into the security posture of your Google environment, as explained here:
 - <https://cloud.google.com/security-command-center/docs/activate-scc-for-an-organization>
 - <https://cloud.google.com/architecture/security-foundations/detective-controls>
- Use the **Organization Policy Service** to centrally enforce settings over your entire Google organization. Organizational policies will be explained later in this chapter.

In this section, we reviewed managing and governance at a large scale. Next, we will review how to implement IaC and PaC on AWS, Azure, and GCP.

Automation using IaC and PaC

Managing large cloud environments requires a change in mindset. Manual deployment is prone to human mistakes and drifts (deviation) from configuration standards and is hard to maintain in the long run since it does not scale. The idea behind IaC is to switch from a manual to an automated way to deploy or make changes to resources in the cloud environment using code.

PaC complements IaC and allows organizations to enforce standards (such as disabling the creation of public resources, allowing only specific VM types, etc.).

IaC

IaC helps bridge multiple organizations within a cloud service provider or even multiple cloud service providers. There are two ways to automate repetitive tasks:

- **Declarative:** Define what the desired state is (for example, object storage must be encrypted at rest), and the IaC makes the required changes (while understanding the order to make each change)
- **Imperative:** Define how the infrastructure needs to be changed to your requirements (for example, ordered steps to create networking configuration and then the compute resources within it)

The following are the benefits of using IaC:

- **Cost reduction:** People can focus on tasks other than resource provisioning or resource change.
- **Fast deployment:** Automation is always faster than manual work.
- **Risk reduction:** Once the deployment has been tested, all future deployments will achieve the same results – no human errors.

- **Stable and scalable environments:** Once the deployment has been tested, it is simple to deploy new environments of any size (simply change the scale inside the configuration files).
- **Documentation:** Since every setting or change is written inside the code, you have full visibility of which configurations were implemented. You can store all the configurations in a central source repository and track configuration changes or drift detection from preconfigured policies.
- **Security misconfigurations:** You can scan for non-compliance before provisioning.

PaC

PaC is a method of defining, managing, and enforcing organizational policies using code. It allows teams to represent rules, best practices, and compliance standards programmatically.

PaC allows us to configure guardrails, where a developer (or DevOps) can deploy resources (such as create new object storage) while keeping expected company standards (for example, do not allow creating publicly accessible object storage).

PaC, similar to IaC, is code that makes it easy to store in a central code repository, make changes (and audit who made them), deploy it, and, in case of issues, roll back to a previous version of the code.

PaC is not just limited to restrictions (such as prohibit public access to resources, or enforcement of using predefined VM instance size), but can also be used to enforce security configuration on Kubernetes clusters, and enforcement of authorization mechanisms to applications.

Let us further discuss automation using IaC and PaC in AWS, Azure, and Google Cloud services.

Common best practices for IaC and PaC in AWS, Google, and Azure

IaC and PaC are pieces of code and, as such, they share similarities.

The following are common best practices for IaC and PaC:

- **Use the least privilege principle:** Grant only the minimum necessary permissions for users to perform their tasks
- **Limit access to sensitive data:** Create deny lists for highly sensitive data and services
- **Use modular templates:** Break down complex templates into smaller, reusable components
- **Leverage parameters:** Make templates flexible by using parameters for customization during deployment
- **Implement proper naming conventions:** Use consistent and descriptive names for resources
- **Utilize variables:** Reduce redundancy and improve maintainability by using variables for repeated values

- **Document your code:** Add comments and provide documentation to improve understanding and maintenance
- **Implement version control:** Use systems such as Git to manage template changes and versioning
- **Regularly review and update IaC and PaC:** Keep policies aligned with your organization's evolving security needs
- **Test IaC and PaC before application:** Implement in a controlled environment to avoid disrupting critical processes

Next, we will review how AWS, Azure, and GCP implement IaC and PaC.

AWS CloudFormation

AWS CloudFormation is the Amazon IaC service. CloudFormation allows you to deploy and update compute, database, storage, network, and many other AWS resources in a standard way.

Note

For developers who are more comfortable writing code in Java, JavaScript, Python, and so on, AWS offers **AWS Cloud Development Kit (CDK)**, which wraps AWS CloudFormation code inside the mentioned programming languages. The result will execute AWS CloudFormation in the background.

Some of the best practices of AWS CloudFormation are as follows:

- Use AWS IAM to control access to deploy or update resources.
- Avoid embedding credentials inside AWS CloudFormation code. Use **AWS Secrets Manager** to store credentials and secrets.
- Use AWS CloudTrail to audit CloudFormation API activity.
- Use an AWS Config rule to check for compliance against the predefined policy on resources stored inside a CloudFormation Registry, as explained here: <https://docs.aws.amazon.com/config/latest/developerguide/aws-config-managed-rules-cloudformation-templates.html>.
- Use **AWS CloudFormation Guard** to validate CloudFormation templates against rulesets (for example, all EBS volumes must be encrypted at rest). For more information, see <https://docs.aws.amazon.com/cfn-guard/latest/ug/what-is-guard.html>.
- Activate **AWS CloudFormation Guard Hooks** to detect configuration that drifts from the organization's standards (such as creating publicly exposed resources) before the CloudFormation code executes, as explained here:
 - <https://docs.aws.amazon.com/cloudformation-cli/latest/hooks-userguide/hooks-concepts.html>

- <https://docs.aws.amazon.com/cloudformation-cli/latest/hooks-userguide/guard-hooks.html>
- Use static code analysis to review your CloudFormation code for security vulnerabilities.

For more information, please refer to the following resource:

- *Security best practices for AWS CloudFormation*: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/security-best-practices.html>

AWS SCPs

SCPs are organizational policies that serve as PaC, allowing customers to manage the maximum available permissions for IAM users and IAM roles in the organization.

SCPs can be attached at the AWS Organizations level, on an OU level, or to a specific AWS account, and, as such, they apply restrictions outside the boundaries (or outside the control) of a developer/DevOps who wishes to make infrastructure configuration changes (such as deploy a new service, delete an existing service, or make configuration changes to a service).

The following are some of the best practices for using SCPs:

- **Keep default permissions:** Leave the default **FullAWSAccess** SCP policy or all allowed actions to the root account of the AWS organization, to allow all child AWS accounts to be able to consume AWS resources.

Note

Even if an SCP is applied to a management account, the SCP applies to the entire organization but not the management account.

- **Apply fine-grained policies:** To create a fine-grained SCP policy for an AWS account, you can use the *view last accessed* service, as explained here: https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_last-accessed-view-data-orgs.html.
- **Apply less granular policies higher in the hierarchy:** Use more restrictive SCPs at lower-level OUs and accounts. Use * on the Action and Resource elements to keep the SCP generic on the OU level, and control the specific actions or resources as part of IAM policies, as explained here: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps_syntax.html.

For more information, please refer to the following resources:

- *Service control policies (SCPs):*

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

- *Service control policy examples:*

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps_examples.html

- *How to use service control policies to set permission guardrails across accounts in your AWS Organization:*

<https://aws.amazon.com/blogs/security/how-to-use-service-control-policies-to-set-permission-guardrails-across-accounts-in-your-aws-organization/>

AWS resource control policies (RCPs)

Resource control policies are organizational policies that serve as PaC, allowing customers to manage the maximum available permissions for resources in the organization.

Similar to SCPs, RCPs can be attached at the AWS Organizations level, on an OU level, or to a specific AWS account.

Here are some of the best practices for using RCPs:

- **Use with other policies:** Combine RCPs with SCPs and IAM policies for comprehensive security
- **Leverage CloudTrail:** Use AWS CloudTrail logs to review *Access Denied* errors and understand the impact of RCPs

For more information, please refer to the following resource:

- *Resource control policies (RCPs):* https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_rcps.html

Azure Resource Manager (ARM) templates

ARM templates are an Azure IaC solution that allows you to create or update Azure resources.

Note

For developers, Azure offers an easier alternative for ARM templates called Azure Bicep, which is an abstraction layer over ARM, written in a **domain-specific language (DSL)** with a simpler, more readable syntax than ARM templates. The result will execute ARM templates in the background.

Some of the best practices of ARM templates are as follows:

- **Validate templates:** Use tools such as **ARM Template Toolkit (arm-ttk)** to validate templates before deployment, as explained here: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/templates/test-toolkit>
- **Handling static credentials:** Avoid using embedding credentials inside Azure ARM templates – use Azure Key Vault to store credentials and secrets
- Use `securestring` for all passwords and secrets
- Use **Protected settings** to make sure your secrets are encrypted when they are passed as parameters to VMs
- **Code review:** Use static code analysis to review your ARM template's code for security vulnerabilities

For more information, please refer to the following resources:

- *What are ARM templates?:*
<https://docs.microsoft.com/en-us/azure/azure-resource-manager/templates/overview>
- *Tutorial: Integrate Azure Key Vault in your ARM template deployment:*
<https://docs.microsoft.com/en-us/azure/azure-resource-manager/templates/template-tutorial-use-key-vault>

Azure Policy

Azure Policy provides policies that allow customers to manage the maximum available permissions for all the subscriptions in the tenant (or in a management group). Similar to AWS SCPs, Azure policies are enforced outside the boundaries of an Azure subscription, and they can be assigned to an Azure management group, a subscription, a resource group, or a specific resource level.

Here are some of the best practices for using Azure Policy:

- **Define a clear governance strategy:** Identify key policies that need to be enforced and ensure alignment with organizational goals
- **Leverage built-in policies:** Use Azure's predefined policy definitions as a foundation and customize them to fit specific requirements
- **Group related policies into initiatives:** Create initiatives to simplify management and assignment of multiple policies
- **Use management groups for scalability:** Organize subscriptions into management groups to apply policies consistently across the entire organization

- **Use an enterprise-wide policy:** Consider using **Enterprise Azure Policy as Code (EPAC)** to deploy Azure Policy as part of a CI/CD pipeline, as explained here: <https://azure.github.io/enterprise-azure-policy-as-code/>
- **Automate remediation:** Use Azure Policy's remediation tasks to automatically bring non-compliant resources into compliance

For more information, please refer to the following resources:

- *What is Azure Policy?:* <https://docs.microsoft.com/en-us/azure/governance/policy/overview>
- *Azure Policy built-in policy definitions:* <https://docs.microsoft.com/en-us/azure/governance/policy/samples/built-in-policies>

Google Cloud Deployment Manager

Google Cloud Deployment Manager is the GCP IaC solution that allows you to create or update GCP resources.

Some of the best practices of Google Cloud Deployment Manager are as follows:

- Use the preview mode to check potential changes before applying them to your infrastructure
- Use static code analysis to review your Deployment Manager code for security vulnerabilities

For more information, please refer to the following resources:

- *Manage Google Cloud resources as a deployment:*
<https://cloud.google.com/deployment-manager/docs/manage-cloud-resources-deployment>
- *Setting access control in a configuration:*
<https://cloud.google.com/deployment-manager/docs/configuration/set-access-control-resources>

Organization Policy Service

Organization Policy Service allows you to configure restrictions on how resources can be used across your Google organization and define guardrails for your employees to stay compliant. Similar to AWS SCPs and Azure policies, organization policies are enforced outside the boundaries of a GCP project, and they can be applied at a GCP organization level, folder level, or project level.

Here are some of the best practices for using Organization Policy Service:

- **Implement essential policies first:** Prioritize policies that address critical security risks, such as public access restrictions and encryption requirements.

- **Use hierarchical application:** Apply policies at the organization level, with exceptions only where necessary. This ensures consistent security controls across your entire cloud environment.
- **Leverage policy inheritance:** Utilize the hierarchical nature of policies to streamline management and reduce the need for manual interventions on individual resources.
- **Test in a sandbox environment:** Experiment with policies in a non-production environment before implementing them across your organization.
- **Use conditional enforcement:** Leverage tags to conditionally enforce constraints based on specific resource attributes, providing more granular control.
- **Utilize dry-run mode:** Test policy changes using dry-run mode to monitor potential impacts on workflows before full enforcement.
- **Leverage Policy Intelligence tools:** Use Google Cloud's Policy Intelligence suite to test, analyze, and improve your organization's policies, as mentioned here: <https://cloud.google.com/policy-intelligence/docs/overview>.
- **Monitor compliance:** Regularly assess the compliance state of resources using available monitoring and auditing tools.

For more information, please refer to the following resources:

- *Introduction to the Organization Policy Service:*
<https://cloud.google.com/resource-manager/docs/organization-policy/overview>
- *Organization policy constraints:*
<https://cloud.google.com/resource-manager/docs/organization-policy/org-policy-constraints>
- *Preventative controls for acceptable resource configurations:*
<https://cloud.google.com/architecture/security-foundations/preventative-controls>

Next, we will review how to use IaC and PaC for organizations looking for a cloud-agnostic solution or a solution for enforcing governance while working with multiple cloud providers.

HashiCorp Terraform

Terraform is considered the most used IaC solution. Terraform provides standardized syntax across multiple cloud providers to deploy and update resources. It has a large community that provides support and has direct integration with cloud APIs.

Note

An open source fork of the HashiCorp commercial addition was introduced in 2023, called OpenTofu, which offers similar (almost identical) capabilities and is backed by the Linux Foundation.

Some of the best practices of Terraform are as follows:

- Enable logging for both troubleshooting and audit trails (check which user ran the scripts and made changes to a specific environment or resource)
- Always deploy secured protocols (such as HTTPS/TLS 1.2)
- Store the Terraform state (i.e., the current resource configuration on the remote cloud environment) remotely, to enable a team of developers to use the same Terraform state
- To secure a Terraform state file, it is essential to store it in a remote backend with encryption enabled (both at rest and in transit), restrict access using fine-grained permissions, and implement state locking to prevent concurrent modifications
- Provision Terraform code using a CI/CD pipeline
- Avoid storing credentials in code; instead, use solutions such as HashiCorp Vault to store all sensitive information (credentials, secrets, passwords, and more)
- Use static code analysis to review your Terraform code for security vulnerabilities

For more information, please refer to the following resource:

- *Introduction to Terraform*: <https://www.terraform.io/intro/index.html>

HashiCorp Sentinel policies

Sentinel policies are the HashiCorp complementary PaC for commercial customers of Terraform Cloud. They integrate with Terraform during the build process. At build time, any Terraform request for changes on cloud resources is been evaluated against Sentinel policies. If the change does not contradict the Sentinel policy, it will be applied to the target resource.

Here are some of the best practices for using Sentinel policies:

- **Use modular design:** Break down complex policies into smaller, reusable components for improved maintainability
- **Utilize multiple enforcement levels:** Use advisory, soft-mandatory, and hard-mandatory levels to provide flexibility in policy enforcement
- **Implement fine-grained, condition-based policies:** Create policies that can reject actions based on specific inputs rather than using coarse-grained permissions
- **Use policy sets:** Group related policies into sets for easier management and application to workspaces or projects

For more information, please refer to the following resource:

- *Introduction to Sentinel*: <https://developer.hashicorp.com/sentinel/docs/intro>

In this section, we reviewed common IaC solutions – both cloud vendor-oriented solutions and HashiCorp Terraform, which provide an IaC solution for different cloud providers. We have also reviewed the security best practices for each of the alternatives.

Summary

In this chapter, we focused on large-scale cloud environments that contain multiple accounts. We discussed how to deploy a multi-account organizational structure and how to set policies to check and enforce compliance over our entire organization.

Then, we learned how to achieve automation in cloud environments using IaC and PaC – from built-in services from AWS, Azure, and GCP to a more cloud-agnostic solution called Terraform.

The information in this chapter should have helped you learn how to manage security in large cloud environments.

What is next?

Now that you have reached the end of this book, we thought we would provide some recommendations regarding what you should do next.

Plan ahead

For each new project you review, from a cloud security point of view, thoroughly read through both the business and technological goals. Then, after understanding both, invest time into adding the relevant security controls.

Automate

This book covers many recommendations and best practices for using AWS, Azure, and GCP built-in services.

Invest some time in learning how to automate the recommended settings using IaC and PaC.

Consider learning about and using Terraform (or OpenTofu) for IaC since it provides connectivity for all the major cloud providers and since it is becoming a standard that most organizations are using for automation.

Think big

Instead of looking at a specific cloud environment and how to secure it, think about highly scalable cloud environments that are spread across multiple data centers (or availability zones), and sometimes even across multiple regions.

Design environments that can serve thousands or even millions of concurrent customers, from day 1, to avoid having to re-architect at a later phase.

Continue learning

This book covers a lot of resources, but do not stop here.

Invest time in taking a deep dive into each service presented in this book, look at the references in this book, continue learning about new products that are released each month to become a cloud security expert, and do not forget to share what you learned through blog posts, podcasts, and platforms such as YouTube.

Index

A

abuse of cloud services 297

- consequences 298
- detecting and mitigating, best practices 299, 300
- example 298

Access Context Manager 150

access control lists (ACLs) 78, 368

access control, to CSP support engineers 219

- AWS Support 219
- Customer Lockbox 221, 222
- Google Access Approval 223, 224
- Google Access Transparency 223, 224

account creation fraud prevention (ACFP) 160

account hijacking 287

- methods 287

account hijacking, in cloud services 287

- consequences 287
- detecting and mitigating, best practices 288, 289, 290
- example 288

account takeover prevention (ATP) feature 161

Active Directory (AD) 17, 139

Active Directory Domain

- Services (AD DS) 102

Active Directory services

- best practices 204

activity audit logs 307

AKS

- CSI driver, securing 112, 113

Amazon API Gateway 70

Amazon Bedrock 172

Amazon CloudWatch 53, 64, 73, 180, 412

Amazon CloudWatch Container Insights 47

Amazon CloudWatch Lambda Insights 74

Amazon CodeGuru Security 401

Amazon Cognito 37

Amazon ECR 38

Amazon ECS Service Connect 53

Amazon ECS VPC endpoint 53

Amazon Elastic Block Store (Amazon EBS)

- securing, best practices 93-95

Amazon Elastic Compute Cloud (EC2) 6, 17

Amazon Elastic File System

- (Amazon EFS) 64, 98

Amazon Elastic Kubernetes Service

- (Amazon EKS) 35

- best practices 35, 350

- CSI driver, securing 111, 112

- reference link 35

- Amazon FSx for Windows File Server** 99
- Amazon GuardDuty** 90, 180, 224-226, 414
- Amazon GuardDuty Malware Protection** 95
- Amazon Inspector** 76, 402
- Amazon Macie** 90
- Amazon Managed Service for Prometheus** 47
- Amazon Q Developer** 401
- Amazon Relational Database Service (RDS)** 360
- Amazon Route 53** 131
 - best practices 131, 132
 - securing 131, 132
- Amazon S3 access points** 82
- Amazon S3 Inventory** 90
- Amazon S3 Protection** 90
- Amazon Simple Storage Service (Amazon S3)** 64, 79
- Amazon Virtual Private Cloud (Amazon VPC)** 43, 69, 116, 117, 253
 - network ACL 121
 - reference link 117
 - security group 121
- Amazon VPC CNi plugin** 43
- Amazon VPC monitoring**
 - best practices 127
- Amazon Web Services (AWS)** 4, 16, 277
 - IaC best practices 424
 - network connectivity, creating 375
 - PaC best practices 424
- application firewall (WAF)** 279
- Application Gateway Ingress Controller (AGIC)** 44
- application performance management (APM)** 307
- application programming interface (API)** 13
- App Service Environment (ASE)** 71
- ARM Template Toolkit (arm-ttk)** 427
- Artifact Analysis** 402
- Artifact Registry** 57, 402
- artificial intelligence (AI)** 233
- asymmetric encryption** 241, 245
 - elliptical curve cryptography (ECC) 246
 - public-key cryptography 245
 - public-key encryption 245, 246
 - Rivest-Shamir-Adleman (RSA) 246
- asynchronous invocation** 63
- attribute-based access control (ABAC)** 101, 188, 285, 367
- audit and security monitoring practices, in GenAI services** 179
 - Amazon Bedrock 180
 - Azure OpenAI 180
 - Google Vertex AI 181
- auditing** 46, 72
- auditing and security monitoring, aspects**
 - abnormal pod activity 46
 - Kubernetes cluster admin activity 46
 - Kubernetes control plane logs 46
- auditing and security monitoring, best practices** 46
 - Amazon EKS 46, 47
 - AWS Lambda 73
 - Azure AKS 47, 48
 - Azure functions 74
 - Google Cloud Run functions 75
 - Google Kubernetes Engine (GKE) 48, 49
 - in function as a service (FaaS) 72, 73
 - in Kubernetes 46
- audit trails**
 - AWS CloudTrail 213-215
 - Azure Monitor 215-217
 - best practices, in cloud environments 212
 - Google Cloud Audit Logs 217, 218
 - maintaining 211, 212
 - purposes 212

authentication, best practices 18

Amazon EC2 19

best practices 18

authentication process

human authentication 187

non-human authentication 187

auto-scaling groups 152**availability zones (AZs) 342****AWS Account Factory 412****AWS App Runner 6****AWS Backup 27****AWS Certificate Manager (ACM) 137, 339****AWS CLI 13**

reference link 13

AWS Client VPN 138

best practices 138-140

reference link 138

securing 138

AWS Client VPN, components

client VPN endpoint 138

target network 138

AWS CloudFormation 412, 424

resource control policies (RCPs) 426

service control policies (SCPs) 425

AWS CloudFormation Guard 424**AWS CloudFormation Guard Hooks 424****AWS CloudTrail 47, 53, 73,**

146, 180, 213, 412

securing, best practices 213-215

AWS Config 76, 90, 414**AWS Control Tower 412****AWS DataSync**

best practices 345

AWS Direct Connect

best practices 340

AWS Directory Service 139

best practices 205, 206

deployment options 205

reference link 205

securing 205

AWS DX 373**AWS Firewall Manager 122****AWS governance 410**

AWS Control Tower 412

AWS Landing Zone Accelerator 412

AWS Organizations 411

AWS Organizations, best practices 412-414

AWS IAM 64

best practices 192-194

policy evaluation logic 190, 191

securing 189, 192

terminology 189

AWS IAM Identity Center 145

best practices 333

**AWS Key Management Service
(SSE-KMS) 215****AWS KMS 413**

AWS-managed keys 251

AWS-owned keys 251

customer-managed keys (CMKs) 251

AWS Lambda 64

reference link 64

AWS Landing Zone Accelerator 412**AWS Load Balancer Controller 43****AWS Managed Services 122****AWS Nitro Enclaves 264****AWS Nitro System 264**

Nitro cards 264

Nitro hypervisor 264

Nitro security chip 264

AWS Organizations 411

best practices 412-414

AWS organization structure 409**AWS Outposts**

best practices 349, 350

AWS Parameters and Secrets**Lambda Extension 66****AWS PrivateLink 84, 122, 177****AWS root user**

best practices 192

AWS root user security

reference link 191

AWS Secrets Manager 39, 66, 260, 424**AWS Security Hub 155, 194, 414****AWS Security Token Service (AWS STS) 192****AWS Service Catalog 412, 414****AWS shared responsibility model 10**

reference link 11

AWS Shield

using 153-155

AWS Signer 76**AWS Site-to-Site VPN 373**

best practices 137, 138

reference link 137

securing 136

AWS Site-to-Site VPN, components

customer gateway (CGW) 136

virtual private gateway 136

AWS Storage Gateway

best practices 345

AWS Systems Manager 53**AWS Systems Manager Parameter****Store 39, 53****AWS Transfer Family**

best practices 346

AWS Transit Gateway 123, 338**AWS Transit Gateway connections**

best practices 340

AWS Verified Access

best practices 145

components 144

connection workflow 144

securing 144

AWS WAF 70

protection mechanisms 159

securing, best practices 160, 161

using 159

Azure

IaC best practices 424

PaC best practices 424

Azure API Management 71**Azure Application Gateway WAF v2**

using 162, 163

Azure App Service 6**Azure Arc**

best practices 352

Azure Backup 28**Azure Bastion 23****Azure Blob Storage 64, 79, 418****Azure CLI 13**

reference link 13

Azure CNI plugin 44**Azure confidential computing 266**

best practices 268

Intel SGX 266

Intel TDX 266

security benefits 267

SEV-SNP 266

Azure Container Registry (ACR) 40**Azure Data Box Gateway**

best practices 346, 347

Azure DDoS Protection

using 155, 156

Azure DNS

best practices 133, 134

securing 133

Azure DNS security

reference link 133

Azure Enterprise Agreement**(Azure EA) 317**

-
- Azure environments**
 - best practices 417, 419
 - Azure ExpressRoute 373**
 - best practices 342
 - Azure File Sync**
 - best practices 347
 - Azure Front Door**
 - using 163-165
 - Azure functions 64**
 - reference link 64
 - Azure governance 415**
 - Azure environments, best practices 417, 419
 - landing zones 416
 - Lighthouse 417
 - management groups 416
 - Azure Key Vault 39, 67, 254, 261, 418**
 - best practices 255
 - CMKs 254
 - customer-provided keys 254
 - Microsoft-managed keys 254
 - Azure Kubernetes Service (AKS) 28, 36, 95, 162, 360**
 - best practices 36
 - Azure Load Balancer 123**
 - Azure-managed disks**
 - securing, best practices 95, 96
 - Azure migration**
 - reference link 308
 - Azure Monitor 156, 180, 215**
 - securing, best practices 215-217
 - Azure Monitor Application Insights 64**
 - Azure OpenAI 173**
 - Azure Policy 427**
 - best practices 427, 428
 - reference link 428
 - Azure Private Link 124**
 - Azure RBAC 418**
 - Azure Resource Manager (ARM) 196**
 - Azure Resource Manager (ARM) templates 426**
 - Azure Policy 427
 - best practices 427
 - reference link 427
 - Azure shared responsibility model 11**
 - reference link 11
 - Azure SQL Database 418**
 - Azure Stack Hub**
 - best practices 351
 - Azure Virtual Machines (Azure VMs) 6, 17**
 - Azure Virtual Machine Scale Sets (VMSS) 162**
 - Azure Virtual Network (Azure VNet) 118, 119**
 - reference link 119
 - Azure virtual network NAT gateway 71**
 - Azure Virtual WAN 341**
 - best practices 342
 - Azure VNet 116**
 - application security group (ASG) 123
 - network security group (NSG) 123
 - Azure VNet monitoring**
 - best practices 128
 - Azure VNet NAT 124**
 - Azure VPN Gateway 373**
 - Azure VPN Gateway (Site-to-Site) 140**
 - best practices 140
 - reference link 140
 - securing 140
 - Azure VPN Gateway (Site-to-Site), components**
 - VPN device 140
 - VPN Gateway 140
 - Azure VPN Point-to-Site (P2S) 141**
 - best practices 141, 142
 - securing 141

Azure VPN Point-to-Site (P2S), components

- VPN client 141
- VPN gateway 141

B**Backup Audit Manager 27****BeyondCorp Enterprise**

- best practices 148-150
- connection workflow 148
- securing 148

Binary Authorization 36**Binary Authorization for Google
Distributed Cloud 353****block storage 77**

- Amazon EBS, securing best practices 93-95
- Azure-managed disks, securing
 - best practices 95, 96
- Google's Persistent Disk, securing
 - best practices 96, 97
- securing 92
- securing, best practices 93

Bucket Lock feature 87**business continuity (BC) 290, 361****business unit (BU) 363****C****California Consumer Privacy
Act (CCPA) 277****California Privacy Rights Act (CPRA) 311****Center for Internet Security
(CIS) 17, 50, 386****Center for Internet Security
(CIS) benchmarks 315****CentOS 13****Certificate Authority (CA) 137, 339****certificate-based authentication 139****chief information security
officer (CISO) 315****chief strategy officer (CSO) 315****Chrome Enterprise connector 150****Chrome Enterprise Premium 148****ciphertext 242****Classless Inter-Domain Routing
(CIDR) 20, 120, 338, 372****cloud-based IAM services**

- AWS IAM, securing 189, 192
- best practices 188
- IAM in GCP, securing 199
- Microsoft Entra ID, securing 194
- securing 188

cloud bursting 327

- examples 327

Cloud Controls Matrix (CCM) 309

- reference link 309

cloud deployment models 5

- hybrid cloud 6
- multi-cloud 6
- private cloud 5
- public cloud 6

Cloud Development Kit (CDK) 424**cloud environments**

- audit trails, best practices 212
- penetration testing, conducting 319-321

cloud environments identities

- cloud-native identity 187
- hybrid identity 187

Cloud IAM 114, 348**Cloud Infrastructure Entitlement
Management (CIEM) 230**

- use cases 383

cloud migration strategies

- reference link 308

cloud-native identity 187

cloud-native SIEM

Google Security Operations
(SecOps) 237-239

cloud-native SIEM(SIEM)

managing 232, 233

Microsoft Sentinel 233-236

**Cloud Next Generation Firewall
(Cloud NGFW) 125**

cloud provider

auditing and monitoring 307, 308

authentication 309

data privacy 306, 307

data sovereignty 306, 307

migration capabilities 308

selecting 304

cloud provider questionnaire

API-related 311

architecture-related 310

audit and monitoring 314

authentication- and authorization-
related 312

business continuity (BC) 314

cloud infrastructure-related 310

compliance-related 315

data-related 311

encryption at rest 313

encryption at transit 312

IR 313

network-related 310

penetration testing and security
assessment 313

secure development life cycle 314

session management 312

cloud providers

tips for contracts 317-319

Cloud Run 57

auditing and security monitoring,
best practices 59

best practices 57

identity and access management,
best practices 57, 58

network access security, best practices 58

securing 57

cloud security

need for 7, 8

Cloud Security Alliance (CSA) 309

**cloud security posture management
(CSPM) 283, 320, 382, 400**

cloud service 5

Cloud Service Mesh 45

cloud service model 6, 304, 305

comparing 306

infrastructure as a service (IaaS) 6

platform as a service (PaaS) 6

software as a service (SaaS) 6

cloud service provider

(CSP) 7, 16, 276, 303, 330, 359

cloud services

data breaches 276

Cloud Storage Transfer Service 348

CloudWatch Events 66

cluster mode 33

command and control (C&C) 127

command-line interface (CLI) 13

command-line tools 13

AWS CLI 13

Azure CLI 13

Google Cloud SDK 14

**Common Internet File System
(CIFS) 21, 313**

**Common Vulnerabilities and
Exposures (CVE) 25**

- compliance, best practices**
 - AWS Lambda 76
 - in function as a service (FaaS) 75
- conduct patch management**
 - best practices 25
- conduct patch management, best practices**
 - Amazon EC2 25
 - GCE 26
- Confidential Google Kubernetes Engine Nodes 36**
- Consensus Assessments Initiative Questionnaire (CAIQ) 309**
 - reference link 309
- Container Apps 54**
 - auditing and monitoring, best practices 56
 - best practices 54
 - data protection, best practices 56
 - identity and access management, best practices 54, 55
 - securing 54
- containers**
 - benefits 31
 - securing 31, 32
- Container Storage Interface (CSI) 77**
- content delivery network (CDN) 116**
- continuous delivery (CD) 390**
- continuous deployment (CD) 390**
- continuous integration/continuous delivery (CI/CD) 305, 385, 390-392**
- control plane components 33**
 - etcd 33
 - kube-controller-manager 33
 - Kubernetes API server 33
 - kube-scheduler 33
- cross-site scripting (XSS) 159, 279, 320**
- customer gateway (CGW) 136**
- customer IAM (CIAM) 368**
- Customer Lockbox 221, 222**

- customer-managed encryption keys (CMEKs) 96**
- customer-managed keys (CMKs) 86, 347, 404**
 - using 216
- customer relationship management (CRM) 4, 305**

D

- data access audit logs 218, 258**
- data at rest 349**
- data breach 276**
 - best practices 278
 - in cloud services 276
- data breach, in cloud services**
 - consequences 277
 - detecting and mitigating, best practices 278-280
 - example 277
- data encryption key (DEK) 243**
- data processing agreement 318**
- data protection, best practices**
 - Amazon EKS 42, 43
 - AWS Lambda 69
 - Azure AKS 44, 45
 - Azure functions 70-72
 - Google Cloud Run functions 72
 - Google Kubernetes Engine (GKE) 45
 - in function as a service (FaaS) 68, 69
 - in Kubernetes 41
- data protection, in file storage**
 - best practices 103-107
- data protection, in object storage**
 - best practices 83-87
- data protection officer (DPO) 306**

data protection practices, in**GenAI services 176**

Amazon Bedrock 177

Azure OpenAI 177, 178

Google Vertex AI 178, 179

data security

examples 363

data security, in multi-cloud**environments 378**

data classification and government 378

encryption at rest 379

encryption in transit 379

encryption in use 380

DDoS protection service

AWS Shield, using 153-155

Azure DDoS Protection, using 155, 156

Google Cloud Armor, using 157, 158

DDoS protection services

using 151, 152

Debian 13**decryption 243****Denial of Service (DoS) attack 287, 321**

types 152

using 151

Denial of Wallet 152**DevOps 390-392****DevSecOps 389, 390**

DevOps and CI/CD 390-392

goals 392

goals, benefits 392, 393

DevSecOps, best practices

people 393, 394

processes 395-400

technology 401-405

Direct Connect (DX) 338, 372, 376**directory services**

AWS Directory Service, securing 205

Microsoft Entra Domain

Services, securing 207

securing 203, 204

disaster recovery (DR) 26, 29, 308, 361**disaster recovery plan (DRP) 278****Distributed Denial of Service****(DDoS) attack 8, 69, 130, 152, 362, 396**

types 152

using 151

distributed DoS (DDoS) protection 296**DNS services**

Amazon Route 53, securing 131, 132

Azure DNS, securing 133

best practices 130

Google Cloud DNS, securing 134, 135

securing 130

security risks 130

Domain Name System (DNS) 115**Domain Name System Security****Extensions (DNSSEC) 130****domain-specific language (DSL) 426****dynamic application security testing****(DAST) 279, 314, 399****E****EC2 Instance Connect Endpoint 21****ECS capacity 51****ECS cluster 51****ECS components**

ECS cluster 51

ECS task 51

ECS task 51**egress data 380**

- egress-only internet gateway** 70
- EKS Pod Identities** 38
- Elastic Block Store (EBS)** 27
- Elastic Compute Cloud (EC2)** 291
- Elastic Container Registry (ECR)** 35
- Elastic Container Service (ECS)**
 - auditing and security monitoring,
 - best practices 53, 54
 - best practices 52
 - identity and access management (IAM), best practices 52, 53
 - securing 51
- Elastic File System (EFS)** 27
- Elastic Kubernetes Service (EKS)** 360
- Elastic Load Balancing (ELB)** 153
- elliptical curve cryptography (ECC)** 246
- encrypted protocols** 349
- encryption** 242, 243
 - asymmetric encryption 245
 - symmetric encryption 243, 244
 - terminology 242, 243
- encryption algorithm** 243
- encryption at rest** 249
 - encryption/decryption process 249
 - object storage encryption 250
- encryption in transit**
 - best practices 247
 - Internet Protocol security (IPSec) 247
 - Transport Layer Security (TLS) 247, 248
- encryption in use** 264
 - AWS Nitro Enclaves 264, 265
 - Azure confidential computing 266-268
 - Google Confidential Computing 268-270
- encryption key** 243
- Enterprise Azure Policy as Code (EPAC)** 428
- Enterprise Discount Program (EDP)** 317
- enterprise resource planning (ERP)** 4, 305
- Event Threat Detection** 203

- Exchange Online logs** 221
- Extensible Service Proxy (ESP)** 356

F

- FaaS best practices**
 - AWS Lambda 64
 - Azure functions 64
 - Google Cloud Run functions 64
- Fast IDentity Online 2 (FIDO2)** 288
- Federal Risk and Authorization Management Program (FedRAMP)** 306
- file storage** 77
 - auditing and security monitoring,
 - best practices 107-110
 - securing 97
 - securing, best practices 98, 99
- File Transfer Protocol (FTP)** 21, 71
- financial management (FinOps)** 380
- foundation models (FM)** 167
- fully qualified domain names (FQDNs)** 146
- function as a service**
 - (FaaS) 61, 283, 305, 328, 361, 396
 - asynchronous invocation 63
 - auditing and security monitoring,
 - best practices 72, 73
 - best practices 63
 - compliance, best practices 75
 - data protection, best practices 68
 - IAM best practices 64, 65
 - securing 62
 - synchronous invocation 62

G

- gcloud CLI** 14
- GCP shared responsibility model** 11, 12
 - reference link 12

GenAI 167

- audit and security monitoring,
 - best practices 179
- data protection, best practices 176
- example workflow 169, 170
- foundation models (FM) 167
- IAM best practices 173
- large language models (LLMs) 168
- prompt 168
- response 168

General Data Protection Regulation (GDPR) 277, 311, 382**generative AI (GenAI)-based tools 397****generative artificial intelligence (GenAI) 359****GitHub Advanced Security 401****GitHub Copilot 401****GKE**

- CSI driver, securing 113, 114

GKE Dataplane V2 45**GKE Gateway controller 45****Global Secure Access**

- components 146
- securing 146

Google

- IaC best practices 423
- PaC best practices 423

Google Access Approval 223, 224**Google Access Transparency 223, 224****Google App Engine (GAE) 6, 149****Google Artifact Registry 36, 41****Google Binary Authorization 57****Google Cloud Armor 59**

- using 157, 158, 165, 166

Google Cloud Audit Logs 59, 64, 110, 421

- securing, best practices 217, 218

Google Cloud Audit Logs (CSI) driver

- securing 110
- securing, best practices 110, 111

- securing, in AKS 112, 113

- securing, in Amazon EKS 111, 112

- securing, in GKE 113, 114

Google Cloud Backup 29**Google Cloud Deployment Manager 428**

- best practices 428

- Organization Policy Service 428

Google Cloud Directory Sync (GCDS) 335

- best practices 336

Google Cloud DNS 134

- best practices 134, 135

- securing 134, 135

Google Cloud governance 419, 420

- Google environments, best practices 420, 421

- Google Resource Manager 420

- landing zones 420

Google Cloud IAM 40, 57, 64, 199, 421

- permission evaluation logic 200

- terminology 199

Google Cloud Identity 199, 238, 370, 421

- best practices 336

Google Cloud Interconnect 373

- best practices 343

Google Cloud KMS 257

- customer-managed encryption keys (CMEKs) 257

- customer-supplied encryption keys (CSEKs) 257

- default encryption keys 257

Google Cloud KMS inventory service 258**Google Cloud Logging 59****Google Cloud NAT 125****Google Cloud Platform (GCP) 4, 16**

- network connectivity, creating 373-376

Google Cloud Platform VPC

- (GCP VPC) 116

Google Cloud Run functions 64
 reference link 64

Google Cloud SDK 14
 reference link 14

Google Cloud Storage 64

Google Cloud VPC 119
 reference link 119

Google Cloud VPC monitoring
 best practices 129

Google Cloud VPN 373
 best practices 143
 reference link 142
 securing 142, 143

Google Cloud VPN, components
 Cloud VPN gateway 142
 Peer VPN gateway 142

Google Compute Engine
 (GCE) 6, 17, 96, 149

Google Confidential Computing 268
 AMD Secure Encrypted
 Virtualization (SEV) 268
 best practices 270
 Intel TDX 268
 security benefits 269
 SEV-SNP 268

Google Distributed Cloud (GDC) 352
 best practices 352

Google Firestore 57

Google Identity Platform 37, 371

Google Kubernetes Engine
 (GKE) 36, 96, 149, 229, 402
 best practices 36

Google Managed Service for
 Microsoft Active Directory
 best practices 208, 209
 securing 208

Google Privileged Access
 Manager (PAM) 421

Google Resource Manager 420
 reference link 420

Google Secret Manager 40, 58, 68, 262, 263

Google Security Command Center
 (SCC) 49, 130, 135, 229, 259, 422
 capabilities 229-232

Google Security Operations 230, 421

Google's Persistent Disk
 securing, best practices 96, 97

Google Vertex AI 173

Google Workforce Identity Federation 238

Google Workspace 6

governance
 guardrails 408
 landing zones 408

GuardDuty Runtime Monitoring 47
guardrails 408

H

hardware security module (HSM) 379

Health Insurance Portability
 and Accountability Act
 (HIPAA) 281, 311, 382

host log 307

HTTP Strict Transport Security (HSTS) 248

human authentication 187

hybrid cloud 6, 325

hybrid cloud environments
 securing 354
 securing, via AWS hybrid
 environments 354, 355
 securing, via Azure hybrid
 environments 355
 securing, via GCP hybrid
 environments 356, 357

**hybrid cloud environments,
computing services 349**

for AWS hybrid environments 349

for Azure hybrid environments 351

for GCP hybrid environments 352

**hybrid cloud environments,
identity management 329**

best practices 330

GCP hybrid environments 335, 336

hybrid AWS environments 330-332

hybrid Azure environments 333, 334

**hybrid cloud environments, network
architecture 337, 338**

on-premises environment,
connecting to AWS 338, 339

on-premises environment,
connecting to Azure 340, 341

on-premises environment,
connecting to GCP 342, 343

**hybrid cloud environments,
storage services 344**

best practices 344

connecting, via AWS hybrid
environments 344

connecting, via Azure hybrid
environments 346

connecting, via GCP hybrid
environments 348

hybrid cloud strategy 327

application modernization 328

archive and data retention 328

backup and disaster recovery 327

cloud bursting 327

distributed data processing 328

hybrid identity 187

IAM Access Analyzer 66

IAM approach

AWS IAM 186

Google Cloud IAM 186

Microsoft Entra ID 186

IAM best practices

AWS Lambda 65, 66

Azure functions 67

Google Cloud Run functions 68

in function as a service (FaaS) 64, 65

IAM best practices, in GenAI services 173

Amazon Bedrock 174

Azure OpenAI 174, 175

Google Vertex AI 175, 176

IAM, in file storage

best practices 99-103

IAM, in GCP

best practices 201-203

securing 199

IAM, in object storage

best practices 80-83

IAM, multi-cloud environment

authorization phase 365

central IdP phase 365

IAM policies

identity-based policies 189

resource-based policies 189

IAM recommender 202

identity and access management, aspects

authentication 37

authorization 37

pods permission 37

- identity and access management,**
 - best practices** 37
 - Amazon EKS 38, 39
 - Azure AKS 39, 40
 - Google Kubernetes Engine (GKE) 40
 - in Kubernetes 37
- Identity and Access Management (IAM)** 25, 78, 145, 185, 275, 312
 - authentication process 187
 - authorization process 187
 - cloud environments identities 187
 - identities management failure 186
 - versus multi-cloud environments 364-371
- Identity-Aware Proxy (IAP)** 24, 58
- identity in AWS, over multi-cloud environments**
 - managing 366-368
- identity in Azure, over multi-cloud environments**
 - managing 368-370
- identity in GCP, over multi-cloud environments**
 - managing 370-372
- identity management**
 - (IdM) 185, 309, 362, 363
- identity provider (IdP)** 18, 145, 309, 396
- incident response (IR)** 275, 307, 366
- indicators of compromise (IOCs)** 236, 280, 380
- information security** 7
- Infrastructure as a Service (IaaS)** 6, 15, 276, 304, 320, 387
- infrastructure as code (IaC)** 278, 395, 408, 422
 - benefits 422
 - best practices, in AWS 423
 - best practices, in Azure 423
 - best practices, in Google 423
 - declarative 422
 - imperative 422
- ingress controller** 34
- insecure APIs, in cloud services** 294
 - consequences 294
 - detecting and mitigating, best practices 295-297
 - example 294
- insider threat** 290
- insider threat, in cloud services** 290
 - consequences 291
 - detecting and mitigating, best practices 291-294
 - example 291
- instance-attached storage** 77
- Instance Metadata Service (IMDS)** 298
- insufficient IAM** 284
- insufficient IAM, secrets and key management**
 - consequences 284
 - detecting and mitigating, best practices 285, 286
 - example 284
- integrated development environment (IDE)** 396
- interactive application security testing (IAST)** 279, 314, 399
- interactive authentication** 187
- International Organization for Standardization (ISO)** 278, 315
- Internet Key Exchange version 2 (IKEv2)** 141
- Internet of Things (IoT)** 305
- Internet Protocol security (IPSec)** 247
- internet service providers (ISPs)** 5
- IR team (IRT)** 313

J**JSON Web Token (JWT)** 295**just-in-time (JIT)** 23, 148, 228, 255, 369, 418**K****Kerberos-constrained delegation (KCD)** 208**key encryption key (KEK)** 243**key generation** 243**Key Management Service (KMS)** 27, 241, 379**KMSs deployment, best practices** 250

AWS KMS 251-253

Azure Key Vault 254-256

general best practices 250

Google Cloud KMS 257-259

kube-bench tool 49**Kubernetes** 32auditing and security monitoring,
best practices 46

data protection, best practices 41

identity and access management,
best practices 37

secure configuration, implementing 49

securing 33

terminology 33

Kubernetes, best practices 35Amazon Elastic Kubernetes
Service (Amazon EKS) 35

Azure Kubernetes Service (AKS) 36

Google Kubernetes Engine (GKE) 36

Kubernetes cluster components

control plane components 33

ingress controller 34

Kubernetes service 34

namespace 34

worker node components 33

Kubernetes, identity types

pod identity 37

service account 37

user identity 37

Kubernetes network access

best practices 41, 42

Kubernetes service 34**Kubernetes terminology** 35**L****labeling** 121**Lambda Protection in Amazon
GuardDuty** 74**landing zones** 408

benefits 408

large language model (LLM) 168, 359**LetsEncrypt certificates** 248**Lightweight Directory Access
Protocol (LDAP)** 285**local gateway (LGW)** 349**Log Analytics workspace** 74, 418**M****machine learning (ML)** 280, 305**managed GenAI services** 170

Amazon Bedrock 172

Azure OpenAI 173

best practices, for mitigating risks 171, 172

considerations, for data usage 170

Google Vertex AI 173

risk 171

managed identity type

system-assigned 194

user-assigned 194

master encryption key (MEK) 243**MFA delete** 82

- Microsoft 365** 6
- Microsoft Azure** 4
- Microsoft Cloud Security**
 - Benchmark (MCSB)** 228
- Microsoft Copilot for Security** 236
- Microsoft Defender for App Service** 75
- Microsoft Defender for**
 - Cloud** 226-229, 402, 418
- Microsoft Defender for Key Vault** 256
- Microsoft Entra authentication** 142
- Microsoft Entra Conditional Access** 196
- Microsoft Entra Conditional**
 - Access policies** 255
- Microsoft Entra Connect** 333
- Microsoft Entra Domain Services**
 - best practices 207, 208
 - reference link 207
 - securing 207
- Microsoft Entra Global Administrator**
 - best practices 196
 - reference link 196
- Microsoft Entra ID** 37, 39, 54, 64, 67, 82, 141, 418
 - cloud-only identity 194
 - hybrid identity 194
 - permissions evaluation logic 196
 - securing 194
 - terminology 194, 195
- Microsoft Entra ID security**
 - best practices 197, 198
 - reference link 197
- Microsoft Entra Internet Access** 146
- Microsoft Entra Password Protection** 197
- Microsoft Entra PIM** 418
- Microsoft Entra Private Access** 146
 - best practices 147, 148
 - components 146
 - connection workflow 146
- Microsoft Entra Privileged Identity Management (PIM)** 23
- Microsoft Power Platform** 236
- Microsoft Purview** 235
- Microsoft Sentinel** 156
- misconfigurations** 280
- misconfigurations, in cloud**
 - services** 280, 281
 - consequences 281
 - detecting and mitigating, best practices 282, 283
 - example 281, 282
- MITRE ATT&CK framework** 301
- multi-cloud** 6
- multi-cloud environments**
 - cost management 380, 381
 - data security 378
 - monitoring and auditing 387
 - network architecture 372, 373
 - patch and configuration management 384-386
 - versus IAM 364-366
- multi-cloud strategy** 360
 - asset management 363
 - BC, achieving 362
 - data security 363
 - data sovereignty 361
 - DR, achieving 362
 - freedom, to select cloud provider 361
 - freedom, to select services 361
 - identity management (IdM) 362, 363
 - reliability, improving 362
 - skills gap 364
- multi-factor authentication (MFA)** 65, 80, 136, 146, 276, 312, 336, 364
- mutual TLS (mTLS)** 43

N

namespace 34

NAT Gateway 70

National Institute of Standards and Technology (NIST) 5, 278, 386

broad network access 5

measured service 5

on-demand self-service 5

rapid elasticity 5

reference link 5

resource pooling 5

National Institute of Standards and Technology Secure Software Development Framework (NIST SSDF) 314

network access control lists (NACLs) 20, 278

network access controls

stateful 117

stateless 117

working 117

network access, in Azure VNet

best practices 123

network access security, best practices 20, 21

Amazon EC2 21, 22

Azure VMs 22, 23

GCE 23, 24

network ACLs (NACLs) 338, 372

network architecture, for multi-cloud environments 372

considerations 372

network connectivity, between
AWS and Azure 375-378

network connectivity, between
AWS and GCP 373-375

network-attached storage (NAS) 97

network connectivity

creating, between AWS and Azure 375

creating, between AWS and GCP 373-377

creating, between Azure and GCP 377

Network File System (NFS) 313

network interface controller (NIC) 22

network security group

(NSG) 22, 56, 71, 376

New Technology LAN Manager (NTLM) 285

node controller 33

non-disclosure agreement (NDA) 316

non-human authentication 187

non-interactive authentication 187

non-Kubernetes workloads

Cloud Run, securing 57

Elastic Container Service (ECS), securing 51

securing 50

O

object storage 77

auditing and security monitoring,

best practices 88-92

securing 78

securing, best practices 78, 79

one-time password (OTP) 279, 370

Open Authorization 2.0

(OAuth 2.0) 295, 309

Open Authorization (OAuth) 187

Open Container Initiative (OCI) 32

OpenID Connect

(OIDC) 37, 67, 112, 187, 309, 329

Open Service Mesh (OSM) 44

open source tools 403-405

OpenSUSE 13

OpenVPN 138

Oracle Fusion Cloud HCM (Human Capital Management) 6

orchestrator 32

Organization Policy Service 422, 428
 best practices 428
 reference link 429

P

Payment Card Industry Data Security Standard (PCI DSS) 242, 311, 379

Payment Card Industry (PCI) 306

penetration testing
 conducting, in cloud environments 319-321

perfect forward secrecy (PFS) 248

Personally Identifiable Information (PII) 85, 145, 241, 284, 311

plaintext 242

platform as a service (PaaS) 6, 276, 304, 320, 387

Point-to-Site VPN 135

policy as code (PaC) 394, 408, 423
 best practices, in AWS 423
 best practices, in Azure 423
 best practices, in Google 423

Policy Intelligence 203, 230

Policy Simulator 202

principle of least privilege (PoLP) 27, 284, 312, 365

private cloud 5

Private Service Connect (PSC) 87, 125

Privileged Access Manager (PAM) 202

Privileged Identity Management (PIM) 148, 196, 255, 369

prompt 168

public cloud 6

R

Recovery Point Objective (RPO) 315

Recovery Time Objective (RTO) 314

Relational Database Service (RDS) 27, 277

Remote Desktop Protocol (RDP) 21, 281

replication controller 33

Resolver query logging 132

resource control policies (RCPs) 190, 426
 best practices 426
 reference link 426

resource group 118

RHEL 13

Rivest-Shamir-Adleman (RSA) 246

role-based access control (RBAC)
 28, 29, 39, 80, 171, 188, 212,
 250, 278, 312, 368, 396

route tables 338

S

S3 Object Lock 85

S3 protection 225

Salesforce CRM 6

SAP SuccessFactors 6

secrets 259

secrets management service,
 best practices 259

 AWS Secrets Manager 260, 261

 Azure Key Vault 261

 Google Secret Manager 262, 263

secure configuration

 Amazon EKS 49, 50

 Azure AKS 50

 Google Kubernetes Engine (GKE) 50
 implementing, in Kubernetes 49

Secure Shell (SSH) 18, 281, 314, 383

Secure Sockets Layer (SSL) protocol 247

- Secure Socket Tunneling Protocol (SSTP)** 141
- security backups, best practices** 26
 - AWS Backup 27
 - Azure Backup 28
 - Google Cloud Backup and DR 29
- Security Assertion Markup Language (SAML)** 187, 276, 309, 329, 330, 366
- security association (SA)** 247
- Security Development Lifecycle (SDL)** 314
- security information and event management (SIEM)** 73, 232, 307, 330, 366, 400
- Security Operations Center (SOC)** 192, 283, 400
- Security Orchestration, Automation, and Response (SOAR)** 230
- security posture** 231
- Security Token Service (STS)** 286
- segregation of duties (SoD)** 286
- Sentinel policies** 430
 - best practices 430
 - reference link 431
- serverless** 61
 - securing 61
- Serverless VPC Access** 72
- Server Message Block (SMB)** 21, 313
- server-side encryption** 215
- service control policies (SCPs)** 81, 190, 414, 425
 - best practices 425, 426
- service-level agreement (SLA)** 303, 361
- service log** 307
- service provider (SP)** 283, 305
- shared access signature (SAS)** 82
- shared or preconfigured account baselines**
 - identity and access management 408
 - management and monitoring 408
 - network 408
 - preconfigured account baselines 409
- shared responsibility model (SRM)** 8-10, 16, 115, 310
 - AWS shared responsibility model 10
 - Azure shared responsibility model 11
 - GCP shared responsibility model 11, 12
 - reference link 10
- Signaling System No. 7 (SS7)** 289
- Simple Storage Service (S3)** 27, 146, 281, 365
- single sign-on (SSO)** 139, 363
- Site-to-Site VPN** 135
- SOC 1 reports** 316
- SOC 1 Type 1 report** 316
- SOC 2 reports** 316
- SOC 2 Type 1** 316
- SOC 2 Type 2** 316
- SOC 3 reports** 316
- SOC reports** 316
- software as a service (SaaS)** 6, 276, 304, 320, 364
- software bill of material (SBOM) tools** 398
- software composition analysis (SCA)** 296, 314, 385, 398
- software-defined networking (SDN)** 116
- software development kit (SDK)** 13, 282
- software development life cycle (SDLC)** 295, 392
- source network address translation (SNAT)** 124
- SSH keys, in Azure portal**
 - reference link 19
- SSL Labs test**
 - reference link 312
- stateful** 117
- stateless** 117
- static application security testing (SAST)** 279, 314, 397
- storage area network (SAN)** 92

Storage Transfer Service

- best practices 348

subnets 116**symmetric encryption 241-244**

- Advanced Encryption Standard (AES) 244
- cons 244
- pros 244

synchronous invocation 62**System and Organization Controls (SOC) 2 Type 2 reports 303****system-assigned 194****T****tactics, techniques, and**

- procedures (TTPs) 300

tagging 121**Temporary Access Pass (TAP) 197****temporary OTPs (TOTPs) 288****Terraform 429**

- best practices 430
- reference link 430

threat detection and response (TDR)

- Amazon GuardDuty 224-226
- conducting 224
- Google Security Command Center (SCC) 229-232
- Microsoft Defender for Cloud 226-229

threat intelligence (TI) 300**Transmission Control Protocol (TCP) 20****Transport Layer Security (TLS) 69, 247, 379****Transport Layer Security (TLS) 1.2 312****trusted execution environment (TEE) 264****two-factor authentication (2FA) 19****U****Ubuntu 13****user acceptance testing (UAT) 392****User and Entity Behavior**

- Analytics (UEBA) 236

user-assigned 194**User Datagram Protocol (UDP) 20****user-defined routes (UDRs) 56, 123****V****version control systems (VCSs) 391****virtual CPU (vCPU) 16****virtual local area networks (VLANs) 116****virtual machines (VMs) 15, 304, 361, 396**

- fundamental, best practices 16, 17
- securing 15, 16

virtual machines (VMs), best practices

- Amazon EC2 17
- Azure VMs 17
- GCE 18

virtual network flow logs 128**virtual networking**

- Azure Virtual Network (Azure VNet) 118, 119
- failure consequences 117
- features 116
- Google Cloud VPC 119
- recommendations 116
- securing 115, 116

virtual networking, best practices 120, 121

- Amazon VPC 121, 122
- Azure VNet 123, 124
- Google Cloud VPC 124, 125

**virtual networks monitoring,
best practices 126**

Amazon VPC 126, 127
Azure VNet 128
Google Cloud VPC 129, 130

virtual network (VNet) 85**Virtual Private Cloud (VPC) 22, 84****virtual private gateway 136****virtual private networks (VPNs) 115****VPC firewall rules 124****VPC Service Controls 24, 59,
72, 87, 125, 150, 357****VPN services**

AWS Client VPN, securing 138
AWS Site-to-Site VPN, securing 136
Azure VPN Point-to-Site
(P2S), securing 141
best practices 136
Google Cloud VPN, securing 142, 143
Point-to-Site VPN 135
securing 135
Site-to-Site VPN 135

W

WAF services

AWS WAF, using 159-161
Azure Application Gateway
WAF v2, using 162, 163
Azure Front Door, using 163-165
best practices 158
Google Cloud Armor, using 165, 166
using 158

**Web Application Firewall on
Application Gateway 72****Web Application Firewall
(WAF) 158, 368, 396****Web Security Scanner 232****worker node components 33**

kubelet 33
kube-proxy 33
Kubernetes runtime 33
pod 33

Workload Identity Federation 353

Z

Zero Trust services 115

AWS Verified Access, securing 144
BeyondCorp Enterprise, securing 148
Global Secure Access, securing 146
securing 143, 144



packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

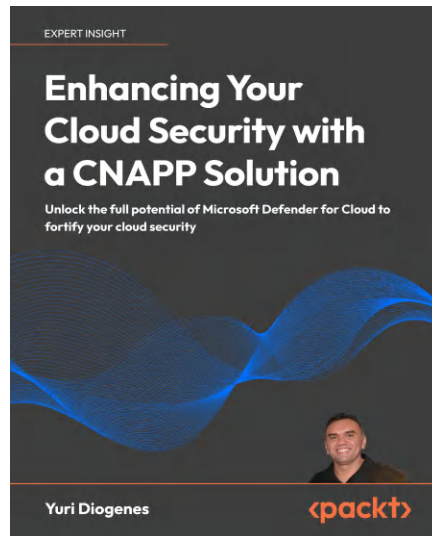
- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packtpub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

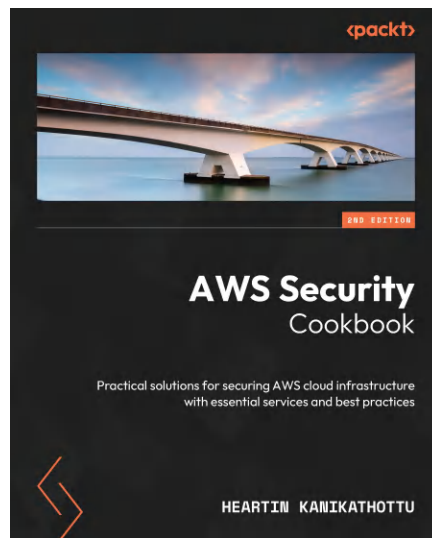


Enhancing Your Cloud Security with a CNAPP Solution

Yuri Diogenes

ISBN: 978-1-83620-487-9

- Implement Microsoft Defender for Cloud across diverse IT environments
- Harness DevOps security capabilities to tighten cloud operations
- Leverage AI tools such as Microsoft Copilot for Security to help remediate security recommendations at scale
- Integrate Microsoft Defender for Cloud with other XDR, SIEM (Microsoft Sentinel) and Microsoft Security Exposure Management
- Optimize your cloud security posture with continuous improvement practices
- Develop effective incident response plans and proactive threat hunting techniques



AWS Security Cookbook

Heartin Kanikathottu

ISBN: 978-1-83508-189-1

- Manage AWS accounts and users with AWS Organizations and IAM Identity Center
- Secure data and infrastructure with IAM policies, RBAC, and encryption
- Enhance web security with TLS, load balancers, and firewalls
- Use AWS services for logging, monitoring, and auditing
- Ensure compliance with machine-learning-powered AWS services
- Explore identity management with Cognito, AWS directory services, and external providers such as Entra ID
- Follow best practices to securely share data across accounts

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packtpub.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Now you've finished *Cloud Security Handbook Second Edition*, we'd love to hear your thoughts! If you purchased the book from Amazon, please [click here](#) to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/9781836200017>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

